Astra Trident 22.04文件

Astra Trident

NetApp
November 17, 2022

This PDF was generated from https://docs.netapp.com/zh-tw/trident-2204/index.html on November 17, 2022. Always check docs.netapp.com for the latest.

目錄

Astra Trident 22.04文件 1
版本資訊 2
22.04的新功能 2
22.01.1中的變更 3
22.01.0版的變更 3
210.1的變更 3
21.0 4
已知問題 4
如需詳細資訊、請參閱 5
概念 6
Astra Trident簡介 6
驅動程式ONTAP 7
資源配置 7
Volume快照 8
虛擬儲存資源池 8
Volume存取群組 9
開始使用 11
歡迎試用 11
需求 11
部署總覽 15
與Trident營運者一起部署 18
使用tridentctl部署 25
接下來呢？ 29
管理Astra Trident 34
升級Astra Trident 34
與營運者一起升級 35
使用tridentctl進行升級 43
解除安裝Astra Trident 46
降級Astra Trident 48
使用Astra Trident 52
設定後端 52
使用kubecl建立後端 114
以KECBECVL執行後端管理 119
使用tridentctl執行後端管理 121
在後端管理選項之間切換 122
管理儲存類別 129
執行Volume作業 131
準備工作節點 156
自動準備工作節點 159

監控Astra Trident 159
適合Docker的Astra Trident 164
部署的先決條件 164
部署Astra Trident 167
升級或解除安裝Astra Trident 171
使用Volume 173
收集記錄 181
管理多個Astra Trident執行個體 182
儲存組態選項 183
已知問題與限制 197
常見問題集 199
一般問題 199
在Kubernetes叢集上安裝及使用Astra Trident 199
疑難排解與支援 200
升級Astra Trident 201
管理後端和磁碟區 202
支援 206
疑難排解 207
一般疑難排解 207
使用操作員疑難排解不成功的Trident部署 208
疑難排解使用tridentctl進行不成功的Trident部署 210
最佳實務做法與建議 211
部署 211
儲存組態 211
整合Astra Trident 217
資料保護 225
安全性 230
參考資料 232
Astra Trident連接埠 232
Astra Trident REST API 232
命令列選項 233
與Kubernetes整合的NetApp產品 234
Kubernetes和Trident物件 235
tridentctl命令和選項 245
較早版本的文件 251
法律聲明 252
版權 252
商標 252
專利 252
隱私權政策 252
開放原始碼 252

Astra Trident 22.04文件

版本資訊

版本資訊提供最新版Astra Trident的新功能、增強功能及錯誤修正資訊。

> 安裝程式壓縮檔中提供的Linux試用版（tridentctl）二進位檔是經過測試且受支援的版本。請注意、不會測試或支援壓縮檔中「/Extras」部分提供的「macos」二進位檔。

22.04的新功能

NetApp持續改善及強化其產品與服務。以下是Astra Trident的一些最新功能。如需舊版的資訊、請參閱 "較早版本的文件"。

> 如果您要從任何先前的Trident版本升級並使用Azure NetApp Files 更新版本、則「位置」組態參數現在是必填的單一欄位。

修正

- 改善iSCSI啟動器名稱的剖析。（問題[#681](https://github.com/NetApp/trident/issues/681))
- 修正不允許使用csi儲存類別參數的問題。（問題[#598](https://github.com/NetApp/trident/issues/598))
- 修復Trident CRD中的重複金鑰宣告。（問題[#671](https://github.com/NetApp/trident/issues/671))
- 修正不正確的「csi Snapshot記錄」。（問題[#629](https://github.com/NetApp/trident/issues/629))
- 已修正在刪除節點上解除發佈磁碟區的問題。（問題[#691](https://github.com/NetApp/trident/issues/691))
- 新增區塊裝置上檔案系統不一致的處理方式。（問題[#656](https://github.com/NetApp/trident/issues/656))
- 修正在安裝期間設定「imageRegistry（影像登錄）」旗標時拉出自動支援映像的問題。（問題[#715](https://github.com/NetApp/trident/issues/715))
- 修正ANF驅動程式無法複製具有多個匯出規則的磁碟區的問題。

增強功能

- 若要連入Trident的安全端點、現在至少需要TLS 1.3。（問題[#698](https://github.com/NetApp/trident/issues/698))
- Trident現在將HSTC標頭新增至其安全端點的回應。
- Trident現在會自動嘗試啟用Azure NetApp Files 「UNIX權限」功能。
- * Kubernetes*：Trident取消程式集現在以系統節點關鍵優先順序類別執行。（問題[#694](https://github.com/NetApp/trident/issues/694))

移除

E系列驅動程式（自20.07起停用）已移除。

22.01.1中的變更

修正

- 已修正在刪除節點上解除發佈磁碟區的問題。（"GitHub問題#691"）
- 存取零欄位以取得ONTAP 靜止API回應中的集合空間時、會出現固定的恐慌。

22.01.0版的變更

修正

- * Kubernetes：*增加大型叢集的節點登錄回退重試時間。
- 已解決以下問題：azure-NetApp-Files驅動程式可能會被同名的多個資源混淆。
- 如果使用方括弧指定SAN IPv6資料生命量、現在就能正常運作。ONTAP
- 修正嘗試匯入已匯入磁碟區傳回EOF、使PVc處於擱置狀態的問題。（"GitHub問題#489"）
- 修正當在SolidFire 一個穩定區上建立超過32個快照時、Astra Trident效能變慢的問題。
- 在建立SSL憑證時、以SHA-256取代SHA-1。
- 固定ANF驅動程式、允許重複的資源名稱、並將作業限制在單一位置。
- 固定ANF驅動程式、允許重複的資源名稱、並將作業限制在單一位置。

增強功能

- Kubernetes增強功能：
  - 新增對Kubernetes 1.23的支援。
  - 透過Trident運算子或Helm安裝Trident Pod時、請新增排程選項。（"GitHub問題#65"）
- 允許GCP驅動程式中的跨區域磁碟區。（"GitHub問題#633"）
- 新增對anf磁碟區「unix權限」選項的支援。（"GitHub問題#6666"）

棄用

Trident REST介面只能以127.0.0.1或[:1]位址接聽和使用

210.1的變更

> v21.10.0版本發生問題、可在移除節點後將Trident控制器重新新增回Kubernetes叢集時、將其置於CrashLooper BackOff狀態。此問題已在版本210.1中修正（GitHub問題669）。

修正

- 修正在GCP CVS後端匯入磁碟區時可能發生的競爭狀況、導致無法匯入。
- 修正刪除節點後、將Trident控制器重新加入Kubernetes叢集（GitHub問題669）時、使Trident控制器進

入CrashLooper BackOff狀態的問題。

- 修正未指定SVM名稱時不再探索SVM的問題（GitHub問題612）。

21.0

修正

- 修正XFS磁碟區的複本無法與來源磁碟區掛載在同一個節點上的問題（GitHub問題514）。
- 修正Astra Trident在關機時記錄嚴重錯誤的問題（GitHub問題597）。
- Kubernetes相關修正：
  - 使用「ONTAP-NAS」和「ONTAP-NAS-flexgroup」驅動程式建立快照時、傳回磁碟區的已用空間作為最小重新設定大小（GitHub問題645）。
  - 修正磁碟區調整大小後記錄「無法擴充檔案系統」錯誤的問題（GitHub問題560）。
  - 已解決Pod可能陷入「終止」狀態的問題（GitHub問題572）。
  - 修正「ONTAP-san經濟」FlexVol 的情況、即快照LUN可能已滿（GitHub問題533）。
  - 修正不同映像的自訂Yaml安裝程式問題（GitHub問題613）。
  - 修正快照大小計算（GitHub問題611）。
  - 修正所有Astra Trident安裝程式可將純Kubernetes識別為OpenShift的問題（GitHub問題639）。
  - 修正Trident運算子、在Kubernetes API伺服器無法連線時停止協調（GitHub問題599）。

增強功能

- 新增了對GCP-CVS Performance Volume的「unixPermissions」選項支援。
- 在GCP中新增對大規模最佳化的CVS磁碟區的支援、範圍介於600 GiB到1 TiB之間。
- Kubernetes相關增強功能：
  - 新增對Kubernetes 1.22的支援。
  - 讓Trident運算子和Helm圖表能與Kubernetes 1.22搭配使用（GitHub問題628）。
  - 將操作員映像新增至「tridentctl」映像命令（GitHub Issue 570）。

實驗性增強功能

- 在「ONTAP-san」驅動程式中新增了對Volume複寫的支援。
- 新增*技術預覽* REST支援功能、支援「ONTAP-NAA-flexgroup」、「ONTAP-SAN」和「ONTAP-NAS-P節約」驅動程式。

已知問題

已知問題可識別可能導致您無法成功使用產品的問題。

- 對於未在StorageClass中指定「FSType」的磁碟區、Astra Trident現在會執行空白的「FSType」（「FSType =」）。使用Kubernetes 1.17或更新版本時、Trident支援為NFS磁碟區提供空白的「

FSType」。對於iSCSI磁碟區、您必須在使用安全性內容強制執行「fsGroup」時、在StorageClass上設定「FSType」。

- 在多個Astra Trident執行個體之間使用後端時、每個後端組態檔都應該有不同的「儲存前置字元」值、以供ONTAP 後端使用、或是使用不同的「TenantName」作為SolidFire 後端。Astra Trident無法偵測其他Astra Trident執行個體所建立的磁碟區。嘗試在ONTAP 不穩定或SolidFire 不穩定的後端上建立現有的Volume是成功的、因為Astra Trident將建立Volume視為強大的作業。如果「儲存前置詞」或「TenantName」沒有不同、則在相同後端上建立的磁碟區可能會發生名稱衝突。
- 安裝Astra Trident（使用「tridentctl」或Trident運算子）並使用「tridentctl」來管理Astra Trident時、您應確保設定「KUBECCONFIG」環境變數。這是表示Kubernetes叢集「tridentctl」應該與之搭配使用的必要項目。在使用多個Kubernetes環境時、您應該確保「KUBECCONFIG」檔案的來源正確無誤。
- 若要執行iSCSI PV的線上空間回收、工作節點上的基礎作業系統可能需要將掛載選項傳遞至磁碟區。對於需要「disccard」的RHEL/RedHat CoreOS執行個體來說、這是正確的做法 "掛載選項"；請確定您的隨附了捨棄掛載選項 "d4b9b9554fd820f43eae492d33e41167" 支援線上區塊捨棄。
- 如果每個Kubernetes叢集有一個以上的Astra Trident執行個體、Astra Trident就無法與其他執行個體通訊、也無法探索他們所建立的其他磁碟區、如果叢集內有多個執行個體執行、就會導致非預期和不正確的行為。每個Kubernetes叢集只應有一個Astra Trident執行個體。
- 如果在Astra Trident離線時、從Kubernetes刪除Astra Trident型的「torageClass」物件、則Astra Trident在其資料庫恢復連線時、不會從資料庫中移除對應的儲存類別。您應該使用「tridentctl」或REST API來刪除這些儲存類別。
- 如果使用者刪除由Astra Trident提供的PV、然後再刪除對應的PVC、則Astra Trident不會自動刪除備份Volume。您應該透過「tridentctl」或REST API來移除磁碟區。
- 除非集合體是每個資源配置要求的唯一集合體、否則無法同時配置多個支援區。ONTAP FlexGroup
- 使用Astra Trident over IPv6時、您應該在後端定義的方括弧內指定「managementLIF」和「data LIF」。例如：「[FD20：8b1e：b258：2000：f816：3eff：feec：0]。
- 如果OpenShift 4.5使用「Poolidfire - san」驅動程式、請確定基礎工作者節點使用MD5做為CHAP驗證演算法。

如需詳細資訊、請參閱

- "Astra Trident GitHub"
- "Astra Trident部落格"

概念

Astra Trident簡介

Astra Trident是由NetApp維護的完整支援開放原始碼專案、也是其中的一部分 "Astra產品系列"。其設計旨在協助您使用業界標準介面（例如Container Storage介面（SCSI））來滿足容器化應用程式的持續需求。

Astra Trident將Kubernetes叢集部署為Pod、為Kubernetes工作負載提供動態儲存協調服務。它可讓您的容器化應用程式快速輕鬆地使用NetApp廣泛產品組合中的持續儲存設備、包括ONTAP ：VMware的VMware產品組合（AFF/FAS/Select/Cloud / Amazon FSX for NetApp ONTAP ）、Element軟體（NetApp HCI / SolidFire ）、Astra Data Store、Azure NetApp Files 以及VMware服務、Cloud Volumes Service 以及Google Cloud上的VMware。

Astra Trident也是NetApp Astra的一項基礎技術、運用NetApp領先業界的資料管理技術、提供快照、備份、複寫及複製等快照、備份、複寫及複製功能、為Kubernetes工作負載提供資料保護、災難恢復、可攜性及移轉使用案例。

支援的Kubernetes叢集架構

下列Kubernetes架構支援Astra Trident：

Kubernetes叢集架構	支援	預設安裝
單一主機、運算	是的	是的
多重主機、運算	是的	是的
Master、「etcd」、運算	是的	是的
主要、基礎架構、運算	是的	是的

什麼是Astra？

Astra可讓企業更輕鬆地管理、保護及搬移在公有雲內部及內部部署的Kubernetes上執行的資料豐富的容器化工作負載。Astra透過NetApp在公有雲和內部部署環境中備受肯定且廣泛的儲存產品組合、運用Astra Trident來提供持續的容器儲存設備。此外、它還提供一組豐富的進階應用程式感知資料管理功能、例如Snapshot、備份與還原、活動記錄、以及資料保護、災難/資料恢復、資料稽核、以及Kubernetes工作負載的移轉使用案例等主動複製功能。

您可以在Astra頁面註冊免費試用。

以取得更多資訊

- "NetApp Astra產品系列"
- "Astra Control Service文件"
- "Astra Control Center文件"

- "Astra Data Store文件"
- "Astra API文件"

驅動程式ONTAP

Astra Trident提供五ONTAP 種獨特的Intel儲存驅動程式、可與ONTAP 各種叢集進行通訊。深入瞭解每個驅動程式如何處理磁碟區的建立、存取控制及其功能。

驅動程式	傳輸協定	**Volume**模式	支援的存取模式	支援的檔案系統
「ONTAP-NAS」	NFS	檔案系統	rwo、rwx、ROX	」、NFS
《ONTAP-NANAS經濟》	NFS	檔案系統	rwo、rwx、ROX	」、NFS
「ONTAP-NAA-flexgroup」	NFS	檔案系統	rwo、rwx、ROX	」、NFS
「ONTAP-SAN」	iSCSI	區塊	rwo、ROX、rwx	無檔案系統。原始區塊裝置
「ONTAP-SAN」	iSCSI	檔案系統	Rwo、ROX	《xfs》、《ext3》、《ext4》
《ONTAP-san經濟》	iSCSI	區塊	rwo、ROX、rwx	無檔案系統。原始區塊裝置
《ONTAP-san經濟》	iSCSI	檔案系統	Rwo、ROX	《xfs》、《ext3》、《ext4》

使用安全角色（使用者名稱/密碼）的登入認證、或使用安裝在叢集上的私密金鑰和憑證、即可驗證支援服務。ONTAP ONTAP您可以更新現有的後端、以「tridentctl update backend」從一個驗證模式移到另一個驗證模式。

資源配置

Astra Trident的資源配置有兩個主要階段。第一階段會將儲存類別與一組適當的後端儲存資源池建立關聯、並在進行資源配置之前做好必要準備。第二階段包括磁碟區建立本身、並要求從與擱置磁碟區儲存類別相關的儲存資源池中選擇儲存資源池。

將後端儲存資源池與儲存類別建立關聯、取決於儲存類別所要求的屬性及其「儲存資源池」、「附加儲存資源池」及「排除儲存資源池」清單。當您建立儲存類別時、Trident會比較每個後端所提供的屬性和集區、以及儲存類別所要求的屬性和集區。如果儲存資源池的屬性和名稱符合所有要求的屬性和資源池名稱、Astra Trident會將該儲存資源池新增至該儲存類別適用的儲存資源池集合。此外、Astra Trident也會將「addionalStoragePools」清單中列出的所有儲存集區新增至該集、即使其屬性無法滿足所有或任何儲存類別的要求屬性。您應該使用「排除StoragePools」清單來覆寫及移除儲存類別的儲存資源池。每次新增後端時、Astra Trident都會執行類似的程序、檢查其儲存集區是否符合現有儲存類別的儲存集區、以及是否移除任何已標記為「已排除」的儲存資源池。

Astra Trident接著會使用儲存類別與儲存資源池之間的關聯、來決定要在何處配置磁碟區。當您建立Volume

時、Astra Trident會先取得該磁碟區儲存類別的儲存集區集區集區、 此外、如果您指定磁碟區的傳輸協定、Astra Trident會移除無法提供所需傳輸協定的儲存資源池（例如、NetApp HCI / SolidFire後端無法提供檔案型磁碟區、ONTAP 而非由NAS後端提供區塊型磁碟區）。Astra Trident會隨機排列此結果集的順序、以利均勻散佈磁碟區、然後透過磁碟區重複資料、進而嘗試在每個儲存集區上配置磁碟區。如果某個項目成功、則會成功傳回、並記錄程序中發生的任何故障。Astra Trident返回故障*僅當*無法在* all *上配置所需儲存類別和傳輸協定可用的儲存資源池時。

Volume快照

深入瞭解Astra Trident如何為其驅動程式建立磁碟區快照。

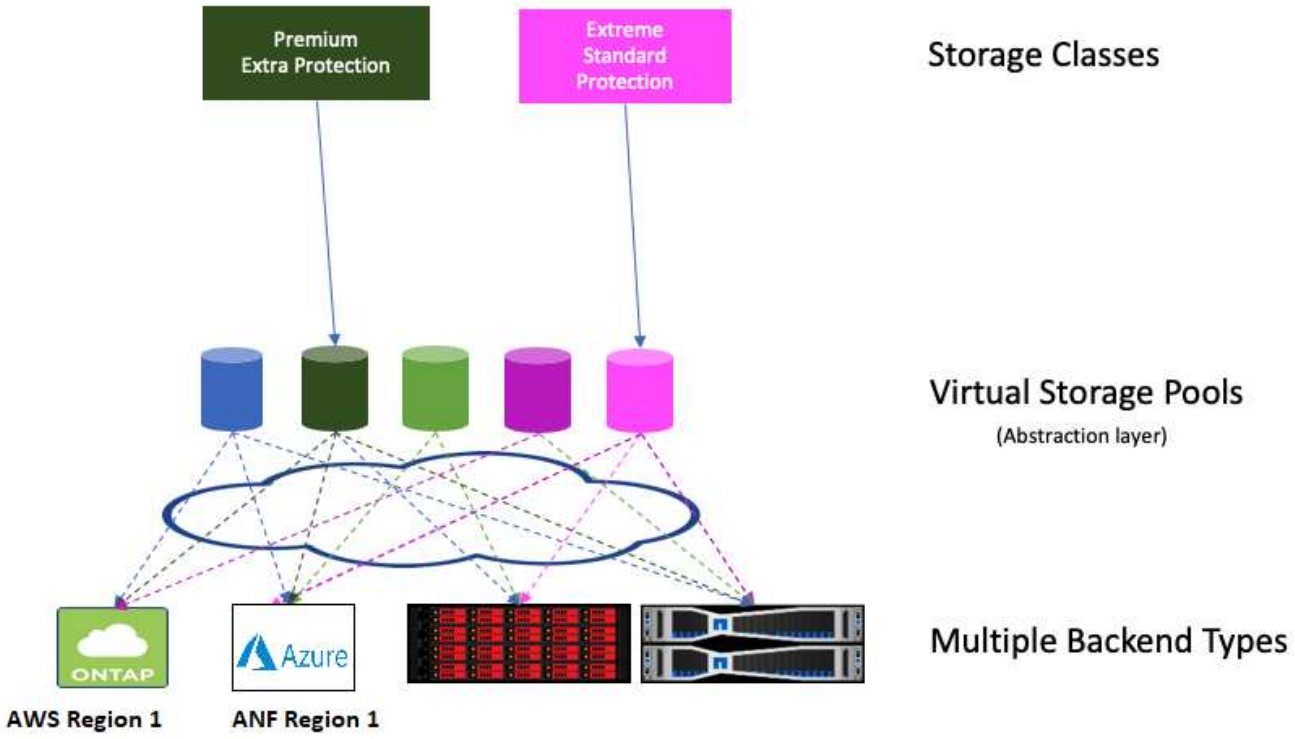
- 對於「ONTAP-NAS」、「ONTAP-SAN」、「GCP-CVS」和「azure-NetApp-files'」等驅動程式、每個「持續Volume（PV）」都會對應FlexVol 到一個版本。因此、磁碟區快照會建立為NetApp快照。NetApp的Snapshot技術提供比競爭快照技術更高的穩定性、擴充性、可恢復性和效能。這些Snapshot複本無論在建立所需的時間、還是在儲存空間中、都能發揮極高的效率。
- 對於「ONTAP-NAS-flexgroup」驅動程式、每個「持續Volume」（PV）都會對應FlexGroup 到一個VMware。因此、磁碟區快照會建立為NetApp FlexGroup 的「資訊快照」。NetApp的Snapshot技術提供比競爭快照技術更高的穩定性、擴充性、可恢復性和效能。這些Snapshot複本無論在建立所需的時間、還是在儲存空間中、都能發揮極高的效率。
- 對於「ontap-san經濟」驅動程式、PV會對應至在共享FlexVols上建立的LUN。PV的Volume Snapshot是透過執行相關LUN的FlexClones來達成的。ONTAP的FlexClone技術幾乎可以即時建立最大資料集的複本。複本會與其父實體共用資料區塊、除了中繼資料所需的儲存空間外、不需要使用任何儲存設備。
- 對於「Poolidfire - san」驅動程式、每個PV對應至NetApp Element 在該軟體/NetApp HCI叢集上建立的LUN。Volume Snapshot以基礎LUN的元素快照來表示。這些快照是時間點複本、只佔用少量系統資源和空間。
- 使用「ONTAP-NAS」和「ONTAP-SAN」驅動程式時、ONTAP 不只是FlexVol 指時間點的複本、還會佔用FlexVol 整個過程中的空間。這可能會產生磁碟區中的可寫入空間量、以便在建立/排程快照時縮短時間。解決此問題的一種簡單方法、就是透過Kubernetes調整大小來擴充磁碟區。另一個選項是刪除不再需要的快照。當透過Kubernetes建立的Volume Snapshot被刪除時、Astra Trident會刪除相關的ONTAP 功能快照。不透過Kubernetes建立的支援快照也可以刪除。ONTAP

有了Astra Trident、您可以使用Volume Snapshot來建立新的PV。使用FlexClone技術建立這些快照的PV、以支援ONTAP 支援的支援的支援功能和CVS後端。從快照建立PV時、備份磁碟區是Snapshot父磁碟區的FlexClone。「Poolidfire - san」驅動程式使用Element軟體磁碟區複製、從快照建立PV。在此、它會從元素快照建立複本。

虛擬儲存資源池

虛擬儲存資源池可在Astra Trident的儲存後端與Kubernetes的「torageClass」之間提供抽象層。系統管理員可透過通用的後端不受後端限制的方式、定義位置、效能及每個後端的保護等層面、而無需設定「儲存類別」、以指定要使用何種實體後端、後端集區或後端類型來滿足所需的條件。

儲存管理員可在Json或Yaml定義檔案的任何Astra Trident後端上定義虛擬儲存池。



在虛擬資源池清單之外指定的任何層面、都會對後端進行全域設定、並套用至所有虛擬資源池、而每個虛擬資源池則可個別指定一個或多個層面（覆寫任何後端全域層面）。

> 定義虛擬儲存集區時、請勿嘗試重新排列後端定義中現有虛擬集區的順序。此外、建議您不要編輯/修改現有虛擬資源池的屬性、改為定義新的虛擬資源池。

大部分方面都是以後端特定的詞彙來指定。最重要的是、高寬比值不會暴露在後端驅動程式之外、也無法在「儲存類別」中進行比對。而是由系統管理員為每個虛擬資源池定義一或多個標籤。每個標籤都是「金鑰：值配對」、而且標籤可能在獨特的後端之間通用。如同個別層面、標籤可依資源池指定、也可全域指定至後端。不同於具有預先定義名稱和值的各個層面、系統管理員有充分的判斷權、可視需要定義標籤金鑰和值。

「torageClass」會參照選取元參數中的標籤、來識別要使用的虛擬資源池。虛擬資源池選取器支援六個運算子：

營運者	範例	說明
`='	效能=優異	資源池的標籤值必須相符
！！=	效能！=極致	集區的標籤值不得相符
《in》	位置（東部、西部）	集區的標籤值必須位於一組值中
《不」	效能附註（銀、銅）	集區的標籤值不得為b

Volume存取群組

深入瞭解Astra Trident的使用方式 "Volume存取群組"。

> 如果您使用的是CHAP、建議您略過本節、以簡化管理並避免以下所述的擴充限制。此外、如果您在「csi」模式中使用Astra Trident、則可以忽略本節。Astra Trident在安裝為增強的SCSI資源配置程式時、會使用CHAP。

Astra Trident可使用Volume存取群組來控制其所配置之磁碟區的存取。如果停用CHAP、除非您在組態中指定一或多個存取群組ID、否則會尋找稱為「Trident」的存取群組。

雖然Astra Trident會將新磁碟區與已設定的存取群組建立關聯、但它不會自行建立或管理存取群組。存取群組必須先存在、儲存後端才會新增至Astra Trident、而且必須包含Kubernetes叢集中每個節點的iSCSI IQN、這些節點可能會掛載該後端所配置的磁碟區。在大多數安裝中、這包括叢集中的每個工作節點。

對於具有超過64個節點的Kubernetes叢集、您應該使用多個存取群組。每個存取群組最多可包含64個IQN、每個磁碟區可屬於四個存取群組。在設定最多四個存取群組的情況下、叢集中最多256個節點的任何節點都能存取任何磁碟區。如需Volume存取群組的最新限制、請參閱 "請按這裡"。

如果您要將組態從使用預設的「三叉」存取群組修改為使用其他存取群組、請在清單中加入「三叉」存取群組的ID。

開始使用

歡迎試用

NetApp提供立即可用的實驗室映像、您可以透過申請 "NetApp試用"。測試磁碟機提供您一個沙箱環境、其中安裝並設定了三節點Kubernetes叢集和Astra Trident。這是熟悉Astra Trident並探索其特色的絕佳方式。

另一個選項是查看 "Kubeadm安裝指南" 由Kubernetes提供。

> 您不應該在正式作業中使用這些指令所建置的Kubernetes叢集。請使用經銷商提供的正式作業部署指南、建立正式作業就緒的叢集。

如果這是您第一次使用Kubernetes、請熟悉這些概念和工具 "請按這裡"。

需求

請先檢閱支援的前端、後端及主機組態、開始著手。

> 若要瞭解Astra Trident使用的連接埠、請參閱 "請按這裡"。

支援的前端（協調器）

Astra Trident支援多個容器引擎和協調器、包括：

- Anthos on -Prem（VMware）和Anthos on裸機1.9、1.10、1.11
- Kubernetes 1.18 - 1.24
- Mirantis Kubernetes Engine 3.4
- OpenShift 4.7、4.8、4.9、4.10.

這些版本支援Trident運算子：

- Anthos on -Prem（VMware）和Anthos on裸機1.9、1.10、1.11
- Kubernetes 1.18 - 1.24
- OpenShift 4.7、4.8、4.9、4.10.

Astra Trident也能與其他全管理且自我管理的Kubernetes產品搭配使用、包括Google Kubernetes Engine（GKE）、Amazon Elastic Kubernetes Services（EKS）、Azure Kubernetes Service（KS）、Rancher及VMware Tanzu Portfolio。

支援的後端（儲存）

若要使用Astra Trident、您需要下列一或多個支援的後端：

- Amazon FSX for NetApp ONTAP 產品
- Azure NetApp Files

- Astra資料儲存區
- Cloud Volumes ONTAP
- 適用於 GCP Cloud Volumes Service
- FAS/AFF/選取9.3或更新版本
- NetApp All SAN Array ASA （ESAN）
- NetApp HCI / Element軟體11或更新版本

功能需求

下表摘要說明此Astra Trident版本的可用功能及其支援的Kubernetes版本。

功能	**Kubernetes**版本	需要功能閘道？
csi Trident	1.18 - 1.24	否
Volume Snapshot	1.18 - 1.24	否
來自Volume Snapshot的PVc	1.18 - 1.24	否
iSCSI PV調整大小	1.18 - 1.24	否
資訊雙向CHAP ONTAP	1.18 - 1.24	否
動態匯出原則	1.18 - 1.24	否
Trident運算子	1.18 - 1.24	否
自動工作者節點準備（試用版）	1.18 - 1.24	否
csi拓撲	1.18 - 1.24	否

已測試的主機作業系統

依預設、Astra Trident會在容器中執行、因此會在任何Linux工作者上執行。不過、這些員工必須能夠使用標準NFS用戶端或iSCSI啟動器來掛載Astra Trident提供的磁碟區、視您使用的後端而定。

儘管Astra Trident並未正式「支援」特定作業系統、但下列Linux套裝作業系統仍可正常運作：

- 受OpenShift Container Platform支援的RedHat CoreOS（RMCOS）版本
- RHEL或CentOS 7.4或更新版本
- Ubuntu 18.04或更新版本

「tridentctl」公用程式也可在任何這些Linux版本上執行。

主機組態

視使用中的後端而定、應該在叢集中的所有員工上安裝NFS和/或iSCSI公用程式。請參閱 "請按這裡" 以取得更多資訊。

儲存系統組態

Astra Trident可能需要對儲存系統進行一些變更、才能使用後端組態。請參閱 "請按這裡" 以取得詳細資料。

Container映像和對應的Kubernetes版本

對於空拍安裝、下列清單是安裝Astra Trident所需的容器映像參考資料。使用「tridentctl images)」命令來驗證所需的容器映像清單。

Kubernetes版本	Container映像
v1.18.0	• NetApp/Trident：22.04.0 • NetApp/Trident自動支援：22.04 • k8s.gcr.io/sig-storage / csi佈建程式：v2.2.2 • k8s.gcr.io/sig-storage / csi附加程式：v3.4.0 • k8s.gcr.io/sig-storage / csi大小調整程式：v1.4.1 • k8s.gcr.io/sig-storage / csi快照記錄：v3.0.3 • k8s.gcr.io/sig-storage / csi節點驅動程式登錄程式：v2.5.0 • NetApp/Trident營運者：22.04.0（選用）
v1.19.0版	• NetApp/Trident：22.04.0 • NetApp/Trident自動支援：22.04 • k8s.gcr.io/sig-storage / csi佈建程式：v2.2.2 • k8s.gcr.io/sig-storage / csi附加程式：v3.4.0 • k8s.gcr.io/sig-storage / csi大小調整程式：v1.4.1 • k8s.gcr.io/sig-storage / csi快照記錄：v3.0.3 • k8s.gcr.io/sig-storage / csi節點驅動程式登錄程式：v2.5.0 • NetApp/Trident營運者：22.04.0（選用）

Kubernetes版本	Container映像
v1.20.0	• NetApp/Trident：22.04.0 • NetApp/Trident自動支援：22.04 • k8s.gcr.io/sig-storage / csi佈建程式：v2.2.2 • k8s.gcr.io/sig-storage / csi附加程式：v3.4.0 • k8s.gcr.io/sig-storage / csi大小調整程式：v1.4.1 • k8s.gcr.io/sig-storage / csi快照記錄：v3.0.3 • k8s.gcr.io/sig-storage / csi節點驅動程式登錄程式：v2.5.0 • NetApp/Trident營運者：22.04.0（選用）
1.21.0版	• NetApp/Trident：22.04.0 • NetApp/Trident自動支援：22.04 • k8s.gcr.io/sig-storage / csi佈建程式：v2.2.2 • k8s.gcr.io/sig-storage / csi附加程式：v3.4.0 • k8s.gcr.io/sig-storage / csi大小調整程式：v1.4.1 • k8s.gcr.io/sig-storage / csi快照記錄：v3.0.3 • k8s.gcr.io/sig-storage / csi節點驅動程式登錄程式：v2.5.0 • NetApp/Trident營運者：22.04.0（選用）
1.22.0版	• NetApp/Trident：22.04.0 • NetApp/Trident自動支援：22.04 • k8s.gcr.io/sig-storage / csi佈建程式：v2.2.2 • k8s.gcr.io/sig-storage / csi附加程式：v3.4.0 • k8s.gcr.io/sig-storage / csi大小調整程式：v1.4.1 • k8s.gcr.io/sig-storage / csi快照記錄：v3.0.3 • k8s.gcr.io/sig-storage / csi節點驅動程式登錄程式：v2.5.0 • NetApp/Trident營運者：22.04.0（選用）

Kubernetes版本	Container映像
1.23.0版	• NetApp/Trident：22.04.0 • NetApp/Trident自動支援：22.04 • k8s.gcr.io/sig-storage / csi佈建程式：v2.2.2 • k8s.gcr.io/sig-storage / csi附加程式：v3.4.0 • k8s.gcr.io/sig-storage / csi大小調整程式：v1.4.1 • k8s.gcr.io/sig-storage / csi快照記錄：v3.0.3 • k8s.gcr.io/sig-storage / csi節點驅動程式登錄程式：v2.5.0 • NetApp/Trident營運者：22.04.0（選用）

在Kubernetes版本1.20及更新版本上、只有在「v1」版本提供「volumesnapshots.snapshot.storage」CRD的情況下、才能使用已驗證的「k8s.gcr.io/sig-storage / csi快照：v5.x」影像。如果「v1Beta1」版本使用/不使用「v1」版本來提供CRD、請使用已驗證的「k8s.gcr.IO/SIG-storage / cscs-snapshotter：v3.x」影像。

部署總覽

您可以使用Trident運算子或Tridentctl來部署Astra Trident。

從22.04版開始、每次安裝Astra Trident時、AES金鑰就不會再重新產生。在這個版本中、Astra Trident會安裝一個新的秘密物件、並在安裝過程中持續存在。這表示22.04中的「tridentctl」可以解除安裝Trident的舊版、但舊版無法解除安裝22.04安裝。

選擇部署方法

若要判斷要使用哪種部署方法、請考慮下列事項：

為什麼我應該使用**Trident**運算子？

。 "Trident運算子" 是動態管理Astra Trident資源及自動化設定階段的絕佳方法。必須滿足一些先決條件。請參閱 "需求"。

Trident營運者提供以下幾項優點：

自我修復功能

您可以監控Astra Trident安裝、並主動採取措施來處理問題、例如刪除部署或意外修改部署。當操作員設定為部署時、會建立一個「Trident - oper-<generated -id>' pod」。此Pod可將「TridentOrchestrator」的CR與Astra Trident安裝建立關聯、確保只有一個作用中的「TridentOrchestrator」。換句話說、操作人員可確保叢集中只有一個Astra Trident執行個體、並控制其設定、確保安裝具有冪等特性。當對安裝進行變更（例如刪除部署或節點取消設定）時、操作員會分別識別並修正這些變更。

輕鬆更新現有安裝

您可以輕鬆地與營運者一起更新現有的部署。您只需要編輯「TridentOrchestrator」、就能更新安裝內容。例如、假設您需要啟用Astra Trident來產生偵錯記錄的案例。

若要這麼做、請修補您的「TridentOrchestrator」、將「pec.debug`」設為「true」：

```
kubectl patch torc <trident-orchestrator-name> -n trident --type=merge -p
'{"spec":{"debug":true}}'
```

更新「TridentOrchestrator」之後、營運者會處理更新並修補現有安裝。這可能會觸發建立新的Pod、以據此修改安裝。

自動處理**Kubernetes**升級

當叢集的Kubernetes版本升級至支援的版本時、營運者會自動更新現有的Astra Trident安裝、並加以變更、以確保其符合Kubernetes版本的要求。

> 如果叢集升級至不受支援的版本、則操作員將無法安裝Astra Trident。如果操作員已安裝Astra Trident、則會顯示警告、指出Astra Trident安裝在不受支援的Kubernetes版本上。

使用**Cloud Manager**管理**Kubernetes**叢集

與 "使用Cloud Manager的Astra Trident"、您可以升級至最新版的Astra Trident、新增及管理儲存類別、並將其連線至工作環境、以及使用Cloud Backup Service NetApp備份持續的Volume。Cloud Manager可手動或使用Helm、使用Trident營運者來支援Astra Trident部署。

為什麼要使用**Helm**？

如果您有其他使用Helm管理的應用程式、從Astra Trident 21.01開始、您也可以使用Helm來管理部署。

何時應該使用**tridenctl**？

如果您現有的部署必須升級至、或是想要高度自訂部署、請參閱使用 "試用"。這是部署Astra Trident的傳統方法。

在部署方法之間移動的考量

不難想像需要在部署方法之間移動的情況。在嘗試從「tridentctl」部署移至以營運者為基礎的部署之前、您應該考慮下列事項：

- 請務必使用相同的方法來解除安裝Astra Trident。如果您已使用「tridentctl」部署、您應該使用適當版本的「tridentctl」二進位來解除安裝Astra Trident。同樣地、如果您是與營運者一起部署、您也應該編輯「TridentOrchestrator」、並設定「shpec.uninstall=true」來解除安裝Astra Trident。
- 如果您想要移除並使用「tridentctl」來部署Astra Trident、請先編輯「TridentOrchestrator」、然後設定「shipec.uninstall=true」來解除安裝Astra Trident。然後刪除「TridentOrchestrator」和營運者部署。然後您可以使用「tridentctl」安裝。
- 如果您有手動的操作員型部署、而且想要使用以Helm為基礎的Trident操作員部署、您應該先手動解除安裝操作員、然後再執行Helm安裝。如此一來、Helm就能部署具有所需標籤和註釋的Trident運算子。如果您不

這麼做、則Helm型Trident營運者部署將會失敗、並顯示標籤驗證錯誤和註釋驗證錯誤。如果您的部署是「tridentctl」型部署、則可以使用Helm型部署、而不會發生問題。

瞭解部署模式

有三種方法可以部署Astra Trident。

標準部署

在Kubernetes叢集上部署Trident會導致Astra Trident安裝程式執行兩項作業：

- 透過網際網路擷取Container映像
- 建立部署和/或節點取消設定、在Kubernetes叢集中的所有合格節點上執行Astra Trident Pod。

這類的標準部署可透過兩種不同方式執行：

- 使用「tridentctl install」
- 使用Trident運算子。您可以手動或使用Helm來部署Trident運算子。

這種安裝模式是安裝Astra Trident的最簡單方法、適用於大多數不受網路限制的環境。

離線部署

若要執行無線部署、您可以在叫用「tridentctl install」時、使用「-image-registry」旗標來指向私有映像登錄。如果使用Trident運算子進行部署、您也可以在「TridentOrchestrator」中指定「s.pec.imageRegistry」。此登錄應包含 "Trident影像"、 "Trident AutoSupport 的圖片"以及Kubernetes版本所需的csi sidecar映像。

若要自訂部署、您可以使用「tridentctl」來產生Trident資源的資訊清單。這包括部署、取消程式集、服務帳戶、以及Astra Trident在安裝過程中所建立的叢集角色。

如需自訂部署的詳細資訊、請參閱下列連結：

- "自訂您的營運者型部署"
- "85cda507040c129ee606d4d8df583b90"

> 如果您使用的是私有映像儲存庫、則應在私有登錄URL的結尾加上「/sig-storage」。在使用私有註冊表進行「tridentctl」部署時、您應該搭配「-trident映像」和「-autocomple-image」來搭配「-image登錄」。如果您使用Trident運算子來部署Astra Trident、請確定Orchestrator CR在安裝參數中包含「tridentImage」和「autosupportImage」。

遠端部署

以下是遠端部署程序的高階概觀：

- 在遠端機器上部署適當版本的「kubecll」、以便部署Astra Trident。
- 從Kubernetes叢集複製組態檔、然後在遠端機器上設定「KUBECCONFIG」環境變數。
- 啟動「kubectl Get nodes」命令、確認您可以連線至所需的Kubernetes叢集。
- 使用標準安裝步驟、從遠端機器完成部署。

其他已知組態選項

在VMware Tanzu產品組合產品上安裝Astra Trident時：

- 叢集必須支援特殊權限的工作負載。
- 「-kubelet-dir」旗標應設定為kubelet目錄的位置。依預設、這是「/var/vcap/data/kubelet」。

  使用「-kubelet-dir」指定kubelet位置、已知適用於Trident運算子、Helm及「tridentctl」部署。

與Trident營運者一起部署

您可以使用Trident運算子來部署Astra Trident。您可以使用下列兩種方式之一來部署Trident運算子：

- 使用Trident "掌舵表"：Helm圖表可部署Trident運算子、並在單一步驟中安裝Trident。
- 手動：Trident提供 "bunder.yaml" 可用於安裝運算子及建立關聯物件的檔案。

> 如果您尚未熟悉 "基本概念"現在正是這麼做的好時機。

若要部署Astra Trident、必須符合下列先決條件：

- 您擁有執行Kubernetes 1.18 - 1.24之受支援Kubernetes叢集的完整權限。
- 您可以存取支援的NetApp儲存系統。
- 您可以從所有Kubernetes工作節點掛載磁碟區。
- 您的Linux主機已安裝並設定為管理您要使用的Kubernetes叢集。
- 您已將「KUBECCONFIG」環境變數設定為指向Kubernetes叢集組態。
- 您已啟用 "具備Astra Trident所需的閘道"。
- 如果您使用Kubernetes搭配Docker Enterprise、 "請依照他們的步驟啟用CLI存取"。

您知道嗎？太棒了！讓我們開始吧。

部署Trident操作員、並使用Helm安裝Astra Trident

使用Helm執行列出的步驟來部署Trident運算子。

除了上述先決條件之外、若要使用Helm部署Trident運算子、您還需要下列項目：

- Kubernetes 1.18 - 1.24
- Helm版本3

步驟

1. 新增Trident的Helm儲存庫：

```
helm repo add netapp-trident https://netapp.github.io/trident-helm-chart
```

2. 使用「helm install」命令並指定部署名稱。請參閱下列範例：

```
helm install <release-name> netapp-trident/trident-operator --version
22.4.0 --namespace <trident-namespace>
```

> 如果您尚未建立Trident的命名空間、可以將「-cree-namespace"參數新增至「helm install」命令。然後、Helm會自動為您建立命名空間。

安裝期間有兩種傳遞組態資料的方法：

- `--Values`（或-f'）：指定具有置換的Yaml檔案。這可以多次指定、最右邊的檔案會優先。
- -Set'：在命令列上指定置換項目。

例如、若要變更預設值「debug」、請執行下列「-set」命令：

```
$ helm install <name> netapp-trident/trident-operator --version 22.4.0
--set tridentDebug=true
```

Helm圖表中的「values.yaml」檔案提供金鑰清單及其預設值。

「helm清單」會顯示安裝的詳細資料、例如名稱、命名空間、圖表、狀態、 應用程式版本、修訂編號等。

手動部署Trident運算子

執行所列步驟、手動部署Trident運算子。

步驟1：判斷Kubernetes叢集的資格

首先您需要登入Linux主機、然後確認它正在管理_運作_、 "支援的Kubernetes叢集" 您擁有必要的權限。

> 使用OpenShift時、請在所有範例中使用「oc"而非「kubecll」、然後先執行「ocLogin -u system:admin」或「occ login-u kube-admin」、以*系統：admin*登入。

若要驗證Kubernetes版本、請執行下列命令：

```
kubectl version
```

若要查看您是否具有Kubernetes叢集管理員權限、請執行下列命令：

```
kubectl auth can-i '*' '*' --all-namespaces
```

若要驗證是否可以從Docker Hub啟動使用映像的Pod、並透過Pod網路連線至儲存系統、請執行下列命令：

```
kubectl run -i --tty ping --image=busybox --restart=Never --rm -- \
  ping <management IP>
```

步驟**2**：下載並設定營運者

> 從21.01開始、Trident運算子就是叢集範圍。若要使用Trident營運者來安裝Trident、必須建立「TridentOrchestrator」自訂資源定義（CRD）並定義其他資源。在安裝Astra Trident之前、您應該先執行這些步驟來設定操作員。

1. 下載最新版本的 "Trident安裝程式套裝組合" 從_下載_區段擷取。

```
wget
https://github.com/NetApp/trident/releases/download/v22.04.0/trident-
installer-22.04.0.tar.gz
tar -xf trident-installer-22.04.0.tar.gz
cd trident-installer
```

2. 使用適當的CRD資訊清單來建立「TridentOrchestrator」CRD。之後您將建立一個「TridentOrchestrator」自訂資源、以產生操作者的安裝。

   執行下列命令：

```
kubectl create -f
deploy/crds/trident.netapp.io_tridentorchestrators_crd_post1.16.yaml
```

3. 建立「TridentOrchestrator」CRD之後、請建立下列作業者部署所需的資源：

   - 營運者的服務帳戶
   - 叢集角色和叢集角色繫結至服務帳戶
   - 專屬的PodSecurity原則
   - 營運者本身

     Trident安裝程式包含定義這些資源的資訊清單。根據預設、運算子會部署在「Trident」命名空間中。如果「Trident」命名空間不存在、請使用下列資訊清單來建立名稱空間。

```
$ kubectl apply -f deploy/namespace.yaml
```

4. 若要在預設的「三叉」命名空間以外的命名空間中部署運算子、您應該更新「serviceaccount.yaml」、「clusterrolebind.yaml」和「oper.yaml」等資訊清單、然後產生「bundle.yaml」。

   執行下列命令、以更新Yaml清單、並使用「kustomization、yaml」產生您的「bunder.yaml」：

```
kubectl kustomize deploy/ > deploy/bundle.yaml
```

執行下列命令以建立資源並部署營運者：

```
kubectl create -f deploy/bundle.yaml
```

5. 若要在部署後驗證操作員的狀態、請執行下列步驟：

```
$ kubectl get deployment -n <operator-namespace>
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
trident-operator    1/1     1            1           3m

$ kubectl get pods -n <operator-namespace>
NAME                                READY   STATUS             RESTARTS
AGE
trident-operator-54cb664d-lnjxh     1/1     Running            0
3m
```

營運者部署成功建立一個在叢集中其中一個工作節點上執行的Pod。

> Kubernetes叢集中只應有*一個運算子執行個體*。請勿建立Trident營運者的多個部署。

步驟3：建立TridentOrchestrator並安裝Trident

您現在可以使用運算子來安裝Astra Trident！這需要建立「TridentOrchestrator」。Trident安裝程式隨附建立「TridentOrchestrator」的範例定義。這將開始在「Trident」命名空間中安裝。

```
$ kubectl create -f deploy/crds/tridentorchestrator_cr.yaml
tridentorchestrator.trident.netapp.io/trident created

$ kubectl describe torc trident
Name:        trident
Namespace:
Labels:      <none>
Annotations: <none>
API Version: trident.netapp.io/v1
Kind:        TridentOrchestrator
...
Spec:
  Debug:     true
  Namespace: trident
Status:
  Current Installation Params:
    IPv6:                      false
    Autosupport Hostname:
    Autosupport Image:         netapp/trident-autosupport:21.04
    Autosupport Proxy:
    Autosupport Serial Number:
    Debug:                     true
    Enable Node Prep:          false
    Image Pull Secrets:
    Image Registry:
    k8sTimeout:           30
    Kubelet Dir:          /var/lib/kubelet
    Log Format:           text
    Silence Autosupport:  false
    Trident Image:        netapp/trident:21.04.0
  Message:                  Trident installed  Namespace:
trident
  Status:                   Installed
  Version:                  v21.04.0
Events:
    Type Reason Age From Message ---- ------ ---- ---- -------Normal
    Installing 74s trident-operator.netapp.io Installing Trident Normal
    Installed 67s trident-operator.netapp.io Trident installed
```

Trident運算子可讓您使用「TridentOrchestrator」規格中的屬性、自訂Astra Trident的安裝方式。請參閱 "自訂您的Trident部署"。

「TridentOrchestrator」的狀態會指出安裝是否成功、並顯示安裝的Trident版本。

狀態	說明
安裝	營運者使用此「TridentOrchestrator」CR來安裝Astra Trident。
已安裝	Astra Trident已成功安裝。
正在解除安裝	操作員正在解除安裝Astra Trident、因為是「shpec.uninstall=true」。
已解除安裝	Astra Trident已解除安裝。
失敗	營運者無法安裝、修補、更新或解除安裝Astra Trident；營運者將自動嘗試從此狀態恢復。如果此狀態持續存在、您將需要疑難排解。
正在更新	營運者正在更新現有的安裝。
錯誤	不使用「TridentOrchestrator」。另一個已經存在。

在安裝過程中、「TridentOrchestrator」的狀態會從「安裝」變更為「安裝」。如果您觀察到「失敗」狀態、而操作者無法自行恢復、則應檢查操作者的記錄。請參閱 "疑難排解" 區段。

您可以查看已建立的Pod、確認Astra Trident安裝是否已完成：

```
$ kubectl get pod -n trident
NAME                                READY   STATUS    RESTARTS   AGE
trident-csi-7d466bf5c7-v4cpw        5/5     Running   0           1m
trident-csi-mr6zc                   2/2     Running   0           1m
trident-csi-xrp7w                   2/2     Running   0           1m
trident-csi-zh2jt                   2/2     Running   0           1m
trident-operator-766f7b8658-ldzsv   1/1     Running   0           3m
```

您也可以使用「tridentctl」來檢查安裝的Astra Trident版本。

```
$ ./tridentctl -n trident version
+----------------+----------------+
| SERVER VERSION | CLIENT VERSION |
+----------------+----------------+
| 21.04.0        | 21.04.0        |
+----------------+----------------+
```

現在您可以繼續建立後端。請參閱 "部署後工作"。

如需部署期間的疑難排解問題、請參閱 "疑難排解" 區段。

自訂Trident營運者部署

Trident運算子可讓您使用「TridentOrchestrator」規格中的屬性、自訂Astra Trident的安裝方式。

請參閱下表以取得屬性清單：

參數	說明	預設
命名空間	用於安裝Astra Trident的命名空間	"預設"
「Debug」	啟用Astra Trident的偵錯功能	錯
《IPv6》	透過IPv6安裝Astra Trident	錯
「k8stimeout」	Kubernetes作業逾時	30秒
「ilenceAutoSupport」	請勿AutoSupport 自動將此套裝組合傳送至NetApp	錯
「enableNodePrep」	自動管理工作節點相依性（* BETA * ）	錯
「autosupportImage」	遙測的容器影像AutoSupport	「NetApp/Trident自動支援：21.04.0」
「autosupportProxy」	代理伺服器的位址/連接埠、用於傳送AutoSupport 「遙測」功能	"<a href="http://proxy.example.com:8888"" class="bare">http://proxy.example.com:8888"</a>
解除安裝	用來解除安裝Astra Trident的旗標	錯
記錄格式	要使用的Astra Trident記錄格式[text、json]	"文字"
「TridentImage」	要安裝的Astra Trident映像	「NetApp/Trident：21.04」
「影像登錄」	內部登錄的路徑、格式為「<登錄FQDN >[:port]（/子路徑）」	「k8s.gcr.io/sig-storage（k8s 1.18以上）或que.io/k8scsi」
《kubeletDir	主機上的kubelet目錄路徑	"/var/lib/kubelet"
《有線》	要刪除以執行完整移除Astra Trident的資源清單	
「imagePullSecrets」	從內部登錄擷取映像的機密	
「controllerPluginNodeSelector」	執行Trident控制器csi外掛程式之Pod的其他節點選取器。格式與pod.spec.nodeSelector相同。	無預設值；選用
「控制器插件」	覆寫執行Trident控制器csi外掛程式之Pod的容錯能力。格式與po.spec.TBolerations相同。	無預設值；選用
「nodePluginNodeSelector」	執行Trident Node SCSI外掛程式之Pod的其他節點選取器。格式與pod.spec.nodeSelector相同。	無預設值；選用
「nodePluginBolerations」	覆寫執行Trident Node SCSI外掛程式之Pod的容錯能力。格式與po.spec.TBolerations相同。	無預設值；選用

「pec.namespace`」在「TridentOrchestrator」中指定、以表示要安裝哪個命名空間Astra Trident。此參數*無法在安裝Astra Trident之後更新*。嘗試這樣做會導致「TridentOrchestrator」狀態變更為「失敗」。Astra Trident不打算跨命名空間移轉。

自動工作節點準備是*測試版功能*、僅適用於非正式作業環境。

如需格式化Pod參數的詳細資訊、請參閱 "將Pod指派給節點"。

您可以在定義「TridentOrchestrator」時、使用上述屬性來自訂安裝。範例如下：

```
$ cat deploy/crds/tridentorchestrator_cr_imagepullsecrets.yaml
apiVersion: trident.netapp.io/v1
kind: TridentOrchestrator
metadata:
  name: trident
spec:
  debug: true
  namespace: trident
  imagePullSecrets:
  - thisisasecret
```

以下是另一個範例、說明如何使用節點選取器來部署Trident：

```
apiVersion: trident.netapp.io/v1
kind: TridentOrchestrator
metadata:
  name: trident
spec:
  debug: true
  namespace: trident
  controllerPluginNodeSelector:
    nodetype: master
  nodePluginNodeSelector:
    storage: netapp
```

如果您想要自訂超出「TridentOrchestrator」引數範圍的安裝、您應該考慮使用「tridentctl」來產生可視需要修改的自訂Yaml資訊清單。

使用tridentctl部署

您可以使用「tridentctl」來部署Astra Trident。

如果您尚未熟悉 "基本概念"現在正是這麼做的好時機。

若要自訂部署、請參閱 "請按這裡"。

若要部署Astra Trident、必須符合下列先決條件：

- 您擁有支援的Kubernetes叢集的完整權限。
- 您可以存取支援的NetApp儲存系統。
- 您可以從所有Kubernetes工作節點掛載磁碟區。
- 您的Linux主機已安裝並設定為管理您要使用的Kubernetes叢集。
- 您已將「KUBECCONFIG」環境變數設定為指向Kubernetes叢集組態。
- 您已啟用 "具備Astra Trident所需的閘道"。
- 如果您使用Kubernetes搭配Docker Enterprise、 "請依照他們的步驟啟用CLI存取"。

您知道嗎？太棒了！讓我們開始吧。

如需自訂部署的相關資訊、請參閱 "請按這裡"。

步驟1：判斷Kubernetes叢集的資格

首先您需要登入Linux主機、然後驗證它是否正在管理_工作_、 "支援的Kubernetes叢集" 您擁有必要的權限。

使用OpenShift時、您可以在所有範例中使用「oc"而非「kubecll」、而且您應該先執行「ocLogin -u system:admin」或「ocLogin -u kube-admin」、以*系統：admin*登入。

若要檢查Kubernetes版本、請執行下列命令：

```
kubectl version
```

若要查看您是否具有Kubernetes叢集管理員權限、請執行下列命令：

```
kubectl auth can-i '*' '*' --all-namespaces
```

若要驗證是否可以從Docker Hub啟動使用映像的Pod、並透過Pod網路連線至儲存系統、請執行下列命令：

```
kubectl run -i --tty ping --image=busybox --restart=Never --rm -- \
  ping <management IP>
```

識別Kubernetes伺服器版本。安裝Astra Trident時會使用此功能。

步驟2：下載並解壓縮安裝程式

Trident安裝程式會建立Trident pod、設定用來維護其狀態的CRD物件、並初始化執行動作的csi sidecars、例如資源配置和將磁碟區附加至叢集主機。

您可以下載的最新版本 "Trident安裝程式套裝組合" 從_下載_區段擷取。

例如、如果最新版本為21.07.1：

```
wget https://github.com/NetApp/trident/releases/download/v21.07.1/trident-
installer-21.07.1.tar.gz
tar -xf trident-installer-21.07.1.tar.gz
cd trident-installer
```

步驟3：安裝Astra Trident

執行「tridentctl install」命令、在所需的命名空間中安裝Astra Trident。

```
$ ./tridentctl install -n trident
....
INFO Starting Trident installation.                namespace=trident
INFO Created service account.
INFO Created cluster role.
INFO Created cluster role binding.
INFO Added finalizers to custom resource definitions.
INFO Created Trident service.
INFO Created Trident secret.
INFO Created Trident deployment.
INFO Created Trident daemonset.
INFO Waiting for Trident pod to start.
INFO Trident pod started.                          namespace=trident
pod=trident-csi-679648bd45-cv2mx
INFO Waiting for Trident REST interface.
INFO Trident REST interface is up.                 version=21.07.1
INFO Trident installation succeeded.
....
```

安裝程式完成後、看起來會是這樣。根據Kubernetes叢集中的節點數量、您可能會看到更多的Pod：

```
$ kubectl get pod -n trident
NAME                              READY   STATUS    RESTARTS   AGE
trident-csi-679648bd45-cv2mx      4/4     Running   0          5m29s
trident-csi-vgc8n                 2/2     Running   0          5m29s

$ ./tridentctl -n trident version
+----------------+----------------+
| SERVER VERSION | CLIENT VERSION |
+----------------+----------------+
| 21.07.1        | 21.07.1        |
+----------------+----------------+
```

如果您看到類似上述範例的輸出、表示您已完成此步驟、但尚未完整設定Astra Trident。請繼續進行下一步。請參閱 "部署後工作"。

但是、如果安裝程式未成功完成、或您看不到*執行中*的「Trident - csi -<產生的id>」、則表示平台未安裝。

> 如需部署期間的疑難排解問題、請參閱 "疑難排解" 區段。

自訂試用部署

Trident安裝程式可讓您自訂屬性。例如、如果您已將Trident映像複製到私有儲存庫、則可以使用「-trident映像」來指定映像名稱。如果您已將Trident映像及所需的csi sidecar映像複製到私有儲存庫、最好使用「-image -registry」交換器來指定該儲存庫的位置、此開關的格式為「<registry FQDN（登錄FQDN）>[:port]」。

若要讓Astra Trident自動為您設定工作節點、請使用「-enable節點準備」。如需其運作方式的詳細資訊、請參閱 "請按這裡"。

> 自動工作節點準備是*試用版功能*、僅適用於非正式作業環境。

如果您使用Kubernetes的發佈方式、而其中的「kubelet」將資料保留在通常「/var/lib/kubelet」以外的路徑上、您可以使用「-kubelet-dir」來指定替代路徑。

如果您需要自訂安裝、而不需要安裝程式的引數允許、也可以自訂部署檔案。使用「-generame-custom-yaml」參數、可在安裝程式的「設置」目錄中建立下列Yaml檔案：

- 《三叉叢集繫結.yaml》
- 《三叉部署：yaml》
- 《三叉怪怪怪怪怪怪怪怪怪怪怪怪怪怪怪怪怪怪怪怪怪怪怪怪怪
- 《Trident - clusterroy.yaml》
- 《三叉魔魔怪、yaml》
- 《Trident服務：yaml》
- 《Trident命名空間.yaml》
- 「Trident服務帳戶」。yaml

產生這些檔案之後、您可以根據自己的需求加以修改、然後使用「-use自訂-yaml」來安裝自訂部署。

```
./tridentctl install -n trident --use-custom-yaml
```

接下來呢？

部署Astra Trident之後、您可以繼續建立後端、建立儲存類別、配置磁碟區、以及將磁碟區掛載至Pod。

步驟1：建立後端

您現在可以繼續建立後端、由Astra Trident用來配置磁碟區。若要這麼做、請建立包含必要參數的「backend.json」檔案。不同後端類型的組態檔範例可在「shame-INPUT」目錄中找到。

請參閱 "請按這裡" 如需如何為後端類型設定檔案的詳細資訊、請參閱。

```
cp sample-input/<backend template>.json backend.json
vi backend.json
```

```
./tridentctl -n trident create backend -f backend.json
+-------------+----------------+--------------------------------------
+--------+---------+
|    NAME     | STORAGE DRIVER |                 UUID                 |
STATE   | VOLUMES |
+-------------+----------------+--------------------------------------
+--------+---------+
| nas-backend | ontap-nas      | 98e19b74-aec7-4a3d-8dcf-128e5033b214 |
online |       0 |
+-------------+----------------+--------------------------------------
+--------+---------+
```

如果建立失敗、表示後端組態有問題。您可以執行下列命令來檢視記錄、以判斷原因：

```
./tridentctl -n trident logs
```

解決問題之後、只要返回此步驟的開頭、然後再試一次即可。如需更多疑難排解秘訣、請參閱 "疑難排解" 區段。

步驟2：建立儲存類別

Kubernetes使用者使用指定的持續磁碟區宣告（PVCS）來配置磁碟區 "儲存類別" 依名稱。使用者會隱藏詳細資料、但儲存類別會識別該類別所使用的資源配置程式（本例中為Trident）、以及該類別對資源配置程式的意義。

建立儲存類別Kubernetes使用者要指定何時需要磁碟區。類別的組態需要建構您在上一個步驟中建立的後端、

以便Astra Trident使用它來配置新的磁碟區。

最簡單的儲存類別是以安裝程式隨附的「slam-filed/storage－class-csi・yaml.templ」檔案為基礎、以儲存驅動程式名稱取代「*backend_typ_*」。

```
./tridentctl -n trident get backend
+-------------+----------------+-------------------------------------
+--------+---------+
|    NAME     | STORAGE DRIVER |                 UUID                |
STATE  | VOLUMES |
+-------------+----------------+-------------------------------------
+--------+---------+
| nas-backend | ontap-nas      | 98e19b74-aec7-4a3d-8dcf-128e5033b214 |
online |       0 |
+-------------+----------------+-------------------------------------
+--------+---------+

cp sample-input/storage-class-csi.yaml.templ sample-input/storage-class-
basic-csi.yaml

# Modify __BACKEND_TYPE__ with the storage driver field above (e.g.,
ontap-nas)
vi sample-input/storage-class-basic-csi.yaml
```

這是Kubernetes物件、所以您可以使用「kubectl」在Kubernetes中建立。

```
kubectl create -f sample-input/storage-class-basic-csi.yaml
```

現在您應該會看到Kubernetes和Astra Trident中的* basic、csi *儲存類別、而Astra Trident應該已經在後端探索集區。

```
kubectl get sc basic-csi
NAME         PROVISIONER             AGE
basic-csi    csi.trident.netapp.io   15h

./tridentctl -n trident get storageclass basic-csi -o json
{
  "items": [
    {
      "Config": {
        "version": "1",
        "name": "basic-csi",
        "attributes": {
          "backendType": "ontap-nas"
        },
        "storagePools": null,
        "additionalStoragePools": null
      },
      "storage": {
        "ontapnas_10.0.0.1": [
          "aggr1",
          "aggr2",
          "aggr3",
          "aggr4"
        ]
      }
    }
  ]
}
```

步驟3：配置第一個Volume

現在您可以動態配置第一個Volume了。這是透過建立Kubernetes來完成 "持續磁碟區宣告" （PVc）物件。

為使用您剛建立之儲存類別的磁碟區建立一個永久虛擬磁碟。

請參閱「shame-INPUT / PVC-base-csi。yaml」範例。請確定儲存類別名稱符合您所建立的名稱。

```
kubectl create -f sample-input/pvc-basic-csi.yaml

kubectl get pvc --watch
NAME      STATUS    VOLUME                                     CAPACITY
ACCESS MODES  STORAGECLASS   AGE
basic     Pending
basic          1s
basic     Pending   pvc-3acb0d1c-b1ae-11e9-8d9f-5254004dfdb7   0
basic          5s
basic     Bound     pvc-3acb0d1c-b1ae-11e9-8d9f-5254004dfdb7   1Gi
RWO           basic          7s
```

步驟4：在Pod中掛載磁碟區

現在讓我們掛載磁碟區。我們將推出一個將PV掛載到「/usr/share/ngines/html」下的「恩靈思」Pod。

```
cat << EOF > task-pv-pod.yaml
kind: Pod
apiVersion: v1
metadata:
  name: task-pv-pod
spec:
  volumes:
    - name: task-pv-storage
      persistentVolumeClaim:
       claimName: basic
  containers:
    - name: task-pv-container
      image: nginx
      ports:
        - containerPort: 80
          name: "http-server"
      volumeMounts:
        - mountPath: "/usr/share/nginx/html"
          name: task-pv-storage
EOF
kubectl create -f task-pv-pod.yaml
```

```
# Wait for the pod to start
kubectl get pod --watch

# Verify that the volume is mounted on /usr/share/nginx/html
kubectl exec -it task-pv-pod -- df -h /usr/share/nginx/html

# Delete the pod
kubectl delete pod task-pv-pod
```

此時、Pod（應用程式）不再存在、但磁碟區仍然存在。如果需要、您可以從其他Pod使用。

若要刪除磁碟區、請刪除請款：

```
kubectl delete pvc basic
```

您現在可以執行其他工作、例如：

- "設定其他後端。"
- "建立其他儲存類別。"

管理Astra Trident

升級Astra Trident

Astra Trident依照每季發行時段進行、每一日曆年發行四個主要版本。每個新版本均以舊版為基礎、提供新功能與效能增強功能、以及錯誤修正與改善功能。我們建議您每年至少升級一次、以善用Astra Trident的新功能。

> 若要升級至未來五個版本的版本、您必須執行多步驟升級。

判斷要升級的版本

- 您可以從「yY-1.MM」版本和任何中間版本升級至「y.MM」版本。例如、您可以直接從19.07或更新版本升級至20.07（包括點版本、例如19.07.1）。
- 如果您有較早的版本、應該執行多步驟升級。這需要您先升級至最新版本、以符合您的四個版本。例如、如果您執行的是18.07、想要升級至20.07版本、請依照下列指示執行多步驟升級程序：
  - 第一次從18.07升級至19.07。請參閱個別版本的文件、以取得升級的特定指示。
  - 然後從19.07升級至20.07。

> 所有19.04版及更早版本的升級、都需要將Astra Trident的中繼資料從自己的「etcd」移轉至CRD物件。請務必查看版本文件、瞭解升級的運作方式。

> 升級時、務必在Astra Trident使用的「torageClasses」中提供「參數.FSType」。您可以刪除並重新建立「儲存類別」、而不會中斷現有的磁碟區。這是強制實施的一項**要求 "安全性內容" 適用於SAN磁碟區。。 "範例輸入" 目錄包含範例、例如 "9d267d893296d1b56789c03ad196afc8" 和 "4e4578e23d9d48bd9338b1581f884392"。如需詳細資訊、請參閱 "已知問題"。

我應該選擇哪種升級途徑？

您可以使用下列其中一種方法來升級：

- 使用Trident運算子。
- 使用「tridentctl」。

> 現在、自Kubernetes 1.20開始、「csi Volume Snapshot」就是GA的一項功能。升級Astra Trident時、必須先移除所有先前的Alpha Snapshot CRS和CRD（Volume Snapshot類別、Volume Snapshot和Volume Snapshot內容）、才能執行升級。請參閱 "這篇部落格" 瞭解將Alpha快照移轉至測試版/GA規格所需的步驟。

如果符合下列條件、您可以使用Trident運算子進行升級：

- 您執行的是「csi Trident」（19.07及更新版本）。
- 您有一個CRD型Trident版本（19.07及更新版本）。
- 您正在執行自訂安裝（使用自訂YAML）。

> 如果您使用的是以「etcd」為基礎的Trident版本（19.04或更早版本）、請勿使用運算子來升級Trident。

如果您不想使用操作員、或是擁有操作員無法支援的自訂安裝、您可以使用「tridentctl」進行升級。這是Trident 19.04及更早版本的首選升級方法。

操作員變更

Astra Trident的21.01版為營運者帶來了一些重要的架構變更、如下所示：

- 運算子現在*叢集範圍*。Trident運算子先前的執行個體（20.04到20.10版）為*命名空間範圍*。叢集範圍內的運算子具有下列優點：
  - 資源責任：營運者現在可在叢集層級管理與Astra Trident安裝相關的資源。在安裝Astra Trident時、營運者使用「擁有者參考資料」來建立及維護數項資源。在叢集範圍內的資源上維護「擁有者參考資料」、可能會在OpenShift等Kubernetes經銷商上產生錯誤。叢集範圍的運算子可減輕此問題。對於自動修復和修補Trident資源、這是必要的需求。
  - 卸載期間清理：完整移除Astra Trident將需要刪除所有相關資源。命名空間範圍的運算子可能會在移除叢集範圍的資源（例如叢集角色、叢集角色繫結和Podcast安全性原則）時遇到問題、並導致不完整的清理。叢集範圍的運算子可消除此問題。使用者可以完全解除安裝Astra Trident、並視需要重新安裝。
- 現在、「TridentProvisioner」取代為「TridentOrchestrator」、做為用來安裝和管理Astra Trident的自訂資源。此外、「TridentOrchestrator」規格也引進了新的領域。使用者可以指定命名空間Trident必須使用「pec.namespace`」欄位來安裝/升級。您可以參考範例 "請按這裡"。

如需詳細資訊、請參閱

- "使用Trident營運者進行升級"
- "b41f68e89b7b02a9077fb800d31a2fbe"

與營運者一起升級

您可以使用業者輕鬆升級現有的Astra Trident安裝。

若要使用營運者進行升級、應符合下列條件：

- 您應該安裝以「服務整合式」為基礎的Astra Trident。若要檢查您是否正在執行csi Trident、請檢查Trident命名空間中的Pod。如果他們遵循「Trident - csi*」命名模式、您就會執行「csi Trident」。
- 您應該安裝CRD型Trident。這代表19.07及更新版本的所有版本。如果您安裝的是基於SCSI的安裝、則很可能是以CRD為基礎的安裝。
- 如果您已解除安裝了「csi Trident」、而且安裝中的中繼資料持續存在、您可以使用操作者進行升級。
- 在特定Kubernetes叢集中的所有命名空間中、只應有一個Astra Trident安裝。
- 您應該使用執行的Kubernetes叢集 "版本1.18 - 1.24"。
- 如果存在Alpha Snapshot CRD、您應該使用「tridentctl obliviate alpha-snapshot -crd」來移除。這會刪除Alpha Snapshot規格的客戶需求日。如需應刪除/移轉的現有快照、請參閱 "這篇部落格"。

使用OpenShift Container Platform上的運算子來升級Trident時、您應該升級至Trident 21.01.1或更新版本。隨21.01.0一起發行的Trident運算子包含已在21.01.1中修正的已知問題。如需詳細資訊、請參閱 "GitHub問題詳細資料"。

升級叢集範圍內的操作員安裝

若要從* Trident 21.01*及更新版本*升級、請依照下列步驟進行。

步驟

1. 刪除用來安裝目前Astra Trident執行個體的Trident運算子。例如、如果您要從21.01升級、請執行下列命令：

```
kubectl delete -f 21.01/trident-installer/deploy/bundle.yaml -n trident
```

2. （選用）如果您想要修改安裝參數、請編輯您在安裝Trident時所建立的「TridentOrchestrator」物件。這可能包括修改自訂Trident映像、從私有映像登錄擷取容器映像、啟用偵錯記錄或指定映像提取機密等變更。
3. 使用「bunder.yaml」檔案來安裝Astra Trident、該檔案可設定新版本的Trident運算子。執行下列命令：

```
kubectl create -f 21.10.0/trident-installer/deploy/bundle.yaml -n
trident
```

在此步驟中、21.10.0 Trident營運者將識別現有的Astra Trident安裝、並將其升級至與營運者相同的版本。

升級命名空間範圍內的操作員安裝

若要從使用命名空間範圍運算子（20.07到20.10版）安裝的Astra Trident執行個體進行升級、請遵循下列步驟：

步驟

1. 確認現有Trident安裝的狀態。要執行此操作、請查看「TridentProvisioner」的*狀態*。狀態應為「Installed」（已安裝）。

```
$ kubectl describe tprov trident -n trident | grep Message: -A 3
Message:  Trident installed
Status:   Installed
Version:  v20.10.1
```

如果狀態顯示「正在更新」、請務必先解決、再繼續。如需可能狀態值的清單、請參閱 "請按這裡"。

2. 使用Trident安裝程式提供的資訊清單、建立「TridentOrchestrator」CRD。

```
# Download the release required [21.01]
$ mkdir 21.07.1
$ cd 21.07.1
$ wget
https://github.com/NetApp/trident/releases/download/v21.07.1/trident-
installer-21.07.1.tar.gz
$ tar -xf trident-installer-21.07.1.tar.gz
$ cd trident-installer
$ kubectl create -f
deploy/crds/trident.netapp.io_tridentorchestrators_crd_post1.16.yaml
```

3. 使用命名空間範圍的運算子資訊清單來刪除。若要完成此步驟、您需要使用「bunder.yaml」檔案來部署命名空間範圍的運算子。您可以從取得「bunder.yaml」 "Trident儲存庫"。請務必使用適當的分行。

> 刪除命名空間範圍的運算子之後、在安裝叢集範圍的運算子之前、您應該對Trident安裝參數進行必要的變更（例如、變更「tridentImage」、「autosupportImage」、「Private Image儲存庫」的值、以及提供「imagepolSecrets」）。如需可更新的完整參數清單、請參閱 "參數清單"。

```
#Ensure you are in the right directory
$ pwd
$ /root/20.10.1/trident-installer

#Delete the namespace-scoped operator
$ kubectl delete -f deploy/bundle.yaml
serviceaccount "trident-operator" deleted
clusterrole.rbac.authorization.k8s.io "trident-operator" deleted
clusterrolebinding.rbac.authorization.k8s.io "trident-operator" deleted
deployment.apps "trident-operator" deleted
podsecuritypolicy.policy "tridentoperatorpods" deleted

#Confirm the Trident operator was removed
$ kubectl get all -n trident
NAME                                READY   STATUS    RESTARTS   AGE
pod/trident-csi-68d979fb85-dsrmn    6/6     Running   12         99d
pod/trident-csi-8jfhf               2/2     Running   6          105d
pod/trident-csi-jtnjz               2/2     Running   6          105d
pod/trident-csi-lcxvh               2/2     Running   8          105d

NAME                  TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)
AGE
service/trident-csi   ClusterIP   10.108.174.125   <none>
34571/TCP,9220/TCP   105d

NAME                          DESIRED   CURRENT   READY   UP-TO-DATE
AVAILABLE   NODE SELECTOR                                       AGE
daemonset.apps/trident-csi   3         3         3       3            3
kubernetes.io/arch=amd64,kubernetes.io/os=linux   105d

NAME                          READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/trident-csi   1/1     1            1           105d

NAME                                     DESIRED   CURRENT   READY   AGE
replicaset.apps/trident-csi-68d979fb85   1         1         1
105d
```

在這個階段、「Trident - oper-xxxxxx-xxxxx" pod會刪除。

4. （選用）如果需要修改安裝參數、請更新「TridentProvisioner」規格。這些變更可能包括：修改私有映像登錄以從擷取容器映像、啟用偵錯記錄或指定映像提取機密。

```
$  kubectl patch tprov <trident-provisioner-name> -n <trident-namespace>
--type=merge -p '{"spec":{"debug":true}}'
```

5. 安裝叢集範圍的運算子。

> 安裝叢集範圍的運算子會將「TridentProvisioner」物件移轉至「TridentOrchestrator」物件、刪除「TridentProvisioner」物件和「Trident置 備程式」CRD、並將Astra Trident升級至所使用的叢集範圍運算子版本。在接下來的範例中、Trident已升級至21.07.1。

> 使用叢集範圍的運算子來升級Astra Trident、會將「TridentProvisioner」移轉至名稱相同的「tridentOrchestrator」物件。這會由操作員自動處理。升級也會將Astra Trident安裝在與之前相同的命名空間中。

```
#Ensure you are in the correct directory
$ pwd
$ /root/21.07.1/trident-installer

#Install the cluster-scoped operator in the **same namespace**
$ kubectl create -f deploy/bundle.yaml
serviceaccount/trident-operator created
clusterrole.rbac.authorization.k8s.io/trident-operator created
clusterrolebinding.rbac.authorization.k8s.io/trident-operator created
deployment.apps/trident-operator created
podsecuritypolicy.policy/tridentoperatorpods created

#All tridentProvisioners will be removed, including the CRD itself
$ kubectl get tprov -n trident
Error from server (NotFound): Unable to list "trident.netapp.io/v1,
Resource=tridentprovisioners": the server could not find the requested
resource (get tridentprovisioners.trident.netapp.io)

#tridentProvisioners are replaced by tridentOrchestrator
$ kubectl get torc
NAME      AGE
trident   13s

#Examine Trident pods in the namespace
$ kubectl get pods -n trident
NAME                                READY   STATUS    RESTARTS   AGE
trident-csi-79df798bdc-m79dc        6/6     Running   0          1m41s
trident-csi-xrst8                   2/2     Running   0          1m41s
trident-operator-5574dbbc68-nthjv   1/1     Running   0          1m52s

#Confirm Trident has been updated to the desired version
$ kubectl describe torc trident | grep Message -A 3
Message:                Trident installed
Namespace:              trident
Status:                 Installed
Version:                v21.07.1
```

升級Helm型的營運者安裝

請執行下列步驟、升級Helm型的操作員安裝。

步驟

1. 下載最新的Astra Trident版本。
2. 使用「helm升級」命令。請參閱下列範例：

```
$ helm upgrade <name> trident-operator-21.07.1.tgz
```

其中「trident操作者-21.07.1.tgz」代表您要升級的版本。

3. 執行「helm清單」、確認圖表和應用程式版本都已升級。

> 若要在升級期間傳遞組態資料、請使用「-set」。

例如、若要變更預設值「tridentDebug」、請執行下列命令：

```
$ helm upgrade <name> trident-operator-21.07.1-custom.tgz --set
tridentDebug=true
```

如果您執行「$ tridentctl logs」、就會看到偵錯訊息。

> 如果您在初始安裝期間設定任何非預設選項、請確定升級命令中已包含這些選項、否則這些值將會重設為預設值。

從非營運者安裝升級

如果您有符合上述先決條件的「SCSI Trident」執行個體、您可以升級至Trident運算子的最新版本。

步驟

1. 下載最新的Astra Trident版本。

```
# Download the release required [21.07.1]
$ mkdir 21.07.1
$ cd 21.07.1
$ wget
https://github.com/NetApp/trident/releases/download/v21.07.1/trident-
installer-21.07.1.tar.gz
$ tar -xf trident-installer-21.07.1.tar.gz
$ cd trident-installer
```

2. 從資訊清單建立「TridentOrchestrator」CRD。

```
$ kubectl create -f
deploy/crds/trident.netapp.io_tridentorchestrators_crd_post1.16.yaml
```

3. 部署營運者。

```
#Install the cluster-scoped operator in the **same namespace**
$ kubectl create -f deploy/bundle.yaml
serviceaccount/trident-operator created
clusterrole.rbac.authorization.k8s.io/trident-operator created
clusterrolebinding.rbac.authorization.k8s.io/trident-operator created
deployment.apps/trident-operator created
podsecuritypolicy.policy/tridentoperatorpods created

#Examine the pods in the Trident namespace
NAME                                READY   STATUS    RESTARTS   AGE
trident-csi-79df798bdc-m79dc        6/6     Running   0          150d
trident-csi-xrst8                   2/2     Running   0          150d
trident-operator-5574dbbc68-nthjv   1/1     Running   0          1m30s
```

4. 建立「TridentOrchestrator」、以安裝Astra Trident。

```
#Create a tridentOrchestrator to initate a Trident install
$ cat deploy/crds/tridentorchestrator_cr.yaml
apiVersion: trident.netapp.io/v1
kind: TridentOrchestrator
metadata:
  name: trident
spec:
  debug: true
  namespace: trident

$ kubectl create -f deploy/crds/tridentorchestrator_cr.yaml

#Examine the pods in the Trident namespace
NAME                                READY   STATUS    RESTARTS   AGE
trident-csi-79df798bdc-m79dc        6/6     Running   0          1m
trident-csi-xrst8                   2/2     Running   0          1m
trident-operator-5574dbbc68-nthjv   1/1     Running   0          5m41s

#Confirm Trident was upgraded to the desired version
$ kubectl describe torc trident | grep Message -A 3
Message:                Trident installed
Namespace:              trident
Status:                 Installed
Version:                v21.07.1
```

現有的後端和PVCS會自動提供使用。

使用tridentctl進行升級

您可以使用「tridentctl」輕鬆升級現有的Astra Trident安裝。

升級至最新版Astra Trident時、請考慮下列事項：

- 從Trident 20.01開始、只有的試用版 "Volume快照" 支援。Kubernetes系統管理員應謹慎地將Alpha快照物件安全備份或轉換成試用版、以保留舊版Alpha快照。
- Volume快照的試用版推出一組經過修改的CRD和快照控制器、這兩個控制器都應該在安裝Astra Trident之前設定。

> "這篇部落格" 討論將Alpha Volume快照移轉至試用版格式的相關步驟。

解除安裝和重新安裝Astra Trident可做為升級。當您解除安裝Trident時、不會刪除由Astra Trident部署所使用的持續磁碟區宣告（PVc）和持續磁碟區（PV）。當Astra Trident離線時、已配置的PV仍可繼續使用、而Astra Trident會在任何建立於過渡期間的永久虛電路恢復上線後、為其配置磁碟區。

> 升級Astra Trident時、請勿中斷升級程序。確保安裝程式執行完成。

升級後的後續步驟

若要使用較新Trident版本（例如隨需Volume Snapshots）提供的豐富功能集、您可以使用「tridentctl升級」命令來升級磁碟區。

如果有舊版磁碟區、您應該將其從NFS/iSCSI類型升級為「csi」類型、以便使用Astra Trident的完整新功能集。Trident提供的舊PV支援傳統功能集。

決定將磁碟區升級為「csi」類型時、請考慮下列事項：

- 您可能不需要升級所有磁碟區。先前建立的磁碟區將繼續可供存取、並正常運作。
- 在升級時、PV可作為部署/狀態集的一部分掛載。不需要關閉部署/狀態集。
- 您*無法*在升級時將PV附加至獨立式Pod。在升級磁碟區之前、您應該先關閉Pod。
- 您只能升級綁定到PVc的磁碟區。在升級之前、應先移除和匯入未繫結至PVCS的磁碟區。

Volume升級範例

以下範例顯示如何執行Volume升級。

1. 執行「kubectl Get PV」以列出PV。

```
$ kubectl get pv
NAME                          CAPACITY     ACCESS MODES   RECLAIM POLICY
STATUS   CLAIM                   STORAGECLASS    REASON   AGE
default-pvc-1-a8475           1073741824   RWO            Delete
Bound    default/pvc-1           standard                19h
default-pvc-2-a8486           1073741824   RWO            Delete
Bound    default/pvc-2           standard                19h
default-pvc-3-a849e           1073741824   RWO            Delete
Bound    default/pvc-3           standard                19h
default-pvc-4-a84de           1073741824   RWO            Delete
Bound    default/pvc-4           standard                19h
trident                       2Gi          RWO            Retain
Bound    trident/trident                                  19h
```

目前有四部PV是Trident 20.07使用「NetApp.IO/triident」資源配置程式所建立。

2. 執行「kubectl描述PV」以取得PV的詳細資料。

```
$ kubectl describe pv default-pvc-2-a8486

Name:            default-pvc-2-a8486
Labels:          <none>
Annotations:     pv.kubernetes.io/provisioned-by: netapp.io/trident
                 volume.beta.kubernetes.io/storage-class: standard
Finalizers:      [kubernetes.io/pv-protection]
StorageClass:    standard
Status:          Bound
Claim:           default/pvc-2
Reclaim Policy:  Delete
Access Modes:    RWO
VolumeMode:      Filesystem
Capacity:        1073741824
Node Affinity:   <none>
Message:
Source:
    Type:      NFS (an NFS mount that lasts the lifetime of a pod)
    Server:    10.xx.xx.xx
    Path:      /trid_1907_alpha_default_pvc_2_a8486
    ReadOnly:  false
```

PV是使用「NetApp.IO/Trident」資源配置程式所建立、屬於NFS類型。為了支援Astra Trident提供的所有新功能、此PV應升級為「csi」類型。

3. 執行「tridenctl升級Volume <三位角Volume >」命令、將舊版Astra Trident Volume升級至csi規格。

```
$ ./tridentctl get volumes -n trident
+----------------------+---------+---------------+----------
+--------------------------------------+--------+---------+
|            NAME      |  SIZE   | STORAGE CLASS | PROTOCOL |
BACKEND UUID             | STATE  | MANAGED |
+----------------------+---------+---------------+----------
+--------------------------------------+--------+---------+
| default-pvc-2-a8486 | 1.0 GiB | standard      | file     | c5a6f6a4-
b052-423b-80d4-8fb491a14a22 | online | true    |
| default-pvc-3-a849e | 1.0 GiB | standard      | file     | c5a6f6a4-
b052-423b-80d4-8fb491a14a22 | online | true    |
| default-pvc-1-a8475 | 1.0 GiB | standard      | file     | c5a6f6a4-
b052-423b-80d4-8fb491a14a22 | online | true    |
| default-pvc-4-a84de | 1.0 GiB | standard      | file     | c5a6f6a4-
b052-423b-80d4-8fb491a14a22 | online | true    |
+----------------------+---------+---------------+----------
+--------------------------------------+--------+---------+

$ ./tridentctl upgrade volume default-pvc-2-a8486 -n trident
+----------------------+---------+---------------+----------
+--------------------------------------+--------+---------+
|            NAME      |  SIZE   | STORAGE CLASS | PROTOCOL |
BACKEND UUID             | STATE  | MANAGED |
+----------------------+---------+---------------+----------
+--------------------------------------+--------+---------+
| default-pvc-2-a8486 | 1.0 GiB | standard      | file     | c5a6f6a4-
b052-423b-80d4-8fb491a14a22 | online | true    |
+----------------------+---------+---------------+----------
+--------------------------------------+--------+---------+
```

4. 執行「kubecll描述PV」來驗證磁碟區是否為csi Volume。

```
$ kubectl describe pv default-pvc-2-a8486
Name:            default-pvc-2-a8486
Labels:          <none>
Annotations:     pv.kubernetes.io/provisioned-by: csi.trident.netapp.io
                 volume.beta.kubernetes.io/storage-class: standard
Finalizers:      [kubernetes.io/pv-protection]
StorageClass:    standard
Status:          Bound
Claim:           default/pvc-2
Reclaim Policy:  Delete
Access Modes:    RWO
VolumeMode:      Filesystem
Capacity:        1073741824
Node Affinity:   <none>
Message:
Source:
    Type:              CSI (a Container Storage Interface (CSI) volume
source)
    Driver:            csi.trident.netapp.io
    VolumeHandle:      default-pvc-2-a8486
    ReadOnly:          false
    VolumeAttributes:      backendUUID=c5a6f6a4-b052-423b-80d4-
8fb491a14a22

internalName=trid_1907_alpha_default_pvc_2_a8486
                           name=default-pvc-2-a8486
                           protocol=file
Events:                <none>
```

如此一來、您就能將由Astra Trident建立的NFS/iSCSI類型磁碟區、以每個磁碟區為基礎、升級為「csi」類型。

解除安裝Astra Trident

視Astra Trident的安裝方式而定、有多種選項可供您解除安裝。

使用Helm解除安裝

如果您使用Helm安裝Astra Trident、您可以使用「helm uninstall」來解除安裝。

```
#List the Helm release corresponding to the Astra Trident install.
$ helm ls -n trident
NAME          NAMESPACE       REVISION        UPDATED
STATUS          CHART                           APP VERSION
trident       trident         1               2021-04-20
00:26:42.417764794 +0000 UTC deployed        trident-operator-21.07.1
21.07.1

#Uninstall Helm release to remove Trident
$ helm uninstall trident -n trident
release "trident" uninstalled
```

使用Trident運算子解除安裝

如果您使用運算子安裝了Astra Trident、您可以執行下列其中一項來解除安裝：

- 要設定解除安裝旗標、請編輯「TridentOrchestrator」：*您可以編輯「TridentOrchestrator」、然後設定「s.pec.uninstall=true」。編輯「TridentOrchestrator」CR、並設定「uninstall」旗標、如下所示：

  ```
  $  kubectl patch torc <trident-orchestrator-name> --type=merge -p
  '{"spec":{"uninstall":true}}'
  ```

  當「uninstall」旗標設定為「true」時、Trident操作者會解除安裝Trident、但不會移除TridentOrchestrator本身。如果您想要再次安裝Trident、請清理TridentOrchestrator並建立新的Trident。

- *刪除「TridentOrchestrator」：*移除用來部署Astra Trident的「TridentOrchestrator」、您就會指示操作員解除安裝Trident。營運者會處理刪除「TridentOrchestrator」的程序、並繼續移除Astra Trident部署和取消安裝、刪除其在安裝過程中建立的Trident Pod。若要完全移除Astra Trident（包括它所建立的CRD）、並有效清除Slate Clean、您可以編輯「TridentOrchestrator」來通過「wipeout」選項。請參閱下列範例：

```
$ kubectl patch torc <trident-orchestrator-name> --type=merge -p
'{"spec":{"wipeout":["crds"],"uninstall":true}}'
```

這會完全解除安裝Astra Trident、並清除所有與後端及其管理的磁碟區相關的中繼資料。後續安裝會視為全新安裝。

> 執行完整解除安裝時、您只能考慮抹除客戶需求日。此動作無法復原。除非您想要重新開始並建立全新的**Astra Trident**安裝、否則請勿清除客戶需求日。

使用tridentctl解除安裝

執行「tridentctl」中的「uninstall」命令、移除所有與Astra Trident相關的資源、但CRD和相關物件除外、因此可輕鬆再次執行安裝程式、以更新至較新的版本。

```
./tridentctl uninstall -n <namespace>
```

若要完整移除Astra Trident、您應移除由Astra Trident所建立之CRD的微調工具、然後刪除CRD。

降級Astra Trident

瞭解降級至舊版Astra Trident的相關步驟。

您可能會考慮降級的原因有多種、例如：

- 應變規劃
- 立即修正升級後發現的錯誤
- 相依性問題、不成功和不完整的升級

何時降級

移至使用CRD的Astra Trident版本時、您應該考慮降級。由於Astra Trident現在使用CRD來維護狀態、因此所建立的所有儲存實體（後端、儲存類別、PV和Volume快照）都會有相關的CRD物件、而非寫入「Trident」PV（由舊版Astra Trident所使用）的資料。新建立的PV、後端和儲存類別都會維護為CRD物件。如果您需要降級、則只能針對使用CRD（19.07及更新版本）執行的Astra Trident版本嘗試降級。這是為了確保在降級發生後、可以看到目前Astra Trident版本上執行的所有作業。

何時不降級

您不應降級至使用「etcd」來維持狀態的Trident版本（19.04及更早版本）。所有以目前Astra Trident版本執行的作業、都不會在降級後反映出來。移回舊版時、無法使用新建立的PV。當您移回舊版時、Astra Trident不會看到對物件所做的變更、例如後端、PV、儲存類別和Volume快照（已建立/更新/刪除）。返回舊版不會中斷已使用舊版建立之PV的存取、除非已升級。

使用業者安裝**Astra Trident**時的降級程序

對於使用Trident操作員完成的安裝、降級程序會有所不同、不需要使用「tridentctl」。

對於使用Trident營運者完成的安裝、Astra Trident可降級為下列任一項：

- 使用命名空間範圍運算子安裝的版本（20.07 - 20.10）。
- 使用叢集範圍運算子（21.01及更新版本）安裝的版本。

降級至叢集範圍的運算子

若要將Astra Trident降級為使用叢集範圍運算子的版本、請遵循下列步驟。

步驟

1. "解除安裝Astra Trident"。除非您想要完全移除現有的安裝、否則請勿使用**CRD**。
2. 刪除叢集範圍的運算子。為達成此目的、您將需要用於部署營運者的資訊清單。您可以從取得 "Trident GitHub repo"。請務必切換至所需的分公司。

3. 安裝所需版本的Astra Trident、以繼續降級。請遵循所需版本的說明文件。

降級至命名空間範圍的運算子

本節摘要說明降級至Astra Trident版本所需的步驟、範圍介於20.07到20.10之間、此版本將使用命名空間範圍的運算子來安裝。

步驟

1. "解除安裝Astra Trident"。*除非您想要完全移除現有的安裝、否則請勿刪掉CRD。*請確定刪除了「三線管協調器」。

```
#Check to see if there are any tridentorchestrators present
$ kubectl get torc
NAME        AGE
trident     20h

#Looks like there is a tridentorchestrator that needs deleting
$ kubectl delete torc trident
tridentorchestrator.trident.netapp.io "trident" deleted
```

2. 刪除叢集範圍的運算子。為達成此目的、您將需要用於部署營運者的資訊清單。您可以從取得 "Trident GitHub repo"。請務必切換至所需的分公司。
3. 刪除「TridentOrchestrator」CRD。

```
#Check to see if ``tridentorchestrators.trident.netapp.io`` CRD is
present and delete it.
$ kubectl get crd tridentorchestrators.trident.netapp.io
NAME                                      CREATED AT
tridentorchestrators.trident.netapp.io   2021-01-21T21:11:37Z
$ kubectl delete crd tridentorchestrators.trident.netapp.io
customresourcedefinition.apiextensions.k8s.io
"tridentorchestrators.trident.netapp.io" deleted
```

   Astra Trident已解除安裝。

4. 安裝所需的版本、繼續降級。請遵循所需版本的說明文件。

使用**Helm**降級

若要降級、請使用「helm復原」命令。請參閱下列範例：

```
$ helm rollback trident [revision #]
```

使用tridentctl安裝Astra Trident時的降級程序

如果您使用「tridentctl」來安裝Astra Trident、降級程序會涉及下列步驟。此順序將引導您完成從Astra Trident 21.007移至20.07的降級程序。

> 在開始降級之前、您應該擷取Kubernetes叢集的「etcd」快照。這可讓您備份Astra Trident客戶需求日的目前狀態。

步驟

1. 請確定使用「tridentctl」安裝Trident。如果您不確定Astra Trident的安裝方式、請執行以下簡單測試：
   a. 列出Trident命名空間中的Pod。
   b. 識別叢集中執行的Astra Trident版本。您可以使用「tridentctl」或查看Trident Pod中使用的影像。
   c. 如果您*沒有看到*「tridentOrchestrator」、（或）「trident置 備程式」、（或）名為「trident - oper-xxxxx-xxxxX」的Pod、則Astra Trident 會安裝「tridentctl」。
2. 使用現有的「tridentctl」二進位檔來解除安裝Astra Trident。在這種情況下、您將會以21.07二進位檔解除安裝。

```
$ tridentctl version -n trident
+----------------+----------------+
| SERVER VERSION | CLIENT VERSION |
+----------------+----------------+
| 21.07.0        | 21.07.0        |
+----------------+----------------+

$ tridentctl uninstall -n trident
INFO Deleted Trident deployment.
INFO Deleted Trident daemonset.
INFO Deleted Trident service.
INFO Deleted Trident secret.
INFO Deleted cluster role binding.
INFO Deleted cluster role.
INFO Deleted service account.
INFO Deleted pod security policy.
podSecurityPolicy=tridentpods
INFO The uninstaller did not delete Trident's namespace in case it is
going to be reused.
INFO Trident uninstallation succeeded.
```

3. 完成後、請取得所需版本的Trident二進位檔（本範例為20.07）、並使用它來安裝Astra Trident。您可以為產生自訂YAML "自訂安裝" 如有需要。

```
$ cd 20.07/trident-installer/
$ ./tridentctl install -n trident-ns
INFO Created installer service account.
serviceaccount=trident-installer
INFO Created installer cluster role.               clusterrole=trident-
installer
INFO Created installer cluster role binding.
clusterrolebinding=trident-installer
INFO Created installer configmap.                  configmap=trident-
installer
...
...
INFO Deleted installer cluster role binding.
INFO Deleted installer cluster role.
INFO Deleted installer service account.
```

降級程序已完成。

使用Astra Trident

設定後端

後端定義了Astra Trident與儲存系統之間的關係。它告訴Astra Trident如何與該儲存系統通訊、以及Astra Trident如何從該儲存系統配置磁碟區。Astra Trident會自動從後端提供符合儲存類別所定義需求的儲存資源池。深入瞭解如何根據您擁有的儲存系統類型來設定後端。

- "設定Azure NetApp Files 一個靜態後端"
- "設定Cloud Volumes Service 適用於Google Cloud Platform後端的功能"
- "設定NetApp HCI 一個不只是功能的SolidFire 後端"
- "使用ONTAP 功能不一的Cloud Volumes ONTAP NAS驅動程式來設定後端"
- "使用ONTAP 不支援的Cloud Volumes ONTAP SAN驅動程式來設定後端"
- "使用Astra Trident搭配Amazon FSX for NetApp ONTAP 解決方案"

設定Azure NetApp Files 一個靜態後端

瞭Azure NetApp Files 解如何使用提供的範例組態、將靜態（anf）設定為Astra Trident安裝的後端。

> 該支援服務不支援低於100 GB的磁碟區。Azure NetApp Files如果要求較小的磁碟區、Astra Trident會自動建立100-GB磁碟區。

若要設定及使用 "Azure NetApp Files" 後端、您需要下列項目：

- Azure訂閱提供的「SubscriptionID」Azure NetApp Files （含功能不支援的功能）。
- 來自於的「TenantId」、「clientID」和「clientSecret」 "應用程式註冊" 在Azure Active Directory中、具備Azure NetApp Files 充分的權限執行此功能。應用程式登錄應使用Azure預先定義的「擁有者」或「貢獻者」角色。

  > 若要深入瞭解Azure內建角色、請參閱 "Azure文件"。

- Azure的「位置」至少包含一個 "委派的子網路"。從Trident 22.01起、「位置」參數是後端組態檔最上層的必填欄位。會忽略虛擬資源池中指定的位置值。
- 如果Azure NetApp Files 您是第一次使用或是在新的位置使用、則需要進行一些初始組態。請參閱 "快速入門指南"。

Trident會根據後端組態（子網路、虛擬網路、服務層級和位置）、在所要求位置可用的容量集區上建立ANF磁碟區、並符合所要求的服務層級和子網路。

> 附註：Astra Trident不支援手動QoS容量資源池。

後端組態選項

如需後端組態選項、請參閱下表：

參數	說明	預設
「分度」		永遠為1
「torageDriverName」	儲存驅動程式名稱	「Azure - NetApp-Files」
「後端名稱」	自訂名稱或儲存後端	驅動程式名稱+「_」+隨機字元
《訂閱ID》	Azure訂閱的訂閱ID	
「TenantId」	應用程式註冊的租戶ID	
"clientID"	應用程式註冊的用戶端ID	
「客戶機密」	應用程式註冊的用戶端機密	
《服務層級》	其中一種是「標準」、「高級」或「超高」	""（隨機）
位置	要建立新磁碟區的Azure位置名稱	
"來源群組"	用於篩選已探索資源的資源群組清單	「[]」（無篩選器）
《netappAccounts》	篩選探索資源的NetApp帳戶清單	「[]」（無篩選器）
《容量Pools》	用於篩選已探索資源的容量集區清單	「[]」（無篩選器、隨機）
「虛擬化網路」	具有委派子網路的虛擬網路名稱	"
《Subnet》	委派給「icrosoft.Netapp/volumes`」的子網路名稱	"
「nfsMountOptions」	精細控制NFS掛載選項。	"nfsves=3"
《限制Volume大小》	如果要求的磁碟區大小高於此值、則資源配置失敗	""（預設不強制執行）
「DebugTraceFlags」	疑難排解時要使用的偵錯旗標。範例：「\{"API"：假、「方法」：真、「探索」：true} 。除非您正在進行疑難排解並需要詳細的記錄傾印、否則請勿使用此功能。	null

如果嘗試建立永久虛擬基礎架構時發生「找不到容量資源池」錯誤、您的應用程式登錄可能沒有相關的必要權限和資源（子網路、虛擬網路、容量資源池）。Astra Trident會在啟用偵錯功能時、記錄在建立後端時所探索到的Azure資源。請務必檢查是否使用適當的角色。

如果您想使用NFS 4.1版掛載磁碟區、可以在以逗號分隔的掛載選項清單中加入「nfsves=4'」、以選擇NFS v4.1。儲存類別中設定的任何掛載選項、都會覆寫在後端組態檔中設定的掛載選項。

可以使用短名稱或全限定名稱來指定"sourceGroups"、"netappAccounts"、"capacityPools、"virtualNetwork（虛擬化網路）和"shubnet"的值。簡短名稱可能會比對同名的多個資源、因此在大多數情況下、建議使用完整名稱。「資源群組」、「netappAccounts」和「capacityPools」值是篩選器、可將探索到的資源集限制在此儲存後端可用的資源集、並可任意組合指定。完整名稱格式如下：

類型	格式
資源群組	<資源群組>
NetApp帳戶	資源群組//<NetApp帳戶>
容量資源池	資源群組//<NetApp帳戶>/<容量資源池>
虛擬網路	資源群組//<虛擬網路>
子網路	資源群組//<虛擬網路>/<子網路>

您可以在組態檔的特殊區段中指定下列選項、以控制預設的每個Volume佈建方式。請參閱下列組態範例。

參數	說明	預設
「匯出規則」	新磁碟區的匯出規則	「0.00.0.0/0」
「napshotDir	控制.snapshot目錄的可見度	"假"
《大小》	新磁碟區的預設大小	100公克
「unixPermissions」	新磁碟區的UNIX權限（4個八進位數字）	""（預覽功能、訂閱時需要白名單）

「匯出規則」值必須是以逗號分隔的清單、以CIDR表示法列出所有的IPv4位址或IPv4子網路組合。

> 對於在ANF後端上建立的所有磁碟區、Astra Trident會將儲存資源池上的所有標籤複製到資源配置時的儲存磁碟區。儲存管理員可以定義每個儲存資源池的標籤、並將儲存資源池中建立的所有磁碟區分組。這是根據後端組態中提供的一組可自訂標籤、方便區分磁碟區的方法。

範例**1**：最低組態

這是絕對最低的後端組態。使用此組態、Astra Trident會在設定的位置探索所有NetApp帳戶、容量集區和委派給ANF的子網路、並隨機將新磁碟區放在其中一個集區和子網路上。

當您剛開始使用ANF並嘗試各種功能時、這種組態是理想的選擇、但實際上您想要為您所配置的磁碟區提供額外的範圍。

```
{
    "version": 1,
    "storageDriverName": "azure-netapp-files",
    "subscriptionID": "9f87c765-4774-fake-ae98-a721add45451",
    "tenantID": "68e4f836-edc1-fake-bff9-b2d865ee56cf",
    "clientID": "dd043f63-bf8e-fake-8076-8de91e5713aa",
    "clientSecret": "SECRET",
    "location": "eastus"
}
```

範例**2**：使用容量集區篩選器的特定服務層級組態

此後端組態可將磁碟區置於Azure的「東向」位置、位於「超」容量資源池中。Astra Trident會自動探索該位置委派給ANF的所有子網路、並隨機在其中一個磁碟區上放置新磁碟區。

```
    {
        "version": 1,
        "storageDriverName": "azure-netapp-files",
        "subscriptionID": "9f87c765-4774-fake-ae98-a721add45451",
        "tenantID": "68e4f836-edc1-fake-bff9-b2d865ee56cf",
        "clientID": "dd043f63-bf8e-fake-8076-8de91e5713aa",
        "clientSecret": "SECRET",
        "location": "eastus",
        "serviceLevel": "Ultra",
        "capacityPools": [
            "application-group-1/account-1/ultra-1",
            "application-group-1/account-1/ultra-2"
],
    }
```

範例**3**：進階組態

此後端組態可進一步將磁碟區放置範圍縮小至單一子網路、並修改部分Volume資源配置預設值。

```
    {
        "version": 1,
        "storageDriverName": "azure-netapp-files",
        "subscriptionID": "9f87c765-4774-fake-ae98-a721add45451",
        "tenantID": "68e4f836-edc1-fake-bff9-b2d865ee56cf",
        "clientID": "dd043f63-bf8e-fake-8076-8de91e5713aa",
        "clientSecret": "SECRET",
        "location": "eastus",
        "serviceLevel": "Ultra",
        "capacityPools": [
            "application-group-1/account-1/ultra-1",
            "application-group-1/account-1/ultra-2"
],
        "virtualNetwork": "my-virtual-network",
        "subnet": "my-subnet",
        "nfsMountOptions": "vers=3,proto=tcp,timeo=600",
        "limitVolumeSize": "500Gi",
        "defaults": {
            "exportRule": "10.0.0.0/24,10.0.1.0/24,10.0.2.100",
            "snapshotDir": "true",
            "size": "200Gi",
            "unixPermissions": "0777"
=======
        }
    }
```

範例**4**：虛擬儲存池組態

此後端組態可在單一檔案中定義多個儲存集區。當您有多個容量集區支援不同的服務層級、而且想要在Kubernetes中建立代表這些層級的儲存類別時、這很有用。

```
{
    "version": 1,
    "storageDriverName": "azure-netapp-files",
    "subscriptionID": "9f87c765-4774-fake-ae98-a721add45451",
    "tenantID": "68e4f836-edc1-fake-bff9-b2d865ee56cf",
    "clientID": "dd043f63-bf8e-fake-8076-8de91e5713aa",
    "clientSecret": "SECRET",
    "location": "eastus",
    "resourceGroups": ["application-group-1"],
    "nfsMountOptions": "vers=3,proto=tcp,timeo=600",
    "labels": {
        "cloud": "azure"
    },
    "location": "eastus",

    "storage": [
        {
            "labels": {
                "performance": "gold"
            },
            "serviceLevel": "Ultra",
            "capacityPools": ["ultra-1", "ultra-2"]
        },
        {
            "labels": {
                "performance": "silver"
            },
            "serviceLevel": "Premium",
            "capacityPools": ["premium-1"]
        },
        {
            "labels": {
                "performance": "bronze"
            },
            "serviceLevel": "Standard",
            "capacityPools": ["standard-1", "standard-2"]
        }
    ]
}
```

下列「儲存類別」定義是指上述的儲存資源池。您可以使用「parameters.selector`」欄位、為每個「儲存類

別」指定用於裝載磁碟區的虛擬資源池。該磁碟區會在所選的資源池中定義各個層面。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gold
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=gold"
allowVolumeExpansion: true
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: silver
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=silver"
allowVolumeExpansion: true
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: bronze
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=bronze"
allowVolumeExpansion: true
```

接下來呢？

建立後端組態檔之後、請執行下列命令：

```
tridentctl create backend -f <backend-file>
```

如果後端建立失敗、表示後端組態有問題。您可以執行下列命令來檢視記錄、以判斷原因：

```
tridentctl logs
```

識別並修正組態檔的問題之後、您可以再次執行create命令。

針對GCP後端設定CVS

瞭Cloud Volumes Service 解如何使用所提供的範例組態、將NetApp for Google Cloud Platform（GCP）設定為Astra Trident安裝的後端。

> 適用於Google Cloud的NetApp Cloud Volumes Service 支援的CVS效能磁碟區大小不得低於100 GiB、或CVS磁碟區大小不得低於300 GiB。如果所要求的磁碟區小於最小大小、Astra Trident會自動建立最小大小的磁碟區。

以設定及使用 "適用於 Google Cloud Cloud Volumes Service" 後端、您需要下列項目：

- 使用NetApp CVS設定的Google Cloud帳戶
- Google Cloud帳戶的專案編號
- Google Cloud服務帳戶的角色為「netappcloudVolumes.admin」
- CVS服務帳戶的API金鑰檔

Astra Trident現在支援預設的較小磁碟區 "GCP上的CVS服務類型"。對於以「儲存類別=軟體」建立的後端、現在磁碟區的資源配置大小最小為300 GiB。CVS目前在「管制可用度」下提供此功能、並不提供技術支援。使用者必須註冊才能存取低於1TiB的磁碟區 "請按這裡"。NetApp建議客戶使用低於1TiB的磁碟區來處理*非正式作業*的工作負載。

> 使用預設CVS服務類型（「儲存類別=軟體」）部署後端時、使用者必須取得GCP上的子1TiB磁碟區功能、以取得相關專案編號和專案ID的存取權。這是Astra Trident配置子1TiB磁碟區所需的功能。如果沒有、則低於600 GiB的PVCS將無法建立Volume。使用取得對低於1TiB磁碟區的存取權 "這份表格"。

由Astra Trident針對預設CVS服務層級所建立的磁碟區、將會配置如下：

- 小於300 GiB的PVCS會導致Astra Trident建立300 GiB CVS Volume。
- 介於300 GiB到600 GiB之間的PVCS會導致Astra Trident建立一個所需大小的CVS Volume。
- 介於600 GiB和1 TiB之間的PVCS會導致Astra Trident建立1TiB CVS Volume。
- 大於1 TiB的PVCS會導致Astra Trident建立所需大小的CVS Volume。

後端組態選項

如需後端組態選項、請參閱下表：

參數	說明	預設
「分度」		永遠為1
「torageDriverName」	儲存驅動程式名稱	「GCP-CVS」
「後端名稱」	自訂名稱或儲存後端	驅動程式名稱+「_」+ API金鑰的一部分
「torageClass」	儲存類型：可選擇「硬體」（效能最佳化）或「軟體」（CVS服務類型）	

參數	說明	預設
「ProjectNumber」	Google Cloud帳戶專案編號。此值可在Google Cloud入口網站的首頁找到。	
《apiRegion》	CVS帳戶區域。這是後端配置磁碟區的區域。	
「apiKey」	Google Cloud服務帳戶的API金鑰、其角色 為「netappcloudVolumes.admin」。其中包含Google Cloud服務帳戶私密金鑰檔案（逐字複製到後端組態檔）的JSON-格式內容。	
"proxyurl"	Proxy URL（如果需要代理伺服器才能連線至CVS帳戶）。Proxy伺服器可以是HTTP Proxy或HTTPS Proxy。對於HTTPS Proxy、會跳過憑證驗證、以允許在Proxy伺服器中使用自我簽署的憑證。不支援已啟用驗證的Proxy伺服器。	
「nfsMountOptions」	精細控制NFS掛載選項。	"nfsves=3"
《限制Volume大小》	如果要求的磁碟區大小高於此值、則資源配置失敗	""（預設不強制執行）
《服務層級》	新磁碟區的CVS服務層級。這些值包括「標準」、「高級」和「極端」。	"標準"
網路	用於CVS磁碟區的GCP網路	「預設」
「DebugTraceFlags」	疑難排解時要使用的偵錯旗標。例如：「 \{"API":fals,"method":true}」。除非您正在進行疑難排解並需要詳細的記錄傾印、否則請勿使用此功能。	null

如果使用共享VPC網路、則必須同時指定「ProjectNumber」和「hostProjectNumber」。在這種情況下、「專案編號」是服務專案、而「主機專案編號」則是主機專案。

「apiRegion」代表Astra Trident建立CVS磁碟區的GCP區域。在建立跨區域Kubernetes叢集時、在「apiRegion」中建立的CVS磁碟區可用於多個GCP區域的節點上排程的工作負載。請注意、跨區域流量會產生額外成本。

- 若要啟用跨區域存取、您的StorageClass定義「alledtopologies」必須包含所有地區。例如：

```
- key: topology.kubernetes.io/region
  values:
  - us-east1
  - europe-west1
```

- 「torageClass」是選用參數、可用來選取所需的參數 "CVS服務類型"。您可以從基礎CVS服務類型（「儲存類別=軟體」）或CVS效能服務類型（「儲存類別=硬體」）中進行選擇、這是Trident預設使用的類型。請務必指定「apiRegion」、在後端定義中提供各自的CVS「儲存類別」。

Astra Trident與Google Cloud上的基礎CVS服務類型整合、是一項*測試版功能、不適用於正式作業工作負載。Trident 完全支援 CVS效能服務類型、並依預設使用。

每個後端都會在單一Google Cloud區域中配置磁碟區。若要在其他區域建立磁碟區、您可以定義其他後端。

您可以在組態檔的特殊區段中指定下列選項、以控制預設的每個Volume佈建方式。請參閱下列組態範例。

參數	說明	預設
「匯出規則」	新磁碟區的匯出規則	「0.00.0.0/0」
「napshotDir	存取「.snapshot」目錄	"假"
「快照保留區」	保留給快照的磁碟區百分比	""（接受CVS預設值為0）
《大小》	新磁碟區的大小	「100Gi」

「匯出規則」值必須是以逗號分隔的清單、以CIDR表示法列出所有的IPv4位址或IPv4子網路組合。

針對在CVS Google Cloud後端上建立的所有磁碟區、Trident會在儲存資源池上的所有標籤配置時複製到儲存磁碟區。儲存管理員可以定義每個儲存資源池的標籤、並將儲存資源池中建立的所有磁碟區分組。這是根據後端組態中提供的一組可自訂標籤、方便區分磁碟區的方法。

範例**1**：最低組態

這是絕對最低的後端組態。

```
{
    "version": 1,
    "storageDriverName": "gcp-cvs",
    "projectNumber": "012345678901",
    "apiRegion": "us-west2",
    "apiKey": {
        "type": "service_account",
        "project_id": "my-gcp-project",
        "private_key_id": "1234567890123456789012345678901234567890",
```

```
        "private_key": "-----BEGIN PRIVATE KEY-----
\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZ
srrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisI
sAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSa
PIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZN
chRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlz
ZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl
/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kw
s8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY
9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHc
zZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHi
sIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOgu
SaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyA
ZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGz
lzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3
bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4
Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5o
jY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nzn
HczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrt
HisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbO
guSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKe
yAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRA
GzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4j
K3bl/qp8B4Kws8zX5ojY9m\nXsYg6gyxy4zq7OlwWgLwGa==\n-----END PRIVATE
KEY-----\n",
        "client_email": "cloudvolumes-admin-sa@my-gcp-
project.iam.gserviceaccount.com",
        "client_id": "123456789012345678901",
        "auth_uri": "https://accounts.google.com/o/oauth2/auth",
        "token_uri": "https://oauth2.googleapis.com/token",
        "auth_provider_x509_cert_url":
"https://www.googleapis.com/oauth2/v1/certs",
        "client_x509_cert_url":
"https://www.googleapis.com/robot/v1/metadata/x509/cloudvolumes-admin-
sa%40my-gcp-project.iam.gserviceaccount.com"
    }
}
```


範例**2**：基礎**CVS**服務類型組態

此範例顯示使用基本CVS服務類型的後端定義、此服務類型適用於一般用途的工作負載、提供輕度/中度效能、以及高分區可用度。

```
{
    "version": 1,
    "storageDriverName": "gcp-cvs",
```

```
    "projectNumber": "012345678901",
    "storageClass": "software",
    "apiRegion": "us-east4",
    "apiKey": {
        "type": "service_account",
        "project_id": "my-gcp-project",
        "private_key_id": "1234567890123456789012345678901234567890",
        "private_key": "-----BEGIN PRIVATE KEY-----
\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZ
srrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisI
sAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSa
PIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZN
chRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlz
ZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl
/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kw
s8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY
9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHc
zZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHi
sIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOgu
SaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyA
ZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGz
lzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3
bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4
Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5o
jY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nzn
HczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrt
HisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbO
guSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKe
yAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRA
GzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4j
K3bl/qp8B4Kws8zX5ojY9m\nXsYg6gyxy4zq7OlwWgLwGa==\n-----END PRIVATE
KEY-----\n",
        "client_email": "cloudvolumes-admin-sa@my-gcp-
project.iam.gserviceaccount.com",
        "client_id": "123456789012345678901",
        "auth_uri": "https://accounts.google.com/o/oauth2/auth",
        "token_uri": "https://oauth2.googleapis.com/token",
        "auth_provider_x509_cert_url":
"https://www.googleapis.com/oauth2/v1/certs",
        "client_x509_cert_url":
"https://www.googleapis.com/robot/v1/metadata/x509/cloudvolumes-admin-
sa%40my-gcp-project.iam.gserviceaccount.com"
    }
}
```

範例3：單一服務層級組態

此範例顯示後端檔案、可將相同層面套用至Google Cloud us-west2區域中所有由Astra Trident建立的儲存設備。此範例也顯示後端組態檔中使用「proxyURL」的情形。

```
{
    "version": 1,
    "storageDriverName": "gcp-cvs",
    "projectNumber": "012345678901",
    "apiRegion": "us-west2",
    "apiKey": {
        "type": "service_account",
        "project_id": "my-gcp-project",
        "private_key_id": "1234567890123456789012345678901234567890",
        "private_key": "-----BEGIN PRIVATE KEY-----
\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZ
srrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisI
sAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSa
PIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZN
chRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlz
ZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl
/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kw
s8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY
9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHc
zZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHi
sIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOgu
SaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyA
ZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGz
lzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3
bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4
Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5o
jY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nzn
HczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrt
HisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbO
guSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKe
yAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRA
GzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4j
K3bl/qp8B4Kws8zX5ojY9m\nXsYg6gyxy4zq7OlwWgLwGa==\n-----END PRIVATE
KEY-----\n",
        "client_email": "cloudvolumes-admin-sa@my-gcp-
project.iam.gserviceaccount.com",
        "client_id": "123456789012345678901",
        "auth_uri": "https://accounts.google.com/o/oauth2/auth",
        "token_uri": "https://oauth2.googleapis.com/token",
        "auth_provider_x509_cert_url":
"https://www.googleapis.com/oauth2/v1/certs",
```

```
        "client_x509_cert_url":
"https://www.googleapis.com/robot/v1/metadata/x509/cloudvolumes-admin-
sa%40my-gcp-project.iam.gserviceaccount.com"
    },
    "proxyURL": "http://proxy-server-hostname/",
    "nfsMountOptions": "vers=3,proto=tcp,timeo=600",
    "limitVolumeSize": "10Ti",
    "serviceLevel": "premium",
    "defaults": {
        "snapshotDir": "true",
        "snapshotReserve": "5",
        "exportRule": "10.0.0.0/24,10.0.1.0/24,10.0.2.100",
        "size": "5Ti"
    }
}
```

範例**4**：虛擬儲存池組態

此範例顯示使用虛擬儲存資源池設定的後端定義檔、以及回溯到這些資源池的「儲存類別」。

在下圖所示的範例後端定義檔中、會針對所有儲存資源池設定特定的預設值、將「快照保留」設為5%、將「匯出規則」設為0.00.0/0。虛擬儲存資源池是在「儲存區」區段中定義的。在此範例中、每個個別的儲存資源池都會設定自己的「服務層級」、有些資源池則會覆寫預設值。

```
{
    "version": 1,
    "storageDriverName": "gcp-cvs",
    "projectNumber": "012345678901",
    "apiRegion": "us-west2",
    "apiKey": {
        "type": "service_account",
        "project_id": "my-gcp-project",
        "private_key_id": "1234567890123456789012345678901234567890",
        "private_key": "-----BEGIN PRIVATE KEY-----
\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZ
srrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisI
sAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSa
PIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZN
chRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlz
ZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl
/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kw
s8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY
9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHc
zZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHi
sIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOgu
SaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyA
```

```
ZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGz
lzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3
bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4
Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5o
jY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nzn
HczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrt
HisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbO
guSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKe
yAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRA
GzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4j
K3bl/qp8B4Kws8zX5ojY9m\nXsYg6gyxy4zq7OlwWgLwGa==\n-----END PRIVATE
KEY-----\n",
        "client_email": "cloudvolumes-admin-sa@my-gcp-
project.iam.gserviceaccount.com",
        "client_id": "123456789012345678901",
        "auth_uri": "https://accounts.google.com/o/oauth2/auth",
        "token_uri": "https://oauth2.googleapis.com/token",
        "auth_provider_x509_cert_url":
"https://www.googleapis.com/oauth2/v1/certs",
        "client_x509_cert_url":
"https://www.googleapis.com/robot/v1/metadata/x509/cloudvolumes-admin-
sa%40my-gcp-project.iam.gserviceaccount.com"
    },
    "nfsMountOptions": "vers=3,proto=tcp,timeo=600",

    "defaults": {
        "snapshotReserve": "5",
        "exportRule": "0.0.0.0/0"
    },

    "labels": {
        "cloud": "gcp"
    },
    "region": "us-west2",

    "storage": [
        {
            "labels": {
                "performance": "extreme",
                "protection": "extra"
            },
            "serviceLevel": "extreme",
            "defaults": {
                "snapshotDir": "true",
                "snapshotReserve": "10",
                "exportRule": "10.0.0.0/24"
```

```
            }
        },
        {
            "labels": {
                "performance": "extreme",
                "protection": "standard"
            },
            "serviceLevel": "extreme"
        },
        {
            "labels": {
                "performance": "premium",
                "protection": "extra"
            },
            "serviceLevel": "premium",
            "defaults": {
                "snapshotDir": "true",
                "snapshotReserve": "10"
            }
        },

        {
            "labels": {
                "performance": "premium",
                "protection": "standard"
            },
            "serviceLevel": "premium"
        },

        {
            "labels": {
                "performance": "standard"
            },
            "serviceLevel": "standard"
        }
    ]
}
```

下列StorageClass定義係指上述儲存資源池。您可以使用「parameters.selector`」欄位、為每個StorageClass指定用於裝載磁碟區的虛擬集區。該磁碟區會在所選的資源池中定義各個層面。

第一個StorageClass（「CVS極致額外保護」）會對應至第一個虛擬儲存資源池。這是唯一提供極致效能、快照保留率為10%的資源池。最後一個StorageClass（「CVS額外保護」）會呼叫提供快照保留10%的任何儲存資源池。Astra Trident決定選取哪個虛擬儲存池、並確保符合快照保留需求。

```
apiVersion: storage.k8s.io/v1
```

```
kind: StorageClass
metadata:
  name: cvs-extreme-extra-protection
provisioner: netapp.io/trident
parameters:
  selector: "performance=extreme; protection=extra"
allowVolumeExpansion: true
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: cvs-extreme-standard-protection
provisioner: netapp.io/trident
parameters:
  selector: "performance=premium; protection=standard"
allowVolumeExpansion: true
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: cvs-premium-extra-protection
provisioner: netapp.io/trident
parameters:
  selector: "performance=premium; protection=extra"
allowVolumeExpansion: true
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: cvs-premium
provisioner: netapp.io/trident
parameters:
  selector: "performance=premium; protection=standard"
allowVolumeExpansion: true
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: cvs-standard
provisioner: netapp.io/trident
parameters:
  selector: "performance=standard"
allowVolumeExpansion: true
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
```

```
metadata:
  name: cvs-extra-protection
provisioner: netapp.io/trident
parameters:
  selector: "protection=extra"
allowVolumeExpansion: true
```

接下來呢？

建立後端組態檔之後、請執行下列命令：

```
tridentctl create backend -f <backend-file>
```

如果後端建立失敗、表示後端組態有問題。您可以執行下列命令來檢視記錄、以判斷原因：

```
tridentctl logs
```

識別並修正組態檔的問題之後、您可以再次執行create命令。

設定NetApp HCI 一個不只是功能的SolidFire 後端

瞭解如何在安裝Astra Trident時建立及使用元素後端。

您需要的是 **#8217** ；需要的是什麼

- 支援的儲存系統、可執行Element軟體。
- 提供給NetApp HCI / SolidFire叢集管理員或租戶使用者的認證、以管理磁碟區。
- 您所有的Kubernetes工作節點都應該安裝適當的iSCSI工具。請參閱 "工作節點準備資訊"。

「Poolidfire - san」儲存驅動程式支援兩種Volume模式：檔案和區塊。對於「檔案系統」磁碟區模式、Astra Trident會建立一個磁碟區並建立檔案系統。檔案系統類型由StorageClass指定。

驅動程式	傳輸協定	**Volume**模式	支援的存取模式	支援的檔案系統
「olidfire - san」	iSCSI	區塊	rwo、ROX、rwx	無檔案系統。原始區塊裝置。
「olidfire - san」	iSCSI	區塊	rwo、ROX、rwx	無檔案系統。原始區塊裝置。
「olidfire - san」	iSCSI	檔案系統	Rwo、ROX	《xfs》、《ext3》、《ext4》
「olidfire - san」	iSCSI	檔案系統	Rwo、ROX	《xfs》、《ext3》、《ext4》

Astra Trident在做為增強型的csi資源配置程式時使用CHAP。如果您使用的是CHAP（這是「csi」的預設值）、則不需要進一步準備。建議將「UseCHAP」選項明確設定為搭配非「csi Trident」使用CHAP。否則請參閱 "請按這裡"。

Volume存取群組僅受Astra Trident的傳統非csi架構支援。當Astra Trident設定為以「csi」模式運作時、會使用CHAP。

如果未設定「存取群組」或「UseCHAP」、則適用下列其中一項規則：

- 如果偵測到預設的「Trident」存取群組、就會使用存取群組。
- 如果未偵測到存取群組、且Kubernetes版本為1.7或更新版本、則會使用CHAP。

後端組態選項

如需後端組態選項、請參閱下表：

參數	說明	預設
「分度」		永遠為1
「torageDriverName」	儲存驅動程式名稱	永遠是「solidfire-san」
「後端名稱」	自訂名稱或儲存後端	「S指_」+儲存設備（iSCSI）IP位址SolidFire
端點	MVIP、適用於SolidFire 採用租戶認證的不含用戶身分證明的叢集	
《VIP》	儲存設備（iSCSI）IP位址和連接埠	
《標籤》	套用到磁碟區的任意JSON-格式化標籤集。	「」
《天王名稱》	要使用的租戶名稱（如果找不到、請建立）	
《初始器IFACE》	將iSCSI流量限制在特定的主機介面	「預設」
《UseCHAP》	使用CHAP驗證iSCSI	是的
《存取群組》	要使用的存取群組ID清單	尋找名為「Trident」的存取群組ID
《類型》	QoS規格	
《限制Volume大小》	如果要求的磁碟區大小高於此值、則資源配置失敗	「」（預設不強制執行）
「DebugTraceFlags」	疑難排解時要使用的偵錯旗標。範例：｛"API"：假、「方法」：true｝	null

除非您正在進行疑難排解並需要詳細的記錄傾印、否則請勿使用「debugTraceFlags」。

對於所有建立的磁碟區、Astra Trident會在儲存資源池上的所有標籤配置時、複製到備用儲存LUN。儲存管理員可以定義每個儲存資源池的標籤、並將儲存資源池中建立的所有磁碟區分組。這是根據後端組態中提供的一組可自訂標籤、方便區分磁碟區的方法。

範例**1**：三種磁碟區類型的**Solidfire-san**驅動程式後端組態

此範例顯示使用CHAP驗證的後端檔案、並建立具有特定QoS保證的三種Volume類型模型。您很可能會使用「IOPS」儲存類別參數來定義儲存類別、以使用每個類別。

```
{
    "version": 1,
    "storageDriverName": "solidfire-san",
    "Endpoint": "https://<user>:<password>@<mvip>/json-rpc/8.0",
    "SVIP": "<svip>:3260",
    "TenantName": "<tenant>",
    "labels": {"k8scluster": "dev1", "backend": "dev1-element-cluster"},
    "UseCHAP": true,
    "Types": [{"Type": "Bronze", "Qos": {"minIOPS": 1000, "maxIOPS": 2000,
"burstIOPS": 4000}},
              {"Type": "Silver", "Qos": {"minIOPS": 4000, "maxIOPS": 6000,
"burstIOPS": 8000}},
              {"Type": "Gold", "Qos": {"minIOPS": 6000, "maxIOPS": 8000,
"burstIOPS": 10000}}]
}
```

範例**2**：採用虛擬儲存資源池的**Solidfire-san**驅動程式的後端和儲存類別組態

此範例顯示使用虛擬儲存資源池設定的後端定義檔案、以及參照回溯的StorageClass。

在下圖所示的範例後端定義檔中、會針對所有儲存資源池設定特定的預設值、將「type」設為Silver。虛擬儲存資源池是在「儲存區」區段中定義的。在此範例中、有些儲存資源池會設定自己的類型、有些資源池則會覆寫上述設定的預設值。

```
{
    "version": 1,
    "storageDriverName": "solidfire-san",
    "Endpoint": "https://<user>:<password>@<mvip>/json-rpc/8.0",
    "SVIP": "<svip>:3260",
    "TenantName": "<tenant>",
    "UseCHAP": true,
    "Types": [{"Type": "Bronze", "Qos": {"minIOPS": 1000, "maxIOPS": 2000,
"burstIOPS": 4000}},
              {"Type": "Silver", "Qos": {"minIOPS": 4000, "maxIOPS": 6000,
"burstIOPS": 8000}},
              {"Type": "Gold", "Qos": {"minIOPS": 6000, "maxIOPS": 8000,
"burstIOPS": 10000}}],

    "type": "Silver",
    "labels":{"store":"solidfire", "k8scluster": "dev-1-cluster"},
    "region": "us-east-1",

    "storage": [
        {
            "labels":{"performance":"gold", "cost":"4"},
            "zone":"us-east-1a",
            "type":"Gold"
        },
        {
            "labels":{"performance":"silver", "cost":"3"},
            "zone":"us-east-1b",
            "type":"Silver"
        },
        {
            "labels":{"performance":"bronze", "cost":"2"},
            "zone":"us-east-1c",
            "type":"Bronze"
        },
        {
            "labels":{"performance":"silver", "cost":"1"},
            "zone":"us-east-1d"
        }
    ]
}
```

下列StorageClass定義係指上述虛擬儲存資源池。使用「parameters.selector`」欄位、每個StorageClass都會呼叫哪些虛擬資源池可用於裝載磁碟區。磁碟區將會在所選的虛擬資源池中定義各個層面。

第一個StorageClass（「Poolidfire gold-s4」）會對應至第一個虛擬儲存集區。這是唯一提供黃金級效能的「Volume Type QoS」（Volume Type QoS）。最後一個StorageClass（「Poolidfire Silver」）會呼叫任何提

供銀級效能的儲存資源池。Astra Trident將決定選取哪個虛擬儲存資源池、並確保符合儲存需求。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: solidfire-gold-four
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=gold; cost=4"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: solidfire-silver-three
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=silver; cost=3"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: solidfire-bronze-two
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=bronze; cost=2"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: solidfire-silver-one
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=silver; cost=1"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: solidfire-silver
provisioner: csi.trident.netapp.io
parameters:
  selector: "performance=silver"
  fsType: "ext4"
```

如需詳細資訊、請參閱

- "Volume存取群組"

使用ONTAP 不支援的Cloud Volumes ONTAP SAN驅動程式來設定後端

深入瞭解如何使用ONTAP 支援功能的功能和功能性SAN驅動程式來設定功能性的後端。ONTAP Cloud Volumes ONTAP

- "準備"
- "組態與範例"

使用者權限

Astra Trident希望以ONTAP 支援或SVM管理員的身分執行、通常是使用「admin」叢集使用者或「vsadmin」SVM使用者、或使用具有相同角色的不同名稱使用者。對於Amazon FSx for NetApp ONTAP 等部署、Astra Trident預期會使用ONTAP 叢集「fsxadmin」使用者或「vsadmin」SVM使用者、或是具有相同角色之不同名稱的使用者、來執行成一個或多個SVM管理員。「fsxadmin」使用者是叢集管理使用者的有限替代。

> 如果您使用「limitAggregateusage」參數、則需要叢集管理權限。使用Amazon FSx for NetApp ONTAP 支援Astra Trident時、「limitAggregateusage」參數無法搭配「vsadmin」和「fsxadmin」使用者帳戶使用。如果您指定此參數、組態作業將會失敗。

雖然可以在ONTAP 功能區內建立更嚴格的角色、讓Trident驅動程式能夠使用、但我們不建議您這麼做。Trident的大多數新版本都會呼叫額外的API、而這些API必須納入考量、使升級變得困難且容易出錯。

準備

瞭解如何準備使用ONTAP 支援不支援的SAN驅動程式來設定支援功能的後端。ONTAP對於所有ONTAP 的不支援端點、Astra Trident至少需要指派一個集合體給SVM。

請記住、您也可以執行多個驅動程式、並建立指向一個或多個驅動程式的儲存類別。例如、您可以設定使用「ONTAP-SAN」驅動程式的「SAN開發」類別、以及使用「ONTAP-SAN經濟」類別的「SAN預設」類別。

您所有的Kubernetes工作節點都必須安裝適當的iSCSI工具。請參閱 "請按這裡" 以取得更多詳細資料。

驗證

Astra Trident提供兩種驗ONTAP 證功能來驗證支援的後端。

- 認證型：ONTAP 對具備所需權限的使用者名稱和密碼。建議使用預先定義的安全登入角色、例如「admin」或「vsadmin」、以確保與ONTAP 各種版本的最大相容性。
- 憑證型：Astra Trident也能ONTAP 使用安裝在後端的憑證與某個叢集進行通訊。在此處、後端定義必須包含用戶端憑證、金鑰及信任的CA憑證（建議使用）的Base64編碼值。

使用者也可以選擇更新現有的後端、選擇從認證移至憑證型、反之亦然。如果*同時提供認證資料和憑證*、Astra Trident將預設使用憑證、同時發出警告、從後端定義中移除認證資料。

啟用認證型驗證

Astra Trident需要SVM範圍/叢集範圍管理員的認證資料、才能與ONTAP 該後端進行通訊。建議使用預先定義的

標準角色、例如「admin」或「vsadmin」。這可確保與未來ONTAP 的支援版本保持前瞻相容、因為未來的Astra Trident版本可能會使用功能API。您可以建立自訂的安全登入角色、並與Astra Trident搭配使用、但不建議使用。

後端定義範例如下所示：

```
{
  "version": 1,
  "backendName": "ExampleBackend",
  "storageDriverName": "ontap-san",
  "managementLIF": "10.0.0.1",
  "dataLIF": "10.0.0.2",
  "svm": "svm_nfs",
  "username": "vsadmin",
  "password": "secret",
}
```

請記住、後端定義是唯一以純文字儲存認證的位置。建立後端之後、使用者名稱/密碼會以Base64編碼、並儲存為Kubernetes機密。建立/更新後端是唯一需要知道認證資料的步驟。因此、這是一項純管理員操作、由Kubernetes /儲存管理員執行。

啟用憑證型驗證

新的和現有的後端可以使用憑證、並與ONTAP 該後端通訊。後端定義需要三個參數。

- 用戶端憑證：用戶端憑證的Base64編碼值。
- 用戶端私密金鑰：關聯私密金鑰的Base64編碼值。
- 信任的CACertifate：受信任CA憑證的Base64編碼值。如果使用信任的CA、則必須提供此參數。如果未使用信任的CA、則可忽略此問題。

典型的工作流程包括下列步驟。

步驟

1. 產生用戶端憑證和金鑰。產生時、請將Common Name（CN）（一般名稱（CN））設定為ONTAP 驗證身分。

   ```
   openssl req -x509 -nodes -days 1095 -newkey rsa:2048 -keyout k8senv.key
   -out k8senv.pem -subj "/C=US/ST=NC/L=RTP/O=NetApp/CN=admin"
   ```

2. 將信任的CA憑證新增ONTAP 至整個叢集。這可能已由儲存管理員處理。如果未使用信任的CA、請忽略。

```
security certificate install -type server -cert-name <trusted-ca-cert-
name> -vserver <vserver-name>
ssl modify -vserver <vserver-name> -server-enabled true -client-enabled
true -common-name <common-name> -serial <SN-from-trusted-CA-cert> -ca
<cert-authority>
```

3. 在ONTAP 支援叢集上安裝用戶端憑證和金鑰（步驟1）。

```
security certificate install -type client-ca -cert-name <certificate-
name> -vserver <vserver-name>
security ssl modify -vserver <vserver-name> -client-enabled true
```

4. 確認ONTAP 支援「cert」驗證方法的支援功能。

```
security login create -user-or-group-name admin -application ontapi
-authentication-method cert
security login create -user-or-group-name admin -application http
-authentication-method cert
```

5. 使用產生的憑證測試驗證。以ONTAP Management LIF IP和SVM名稱取代<SfManagement LIF>和<vserver name>。

```
curl -X POST -Lk https://<ONTAP-Management-
LIF>/servlets/netapp.servlets.admin.XMLrequest_filer --key k8senv.key
--cert ~/k8senv.pem -d '<?xml version="1.0" encoding="UTF-8"?><netapp
xmlns="http://www.netapp.com/filer/admin" version="1.21"
vfiler="<vserver-name>"><vserver-get></vserver-get></netapp>'
```

6. 使用Base64編碼憑證、金鑰和信任的CA憑證。

```
base64 -w 0 k8senv.pem >> cert_base64
base64 -w 0 k8senv.key >> key_base64
base64 -w 0 trustedca.pem >> trustedca_base64
```

7. 使用從上一步取得的值建立後端。

```
$ cat cert-backend.json
{
"version": 1,
"storageDriverName": "ontap-san",
"backendName": "SanBackend",
"managementLIF": "1.2.3.4",
"dataLIF": "1.2.3.8",
"svm": "vserver_test",
"clientCertificate": "Faaaakkkkeeee...Vaaallllluuuueeee",
"clientPrivateKey": "LS0tFaKE...0VaLuES0tLS0K",
"trustedCACertificate": "QNFinfO...SiqOyN",
"storagePrefix": "myPrefix_"
}

$ tridentctl create backend -f cert-backend.json -n trident
+------------+----------------+--------------------------------------
+--------+---------+
|    NAME    | STORAGE DRIVER |                  UUID                  |
STATE  | VOLUMES |
+------------+----------------+--------------------------------------
+--------+---------+
| SanBackend | ontap-san      | 586b1cd5-8cf8-428d-a76c-2872713612c1 |
online |       0 |
+------------+----------------+--------------------------------------
+--------+---------+
```

更新驗證方法或旋轉認證資料

您可以更新現有的後端、以使用不同的驗證方法、或是旋轉其認證資料。這兩種方法都可行：使用使用者名稱/密碼的後端可更新以使用憑證；使用憑證的後端可更新為使用者名稱/密碼。若要這麼做、請使用更新的「backend.json」檔案、其中包含執行「tridentctl後端更新」所需的參數。

```
$ cat cert-backend-updated.json
{
"version": 1,
"storageDriverName": "ontap-san",
"backendName": "SanBackend",
"managementLIF": "1.2.3.4",
"dataLIF": "1.2.3.8",
"svm": "vserver_test",
"username": "vsadmin",
"password": "secret",
"storagePrefix": "myPrefix_"
}

#Update backend with tridentctl
$ tridentctl update backend SanBackend -f cert-backend-updated.json -n
trident
+------------+----------------+--------------------------------------
+--------+---------+
|    NAME    | STORAGE DRIVER |                  UUID                |
STATE   | VOLUMES |
+------------+----------------+--------------------------------------
+--------+---------+
| SanBackend | ontap-san      | 586b1cd5-8cf8-428d-a76c-2872713612c1 |
online |       9 |
+------------+----------------+--------------------------------------
+--------+---------+
```

> 當您旋轉密碼時、儲存管理員必須先更新ONTAP 使用者的密碼（位於BIOS）。接著是後端更新。在循環憑證時、可將多個憑證新增至使用者。然後更新後端以使用新的憑證、之後可從ONTAP 該叢集刪除舊的憑證。

更新後端不會中斷對已建立之磁碟區的存取、也不會影響之後建立的磁碟區連線。成功的後端更新顯示Astra Trident可以與ONTAP 該後端通訊、並處理未來的Volume作業。

指定**igroup**

Astra Trident使用igroup來控制其所配置的磁碟區（LUN）存取。系統管理員在指定後端的igroup時有兩種選擇：

- Astra Trident可自動建立及管理每個後端的igroup。如果後端定義中未包含「igroupName」、Astra Trident會在SVM上建立名為「trident -」的igroup。如此可確保每個後端都有專屬的igroup、並處理Kubernetes節點IQN的自動新增/刪除作業。
- 或者、也可以在後端定義中提供預先建立的igroup。這可以使用「igroupName」組態參數來完成。Astra Trident會將Kubernetes節點IQN新增/刪除至預先存在的igroup。

對於定義了「igroupName」的後端、可以使用「tridentctl後端更新」刪除「igroupName」、以便使用Astra Trident自動處理igroup。這不會中斷對已附加至工作負載之磁碟區的存取。未來的連線將使用建立的igroup

Astra Trident來處理。

> 針對每個獨特的Astra Trident執行個體指定igroup是最適合Kubernetes管理員和儲存管理員的最佳實務做法。「csi Trident」可自動新增及移除igroup的叢集節點IQN、大幅簡化其管理。在Kubernetes環境中使用相同的SVM（以及Astra Trident安裝）時、使用專屬的igroup可確保對Kubernetes叢集所做的變更不會影響與其他叢集相關的igroup。此外、也必須確保Kubernetes叢集中的每個節點都有唯一的IQN。如上所述、Astra Trident會自動處理IQN的新增與移除。重複使用主機間的IQN可能會導致主機彼此誤用、並拒絕存取LUN的不良情況。

如果將Astra Trident設定為使用「csi資源配置程式」、則Kubernetes節點IQN會自動新增至igroup或從其中移除。當節點新增至Kubernetes叢集時、「Trident - csi」示範Set會在新增的節點上部署pod（「trident - csi-xxxxX」）、並登錄可附加磁碟區的新節點。節點IQN也會新增至後端的igroup。當節點封鎖、排放及從Kubernetes刪除時、類似的一組步驟可處理刪除IQN。

如果Astra Trident並未以csi資源配置程式的形式執行、則必須手動更新igroup、以包含Kubernetes叢集中每個工作節點的iSCSI IQN。加入Kubernetes叢集的節點IQN必須新增至igroup。同樣地、從Kubernetes叢集移除的節點IQN也必須從igroup移除。

使用雙向**CHAP**驗證連線

Astra Trident可以使用雙向CHAP驗證iSCSI工作階段的「ONTAP-SAN」和「ONTAP-san經濟」驅動程式。這需要在後端定義中啟用「useCHAP」選項。當設為「true」時、Astra Trident會將SVM的預設啟動器安全性設定為雙向CHAP、並從後端檔案設定使用者名稱和機密。NetApp建議使用雙向CHAP來驗證連線。請參閱下列組態範例：

```
{
    "version": 1,
    "storageDriverName": "ontap-san",
    "backendName": "ontap_san_chap",
    "managementLIF": "192.168.0.135",
    "svm": "ontap_iscsi_svm",
    "useCHAP": true,
    "username": "vsadmin",
    "password": "FaKePaSsWoRd",
    "igroupName": "trident",
    "chapInitiatorSecret": "cl9qxIm36DKyawxy",
    "chapTargetInitiatorSecret": "rqxigXgkesIpwxyz",
    "chapTargetUsername": "iJF4heBRT0TCwxyz",
    "chapUsername": "uh2aNCLSd6cNwxyz",
}
```

> 「useCHAP」參數是布林選項、只能設定一次。預設值設為假。將其設為true之後、您就無法將其設為假。

除了"useCHAP=true"之外、"chapInitiator Secret（chapInitiator機密）、"chaptarketatorSecret（chaptarketusernamе"）、"chaptarketusernamе"（chaptargetuseamuse"）和"chapusernamе"（chamus在建立後端後端之後、可以執行「tridentctl update」來變更機密。

運作方式

儲存管理員將「useCHAP」設定為true、指示Astra Trident在儲存後端上設定CHAP。這包括下列項目：

- 在SVM上設定CHAP：
  - 如果SVM的預設啟動器安全性類型為「無」（預設設定）和、則磁碟區中已沒有預先存在的LUN、Astra Trident會將預設安全性類型設為「CHAP」、並繼續設定CHAP啟動器和目標使用者名稱和機密。
  - 如果SVM包含LUN、Astra Trident將不會在SVM上啟用CHAP。如此可確保不限制存取SVM上已存在的LUN。
- 設定CHAP啟動器和目標使用者名稱和機密；這些選項必須在後端組態中指定（如上所示）。
- 管理在後端的「igroupName」中新增的初始化。如果未指定、則預設為「Trident」。

建立後端之後、Astra Trident會建立對應的「tridentbackend」CRD、並將CHAP機密和使用者名稱儲存為Kubernetes機密。由Astra Trident在此後端上建立的所有PV、都會掛載並附加於CHAP上。

旋轉認證資料並更新後端

您可以更新「backend.json」檔案中的CHAP參數、以更新CHAP認證。這需要更新CHAP機密、並使用「tridentctl update」命令來反映這些變更。

> 更新後端的CHAP機密時、您必須使用「tridentctl」來更新後端。請勿透過CLI/ONTAP UI更新儲存叢集上的認證資料、因為Astra Trident無法接受這些變更。

```
$ cat backend-san.json
{
    "version": 1,
    "storageDriverName": "ontap-san",
    "backendName": "ontap_san_chap",
    "managementLIF": "192.168.0.135",
    "svm": "ontap_iscsi_svm",
    "useCHAP": true,
    "username": "vsadmin",
    "password": "FaKePaSsWoRd",
    "igroupName": "trident",
    "chapInitiatorSecret": "cl9qxUpDaTeD",
    "chapTargetInitiatorSecret": "rqxigXgkeUpDaTeD",
    "chapTargetUsername": "iJF4heBRT0TCwxyz",
    "chapUsername": "uh2aNCLSd6cNwxyz",
}

$ ./tridentctl update backend ontap_san_chap -f backend-san.json -n
trident
+----------------+----------------+--------------------------------------
+--------+---------+
|   NAME         | STORAGE DRIVER |                 UUID                 |
STATE   | VOLUMES |
+----------------+----------------+--------------------------------------
+--------+---------+
| ontap_san_chap | ontap-san      | aa458f3b-ad2d-4378-8a33-1a472ffbeb5c |
online |       7 |
+----------------+----------------+--------------------------------------
+--------+---------+
```

現有的連線不會受到影響；如果SVM上的Astra Trident更新認證、它們將繼續保持作用中狀態。新連線將使用更新的認證資料、而現有連線仍保持作用中狀態。中斷舊PV的連線並重新連線、將會使用更新的認證資料。

組態選項與範例

瞭解如何透過ONTAP Astra Trident安裝來建立及使用支援NetApp的SAN驅動程式。本節提供後端組態範例、以及如何將後端對應至StorageClass的詳細資料。

後端組態選項

如需後端組態選項、請參閱下表：

參數	說明	預設
「分度」		永遠為1

參數	說明	預設
「torageDriverName」	儲存驅動程式名稱	「ONTAP-NAS」、「ONTAP-NAS-節約 型」、「ONTAP-NAS-flexgroup」、「ONTAP-SAN」、「ONTAP-san經濟型」
「後端名稱」	自訂名稱或儲存後端	驅動程式名稱+「_」+ dataLIF
《馬納格門達利》	叢集或SVM管理LIF的IP位址	「10.0.0.1」、「[2001:1234:abcd:::fefo]」
「DataLIF」	傳輸協定LIF的IP位址。IPv6使用方括弧。設定後無法更新	除非另有說明、否則由SVM衍生
《使用CHAP》	使用CHAP驗證iSCSI以供ONTAP支援不支援的SAN驅動程式使用[布林值]	錯
《chapInitiator機密》	CHAP啟動器密碼。如果是"useCHAP=true"、則為必要項目	「」
《標籤》	套用到磁碟區的任意JSON-格式化標籤集	「」
《chapTargetInitiator機密》	CHAP目標啟動器機密。如果是"useCHAP=true"、則為必要項目	「」
「chapUsername」	傳入使用者名稱。如果是"useCHAP=true"、則為必要項目	「」
《chapTargetUsername》	目標使用者名稱。如果是"useCHAP=true"、則為必要項目	「」
「用戶端憑證」	用戶端憑證的Base64編碼值。用於憑證型驗證	「」
「clientPrivate Key」	用戶端私密金鑰的Base64編碼值。用於憑證型驗證	「」
「可信賴的CACertifcate」	受信任CA憑證的Base64編碼值。選用。用於憑證型驗證	「」
《使用者名稱》	連線至叢集/ SVM的使用者名稱。用於認證型驗證	「」
密碼	連線至叢集/ SVM的密碼。用於認證型驗證	「」
《虛擬機器》	要使用的儲存虛擬機器	如果指定SVM "managementLIF"則衍生
「igroupName」	要使用之SAN磁碟區的igroup名稱	「Trident -<後端-UUID>」
「toragePrefix」	在SVM中配置新磁碟區時所使用的前置碼。設定後無法更新	「Trident」
「限制Aggregateusage」	如果使用率高於此百分比、則無法進行資源配置。*不適用於Amazon FSX for ONTAP Sfor Sfor *	「」（預設不強制執行）

參數	說明	預設
《限制Volume大小》	如果要求的磁碟區大小高於經濟驅動程式的此值、則無法進行資源配置。	「」（預設不強制執行）
《lunsPerFlexvol》	每FlexVol 個LUN的最大LUN數量、範圍必須在[50、200]	「100」
「DebugTraceFlags」	疑難排解時要使用的偵錯旗標。範例： {"API"：假、「方法」：true }	null
《useREST》	使用ONTAP lsrest API的布林參數。技術預覽	錯

> 「useREST」是一種建議用於測試環境、而非用於正式作業工作負載的技術預覽。當設為「true」時、Astra Trident會使用ONTAP 「lsrest API」與後端通訊。此功能需要ONTAP 使用更新版本的版本。此外ONTAP 、所使用的登入角色必須能夠存取「ONTAP 發揮作用」應用程式。這對預先定義的「vsadmin」和「叢集管理」角色感到滿意。

若要與ONTAP 此叢集通訊、您應該提供驗證參數。這可能是安全登入或安裝憑證的使用者名稱/密碼。

> 如果您使用Amazon FSX for NetApp ONTAP Sendbackend、請勿指定「limitAggregateusage」參數。Amazon FSx for NetApp ONTAP 的「fsxadmin」和「vsadmin」角色不包含擷取Aggregate使用量並透過Astra Trident加以限制的必要存取權限。

> 除非您正在進行疑難排解並需要詳細的記錄傾印、否則請勿使用「debugTraceFlags」。

對於「ONTAP-san」驅動程式、預設為使用SVM的所有資料LIF IP、並使用iSCSI多重路徑。為「ONTAP-san」驅動程式的dataLIF指定IP位址、會強制它們停用多重路徑、並只使用指定的位址。

> 建立後端時、請記住、建立後無法修改「dataLIF」和「儲存前置字元」。若要更新這些參數、您需要建立新的後端。

「igroupName」可設定為ONTAP 已在該叢集上建立的igroup。如果未指定、Astra Trident會自動建立名為Trident的igroup -<後端UUID >。如果提供預先定義的igroupName、則如果要在環境之間共用SVM、NetApp建議使用每個Kubernetes叢集的igroup。這是Astra Trident自動維護IQN新增/刪除的必要條件。

後端也可以在建立後更新igroup：

- 可以更新igroupName、以指向在Astra Trident以外的SVM上建立及管理的新igroup。
- 可以省略igroupName。在此案例中、Astra Trident會自動建立及管理Trident -<backend-UUUUID> igroup。

在這兩種情況下、仍可繼續存取Volume附件。未來的Volume附件將使用更新的igroup。此更新不會中斷對後端磁碟區的存取。

可以為「managementLIF」選項指定完整網域名稱（FQDN）。

所有ONTAP 的不完整驅動程式也可設定為IPv6位址。請務必使用「-use IPv6」旗標來安裝Trident。必須謹慎定義方括弧內的「managementLIF」IPv6位址。

> 使用IPv6位址時、請務必在方括弧內定義「managementLIF」和「dataLIF」（若包含在後端定義中）、例如[28e8：d9fb:a825:b7bf:69a8：d02f：9e7b:3555]。如果未提供「data LIF」、Astra Trident會從SVM擷取IPv6資料LIF。

若要讓ONTAP-SAN驅動程式使用CHAP、請在後端定義中將「useCHAP」參數設為「true」。然後Astra Trident會設定並使用雙向CHAP做為後端所指定SVM的預設驗證。請參閱 "請按這裡" 以瞭解其運作方式。

對於「ONTAP-san經濟」驅動程式、「limitVolume Sizes」選項也會限制其管理的qtree和LUN磁碟區大小上限。

> Astra Trident在所有使用「ONTAP-san」驅動程式建立的磁碟區的「Comments」（註解）欄位中設定資源配置標籤。針對所建立的每個Volume、FlexVol 將會在顯示於其儲存資源池中的「Comments」（註解）欄位中填入所有標籤。儲存管理員可以定義每個儲存資源池的標籤、並將儲存資源池中建立的所有磁碟區分組。這是根據後端組態中提供的一組可自訂標籤、方便區分磁碟區的方法。

用於資源配置磁碟區的後端組態選項

您可以在組態的特定區段中、使用這些選項來控制預設配置每個Volume的方式。如需範例、請參閱下列組態範例。

參數	說明	預設
"paceAllocate（配置）"	LUN的空間分配	「真的」
《保護區》	空間保留模式；「無」（精簡）或「Volume」（完整）	「無」
「快照原則」	要使用的Snapshot原則	「無」
「qosPolicy」	要指派給所建立磁碟區的QoS原則群組。選擇每個儲存集區/後端的其中一個qosPolicy或adaptiveQosPolicy	「」
《adaptiveQosPolicy》	要指派給所建立磁碟區的調適性QoS原則群組。選擇每個儲存集區/後端的其中一個qosPolicy或adaptiveQosPolicy	「」
「快照保留區」	保留給快照「0」的磁碟區百分比	如果「快照原則」為「無」、則其他為「」
「PlitOnClone」	建立複本時、從其父複本分割複本	「假」
「PlitOnClone」	建立複本時、從其父複本分割複本	「假」
加密	啟用NetApp Volume加密	「假」
《生態樣式》	新磁碟區的安全樣式	「UNIX」
「分層政策」	分層原則以使用「無」	ONTAP 9.5之前的SVM-DR組態為「純快照」

搭配Astra Trident使用QoS原則群組需要ONTAP 使用更新版本的版本。建議使用非共用的QoS原則群組、並確保原則群組會個別套用至每個組成群組。共享的QoS原則群組將強制所有工作負載的總處理量上限。

以下是已定義預設值的範例：

```
{
 "version": 1,
 "storageDriverName": "ontap-san",
 "managementLIF": "10.0.0.1",
 "dataLIF": "10.0.0.2",
 "svm": "trident_svm",
 "username": "admin",
 "password": "password",
 "labels": {"k8scluster": "dev2", "backend": "dev2-sanbackend"},
 "storagePrefix": "alternate-trident",
 "igroupName": "custom",
 "debugTraceFlags": {"api":false, "method":true},
 "defaults": {
     "spaceReserve": "volume",
     "qosPolicy": "standard",
     "spaceAllocation": "false",
     "snapshotPolicy": "default",
     "snapshotReserve": "10"
 }
}
```

對於使用「ONTAP-san」驅動程式建立的所有磁碟區、Astra Trident會在FlexVol the支援LUN中繼資料的情況下、為該系統的功能增加10%的容量。LUN的配置大小與使用者在PVc中要求的大小完全相同。Astra Trident在FlexVol 整個過程中增加10%的速度（顯示ONTAP 在畫面上可用的尺寸）。使用者現在可以取得所要求的可用容量。此變更也可防止LUN成為唯讀、除非可用空間已充分利用。這不適用於ONTAP-san經濟型。

對於定義「快照保留區」的後端、Astra Trident會計算磁碟區大小、如下所示：

```
Total volume size = [(PVC requested size) / (1 - (snapshotReserve
percentage) / 100)] * 1.1
```

1.1是額外10%的Astra Trident加入FlexVol 到the支援LUN中繼資料的功能。對於「快照保留區」= 5%、而PVc要求= 5GiB、磁碟區總大小為5.79GiB、可用大小為5.5GiB。「volume show（Volume show）命令應顯示類似以下範例的結果：

```
Vserver   Volume       Aggregate    State      Type       Size  Available Used%
--------- ------------ ------------ ---------- ---- ---------- ---------- -----
          _pvc_89f1c156_3801_4de4_9f9d_034d54c395f4
                                   online     RW         10GB     5.00GB    0%
          _pvc_e42ec6fe_3baa_4af6_996d_134adbbb8e6d
                                   online     RW       5.79GB     5.50GB    0%
          _pvc_e8372153_9ad9_474a_951a_08ae15e1c0ba
                                   online     RW          1GB    511.8MB    0%
3 entries were displayed.
```

目前、只有調整大小、才能將新計算用於現有的Volume。

最低組態範例

下列範例顯示基本組態、讓大部分參數保留預設值。這是定義後端最簡單的方法。

> 如果您在NetApp ONTAP 支援Astra Trident的NetApp上使用Amazon FSX、建議您指定lifs的DNS名稱、而非IP位址。

ONTAP - SAN驅動程式搭配憑證型驗證

這是最小的後端組態範例。「clientCertificate」、「clientPrivate Key」和「Trusted CACertifate」（選用、如果使用信任的CA）會分別填入「backend.json」、並採用用戶端憑證、私密金鑰和信任的CA憑證的64編碼值。

```
{
    "version": 1,
    "storageDriverName": "ontap-san",
    "backendName": "DefaultSANBackend",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.3",
    "svm": "svm_iscsi",
    "useCHAP": true,
    "chapInitiatorSecret": "cl9qxIm36DKyawxy",
    "chapTargetInitiatorSecret": "rqxigXgkesIpwxyz",
    "chapTargetUsername": "iJF4heBRT0TCwxyz",
    "chapUsername": "uh2aNCLSd6cNwxyz",
    "igroupName": "trident",
    "clientCertificate": "ZXR0ZXJwYXB...ICMgJ3BhcGVyc2",
    "clientPrivateKey": "vciwKIyAgZG...0cnksIGRlc2NyaX",
    "trustedCACertificate": "zcyBbaG...b3Igb3duIGNsYXNz"
}
```

ONTAP與雙向CHAP的SAN驅動程式

這是最小的後端組態範例。此基本組態會建立一個「ONTAP-SAN」後端、並將「useCHAP」設為「true」。

```
{
    "version": 1,
    "storageDriverName": "ontap-san",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.3",
    "svm": "svm_iscsi",
    "labels": {"k8scluster": "test-cluster-1", "backend": "testcluster1-
sanbackend"},
    "useCHAP": true,
    "chapInitiatorSecret": "cl9qxIm36DKyawxy",
    "chapTargetInitiatorSecret": "rqxigXgkesIpwxyz",
    "chapTargetUsername": "iJF4heBRT0TCwxyz",
    "chapUsername": "uh2aNCLSd6cNwxyz",
    "igroupName": "trident",
    "username": "vsadmin",
    "password": "secret"
}
```

ONTAP-san經濟驅動程式

```
{
    "version": 1,
    "storageDriverName": "ontap-san-economy",
    "managementLIF": "10.0.0.1",
    "svm": "svm_iscsi_eco",
    "useCHAP": true,
    "chapInitiatorSecret": "cl9qxIm36DKyawxy",
    "chapTargetInitiatorSecret": "rqxigXgkesIpwxyz",
    "chapTargetUsername": "iJF4heBRT0TCwxyz",
    "chapUsername": "uh2aNCLSd6cNwxyz",
    "igroupName": "trident",
    "username": "vsadmin",
    "password": "secret"
}
```

虛擬儲存資源池的後端範例

在下圖所示的範例後端定義檔中、會針對所有儲存資源池設定特定的預設值、例如「無的儲存資源保留」、「假的資源配置」、以及「假的加密」。虛擬儲存資源池是在儲存區段中定義。

在此範例中、有些儲存資源池會設定自己的「資源保留」、「資源配置」和「加密」值、有些資源池會覆寫上述設定的預設值。

```
{
```

```
    "version": 1,
    "storageDriverName": "ontap-san",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.3",
    "svm": "svm_iscsi",
    "useCHAP": true,
    "chapInitiatorSecret": "cl9qxIm36DKyawxy",
    "chapTargetInitiatorSecret": "rqxigXgkesIpwxyz",
    "chapTargetUsername": "iJF4heBRT0TCwxyz",
    "chapUsername": "uh2aNCLSd6cNwxyz",
    "igroupName": "trident",
    "username": "vsadmin",
    "password": "secret",

    "defaults": {
          "spaceAllocation": "false",
          "encryption": "false",
          "qosPolicy": "standard"
    },
    "labels":{"store": "san_store", "kubernetes-cluster": "prod-cluster-
1"},
    "region": "us_east_1",
    "storage": [
        {
            "labels":{"protection":"gold", "creditpoints":"40000"},
            "zone":"us_east_1a",
            "defaults": {
                "spaceAllocation": "true",
                "encryption": "true",
                "adaptiveQosPolicy": "adaptive-extreme"
            }
        },
        {
            "labels":{"protection":"silver", "creditpoints":"20000"},
            "zone":"us_east_1b",
            "defaults": {
                "spaceAllocation": "false",
                "encryption": "true",
                "qosPolicy": "premium"
            }
        },
        {
            "labels":{"protection":"bronze", "creditpoints":"5000"},
            "zone":"us_east_1c",
            "defaults": {
                "spaceAllocation": "true",
```

```
                "encryption": "false"
            }
        }
    ]
}
```

以下是「ONTAP-san經濟」驅動程式的iSCSI範例：

```
{
    "version": 1,
    "storageDriverName": "ontap-san-economy",
    "managementLIF": "10.0.0.1",
    "svm": "svm_iscsi_eco",
    "useCHAP": true,
    "chapInitiatorSecret": "cl9qxIm36DKyawxy",
    "chapTargetInitiatorSecret": "rqxigXgkesIpwxyz",
    "chapTargetUsername": "iJF4heBRT0TCwxyz",
    "chapUsername": "uh2aNCLSd6cNwxyz",
    "igroupName": "trident",
    "username": "vsadmin",
    "password": "secret",

    "defaults": {
          "spaceAllocation": "false",
          "encryption": "false"
    },
    "labels":{"store":"san_economy_store"},
    "region": "us_east_1",
    "storage": [
        {
            "labels":{"app":"oracledb", "cost":"30"},
            "zone":"us_east_1a",
            "defaults": {
                "spaceAllocation": "true",
                "encryption": "true"
            }
        },
        {
            "labels":{"app":"postgresdb", "cost":"20"},
            "zone":"us_east_1b",
            "defaults": {
                "spaceAllocation": "false",
                "encryption": "true"
            }
        },
```

```
        {
            "labels":{"app":"mysqldb", "cost":"10"},
            "zone":"us_east_1c",
            "defaults": {
                "spaceAllocation": "true",
                "encryption": "false"
            }
        }
    ]
}
```

將後端對應至**StorageClass**

下列StorageClass定義係指上述虛擬儲存資源池。使用「parameters.selector`」欄位、每個StorageClass都會呼叫哪些虛擬資源池可用於裝載磁碟區。磁碟區將會在所選的虛擬資源池中定義各個層面。

- 第一部StorageClass（「protection黃金級」）將對應至「ONTAP-NAS Flexgroup」後端的第一個、第二個虛擬儲存集區、以及「ONTAP-SAN」後端的第一個虛擬儲存集區。這是唯一提供金級保護的資源池。
- 第二個StorageClass（「protection非gold」）將對應至第三個、第四個虛擬儲存集區、位於「ONTAP-NAS」後端、第二個是第三個虛擬儲存集區、位於「ONTAP-SAN」後端。這是唯一提供金級以外保護層級的資源池。
- 第三個StorageClass（「app-mysqldb」）將對應至「ontap-nas」後端的第四個虛擬儲存資源池、以及「ontap-san經濟」後端的第三個虛擬儲存資源池。這些是唯一提供mysqldb類型應用程式儲存池組態的集區。
- 第四個StorageClass（「protection－silver－creditpoints－20k」）將對應至第三個虛擬儲存資源池（位於「ONTAP－NAS－flexgroup」後端）、第二個虛擬儲存資源池（位於「ONTAP－san」後端）。這些資源池是唯一能以20000個信用點數提供金級保護的資源池。
- 第五個StorageClass（'creditpoint-5k'）將對應到第二個位於「ONTAP-NAS經濟」後端的虛擬儲存資源池、以及位於「ONTAP-SAN」後端的第三個虛擬儲存資源池。這些是唯一提供5000個信用點數的資源池產品。

Astra Trident將決定選取哪個虛擬儲存資源池、並確保符合儲存需求。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: protection-gold
provisioner: netapp.io/trident
parameters:
  selector: "protection=gold"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: protection-not-gold
provisioner: netapp.io/trident
parameters:
  selector: "protection!=gold"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: app-mysqldb
provisioner: netapp.io/trident
parameters:
  selector: "app=mysqldb"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: protection-silver-creditpoints-20k
provisioner: netapp.io/trident
parameters:
  selector: "protection=silver; creditpoints=20000"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: creditpoints-5k
provisioner: netapp.io/trident
parameters:
  selector: "creditpoints=5000"
  fsType: "ext4"
```

使用ONTAP NetApp NAS驅動程式設定後端

深入瞭解如何使用ONTAP 功能性和功能性NAS驅動程式來設定功能性的後端。ONTAP Cloud Volumes ONTAP

- "準備"
- "組態與範例"

使用者權限

Astra Trident希望以ONTAP 支援或SVM管理員的身分執行、通常是使用「admin」叢集使用者或「vsadmin」SVM使用者、或使用具有相同角色的不同名稱使用者。對於Amazon FSx for NetApp ONTAP 等部署、Astra Trident預期會使用ONTAP 叢集「fsxadmin」使用者或「vsadmin」SVM使用者、或是具有相同角色之不同名稱的使用者、來執行成一個或多個SVM管理員。「fsxadmin」使用者是叢集管理使用者的有限替代。

> 如果您使用「limitAggregateusage」參數、則需要叢集管理權限。使用Amazon FSx for NetApp ONTAP 支援Astra Trident時、「limitAggregateusage」參數無法搭配「vsadmin」和「fsxadmin」使用者帳戶使用。如果您指定此參數、組態作業將會失敗。

雖然可以在ONTAP 功能區內建立更嚴格的角色、讓Trident驅動程式能夠使用、但我們不建議您這麼做。Trident的大多數新版本都會呼叫額外的API、而這些API必須納入考量、使升級變得困難且容易出錯。

準備

瞭解如何準備使用ONTAP 不含NetApp功能的NAS驅動程式來設定功能完善的後端。ONTAP對於所有ONTAP 的不支援端點、Astra Trident至少需要指派一個集合體給SVM。

對於所有ONTAP 的不支援端點、Astra Trident至少需要指派一個集合體給SVM。

請記住、您也可以執行多個驅動程式、並建立指向一個或多個驅動程式的儲存類別。例如、您可以設定金級、使用「ONTAP-NAS」驅動程式、以及使用「ONTAP-NAS-經濟」的銅級。

您所有的Kubernetes工作節點都必須安裝適當的NFS工具。請參閱 "請按這裡" 以取得更多詳細資料。

驗證

Astra Trident提供兩種驗ONTAP 證功能來驗證支援的後端。

- 認證型：ONTAP 對具備所需權限的使用者名稱和密碼。建議使用預先定義的安全登入角色、例如「admin」或「vsadmin」、以確保與ONTAP 各種版本的最大相容性。
- 憑證型：Astra Trident也能ONTAP 使用安裝在後端的憑證與某個叢集進行通訊。在此處、後端定義必須包含用戶端憑證、金鑰及信任的CA憑證（建議使用）的Base64編碼值。

使用者也可以選擇更新現有的後端、選擇從認證移至憑證型、反之亦然。如果*同時提供認證資料和憑證*、Astra Trident將預設使用憑證、同時發出警告、從後端定義中移除認證資料。

啟用認證型驗證

Astra Trident需要SVM範圍/叢集範圍管理員的認證資料、才能與ONTAP 該後端進行通訊。建議使用預先定義的標準角色、例如「admin」或「vsadmin」。這可確保與未來ONTAP 的支援版本保持前瞻相容、因為未來的Astra Trident版本可能會使用功能API。您可以建立自訂的安全登入角色、並與Astra Trident搭配使用、但不建議使用。

後端定義範例如下所示：

```
{
  "version": 1,
  "backendName": "ExampleBackend",
  "storageDriverName": "ontap-nas",
  "managementLIF": "10.0.0.1",
  "dataLIF": "10.0.0.2",
  "svm": "svm_nfs",
  "username": "vsadmin",
  "password": "secret"
}
```

請記住、後端定義是唯一以純文字儲存認證的位置。建立後端之後、使用者名稱/密碼會以Base64編碼、並儲存為Kubernetes機密。建立/更新後端是唯一需要知道認證資料的步驟。因此、這是一項純管理員操作、由Kubernetes /儲存管理員執行。

啟用憑證型驗證

新的和現有的後端可以使用憑證、並與ONTAP 該後端通訊。後端定義需要三個參數。

- 用戶端憑證：用戶端憑證的Base64編碼值。
- 用戶端私密金鑰：關聯私密金鑰的Base64編碼值。
- 信任的CACertifate：受信任CA憑證的Base64編碼值。如果使用信任的CA、則必須提供此參數。如果未使用信任的CA、則可忽略此問題。

典型的工作流程包括下列步驟。

步驟

1. 產生用戶端憑證和金鑰。產生時、請將Common Name（CN）（一般名稱（CN））設定為ONTAP 驗證身分。

   ```
   openssl req -x509 -nodes -days 1095 -newkey rsa:2048 -keyout k8senv.key
   -out k8senv.pem -subj "/C=US/ST=NC/L=RTP/O=NetApp/CN=vsadmin"
   ```

2. 將信任的CA憑證新增ONTAP 至整個叢集。這可能已由儲存管理員處理。如果未使用信任的CA、請忽略。

   ```
   security certificate install -type server -cert-name <trusted-ca-cert-
   name> -vserver <vserver-name>
   ssl modify -vserver <vserver-name> -server-enabled true -client-enabled
   true -common-name <common-name> -serial <SN-from-trusted-CA-cert> -ca
   <cert-authority>
   ```

3. 在ONTAP 支援叢集上安裝用戶端憑證和金鑰（步驟1）。

```
security certificate install -type client-ca -cert-name <certificate-
name> -vserver <vserver-name>
security ssl modify -vserver <vserver-name> -client-enabled true
```

4. 確認ONTAP 支援「cert」驗證方法的支援功能。

```
security login create -user-or-group-name vsadmin -application ontapi
-authentication-method cert -vserver <vserver-name>
security login create -user-or-group-name vsadmin -application http
-authentication-method cert -vserver <vserver-name>
```

5. 使用產生的憑證測試驗證。以ONTAP Management LIF IP和SVM名稱取代<SfManagement LIF>和<vserver name>。您必須確保LIF的服務原則設定為「預設資料管理」。

```
curl -X POST -Lk https://<ONTAP-Management-
LIF>/servlets/netapp.servlets.admin.XMLrequest_filer --key k8senv.key
--cert ~/k8senv.pem -d '<?xml version="1.0" encoding="UTF-8"?><netapp
xmlns="http://www.netapp.com/filer/admin" version="1.21"
vfiler="<vserver-name>"><vserver-get></vserver-get></netapp>'
```

6. 使用Base64編碼憑證、金鑰和信任的CA憑證。

```
base64 -w 0 k8senv.pem >> cert_base64
base64 -w 0 k8senv.key >> key_base64
base64 -w 0 trustedca.pem >> trustedca_base64
```

7. 使用從上一步取得的值建立後端。

```
$ cat cert-backend-updated.json
{
"version": 1,
"storageDriverName": "ontap-nas",
"backendName": "NasBackend",
"managementLIF": "1.2.3.4",
"dataLIF": "1.2.3.8",
"svm": "vserver_test",
"clientCertificate": "Faaaakkkkeeee...Vaaallllluuuueeee",
"clientPrivateKey": "LS0tFaKE...0VaLuES0tLS0K",
"storagePrefix": "myPrefix_"
}

#Update backend with tridentctl
$ tridentctl update backend NasBackend -f cert-backend-updated.json -n
trident
+------------+----------------+--------------------------------------
+--------+---------+
|    NAME    | STORAGE DRIVER |                 UUID                 |
STATE  | VOLUMES |
+------------+----------------+--------------------------------------
+--------+---------+
| NasBackend | ontap-nas      | 98e19b74-aec7-4a3d-8dcf-128e5033b214 |
online |       9 |
+------------+----------------+--------------------------------------
+--------+---------+
```

更新驗證方法或旋轉認證資料

您可以更新現有的後端、以使用不同的驗證方法、或是旋轉其認證資料。這兩種方法都可行：使用使用者名稱/密碼的後端可更新以使用憑證；使用憑證的後端可更新為使用者名稱/密碼。若要這麼做、請使用更新的「backend.json」檔案、其中包含執行「tridentctl後端更新」所需的參數。

```
$ cat cert-backend-updated.json
{
"version": 1,
"storageDriverName": "ontap-nas",
"backendName": "NasBackend",
"managementLIF": "1.2.3.4",
"dataLIF": "1.2.3.8",
"svm": "vserver_test",
"username": "vsadmin",
"password": "secret",
"storagePrefix": "myPrefix_"
}

#Update backend with tridentctl
$ tridentctl update backend NasBackend -f cert-backend-updated.json -n
trident
+------------+----------------+--------------------------------------
+--------+---------+
|    NAME    | STORAGE DRIVER |                  UUID                |
STATE   | VOLUMES |
+------------+----------------+--------------------------------------
+--------+---------+
| NasBackend | ontap-nas      | 98e19b74-aec7-4a3d-8dcf-128e5033b214 |
online |       9 |
+------------+----------------+--------------------------------------
+--------+---------+
```

> 當您旋轉密碼時、儲存管理員必須先更新ONTAP 使用者的密碼（位於BIOS）。接著是後端更新。在循環憑證時、可將多個憑證新增至使用者。然後更新後端以使用新的憑證、之後可從ONTAP 該叢集刪除舊的憑證。

更新後端不會中斷對已建立之磁碟區的存取、也不會影響之後建立的磁碟區連線。成功的後端更新顯示Astra Trident可以與ONTAP 該後端通訊、並處理未來的Volume作業。

管理NFS匯出原則

Astra Trident使用NFS匯出原則來控制其所配置之磁碟區的存取。

使用匯出原則時、Astra Trident提供兩種選項：

- Astra Trident可動態管理匯出原則本身；在此作業模式中、儲存管理員會指定代表可接受IP位址的CIDR區塊清單。Astra Trident會自動將這些範圍內的節點IP新增至匯出原則。或者、如果未指定CIDR、則會將節點上找到的任何全域範圍單點傳送IP新增至匯出原則。
- 儲存管理員可以建立匯出原則、並手動新增規則。除非在組態中指定不同的匯出原則名稱、否則Astra Trident會使用預設的匯出原則。

動態管理匯出原則

「csi Trident」的20.04版提供動態管理輸出原則的能力ONTAP 、以利實現幕後。這可讓儲存管理員為工作節點IP指定允許的位址空間、而非手動定義明確的規則。它可大幅簡化匯出原則管理；修改匯出原則不再需要在儲存叢集上進行手動介入。此外、這有助於將儲存叢集的存取限制在指定範圍內有IP的工作者節點、以支援精簡且自動化的管理。

> 只有「csi Trident」才能動態管理匯出原則。請務必確保工作節點未被NATed。

範例

必須使用兩種組態選項。以下是後端定義範例：

```
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "backendName": "ontap_nas_auto_export,
    "managementLIF": "192.168.0.135",
    "svm": "svm1",
    "username": "vsadmin",
    "password": "FaKePaSsWoRd",
    "autoExportCIDRs": ["192.168.0.0/24"],
    "autoExportPolicy": true
}
```

> 使用此功能時、您必須確保SVM中的根連接點具有預先設定的匯出原則、並具有允許節點CIDR區塊（例如預設匯出原則）的匯出規則。請務必遵循NetApp建議的最佳實務做法、為Astra Trident指定SVM。

以下是使用上述範例說明此功能的運作方式：

- 將「AutoExpportPolicy」設為「true」。這表示Astra Trident將為「svm1」SVM建立匯出原則、並使用「AutoExportCIDR」位址區塊來處理新增和刪除規則的作業。例如、在SVM上、UUID為403b5326-8482-40dB/96d0-d83fb3f4daec和「autodportPolicy」設為「true」的後端會建立名為「trident 403b5326-8482-40dB/96d0-d83fb3f4daec」的匯出原則。
- 「AutoExportCIDR」包含位址區塊清單。此欄位為選用欄位、預設為「0.00.0.0/0」、「：/0」。如果未定義、Astra Trident會新增在工作者節點上找到的所有全域範圍單點傳送位址。

在此範例中、會提供「192．168．0．0/24」位址空間。這表示、屬於此位址範圍的Kubernetes節點IP將新增至Astra Trident所建立的匯出原則。當Astra Trident登錄其執行的節點時、會擷取節點的IP位址、並對照「AutoExpportCIDR」中提供的位址區塊進行檢查。篩選IP之後、Astra Trident會針對所探索的用戶端IP建立匯出原則規則、並針對所識別的每個節點建立一個規則。

您可以在建立後端後、更新「AutoExpportPolicy」和「AutoExpportCTR」。您可以為自動管理或刪除現有CIDR的後端附加新的CIDR。刪除CIDR時請務必謹慎、以確保不會中斷現有的連線。您也可以選擇停用後端的「autodportPolicy」、然後回到手動建立的匯出原則。這需要在後端組態中設定「exportPolicy」參數。

Astra Trident建立或更新後端之後、您可以使用「tridentctl」或對應的「tridentbackend」CRD來檢查後端：

```
$ ./tridentctl get backends ontap_nas_auto_export -n trident -o yaml
items:
- backendUUID: 403b5326-8482-40db-96d0-d83fb3f4daec
  config:
    aggregate: ""
    autoExportCIDRs:
    - 192.168.0.0/24
    autoExportPolicy: true
    backendName: ontap_nas_auto_export
    chapInitiatorSecret: ""
    chapTargetInitiatorSecret: ""
    chapTargetUsername: ""
    chapUsername: ""
    dataLIF: 192.168.0.135
    debug: false
    debugTraceFlags: null
    defaults:
      encryption: "false"
      exportPolicy: <automatic>
      fileSystemType: ext4
```

當節點新增至Kubernetes叢集並向Astra Trident控制器註冊時、現有後端的匯出原則會更新（前提是它們位於後端的「AutoExpportCIDR」中指定的位址範圍內）。

移除節點時、Astra Trident會檢查所有線上的後端、以移除節點的存取規則。Astra Trident將此節點IP從託管後端的匯出原則中移除、可防止惡意掛載、除非叢集中的新節點重複使用此IP。

對於先前存在的後端、使用「tridentctl update backend」更新後端、可確保Astra Trident自動管理匯出原則。這會建立以後端UUID命名的新匯出原則、而後端上的磁碟區會在重新掛載時使用新建立的匯出原則。

> 刪除具有自動管理匯出原則的後端、將會刪除動態建立的匯出原則。如果重新建立後端、則會將其視為新的後端、並導致建立新的匯出原則。

如果即時節點的IP位址已更新、您必須重新啟動節點上的Astra Trident Pod。Astra Trident接著會更新其管理的後端匯出原則、以反映此IP變更。

組態選項與範例

瞭解如何透過ONTAP Astra Trident安裝來建立及使用NetApp NAS驅動程式。本節提供後端組態範例、以及如何將後端對應至StorageClass的詳細資料。

後端組態選項

如需後端組態選項、請參閱下表：

參數	說明	預設
「分度」		永遠為1

參數	說明	預設
「torageDriverName」	儲存驅動程式名稱	「ONTAP-NAS」、「ONTAP-NAS-節約 型」、「ONTAP-NAS-flexgroup」、「ONTAP-SAN 」、「ONTAP-san經濟型」
「後端名稱」	自訂名稱或儲存後端	驅動程式名稱+「_」+ dataLIF
《馬納格門達利》	叢集或SVM管理LIF的IP位址	「10.0.0.1」、「[2001:1234:abcd::fefo]」
「DataLIF」	傳輸協定LIF的IP位址。IPv6使用方括弧。設定後無法更新	除非另有說明、否則由SVM衍生
「AutoExpportPolicy」	啟用自動匯出原則建立與更新[布 林值]	錯
《AutoExpportCIDR》（自動匯出CTR）	啟用「AutoExpportPolicy」時、用來篩選Kubernetes節點IP的CIDR清單	[「0.00.0/0」、「：/0」]
《標籤》	套用到磁碟區的任意JSON-格式化標籤集	「」
「用戶端憑證」	用戶端憑證的Base64編碼值。用於憑證型驗證	「」
「clientPrivate Key」	用戶端私密金鑰的Base64編碼值。用於憑證型驗證	「」
「可信賴的CACertifcate」	受信任CA憑證的Base64編碼值。選用。用於憑證型驗證	「」
《使用者名稱》	連線至叢集/ SVM的使用者名稱。用於認證型驗證	
密碼	連線至叢集/ SVM的密碼。用於認證型驗證	
《虛擬機器》	要使用的儲存虛擬機器	如果指定SVM "managementLIF"則衍生
「igroupName」	要使用之SAN磁碟區的igroup名稱	「Trident -<後端-UUID>」
「toragePrefix」	在SVM中配置新磁碟區時所使用的前置碼。設定後無法更新	「Trident」
「限制Aggregateusage」	如果使用率高於此百分比、則無法進行資源配置。*不適用於Amazon FSX for ONTAP Sfor Sfor *	「」（預設不強制執行）
《限制Volume大小》	如果要求的磁碟區大小高於經濟驅動程式的此值、則無法進行資源配置。	「」（預設不強制執行）
《lunsPerFlexvol》	每FlexVol 個LUN的最大LUN數量、範圍必須在[50、200]	「100」
「DebugTraceFlags」	疑難排解時要使用的偵錯旗標。範例： {"API"：假、「方法」：true }	null

參數	說明	預設
「nfsMountOptions」	以逗號分隔的NFS掛載選項清單	「」
"qtreesPerFlexvol"	每FlexVol 個邊的最大qtree數、必須在範圍內[50、300]	「200」
《useREST》	使用ONTAP Isrest API的布林參數。技術預覽	錯

> 「useREST」是一種建議用於測試環境、而非用於正式作業工作負載的技術預覽。當設為「true」時、Astra Trident會使用ONTAP 「Isrest API」與後端通訊。此功能需要ONTAP 使用更新版本的版本。此外ONTAP 、所使用的登入角色必須能夠存取「ONTAP 發揮作用」應用程式。這對預先定義的「vsadmin」和「叢集管理」角色感到滿意。

若要與ONTAP 此叢集通訊、您應該提供驗證參數。這可能是安全登入或安裝憑證的使用者名稱/密碼。

> 如果您使用Amazon FSX for NetApp ONTAP Sendbackend、請勿指定「limitAggregateusage」參數。Amazon FSx for NetApp ONTAP 的「fsxadmin」和「vsadmin」角色不包含擷取Aggregate使用量並透過Astra Trident加以限制的必要存取權限。

> 除非您正在進行疑難排解並需要詳細的記錄傾印、否則請勿使用「debugTraceFlags」。

> 建立後端時、請記住、建立後無法修改「dataLIF」和「儲存前置字元」。若要更新這些參數、您需要建立新的後端。

可以為「managementLIF」選項指定完整網域名稱（FQDN）。也可以為「dataLIF」選項指定FQDN、在這種情況下、FQDN將用於NFS裝載作業。如此一來、您就能建立循環DNS、在多個資料生命期之間實現負載平衡。

所有ONTAP 的不完整驅動程式也可設定為IPv6位址。請務必使用「-use IPv6」旗標來安裝Astra Trident。必須謹慎定義方括弧內的「managementLIF」IPv6位址。

> 使用IPv6位址時、請務必在方括弧內定義「managementLIF」和「dataLIF」（若包含在後端定義中）、例如[28e8：d9fb:a825:b7bf:69a8：d02f：9e7b:3555]。如果未提供「data LIF」、Astra Trident會從SVM擷取IPv6資料LIF。

使用「AutoExport Policy」（自動匯出原則）和「AutoExport CIDR」（自動匯出CTR）選項、「csi Trident」（SCSI Trident）可以自動管理匯出原則。所有的ONTAP-NAS-*驅動程式均支援此功能。

對於「ONTAP-NAS-EAS-經濟」驅動程式、「limitVolume Sizes」選項也會限制其管理的qtree和LUN磁碟區大小上限、而「qtreesPerFlexvol'」選項則可自訂每FlexVol 個經濟體系的最大配額樹數。

可以使用「nfsMountOptions」參數來指定掛載選項。Kubernetes持續磁碟區的掛載選項通常會在儲存類別中指定、但如果儲存類別中未指定掛載選項、則Astra Trident會改回使用儲存後端組態檔中指定的掛載選項。如果儲存類別或組態檔中未指定掛載選項、則Astra Trident不會在相關的持續磁碟區上設定任何掛載選項。

Astra Trident在所有使用「ONTAP-NAS」和「ONTAP-NAS-flexgroup」建立的磁碟區的「Comments」（註解）欄位中設定資源配置標籤。根據所使用的驅動程式、意見會設定在FlexVol 以下項目上：（「ONTAP-NAS」）或FlexGroup （「ONTAP-NAS-flexgroup」）。Astra Trident會在儲存資源池配置時、將儲存資源池上的所有標籤複製到儲存磁碟區。儲存管理員可以定義每個儲存資源池的標籤、並將儲存資源池中建立的所有磁碟區分組。這是根據後端組態中提供的一組可自訂標籤、方便區分磁碟區的方法。

用於資源配置磁碟區的後端組態選項

您可以在組態的特定區段中、使用這些選項來控制預設配置每個Volume的方式。如需範例、請參閱下列組態範例。

參數	說明	預設
"paceAllocate（配置）"	LUN的空間分配	「真的」
《保護區》	空間保留模式；「無」（精簡）或「Volume」（完整）	「無」
「快照原則」	要使用的Snapshot原則	「無」
「qosPolicy」	要指派給所建立磁碟區的QoS原則群組。選擇每個儲存集區/後端的其中一個qosPolicy或adaptiveQosPolicy	「」
《adaptiveQosPolicy》	要指派給所建立磁碟區的調適性QoS原則群組。選擇每個儲存集區/後端的其中一個qosPolicy或adaptiveQosPolicy。不受ONTAP-NAS-經濟支援。	「」
「快照保留區」	保留給快照「0」的磁碟區百分比	如果「快照原則」為「無」、則其他為「」
「PlitOnClone」	建立複本時、從其父複本分割複本	「假」
加密	啟用NetApp Volume加密	「假」
《生態樣式》	新磁碟區的安全樣式	「UNIX」
「分層政策」	分層原則以使用「無」	ONTAP 9.5之前的SVM-DR組態為「純快照」
unix權限	新磁碟區的模式	「777」
snapshotDir	控制「.snapshot」目錄的可見度	「假」
匯出原則	要使用的匯出原則	「預設」
安全性樣式	新磁碟區的安全樣式	「UNIX」

搭配Astra Trident使用QoS原則群組需要ONTAP 使用更新版本的版本。建議使用非共用的QoS原則群組、並確保原則群組會個別套用至每個組成群組。共享的QoS原則群組將強制所有工作負載的總處理量上限。

以下是已定義預設值的範例：

```
{
  "version": 1,
  "storageDriverName": "ontap-nas",
  "backendName": "customBackendName",
  "managementLIF": "10.0.0.1",
  "dataLIF": "10.0.0.2",
  "labels": {"k8scluster": "dev1", "backend": "dev1-nasbackend"},
  "svm": "trident_svm",
  "username": "cluster-admin",
  "password": "password",
  "limitAggregateUsage": "80%",
  "limitVolumeSize": "50Gi",
  "nfsMountOptions": "nfsvers=4",
  "debugTraceFlags": {"api":false, "method":true},
  "defaults": {
    "spaceReserve": "volume",
    "qosPolicy": "premium",
    "exportPolicy": "myk8scluster",
    "snapshotPolicy": "default",
    "snapshotReserve": "10"
  }
}
```

對於「ONTAP-NAS」和「ONTAP-NAS-flexGroups」、Astra Trident現在採用新的計算方法、確保FlexVol 利用snapshotReserve百分比和PVc正確調整其規模。當使用者要求使用PVCs時、Astra Trident會FlexVol 使用新的計算方式、建立原始的包含更多空間的候選區。此計算可確保使用者在永久虛擬磁碟中獲得所要求的可寫入空間、且空間不得小於所要求的空間。在v21.07之前、當使用者要求使用PVC（例如5GiB）、快照保留區達到50%時、他們只能獲得2.5GiB的可寫入空間。這是因為使用者要求的是整個磁碟區、而「快照保留區」則是其中的一個百分比。使用Trident 21.07時、使用者要求的是可寫入空間、而Astra Trident則將「快照保留區」編號定義為整個Volume的百分比。這不適用於「ONTAP-NAS-經濟」。請參閱下列範例以瞭解此功能的運作方式：

計算方式如下：

```
Total volume size = (PVC requested size) / (1 - (snapshotReserve
percentage) / 100)
```

對於snapshotReserve = 50%、而PVc要求= 5GiB、磁碟區總大小為2/0.5 = 10GiB、可用大小為5GiB、這是使用者在PVC要求中要求的大小。「volume show（Volume show）命令應顯示類似以下範例的結果：

```
Vserver   Volume       Aggregate    State      Type       Size  Available Used%
--------- ------------ ------------ ---------- ---- ---------- ---------- -----
                 _pvc_89f1c156_3801_4de4_9f9d_034d54c395f4
                                    online     RW         10GB     5.00GB    0%
                 _pvc_e8372153_9ad9_474a_951a_08ae15e1c0ba
                                    online     RW          1GB    511.8MB    0%
2 entries were displayed.
```

在升級Astra Trident時、先前安裝的現有後端會按照上述說明來配置磁碟區。對於在升級之前建立的磁碟區、您應該調整其磁碟區大小、以便觀察變更。例如、較早前使用「快照保留區=50」的2GiB室早導致磁碟區提供1GiB的可寫入空間。例如、將磁碟區大小調整為3GiB、可讓應用程式在6 GiB磁碟區上擁有3GiB的可寫入空間。

最低組態範例

下列範例顯示基本組態、讓大部分參數保留預設值。這是定義後端最簡單的方法。

> 如果您在NetApp ONTAP 支援Trident的NetApp支援上使用Amazon FSX、建議您指定lifs的DNS名稱、而非IP位址。

ONTAP - NAS驅動程式搭配憑證型驗證

這是最小的後端組態範例。「clientCertificate」、「clientPrivate Key」和「Trusted CACertifate」（選用、如果使用信任的CA）會分別填入「backend.json」、並採用用戶端憑證、私密金鑰和信任的CA憑證的64編碼值。

```
{
  "version": 1,
  "backendName": "DefaultNASBackend",
  "storageDriverName": "ontap-nas",
  "managementLIF": "10.0.0.1",
  "dataLIF": "10.0.0.15",
  "svm": "nfs_svm",
  "clientCertificate": "ZXR0ZXJwYXB...ICMgJ3BhcGVyc2",
  "clientPrivateKey": "vciwKIyAgZG...0cnksIGRlc2NyaX",
  "trustedCACertificate": "zcyBbaG...b3Igb3duIGNsYXNz",
  "storagePrefix": "myPrefix_"
}
```

ONTAP - NAS驅動程式搭配自動匯出原則

本範例說明如何指示Astra Trident使用動態匯出原則來自動建立及管理匯出原則。這對「ONTAP-NAS-節約」和「ONTAP-NAS-Flexgroup」驅動程式的運作方式相同。

```
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "labels": {"k8scluster": "test-cluster-east-1a", "backend": "test1-
nasbackend"},
    "autoExportPolicy": true,
    "autoExportCIDRs": ["10.0.0.0/24"],
    "username": "admin",
    "password": "secret",
    "nfsMountOptions": "nfsvers=4",
}
```

ONTAP-NAS-flexgroup驅動程式

```
{
    "version": 1,
    "storageDriverName": "ontap-nas-flexgroup",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "labels": {"k8scluster": "test-cluster-east-1b", "backend": "test1-
ontap-cluster"},
    "svm": "svm_nfs",
    "username": "vsadmin",
    "password": "secret",
}
```

ONTAP - NAS驅動程式搭配IPv6

```
{
 "version": 1,
 "storageDriverName": "ontap-nas",
 "backendName": "nas_ipv6_backend",
 "managementLIF": "[5c5d:5edf:8f:7657:bef8:109b:1b41:d491]",
 "labels": {"k8scluster": "test-cluster-east-1a", "backend": "test1-ontap-
ipv6"},
 "svm": "nas_ipv6_svm",
 "username": "vsadmin",
 "password": "netapp123"
}
```

ONTAP-NAS-經濟驅動程式

```
{
    "version": 1,
    "storageDriverName": "ontap-nas-economy",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "username": "vsadmin",
    "password": "secret"
}
```

虛擬儲存資源池的後端範例

在下圖所示的範例後端定義檔中、會針對所有儲存資源池設定特定的預設值、例如「無的儲存資源保留」、「假的資源配置」、以及「假的加密」。虛擬儲存資源池是在儲存區段中定義。

在此範例中、有些儲存資源池會設定自己的「資源保留」、「資源配置」和「加密」值、有些資源池會覆寫上述設定的預設值。

ONTAP-NAS驅動程式

```
{
    {
    "version": 1,
    "storageDriverName": "ontap-nas",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "username": "admin",
    "password": "secret",
    "nfsMountOptions": "nfsvers=4",

    "defaults": {
          "spaceReserve": "none",
          "encryption": "false",
          "qosPolicy": "standard"
    },
    "labels":{"store":"nas_store", "k8scluster": "prod-cluster-1"},
    "region": "us_east_1",
    "storage": [
        {
            "labels":{"app":"msoffice", "cost":"100"},
            "zone":"us_east_1a",
            "defaults": {
                "spaceReserve": "volume",
```

```
                "encryption": "true",
                "unixPermissions": "0755",
                "adaptiveQosPolicy": "adaptive-premium"
            }
        },
        {
            "labels":{"app":"slack", "cost":"75"},
            "zone":"us_east_1b",
            "defaults": {
                "spaceReserve": "none",
                "encryption": "true",
                "unixPermissions": "0755"
            }
        },
        {
            "labels":{"app":"wordpress", "cost":"50"},
            "zone":"us_east_1c",
            "defaults": {
                "spaceReserve": "none",
                "encryption": "true",
                "unixPermissions": "0775"
            }
        },
        {
            "labels":{"app":"mysqldb", "cost":"25"},
            "zone":"us_east_1d",
            "defaults": {
                "spaceReserve": "volume",
                "encryption": "false",
                "unixPermissions": "0775"
            }
        }
    ]
}
```

ONTAP-NAS-flexgroup驅動程式

```
{
    "version": 1,
    "storageDriverName": "ontap-nas-flexgroup",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "username": "vsadmin",
    "password": "secret",
```

```
    "defaults": {
          "spaceReserve": "none",
          "encryption": "false"
    },
    "labels":{"store":"flexgroup_store", "k8scluster": "prod-cluster-1"},
    "region": "us_east_1",
    "storage": [
        {
            "labels":{"protection":"gold", "creditpoints":"50000"},
            "zone":"us_east_1a",
            "defaults": {
                "spaceReserve": "volume",
                "encryption": "true",
                "unixPermissions": "0755"
            }
        },
        {
            "labels":{"protection":"gold", "creditpoints":"30000"},
            "zone":"us_east_1b",
            "defaults": {
                "spaceReserve": "none",
                "encryption": "true",
                "unixPermissions": "0755"
            }
        },
        {
            "labels":{"protection":"silver", "creditpoints":"20000"},
            "zone":"us_east_1c",
            "defaults": {
                "spaceReserve": "none",
                "encryption": "true",
                "unixPermissions": "0775"
            }
        },
        {
            "labels":{"protection":"bronze", "creditpoints":"10000"},
            "zone":"us_east_1d",
            "defaults": {
                "spaceReserve": "volume",
                "encryption": "false",
                "unixPermissions": "0775"
            }
        }
    ]
}
```

ONTAP-NAS-經濟驅動程式

```
{
    "version": 1,
    "storageDriverName": "ontap-nas-economy",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "username": "vsadmin",
    "password": "secret",

    "defaults": {
          "spaceReserve": "none",
          "encryption": "false"
    },
    "labels":{"store":"nas_economy_store"},
    "region": "us_east_1",
    "storage": [
        {
            "labels":{"department":"finance", "creditpoints":"6000"},
            "zone":"us_east_1a",
            "defaults": {
                "spaceReserve": "volume",
                "encryption": "true",
                "unixPermissions": "0755"
            }
        },
        {
            "labels":{"department":"legal", "creditpoints":"5000"},
            "zone":"us_east_1b",
            "defaults": {
                "spaceReserve": "none",
                "encryption": "true",
                "unixPermissions": "0755"
            }
        },
        {
            "labels":{"department":"engineering", "creditpoints":"3000"},
            "zone":"us_east_1c",
            "defaults": {
                "spaceReserve": "none",
                "encryption": "true",
                "unixPermissions": "0775"
            }
        },
        {
```

```
            "labels":{"department":"humanresource",
"creditpoints":"2000"},
            "zone":"us_east_1d",
            "defaults": {
                "spaceReserve": "volume",
                "encryption": "false",
                "unixPermissions": "0775"
            }
        }
    ]
}
```

將後端對應至**StorageClass**

下列StorageClass定義係指上述虛擬儲存資源池。使用「parameters.selector`」欄位、每個StorageClass都會呼叫哪些虛擬資源池可用於裝載磁碟區。磁碟區將會在所選的虛擬資源池中定義各個層面。

- 第一部StorageClass（「protection黃金級」）將對應至「ONTAP-NAS Flexgroup」後端的第一個、第二個虛擬儲存集區、以及「ONTAP-SAN」後端的第一個虛擬儲存集區。這是唯一提供金級保護的資源池。
- 第二個StorageClass（「protection非gold」）將對應至第三個、第四個虛擬儲存集區、位於「ONTAP-NAS」後端、第二個是第三個虛擬儲存集區、位於「ONTAP-SAN」後端。這是唯一提供金級以外保護層級的資源池。
- 第三個StorageClass（「app-mysqldb」）將對應至「ontap-nas」後端的第四個虛擬儲存資源池、以及「ontap-san經濟」後端的第三個虛擬儲存資源池。這些是唯一提供mysqldb類型應用程式儲存池組態的集區。
- 第四個StorageClass（「protection－silver－creditpoints－20k」）將對應至第三個虛擬儲存資源池（位於「ONTAP－NAS－flexgroup」後端）、第二個虛擬儲存資源池（位於「ONTAP－san」後端）。這些資源池是唯一能以20000個信用點數提供金級保護的資源池。
- 第五個StorageClass（'creditpoint-5k'）將對應到第二個位於「ONTAP-NAS經濟」後端的虛擬儲存資源池、以及位於「ONTAP-SAN」後端的第三個虛擬儲存資源池。這些是唯一提供5000個信用點數的資源池產品。

Astra Trident將決定選取哪個虛擬儲存資源池、並確保符合儲存需求。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: protection-gold
provisioner: netapp.io/trident
parameters:
  selector: "protection=gold"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: protection-not-gold
provisioner: netapp.io/trident
parameters:
  selector: "protection!=gold"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: app-mysqldb
provisioner: netapp.io/trident
parameters:
  selector: "app=mysqldb"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: protection-silver-creditpoints-20k
provisioner: netapp.io/trident
parameters:
  selector: "protection=silver; creditpoints=20000"
  fsType: "ext4"
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: creditpoints-5k
provisioner: netapp.io/trident
parameters:
  selector: "creditpoints=5000"
  fsType: "ext4"
```

使用Astra Trident搭配Amazon FSX for NetApp ONTAP 解決方案

"Amazon FSX for NetApp ONTAP 產品"是一項完全託管的AWS服務、可讓客戶啟動及執行採用NetApp ONTAP的一套儲存作業系統的檔案系統。Amazon FSX for NetApp ONTAP 功能可讓您運用熟悉的NetApp功能、效能和管理功能、同時充分發揮儲存AWS資料的簡易性、敏捷度、安全性和擴充性。FSX支援ONTAP的許多檔案系統功能和管理API。

檔案系統是Amazon FSX的主要資源、類似ONTAP 於內部部署的一個叢集。在每個SVM中、您可以建立一個或多個磁碟區、這些磁碟區是儲存檔案系統中檔案和資料夾的資料容器。有了Amazon FSX for NetApp ONTAP 的功能、Data ONTAP 即可在雲端以託管檔案系統的形式提供支援。新的檔案系統類型稱為* NetApp ONTAP Sing*。

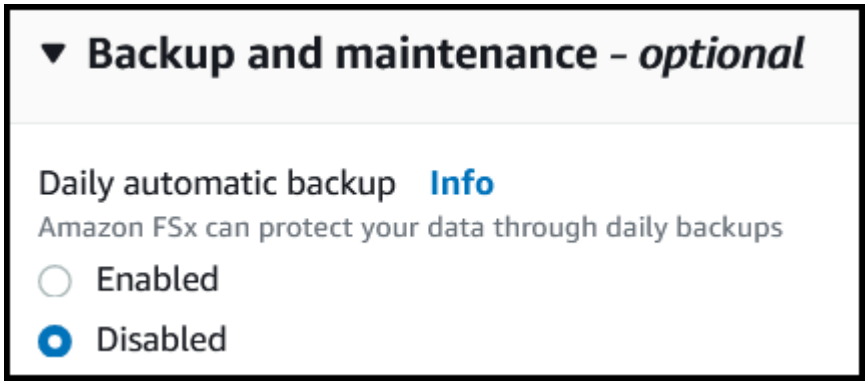
使用Astra Trident搭配Amazon FSX for NetApp ONTAP 供應NetApp時、您可以確保在Amazon Elastic Kubernetes Service（EKS）中執行的Kubernetes叢集、能夠配置區塊和檔案以ONTAP 支援的持續磁碟區。

建立**Amazon FSX for ONTAP Sf**供 檔案系統使用

在啟用自動備份的Amazon FSX檔案系統上建立的磁碟區、無法由Trident刪除。若要刪除PVCs、您需要手動刪除PV和FSXfor ONTAP the Sesfvolume。

若要避免此問題：

- 請勿使用「快速建立」來建立FSX for ONTAP the Suse檔案系統。快速建立工作流程可自動備份、但不提供退出選項。
- 使用「標準建立」時、請停用自動備份。停用自動備份可讓Trident成功刪除磁碟區、而無需進一步手動介入。



瞭解**Astra Trident**

如果您是Astra Trident的新手、請使用下列連結來熟悉您的操作：

- "常見問題集"
- "使用Astra Trident的要求"
- "部署Astra Trident"
- "最佳實務做法、可設定ONTAP 適用於Cloud Volumes ONTAP NetApp ONTAP 的功能、包括功能、功能及Amazon FSX"
- "整合Astra Trident"

- "支援SAN後端組態ONTAP"
- "ASNAS後端組態ONTAP"

深入瞭解驅動程式功能 "請按這裡"。

適用於NetApp ONTAP 的Amazon FSX "FabricPool" 管理儲存層。它可讓您根據資料是否經常存取、將資料儲存在一個層級中。

Astra Trident預期以「vsadmin」SVM使用者或具有相同角色之不同名稱的使用者身分執行。Amazon FSx for NetApp ONTAP 的使用者是「fsxadmin」使用者、只能有限地取代ONTAP 「admin」叢集使用者。不建議將「fsxadmin」使用者搭配Trident使用、因為「vsadmin」SVM使用者可以存取更多Astra Trident功能。

驅動程式

您ONTAP 可以使用下列驅動程式、將Astra Trident與Amazon FSX for NetApp整合：

- 「ONTAP-san」：每個提供的PV都是其專屬Amazon FSX for NetApp ONTAP 的LUN。
- 「ONTAP-san經濟型」：每個配置的PV都是LUN、每個Amazon FSX for NetApp ONTAP 的LUN數量可設定。
- 「ONTAP-NAS」：每個提供的PV都是適用於NetApp ONTAP 的完整Amazon FSX。
- 「ONTAP-NAS-EAS'：每個提供的PV都是qtree、每個Amazon FSX的NetApp ONTAP 功能是可設定的配額樹數。
- 「ONTAP-NAS-Flexgroup」：每個提供的PV都是適用於NetApp ONTAP FlexGroup 的完整Amazon FSX。

驗證

Astra Trident提供兩種驗證模式：

- 憑證型：Astra Trident會使用SVM上安裝的憑證、與FSX檔案系統上的SVM進行通訊。
- 認證型：您可以將「fsxadmin」使用者用於您的檔案系統、或是為SVM設定的「vsadmin」使用者。

> 我們強烈建議您使用「vsadmin」使用者、而非「fssadmin」來設定後端。Astra Trident將使用此使用者名稱和密碼與FSX檔案系統通訊。

若要深入瞭解驗證、請參閱下列連結：

- "NAS ONTAP"
- "SAN ONTAP"

在EKS上部署及設定Astra Trident搭配Amazon FSX for NetApp ONTAP

您需要的是 **#8217** ；需要的是什麼

- 現有的Amazon EKS叢集或自行管理的Kubernetes叢集、已安裝「kubecll」。
- 現有的Amazon FSX-適用於NetApp ONTAP 的支援資料系統和儲存虛擬機器（SVM）、可從叢集的工作節點存取。
- 已準備好的工作節點 "NFS和/或iSCSI"。

請務必遵循Amazon Linux和Ubuntu所需的節點準備步驟 "Amazon機器映像" （AMis）、視您的EKS AMI類型而定。

如需其他Astra Trident需求、請參閱 "請按這裡"。

步驟

1. 使用以下其中一種方式部署Astra Trident：./ Trident入門/ Kubernetes-Deploy .html[部署方法]。
2. 設定Astra Trident如下：
   a. 收集SVM的管理LIF DNS名稱。例如、使用AWS CLI執行下列命令後、在「Endings」→「管理」下找到「DNSName」項目：

```
aws fsx describe-storage-virtual-machines --region <file system
region>
```

3. 建立及安裝驗證憑證。如果您使用的是「ONTAP-SAN」後端、請參閱 "請按這裡"。如果您使用的是「ONTAP-NAS」後端、請參閱 "請按這裡"。

您可以使用SSH從任何位置登入檔案系統（例如安裝憑證）、而該SSH可連至檔案系統。使用「fsxadmin」使用者、您在建立檔案系統時設定的密碼、以及「AWS FSx file-systems」中的管理DNS名稱。

4. 使用您的憑證和管理LIF的DNS名稱建立後端檔案、如下例所示：

```
{
  "version": 1,
  "storageDriverName": "ontap-san",
  "backendName": "customBackendName",
  "managementLIF": "svm-XXXXXXXXXXXXXXXXX.fs-XXXXXXXXXXXXXXXXX.fsx.us-
east-2.aws.internal",
  "svm": "svm01",
  "clientCertificate": "ZXR0ZXJwYXB...ICMgJ3BhcGVyc2",
  "clientPrivateKey": "vciwKIyAgZG...0cnksIGRlc2NyaX",
  "trustedCACertificate": "zcyBbaG...b3Igb3duIGNsYXNz",
 }
```

如需建立後端的相關資訊、請參閱下列連結：

- "使用ONTAP NetApp NAS驅動程式設定後端"
- "使用ONTAP SAN驅動程式設定後端"

不要指定"ontap－san "和"ONTAP－san經濟"驅動程序的"data Lif"允許Astra Trident使用多重路徑。

「limitAggregateusage」參數無法搭配「vsadmin」和「fsxadmin」使用者帳戶使用。如果您指定此參數、組態作業將會失敗。

部署之後、請執行建立的步驟 "儲存類別、配置磁碟區、然後將磁碟區掛載到Pod中"。

如需詳細資訊、請參閱

- "Amazon FSX for NetApp ONTAP 的支援文件"
- "Amazon FSX for NetApp ONTAP 的部落格文章"

使用kubecl建立後端

後端定義了Astra Trident與儲存系統之間的關係。它告訴Astra Trident如何與該儲存系統通訊、以及Astra Trident如何從該儲存系統配置磁碟區。安裝Astra Trident之後、下一步是建立後端。「TridentBackendConfig」自訂資源定義（CRD）可讓您直接透過Kubernetes介面建立及管理Trident後端。您可以使用「kubecll」或Kubernetes發佈的等效CLI工具來執行此作業。

TridentBackendConfig

「TridentBackendConfig」（「tbc」、「tbconfig」、「tbackendconfig」）是前端、名稱為CRD、可讓您使用「kubectl」來管理Astra Trident後端。Kubernetes與儲存管理員現在可以直接透過Kubernetes CLI建立及管理後端、而不需要使用專屬的命令列公用程式（「tridentctl」）。

建立「TridentBackendConfig」物件之後、會發生下列情況：

- Astra Trident會根據您提供的組態自動建立後端。內部代表的是「TridentBackend」（「tbe」、「tridentbackend」）、CR。
- 《TridentBackendConfig》與由Astra Trident所建立的《TridentBackend》有獨特的關聯。

每個「TridentBackendConfig」都有一對一的對應、並有「TridentBackend」。前者是提供給使用者設計及設定後端的介面、後者是Trident代表實際後端物件的方式。

「TridentBackend」CRS是由Astra Trident自動建立的。您*不應該*修改這些項目。如果您想要更新後端、請修改「TridentBackendConfig」物件來執行此動作。

請參閱下列範例、以瞭解「TridentBackendConfig」CR的格式：

```
apiVersion: trident.netapp.io/v1
kind: TridentBackendConfig
metadata:
  name: backend-tbc-ontap-san
spec:
  version: 1
  backendName: ontap-san-backend
  storageDriverName: ontap-san
  managementLIF: 10.0.0.1
  dataLIF: 10.0.0.2
  svm: trident_svm
  credentials:
    name: backend-tbc-ontap-san-secret
```

您也可以查看中的範例 "Trident安裝程式" 所需儲存平台/服務的範例組態目錄。

「show」會採用後端特定的組態參數。在此範例中、後端使用「ONTAP-san」儲存驅動程式、並使用此處表格列出的組態參數。如需所需儲存驅動程式的組態選項清單、請參閱 "儲存驅動程式的後端組態資訊"。

在《TridentBackendConfig》（CRR）中新推出的「sPEC」一節也包含「認證」和「刪除原則」欄位：

- 「認證資料」：此參數為必填欄位、包含用於驗證儲存系統/服務的認證資料。此設定為使用者建立的Kubernetes Secret。認證資料無法以純文字格式傳遞、因此會產生錯誤。
- 「刪除原則」：此欄位可定義刪除「TridentBackendConfig」時應發生的情況。可能需要兩種可能的值之一：
  - 「刪除」：這會同時刪除「TridentBackendConfig」和相關後端。這是預設值。
  - 「保留」：刪除「TridentBackendConfig」（TridentBackendConfig）CR時、後端定義仍會存在、並可使用「tridentctl」進行管理。將刪除原則設為「保留」可讓使用者降級至較早版本（21.04之前）、並保留建立的後端。此欄位的值可在建立「TridentBackendConfig」之後更新。

> 後端名稱是使用「sPEC.backendName」來設定。如果未指定、則會將後端名稱設為「TridentBackendConfig」物件（metadata.name）的名稱。建議使用「sPEC.backendName」明確設定後端名稱。

> 使用「tridentctl」建立的後端沒有關聯的「TridentBackendConfig」物件。您可以建立「TridentBackendConfig」（TridentBackendConfig）的CR、選擇以「kubectl」管理這類後端。必須謹慎指定相同的組態參數（例如「s.pec.backendName」、「sec.storagePrefix」、「sPEec.storageDriverName」等）。Astra Trident會自動將新建立的「TridentBackendConfig」連結至預先存在的後端。

步驟總覽

若要使用「kubecll」建立新的後端、您應該執行下列動作：

1. 建立 "Kubernetes機密"。此機密包含Astra Trident與儲存叢集/服務通訊所需的認證資料。
2. 建立「TridentBackendConfig」物件。其中包含有關儲存叢集/服務的詳細資訊、並參考上一步建立的機密。

建立後端之後、您可以使用「kubecl Get tbc <tbc-name>-n <trident命名空間>」來觀察其狀態、並收集其他詳細資料。

步驟1：建立Kubernetes機密

建立包含後端存取認證的秘密。這是每個儲存服務/平台所獨有的功能。以下是範例：

```
$ kubectl -n trident create -f backend-tbc-ontap-san-secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: backend-tbc-ontap-san-secret
type: Opaque
stringData:
  username: cluster-admin
  password: t@Ax@7q(>
```

下表摘要說明每個儲存平台的機密必須包含的欄位：

儲存平台機密欄位說明	秘密	欄位說明
Azure NetApp Files	ClientID	應用程式註冊的用戶端ID
適用於 GCP Cloud Volumes Service	Private金鑰ID	私密金鑰的ID。GCP服務帳戶API金鑰的一部分、具有CVS管理員角色
適用於 GCP Cloud Volumes Service	Private金鑰	私密金鑰：GCP服務帳戶API金鑰的一部分、具有CVS管理員角色
元素（NetApp HCI / SolidFire）	端點	MVIP、適用於SolidFire 採用租戶認證的不含用戶身分證明的叢集
ONTAP	使用者名稱	連線至叢集/ SVM的使用者名稱。用於認證型驗證
ONTAP	密碼	連線至叢集/ SVM的密碼。用於認證型驗證
ONTAP	用戶端權限金鑰	用戶端私密金鑰的Base64編碼值。用於憑證型驗證
ONTAP	chap使用 者名稱	傳入使用者名稱。如果useCHAP=true則需要。適用於「ONTAP-SAN」和「ONTAP-san經濟」

儲存平台機密欄位說明	秘密	欄位說明
ONTAP	chapInitiator機密	CHAP啟動器密碼。如果useCHAP=true則需要。適用於「ONTAP-SAN」和「ONTAP-san經濟」
ONTAP	chapTargetUsername	目標使用者名稱。如果useCHAP=true則需要。適用於「ONTAP-SAN」和「ONTAP-san經濟」
ONTAP	chapTargetInitiator機密	CHAP目標啟動器機密。如果useCHAP=true則需要。適用於「ONTAP-SAN」和「ONTAP-san經濟」

在此步驟中建立的機密會參照下一步所建立之「TridentBackendConfig」物件的「sapec.ecent」欄位。

步驟2：建立TridentBackendConfig CR

您現在可以建立「TridentBackendConfig」的CR了。在此範例中、使用「ONTAP-SAN」驅動程式的後端是使用「TridentBackendConfig」物件建立、如下所示：

```
$ kubectl -n trident create -f backend-tbc-ontap-san.yaml
```

```
apiVersion: trident.netapp.io/v1
kind: TridentBackendConfig
metadata:
  name: backend-tbc-ontap-san
spec:
  version: 1
  backendName: ontap-san-backend
  storageDriverName: ontap-san
  managementLIF: 10.0.0.1
  dataLIF: 10.0.0.2
  svm: trident_svm
  credentials:
    name: backend-tbc-ontap-san-secret
```

步驟3：確認TridentBackendConfig CR的狀態

現在您已經建立了「TridentBackendConfig」（TridentBackendConfig）CR、您就可以驗證其狀態。請參閱下列範例：

```
$ kubectl -n trident get tbc backend-tbc-ontap-san
NAME                     BACKEND NAME          BACKEND UUID
PHASE   STATUS
backend-tbc-ontap-san   ontap-san-backend     8d24fce7-6f60-4d4a-8ef6-
bab2699e6ab8   Bound   Success
```

已成功建立後端、並連結至「TridentBackendConfig」CR。

階段可以採用下列其中一個值：

- 「綁定」：「TridentBackendConfig」CR與後端相關聯、後端包含「configRef」設定為「TridentBackendConfig」的CR uid。
- 《Unbound》：使用「」表示。「TridentBackendConfig」物件不會繫結至後端。根據預設、所有新建立的「TridentBackendConfig」CRS均處於此階段。階段變更之後、就無法再恢復為Unbound（未綁定）。
- 「刪除」：「TridentBackendConfig」的「刪除原則」已設定為刪除。刪除「TridentBackendConfig」CR時、它會轉換為「刪除」狀態。
  - 如果後端不存在持續磁碟區宣告（PVCS）、刪除「TridentBackendConfig」（TridentBackendConfig）會導致Astra Trident刪除後端、以及刪除「TridentBackendConfig」（TridentBackendConfig）。
  - 如果後端上有一個或多個PVCS、則會進入刪除狀態。隨後、「TridentBackendConfig」CR也會進入刪除階段。只有刪除所有的PVCS之後、才會刪除後端和「TridentBackendConfig」。
- 「遺失」：與「TridentBackendConfig」CR相關的後端意外或刻意刪除、而「TridentBackendConfig」CR仍有刪除後端的參考資料。無論「刪除原則」值為何、「TridentBackendConfig」CR仍可刪除。
- 「未知」：Astra Trident無法判斷與「TridentBackendConfig」CR相關的後端狀態或存在。例如、如果API伺服器沒有回應、或是缺少「tridentbackends.trident.netapp.io` CRD」。這可能需要使用者介入。

在此階段、成功建立後端！還有多種作業可以額外處理、例如 "後端更新和後端刪除"。

（選用）步驟4：取得更多詳細資料

您可以執行下列命令來取得有關後端的詳細資訊：

```
kubectl -n trident get tbc backend-tbc-ontap-san -o wide
```

```
NAME                     BACKEND NAME        BACKEND UUID
PHASE   STATUS    STORAGE DRIVER   DELETION POLICY
backend-tbc-ontap-san   ontap-san-backend   8d24fce7-6f60-4d4a-8ef6-
bab2699e6ab8   Bound   Success   ontap-san        delete
```

此外、您也可以取得「TridentBackendConfig」的YAML/Json傾印。

```
$ kubectl -n trident get tbc backend-tbc-ontap-san -o yaml
```

```
apiVersion: trident.netapp.io/v1
kind: TridentBackendConfig
metadata:
  creationTimestamp: "2021-04-21T20:45:11Z"
  finalizers:
  - trident.netapp.io
  generation: 1
  name: backend-tbc-ontap-san
  namespace: trident
  resourceVersion: "947143"
  uid: 35b9d777-109f-43d5-8077-c74a4559d09c
spec:
  backendName: ontap-san-backend
  credentials:
    name: backend-tbc-ontap-san-secret
  managementLIF: 10.0.0.1
  dataLIF: 10.0.0.2
  storageDriverName: ontap-san
  svm: trident_svm
  version: 1
status:
  backendInfo:
    backendName: ontap-san-backend
    backendUUID: 8d24fce7-6f60-4d4a-8ef6-bab2699e6ab8
  deletionPolicy: delete
  lastOperationStatus: Success
  message: Backend 'ontap-san-backend' created
  phase: Bound
```

《backendInfo》（backendInfo）包含了針對「TridentBackendConfig」（TridentBackendConfig）的CR而建立的後端「backendName」（背端名稱）和「backendUUID」（背端UUID）。「lastoperationStatus」欄位代表上次執行的「TridentBackendConfig」（TridentBackendConfig）CR狀態、可由使用者觸發（例如、使用者在「show」中變更內容）、或由Astra Trident觸發（例如、Astra Trident重新啟動期間）。可能是「成功」或「失敗」。「階段」代表「TridentBackendConfig」與後端之間的關係狀態。在上述範例中、「階段」具有界限值、這表示「TridentBackendConfig」CR與後端相關聯。

您可以執行「kubeclt -n triident描述tbc <tbc-cr-name>」命令、以取得事件記錄的詳細資料。

> 您無法使用「tridentctl」來更新或刪除包含相關「TridentBackendConfig」物件的後端。若要瞭解在「tridentctl」和「TridentBackendConfig」之間切換的步驟、 "請參閱此處"。

以KECBECVL執行後端管理

瞭解如何使用「kubecll」來執行後端管理作業。

刪除後端

刪除「TridentBackendConfig」（TridentBackendConfig）之後、即指示Astra Trident刪除/保留後端（根據「刪除原則」）。若要刪除後端、請確定「刪除原則」已設定為刪除。如果只要刪除「TridentBackendConfig」、請確定「刪除原則」已設定為保留。這可確保後端仍存在、並可使用「tridentctl」進行管理。

執行下列命令：

```
$ kubectl delete tbc <tbc-name> -n trident
```

Astra Trident並不會刪除「TridentBackendConfig」所使用的Kubernetes Secrets。Kubernetes使用者負責清除機密。刪除機密時必須小心。只有在後端未使用機密時、才應刪除這些機密。

檢視現有的後端

執行下列命令：

```
$ kubectl get tbc -n trident
```

您也可以執行「tridentctl Get backend -n trident」或「tridentctl Get backend -o yaml -n trident」、以取得所有後端的清單。這份清單也會包含以「tridentctl」建立的後端。

更新後端

更新後端可能有多種原因：

- 儲存系統的認證資料已變更。若要更新認證資料、必須更新「TridentBackendConfig」物件中使用的Kubernetes Secret。Astra Trident會自動以提供的最新認證資料更新後端。執行下列命令以更新Kubernetes Secret：

```
$ kubectl apply -f <updated-secret-file.yaml> -n trident
```

- 需要ONTAP 更新參數（例如使用的SVM名稱）。在這種情況下、「TridentBackendConfig」物件可直接透過Kubernetes更新。

```
$ kubectl apply -f <updated-backend-file.yaml>
```

或者、您也可以執行下列命令來變更現有的「TridentBackendConfig」CR：

```
$ kubectl edit tbc <tbc-name> -n trident
```

如果後端更新失敗、後端仍會繼續維持其最後已知的組態。您可以執行「kubecl Get tbc <tbc-name>-o yaml -n trident」或「kubecl描述tbc <tbc-name>-n trident」來檢視記錄以判斷原因。

識別並修正組態檔的問題之後、即可重新執行update命令。

使用tridentctl執行後端管理

瞭解如何使用「tridentctl」來執行後端管理作業。

建立後端

建立之後 "後端組態檔"，執行下列命令：

```
$ tridentctl create backend -f <backend-file> -n trident
```

如果後端建立失敗、表示後端組態有問題。您可以執行下列命令來檢視記錄、以判斷原因：

```
$ tridentctl logs -n trident
```

識別並修正組態檔的問題之後、您只需再次執行「create」命令即可。

刪除後端

若要從Astra Trident刪除後端、請執行下列步驟：

1. 擷取後端名稱：

   ```
   $ tridentctl get backend -n trident
   ```

2. 刪除後端：

   ```
   $ tridentctl delete backend <backend-name> -n trident
   ```

> 如果Astra Trident已從這個後端配置磁碟區和快照、但該後端仍存在、則刪除後端會使新的磁碟區無法由其進行資源配置。後端將繼續處於「刪除」狀態、而Trident將繼續管理這些磁碟區和快照、直到它們被刪除為止。

檢視現有的後端

若要檢視Trident知道的後端、請執行下列步驟：

- 若要取得摘要、請執行下列命令：

  ```
  $ tridentctl get backend -n trident
  ```

- 若要取得所有詳細資料、請執行下列命令：

```
$ tridentctl get backend -o json -n trident
```

更新後端

建立新的後端組態檔之後、請執行下列命令：

```
$ tridentctl update backend <backend-name> -f <backend-file> -n trident
```

如果後端更新失敗、表示後端組態有問題、或是您嘗試了無效的更新。您可以執行下列命令來檢視記錄、以判斷原因：

```
$ tridentctl logs -n trident
```

識別並修正組態檔的問題之後、您只需再次執行「update」命令即可。

識別使用後端的儲存類別

這是您可以用Json回答的問題類型範例、其中的「tridentctl」會輸出後端物件。這會使用您需要安裝的「jq」公用程式。

```
$ tridentctl get backend -o json | jq '[.items[] | {backend: .name,
storageClasses: [.storage[].storageClasses]|unique}]'
```

這也適用於使用「TridentBackendConfig」建立的後端。

在後端管理選項之間切換

瞭解Astra Trident管理後端的不同方法。隨著「TridentBackendConfig」的推出、系統管理員現在有兩種獨特的後端管理方法。這會提出下列問題：

- 使用「tridentctl」建立的後端、是否能以「TridentBackendConfig」來管理？
- 使用「TridentBackendConfig」建立的後端、是否可以使用「tridentctl」來管理？

使用**TridentBackendConfig**管理**Tridentctl**後端

本節說明透過Kubernetes介面建立「TridentBackendConfig」物件、直接透過「tridentctl」建立的後端管理所需的步驟。

這將適用於下列案例：

- 由於使用「tridentctl」建立的後端沒有「TridentBackendConfig」、因此它們是預先存在的後端。

- 使用「tridentctl」建立的新後端、而其他「TridentBackendConfig」物件則存在。

在這兩種情況下、後端仍會繼續存在、Astra Trident排程磁碟區會繼續運作。系統管理員有兩種選擇之一：

- 繼續使用「tridentctl」來管理使用它建立的後端。
- 將使用「tridentctl」建立的後端連結至新的「TridentBackendConfig」物件。這樣做將意味着後端將使用“kubedl”而不是“tridentctl”來管理。

若要使用「kubecll」管理預先存在的後端、您需要建立連結至現有後端的「TridentBackendConfig」。以下是如何運作的總覽：

1. 建立Kubernetes機密。此機密包含Astra Trident與儲存叢集/服務通訊所需的認證資料。
2. 建立「TridentBackendConfig」物件。其中包含有關儲存叢集/服務的詳細資訊、並參考上一步建立的機密。必須謹慎指定相同的組態參數（例如「s.pec.backendName」、「sec.storagePrefix」、「sPEec.storageDriverName」等）。必須將「Pec.backendName」設定為現有後端的名稱。

步驟**0**：識別後端

若要建立連結至現有後端的「TridentBackendConfig」、您必須取得後端的組態。在此範例中、假設使用下列Json定義建立後端：

```
$ tridentctl get backend ontap-nas-backend -n trident
+---------------------+----------------
+--------------------------------------+--------+---------+
|          NAME       | STORAGE DRIVER |                 UUID
| STATE   | VOLUMES |
+---------------------+----------------
+--------------------------------------+--------+---------+
| ontap-nas-backend   | ontap-nas      | 52f2eb10-e4c6-4160-99fc-
96b3be5ab5d7 | online |      25 |
+---------------------+----------------
+--------------------------------------+--------+---------+

$ cat ontap-nas-backend.json

{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "managementLIF": "10.10.10.1",
    "dataLIF": "10.10.10.2",
    "backendName": "ontap-nas-backend",
    "svm": "trident_svm",
    "username": "cluster-admin",
    "password": "admin-password",

    "defaults": {
        "spaceReserve": "none",
```

```
        "encryption": "false"
    },
    "labels":{"store":"nas_store"},
    "region": "us_east_1",
    "storage": [
        {
            "labels":{"app":"msoffice", "cost":"100"},
            "zone":"us_east_1a",
            "defaults": {
                "spaceReserve": "volume",
                "encryption": "true",
                "unixPermissions": "0755"
            }
        },
        {
            "labels":{"app":"mysqldb", "cost":"25"},
            "zone":"us_east_1d",
            "defaults": {
                "spaceReserve": "volume",
                "encryption": "false",
                "unixPermissions": "0775"
            }
        }
    ]
}
```

步驟1：建立Kubernetes機密

建立包含後端認證的秘密、如以下範例所示：

```
$ cat tbc-ontap-nas-backend-secret.yaml

apiVersion: v1
kind: Secret
metadata:
  name: ontap-nas-backend-secret
type: Opaque
stringData:
  username: cluster-admin
  passWord: admin-password

$ kubectl create -f tbc-ontap-nas-backend-secret.yaml -n trident
secret/backend-tbc-ontap-san-secret created
```

步驟2：建立TridentBackendConfig CR

下一步是建立一個「TridentBackendConfig」（TridentBackendConfig）CR、它會自動連結至現有的「ONTAP-NAS-backend」（如本範例所示）。確保符合下列要求：

- 相同的後端名稱是在「s.pec.backendName」中定義。
- 組態參數與原始後端相同。
- 虛擬儲存資源池（若有）必須維持與原始後端相同的順序。
- 認證資料是透過Kubernetes Secret提供、而非以純文字提供。

在這種情況下、「TridentBackendConfig」將會如下所示：

```
$ cat backend-tbc-ontap-nas.yaml
apiVersion: trident.netapp.io/v1
kind: TridentBackendConfig
metadata:
  name: tbc-ontap-nas-backend
spec:
  version: 1
  storageDriverName: ontap-nas
  managementLIF: 10.10.10.1
  dataLIF: 10.10.10.2
  backendName: ontap-nas-backend
  svm: trident_svm
  credentials:
    name: mysecret
  defaults:
    spaceReserve: none
    encryption: 'false'
  labels:
    store: nas_store
  region: us_east_1
  storage:
  - labels:
      app: msoffice
      cost: '100'
    zone: us_east_1a
    defaults:
      spaceReserve: volume
      encryption: 'true'
      unixPermissions: '0755'
  - labels:
      app: mysqldb
      cost: '25'
    zone: us_east_1d
    defaults:
      spaceReserve: volume
      encryption: 'false'
      unixPermissions: '0775'

$ kubectl create -f backend-tbc-ontap-nas.yaml -n trident
tridentbackendconfig.trident.netapp.io/tbc-ontap-nas-backend created
```

步驟3：確認TridentBackendConfig CR的狀態

在建立「TridentBackendConfig」之後、其階段必須是「綁定」。它也應反映與現有後端相同的後端名稱和UUID。

```
$ kubectl -n trident get tbc tbc-ontap-nas-backend -n trident
NAME                    BACKEND NAME          BACKEND UUID
PHASE   STATUS
tbc-ontap-nas-backend  ontap-nas-backend     52f2eb10-e4c6-4160-99fc-
96b3be5ab5d7   Bound   Success

#confirm that no new backends were created (i.e., TridentBackendConfig did
not end up creating a new backend)
$ tridentctl get backend -n trident
+----------------------+----------------
+--------------------------------------+--------+---------+
|          NAME        | STORAGE DRIVER |                  UUID
| STATE  | VOLUMES |
+----------------------+----------------
+--------------------------------------+--------+---------+
| ontap-nas-backend   | ontap-nas      | 52f2eb10-e4c6-4160-99fc-
96b3be5ab5d7 | online |      25 |
+----------------------+----------------
+--------------------------------------+--------+---------+
```

現在可以使用「tbc-ontap-nas-backend」「TridentBackendConfig」物件來完全管理後端。

使用tridentctl管理TridentBackendConfig後端

可以使用「tridentctl」來列出使用「TridentBackendConfig」建立的後端。此外、系統管理員也可以刪除「TridentBackendConfig」、並確定「pec.deletionPolicy`」設為「效能」、藉此選擇透過「tridentctl」來完全管理此類後端。

步驟**0**：識別後端

例如、假設使用「TridentBackendConfig」建立下列後端：

```
$ kubectl get tbc backend-tbc-ontap-san -n trident -o wide
NAME                     BACKEND NAME        BACKEND UUID
PHASE   STATUS    STORAGE DRIVER   DELETION POLICY
backend-tbc-ontap-san   ontap-san-backend   81abcb27-ea63-49bb-b606-
0a5315ac5f82   Bound   Success   ontap-san        delete

$ tridentctl get backend ontap-san-backend -n trident
+-------------------+----------------
+--------------------------------------+--------+---------+
|       NAME        | STORAGE DRIVER |                 UUID
| STATE  | VOLUMES |
+-------------------+----------------
+--------------------------------------+--------+---------+
| ontap-san-backend | ontap-san      | 81abcb27-ea63-49bb-b606-
0a5315ac5f82 | online |      33 |
+-------------------+----------------
+--------------------------------------+--------+---------+
```

從輸出中可以看到、「TridentBackendConfig」已成功建立、並繫結至後端（觀察後端的UUID）。

步驟**1**：確認刪除原則已設定為保留

讓我們來看看「改革政策」的價值。這需要設定為「維護」。這將確保刪除「TridentBackendConfig」（TridentBackendConfig）的CR時、後端定義仍會存在、而且可以使用「tridentctl」進行管理。

```
$ kubectl get tbc backend-tbc-ontap-san -n trident -o wide
NAME                     BACKEND NAME        BACKEND UUID
PHASE   STATUS    STORAGE DRIVER   DELETION POLICY
backend-tbc-ontap-san   ontap-san-backend   81abcb27-ea63-49bb-b606-
0a5315ac5f82   Bound   Success   ontap-san        delete

# Patch value of deletionPolicy to retain
$ kubectl patch tbc backend-tbc-ontap-san --type=merge -p
'{"spec":{"deletionPolicy":"retain"}}' -n trident
tridentbackendconfig.trident.netapp.io/backend-tbc-ontap-san patched

#Confirm the value of deletionPolicy
$ kubectl get tbc backend-tbc-ontap-san -n trident -o wide
NAME                     BACKEND NAME        BACKEND UUID
PHASE   STATUS    STORAGE DRIVER   DELETION POLICY
backend-tbc-ontap-san   ontap-san-backend   81abcb27-ea63-49bb-b606-
0a5315ac5f82   Bound   Success   ontap-san        retain
```

除非將「刪除原則」設定為「需要」、否則請勿繼續下一步。

步驟2：刪除TridentBackendConfig CR

最後一個步驟是刪除「TridentBackendConfig」（TridentBackendConfig）。確認「刪除原則」設為「保留」之後、您可以繼續刪除：

```
$ kubectl delete tbc backend-tbc-ontap-san -n trident
tridentbackendconfig.trident.netapp.io "backend-tbc-ontap-san" deleted

$ tridentctl get backend ontap-san-backend -n trident
+-------------------+----------------
+--------------------------------------+--------+---------+
|       NAME        | STORAGE DRIVER |                 UUID
| STATE  | VOLUMES |
+-------------------+----------------
+--------------------------------------+--------+---------+
| ontap-san-backend | ontap-san      | 81abcb27-ea63-49bb-b606-
0a5315ac5f82 | online |      33 |
+-------------------+----------------
+--------------------------------------+--------+---------+
```

刪除「TridentBackendConfig」物件之後、Astra Trident便會移除該物件、而不會實際刪除後端本身。

管理儲存類別

尋找建立儲存類別、刪除儲存類別及檢視現有儲存類別的相關資訊。

設計儲存類別

請參閱 "請按這裡" 以取得有關儲存類別及其設定方式的詳細資訊。

建立儲存類別

取得儲存類別檔案之後、請執行下列命令：

```
kubectl create -f <storage-class-file>
```

「<storage－class file」（儲存設備類別檔案）應以您的儲存設備類別檔案名稱取代。

刪除儲存類別

若要從Kubernetes刪除儲存類別、請執行下列命令：

```
kubectl delete storageclass <storage-class>
```

「<storage－class>'」應改用您的儲存類別。

透過此儲存類別所建立的任何持續磁碟區都將維持不變、Astra Trident將繼續管理這些磁碟區。

> Astra Trident會對其建立的磁碟區執行空白的「FSType」。對於iSCSI後端、建議在StorageClass中強制執行「參數.FSType」。您應該刪除「esixing StorageClass」、然後使用指定的「參數.FSType」重新建立。

檢視現有的儲存類別

- 若要檢視現有的Kubernetes儲存類別、請執行下列命令：

```
kubectl get storageclass
```

- 若要檢視Kubernetes儲存類別詳細資料、請執行下列命令：

```
kubectl get storageclass <storage-class> -o json
```

- 若要檢視Astra Trident的同步儲存類別、請執行下列命令：

```
tridentctl get storageclass
```

- 若要檢視Astra Trident的同步儲存類別詳細資料、請執行下列命令：

```
tridentctl get storageclass <storage-class> -o json
```

設定預設儲存類別

Kubernetes 1.6新增了設定預設儲存類別的功能。如果使用者未在「持續磁碟區宣告」（PVc）中指定一個、則此儲存類別將用於配置「持續磁碟區」。

- 在儲存類別定義中、將「shorageclass.Kubernetes.IO/as-default-Class」註釋設為true、以定義預設儲存類別。根據規格、任何其他值或不存在附註都會解譯為假。
- 您可以使用下列命令、將現有的儲存類別設定為預設的儲存類別：

```
kubectl patch storageclass <storage-class-name> -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```

- 同樣地、您也可以使用下列命令移除預設儲存類別註釋：

```
kubectl patch storageclass <storage-class-name> -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-class":"false"}}}'
```

Trident安裝程式套件中也有包含此附註的範例。

> 在任何指定時間、您的叢集中都只應有一個預設儲存類別。Kubernetes在技術上並不妨礙您擁有多個儲存類別、但它的行為方式就如同完全沒有預設的儲存類別一樣。

識別儲存類別的後端

這是您可以用Json回答的問題類型範例、其中的「tridentctl」輸出用於Astra Trident後端物件。這會使用「jq」公用程式、您可能需要先安裝。

```
tridentctl get storageclass -o json | jq  '[.items[] | {storageClass:
.Config.name, backends: [.storage]|unique}]'
```

執行Volume作業

深入瞭解Astra Trident提供的功能、協助您管理磁碟區。

- "使用「csi拓撲」"
- "使用快照"
- "展開Volume"
- "匯入磁碟區"

使用「csi拓撲」

Astra Trident可以利用、選擇性地建立磁碟區、並將磁碟區附加至Kubernetes叢集中的節點 "「csi拓撲」功能"。使用「csi拓撲」功能、可根據區域和可用性區域、限制對磁碟區的存取、只能存取一部分節點。如今、雲端供應商可讓Kubernetes管理員建立以區域為基礎的節點。節點可位於某個區域內的不同可用度區域、或位於不同區域之間。為了協助在多區域架構中配置工作負載的磁碟區、Astra Trident使用了csi拓撲。

> 深入瞭解「csi拓撲」功能 "請按這裡"。

Kubernetes提供兩種獨特的Volume繫結模式：

- Astra Trident將「Volume BindingMode」設為「Immediate（立即）」、可在沒有任何拓撲感知的情況下建立磁碟區。建立永久虛擬磁碟時、即會處理磁碟區繫結和動態資源配置。這是預設的「Volume BindingMode」、適用於未強制拓撲限制的叢集。建立永續性磁碟區時、不會對要求的Pod排程需求有任何相依性。
- 將「Volume BindingMode」設為「WaitForFirst消費者」時、會延遲建立和繫結永久磁碟區、直到排程並建立使用永久磁碟的Pod為止。如此一來、就能建立磁碟區、以符合拓撲需求所強制執行的排程限制。

> 「等待使用者」繫結模式不需要拓撲標籤。這可獨立於「csi拓撲」功能使用。

若要使用「csi拓撲」、您需要下列項目：

- 執行1.17或更新版本的Kubernetes叢集。

```
$ kubectl version
Client Version: version.Info{Major:"1", Minor:"19",
GitVersion:"v1.19.3",
GitCommit:"1e11e4a2108024935ecfcb2912226cedeafd99df",
GitTreeState:"clean", BuildDate:"2020-10-14T12:50:19Z",
GoVersion:"go1.15.2", Compiler:"gc", Platform:"linux/amd64"}
Server Version: version.Info{Major:"1", Minor:"19",
GitVersion:"v1.19.3",
GitCommit:"1e11e4a2108024935ecfcb2912226cedeafd99df",
GitTreeState:"clean", BuildDate:"2020-10-14T12:41:49Z",
GoVersion:"go1.15.2", Compiler:"gc", Platform:"linux/amd64"}
```

- 叢集中的節點應該有標籤來介紹拓撲認知（「topology、Kubernetes.io/sregion」和「topology、Kubernetes.io/Zone」）。在安裝Astra Trident以識別拓撲之前、這些標籤*應該會出現在叢集*的節點上。

```
$ kubectl get nodes -o=jsonpath='{range .items[*]}[{.metadata.name},
{.metadata.labels}]{"\n"}{end}' | grep --color "topology.kubernetes.io"
[node1,
{"beta.kubernetes.io/arch":"amd64","beta.kubernetes.io/os":"linux","kube
rnetes.io/arch":"amd64","kubernetes.io/hostname":"node1","kubernetes.io/
os":"linux","node-
role.kubernetes.io/master":"","topology.kubernetes.io/region":"us-
east1","topology.kubernetes.io/zone":"us-east1-a"}]
[node2,
{"beta.kubernetes.io/arch":"amd64","beta.kubernetes.io/os":"linux","kube
rnetes.io/arch":"amd64","kubernetes.io/hostname":"node2","kubernetes.io/
os":"linux","node-
role.kubernetes.io/worker":"","topology.kubernetes.io/region":"us-
east1","topology.kubernetes.io/zone":"us-east1-b"}]
[node3,
{"beta.kubernetes.io/arch":"amd64","beta.kubernetes.io/os":"linux","kube
rnetes.io/arch":"amd64","kubernetes.io/hostname":"node3","kubernetes.io/
os":"linux","node-
role.kubernetes.io/worker":"","topology.kubernetes.io/region":"us-
east1","topology.kubernetes.io/zone":"us-east1-c"}]
```

步驟**1**：建立可感知拓撲的後端

Astra Trident儲存後端可根據可用性區域、選擇性地配置磁碟區。每個後端都可以攜帶一個可選的「支援拓撲」區塊、代表必須支援的區域和區域清單。對於使用此類後端的StorageClass、只有在受支援地區/區域中排程的應用程式要求時、才會建立Volume。

以下是後端定義的範例：

```
{
 "version": 1,
 "storageDriverName": "ontap-san",
 "backendName": "san-backend-us-east1",
 "managementLIF": "192.168.27.5",
 "svm": "iscsi_svm",
 "username": "admin",
 "password": "xxxxxxxxxxxx",
 "supportedTopologies": [
{"topology.kubernetes.io/region": "us-east1",
"topology.kubernetes.io/zone": "us-east1-a"},
{"topology.kubernetes.io/region": "us-east1",
"topology.kubernetes.io/zone": "us-east1-b"}
]
}
```

「支援拓撲」是用來提供每個後端的區域和區域清單。這些區域和區域代表StorageClass中可提供的允許值清單。對於包含後端所提供之區域和區域子集的StorageClass、Astra Trident會在後端建立磁碟區。

您也可以定義每個儲存資源池的「支援拓撲」。請參閱下列範例：

```
{"version": 1,
"storageDriverName": "ontap-nas",
"backendName": "nas-backend-us-central1",
"managementLIF": "172.16.238.5",
"svm": "nfs_svm",
"username": "admin",
"password": "Netapp123",
"supportedTopologies": [
      {"topology.kubernetes.io/region": "us-central1",
"topology.kubernetes.io/zone": "us-central1-a"},
      {"topology.kubernetes.io/region": "us-central1",
"topology.kubernetes.io/zone": "us-central1-b"}
    ]
"storage": [
   {
       "labels": {"workload":"production"},
        "region": "Iowa-DC",
        "zone": "Iowa-DC-A",
        "supportedTopologies": [
            {"topology.kubernetes.io/region": "us-central1",
"topology.kubernetes.io/zone": "us-central1-a"}
        ]
    },
    {
        "labels": {"workload":"dev"},
         "region": "Iowa-DC",
         "zone": "Iowa-DC-B",
         "supportedTopologies": [
             {"topology.kubernetes.io/region": "us-central1",
"topology.kubernetes.io/zone": "us-central1-b"}
         ]
     }
]
}
```

在此範例中、「REGion」和「Zone」標籤代表儲存資源池的位置。「topology、Kubernetes.io/region」和「topology、Kubernetes.io/Zone」決定儲存資源池的使用來源。

步驟**2**：定義可感知拓撲的**StorageClass**

根據提供給叢集中節點的拓撲標籤、可以定義StorageClass以包含拓撲資訊。這將決定做為所提出之永久虛擬磁碟要求候選的儲存資源池、以及可以使用Trident所提供之磁碟區的節點子集。

請參閱下列範例：

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
name: netapp-san-us-east1
provisioner: csi.trident.netapp.io
volumeBindingMode: WaitForFirstConsumer
allowedTopologies:
- matchLabelExpressions:
- key: topology.kubernetes.io/zone
  values:
  - us-east1-a
  - us-east1-b
- key: topology.kubernetes.io/region
  values:
  - us-east1
parameters:
  fsType: "ext4"
```

在上述StorageClass定義中、「volumeBindingMode」設定為「WaitForFirst消費者」。在Pod中引用此StorageClass所要求的PVCS之前、系統不會對其採取行動。此外、「opedtopologies」也提供要使用的區域和區域。《NetApp-san—use-east1》StorageClass將在上述定義的「SAN後端-us-east1」後端上建立PVCS。

步驟**3**：建立並使用**PVC**

建立StorageClass並對應至後端後端後端之後、您現在就可以建立PVCS。

請參閱以下「sPEC」範例：

```
---
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
name: pvc-san
spec:
accessModes:
  - ReadWriteOnce
resources:
  requests:
    storage: 300Mi
storageClassName: netapp-san-us-east1
```

使用此資訊清單建立永久虛擬環境可能會產生下列結果：

```
$ kubectl create -f pvc.yaml
persistentvolumeclaim/pvc-san created
$ kubectl get pvc
NAME      STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS
AGE
pvc-san   Pending                                      netapp-san-us-east1
2s
$ kubectl describe pvc
Name:          pvc-san
Namespace:     default
StorageClass:  netapp-san-us-east1
Status:        Pending
Volume:
Labels:        <none>
Annotations:   <none>
Finalizers:    [kubernetes.io/pvc-protection]
Capacity:
Access Modes:
VolumeMode:    Filesystem
Mounted By:    <none>
Events:
  Type    Reason                Age   From                         Message
  ----    ------                ----  ----                         -------
  Normal  WaitForFirstConsumer  6s    persistentvolume-controller  waiting
for first consumer to be created before binding
```

若要Trident建立磁碟區並將其連結至PVc、請在Pod中使用PVc。請參閱下列範例：

```
apiVersion: v1
kind: Pod
metadata:
  name: app-pod-1
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: topology.kubernetes.io/region
            operator: In
            values:
            - us-east1
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 1
        preference:
          matchExpressions:
          - key: topology.kubernetes.io/zone
            operator: In
            values:
            - us-east1-a
            - us-east1-b
  securityContext:
    runAsUser: 1000
    runAsGroup: 3000
    fsGroup: 2000
  volumes:
  - name: vol1
    persistentVolumeClaim:
      claimName: pvc-san
  containers:
  - name: sec-ctx-demo
    image: busybox
    command: [ "sh", "-c", "sleep 1h" ]
    volumeMounts:
    - name: vol1
      mountPath: /data/demo
    securityContext:
      allowPrivilegeEscalation: false
```

此pod化 規範會指示Kubernetes在「us-east1」區域的節點上排程pod、並從「us-east1-a」或「us-east1-b」區域中的任何節點中進行選擇。

請參閱下列輸出：

```
$ kubectl get pods -o wide
NAME        READY   STATUS    RESTARTS   AGE   IP               NODE
NOMINATED NODE   READINESS GATES
app-pod-1   1/1     Running   0          19s   192.168.25.131   node2
<none>           <none>
$ kubectl get pvc -o wide
NAME      STATUS   VOLUME                                     CAPACITY
ACCESS MODES   STORAGECLASS          AGE   VOLUMEMODE
pvc-san   Bound    pvc-ecb1e1a0-840c-463b-8b65-b3d033e2e62b   300Mi
RWO            netapp-san-us-east1   48s   Filesystem
```

更新後端以納入支援的拓撲

您可以使用「tridentctl後端更新」來更新現有的後端、以納入「最上層拓撲」清單。這不會影響已配置的磁碟區、而且只會用於後續的PVCS。

如需詳細資訊、請參閱

- "管理容器的資源"
- "節點選取器"
- "關聯性與反關聯性"
- "污染與容許"

使用快照

從2001版Astra Trident開始、您可以在Kubernetes層建立PV快照。您可以使用這些快照來維護由Astra Trident所建立之磁碟區的時間點複本、並排程建立其他磁碟區（複本）。Volume Snapshot可支援「ONTAP-NAS」、「ONTAP-SAN」、「ONTAP-SAN經濟」、「Poolidfire - SAN」、「GCP - CVS」、 以及「azure-NetApp-file-fil」驅動程式。

> 此功能可從Kubernetes 1.17（試用版）取得、GA版本為1.20。若要瞭解從試用版移轉至GA所涉及的變更、請參閱 "版本部落格"。隨着GA的問世、推出了「v1」API版本、並向下相容於「v1Beta1」快照。

您需要的是 **#8217** ；需要的是什麼

- 建立Volume快照需要建立外部快照控制器和自訂資源定義（CRD）。這是正在使用的Kubernetes Orchestrator的責任（例如：Kubeadm、GKE、OpenShift）。

如果您的Kubernetes發佈版本未包含快照控制器和客戶需求日、您可以依照下列方式進行部署。

1. 建立Volume Snapshot客戶需求日。

   對於Kubernetes 1.20及更新版本、請使用v1 Snapshot CRD搭配v5.0或更新版本的Snapshot元件。對於Kubernetes 1.18和1.19版、請使用v1beta1搭配v3.0.3快照元件。

V5.0元件

```
$ cat snapshot-setup.sh
#!/bin/bash
# Create volume snapshot CRDs
kubectl apply -f https://raw.githubusercontent.com/kubernetes-
csi/external-snapshotter/release-
5.0/client/config/crd/snapshot.storage.k8s.io_volumesnapshotclasses.
yaml
kubectl apply -f https://raw.githubusercontent.com/kubernetes-
csi/external-snapshotter/release-
5.0/client/config/crd/snapshot.storage.k8s.io_volumesnapshotcontents
.yaml
kubectl apply -f https://raw.githubusercontent.com/kubernetes-
csi/external-snapshotter/release-
5.0/client/config/crd/snapshot.storage.k8s.io_volumesnapshots.yaml
```

v3.0.3元件

```
$ cat snapshot-setup.sh
#!/bin/bash
# Create volume snapshot CRDs
kubectl apply -f https://raw.githubusercontent.com/kubernetes-
csi/external-
snapshotter/v3.0.3/client/config/crd/snapshot.storage.k8s.io_volumes
napshotclasses.yaml
kubectl apply -f https://raw.githubusercontent.com/kubernetes-
csi/external-
snapshotter/v3.0.3/client/config/crd/snapshot.storage.k8s.io_volumes
napshotcontents.yaml
kubectl apply -f https://raw.githubusercontent.com/kubernetes-
csi/external-
snapshotter/v3.0.3/client/config/crd/snapshot.storage.k8s.io_volumes
napshots.yaml
```

2. 在所需的命名空間中建立Snapshot控制器。編輯下方的Yaml清單以修改命名空間。

   對於Kubernetes 1.20及更新版本、請使用5.0版或更新版本。對於Kubernetes 1.18和1.19版、請使用v3.0.3

   > 如果在GKE環境中設定隨需磁碟區快照、請勿建立快照控制器。GKE使用內建的隱藏式快照控制器。

V5.0控制器

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes-csi/external-snapshotter/release-5.0/deploy/kubernetes/snapshot-controller/rbac-snapshot-controller.yaml
kubectl apply -f https://raw.githubusercontent.com/kubernetes-csi/external-snapshotter/release-5.0/deploy/kubernetes/snapshot-controller/setup-snapshot-controller.yaml
```

v3.0.3控制器

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes-csi/external-snapshotter/v3.0.3/deploy/kubernetes/snapshot-controller/rbac-snapshot-controller.yaml
kubectl apply -f https://raw.githubusercontent.com/kubernetes-csi/external-snapshotter/v3.0.3/deploy/kubernetes/snapshot-controller/setup-snapshot-controller.yaml
```

「csi Snapshotter」提供 "正在驗證Webhook" 協助使用者驗證現有的v1Beta1快照、並確認它們是有效的資源物件。驗證Webhook會自動標示無效的快照物件、並防止未來建立無效物件。驗證Webhook是由Kubernetes Orchestrator部署。請參閱指示以手動部署驗證Webhook "請按這裡"。尋找無效快照資訊清單的範例 "請按這裡"。

以下範例說明使用快照所需的架構、並說明如何建立及使用快照。

步驟1：設定Volume SnapshotClass

在建立Volume Snapshot之前、請先設定 "d7ca7162c394dee752c35d07a92823da"。

```
$ cat snap-sc.yaml
#Use apiVersion v1 for Kubernetes 1.20 and above. For Kubernetes 1.18 and 1.19, use apiVersion v1beta1.
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshotClass
metadata:
  name: csi-snapclass
driver: csi.trident.netapp.io
deletionPolicy: Delete
```

《driver》指的是Astra Trident的csi驅動程式。「刪除原則」可以是「刪除」或「保留」。當設定為「保留」時、即使刪除「Volume Snapshot」物件、也會保留儲存叢集上的基礎實體快照。

步驟**2**：建立現有**PVc**的快照

```
$ cat snap.yaml
#Use apiVersion v1 for Kubernetes 1.20 and above. For Kubernetes 1.18 and
1.19, use apiVersion v1beta1.
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: pvc1-snap
spec:
  volumeSnapshotClassName: csi-snapclass
  source:
    persistentVolumeClaimName: pvc1
```

快照是針對名為「pvc1」的永久虛擬資料建立、快照名稱則設為「pvc1-snap」。

```
$ kubectl create -f snap.yaml
volumesnapshot.snapshot.storage.k8s.io/pvc1-snap created

$ kubectl get volumesnapshots
NAME                   AGE
pvc1-snap              50s
```

這會建立一個「Volume Snapshot」物件。Volume Snapshot類似於一個PVC、並與代表實際快照的「Volume SnapshotContent」物件相關聯。

您可以透過描述「pvc1-snap」Volume Snapshot的「Volume SnapshotContent」物件來識別該物件。

```
$ kubectl describe volumesnapshots pvc1-snap
Name:         pvc1-snap
Namespace:    default
.
.
.
Spec:
  Snapshot Class Name:    pvc1-snap
  Snapshot Content Name:  snapcontent-e8d8a0ca-9826-11e9-9807-525400f3f660
  Source:
    API Group:
    Kind:       PersistentVolumeClaim
    Name:       pvc1
Status:
  Creation Time:  2019-06-26T15:27:29Z
  Ready To Use:   true
  Restore Size:   3Gi
.
.
```

「快照內容名稱」可識別提供此快照的Volume SnapshotContent物件。「Ready to use」（準備使用）參數表示Snapshot可用於建立新的PVc。

步驟3：從Volume Snapshot建立PVCS

請參閱下列範例、瞭解如何使用快照建立永久虛擬資料：

```
$ cat pvc-from-snap.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-from-snap
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: golden
  resources:
    requests:
      storage: 3Gi
  dataSource:
    name: pvc1-snap
    kind: VolumeSnapshot
    apiGroup: snapshot.storage.k8s.io
```

「Data來源」顯示、必須使用名為「pvc1-snap」的Volume Snapshot建立PVc作為資料來源。這會指示Astra Trident從快照建立一個永久虛擬資料。建立好永久虛擬基礎架構之後、就能將它附加到Pod上、就像使用任何其他永久虛擬基礎架構一樣使用。

> 刪除具有相關快照的持續Volume時、對應的Trident Volume會更新為「刪除狀態」。若要刪除Astra Trident磁碟區、則應移除該磁碟區的快照。

如需詳細資訊、請參閱

- "Volume快照"
- "d7ca7162c394dee752c35d07a92823da"

展開Volume

Astra Trident可讓Kubernetes使用者在建立磁碟區之後擴充磁碟區。尋找擴充iSCSI和NFS磁碟區所需組態的相關資訊。

展開iSCSI Volume

您可以使用「SCSI資源配置程式」來擴充iSCSI持續磁碟區（PV）。

> iSCSI磁碟區擴充支援「ontap－san」、「ONTAP－san經濟」、「Poolidfire－san」等驅動程式、需要Kubernetes 1.16及更新版本。

擴充iSCSI PV包括下列步驟：

- 編輯StorageClass定義、將「owvolVolume Expansion」欄位設定為「true」。
- 編輯PVC定義並更新「s.pec.estices.storage」、以反映新的所需大小、此大小必須大於原始大小。
- 必須將PV附加至Pod、才能調整其大小。調整iSCSI PV的大小有兩種情況：
  - 如果PV附加至Pod、Astra Trident會在儲存後端擴充磁碟區、重新掃描裝置、並重新調整檔案系統的大小。
  - 嘗試調整未附加PV的大小時、Astra Trident會在儲存後端上擴充磁碟區。在將永久虛擬磁碟綁定至Pod之後、Trident會重新掃描裝置並重新調整檔案系統的大小。然後、Kubernetes會在擴充作業成功完成後、更新PVc大小。

以下範例顯示擴充iSCSI PV的運作方式。

步驟1：設定StorageClass以支援Volume擴充

```
$ cat storageclass-ontapsan.yaml
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ontap-san
provisioner: csi.trident.netapp.io
parameters:
  backendType: "ontap-san"
allowVolumeExpansion: True
```

對於已存在的StorageClass、請編輯此類以包含「owalumVolume Expansion」參數。

步驟**2**：使用您建立的**StorageClass**建立一個永久虛擬儲存設備

```
$ cat pvc-ontapsan.yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: san-pvc
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: ontap-san
```

Astra Trident會建立持續磁碟區（PV）、並將其與此持續磁碟區宣告（PVc）建立關聯。

```
$ kubectl get pvc
NAME      STATUS   VOLUME                                     CAPACITY
ACCESS MODES   STORAGECLASS   AGE
san-pvc   Bound    pvc-8a814d62-bd58-4253-b0d1-82f2885db671   1Gi
RWO            ontap-san      8s

$ kubectl get pv
NAME                                       CAPACITY   ACCESS MODES
RECLAIM POLICY   STATUS   CLAIM             STORAGECLASS   REASON   AGE
pvc-8a814d62-bd58-4253-b0d1-82f2885db671   1Gi        RWO
Delete           Bound    default/san-pvc   ontap-san               10s
```

步驟3：定義一個連接至PVc的Pod

在此範例中、會建立使用「shan -PVc」的Pod。

```
$  kubectl get pod
NAME         READY   STATUS    RESTARTS   AGE
centos-pod   1/1     Running   0          65s

$  kubectl describe pvc san-pvc
Name:          san-pvc
Namespace:     default
StorageClass:  ontap-san
Status:        Bound
Volume:        pvc-8a814d62-bd58-4253-b0d1-82f2885db671
Labels:        <none>
Annotations:   pv.kubernetes.io/bind-completed: yes
               pv.kubernetes.io/bound-by-controller: yes
               volume.beta.kubernetes.io/storage-provisioner:
csi.trident.netapp.io
Finalizers:    [kubernetes.io/pvc-protection]
Capacity:      1Gi
Access Modes:  RWO
VolumeMode:    Filesystem
Mounted By:    centos-pod
```

步驟4：展開PV

若要調整從1Gi建立至2Gi的PV大小、請編輯PVc定義、並將「sec.resumes.requests.storage」更新為2Gi。

```
$ kubectl edit pvc san-pvc
# Please edit the object below. Lines beginning with a '#' will be
ignored,
# and an empty file will abort the edit. If an error occurs while saving
this file will be
# reopened with the relevant failures.
#
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  annotations:
    pv.kubernetes.io/bind-completed: "yes"
    pv.kubernetes.io/bound-by-controller: "yes"
    volume.beta.kubernetes.io/storage-provisioner: csi.trident.netapp.io
  creationTimestamp: "2019-10-10T17:32:29Z"
  finalizers:
  - kubernetes.io/pvc-protection
  name: san-pvc
  namespace: default
  resourceVersion: "16609"
  selfLink: /api/v1/namespaces/default/persistentvolumeclaims/san-pvc
  uid: 8a814d62-bd58-4253-b0d1-82f2885db671
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi
 ...
```

步驟5：驗證擴充

您可以檢查PVc、PV和Astra Trident Volume的大小、以正確驗證擴充作業：

```
$ kubectl get pvc san-pvc
NAME      STATUS   VOLUME                                     CAPACITY
ACCESS MODES   STORAGECLASS   AGE
san-pvc   Bound    pvc-8a814d62-bd58-4253-b0d1-82f2885db671   2Gi
RWO            ontap-san      11m
$ kubectl get pv
NAME                                       CAPACITY   ACCESS MODES
RECLAIM POLICY   STATUS   CLAIM             STORAGECLASS   REASON   AGE
pvc-8a814d62-bd58-4253-b0d1-82f2885db671   2Gi        RWO
Delete           Bound    default/san-pvc   ontap-san               12m
$ tridentctl get volumes -n trident
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE   | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-8a814d62-bd58-4253-b0d1-82f2885db671 | 2.0 GiB | ontap-san     |
block    | a9b7bfff-0505-4e31-b6c5-59f492e02d33 | online | true    |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
```

展開NFS Volume

Astra Trident支援在「ONTAP-NAS」、「ONTAP-NAs-P節約」、「ONTAP-NAs-flexgroup」、「GCP-CVS」和「azure-NetApp-fils」後端配置NFS PV的Volume擴充。

步驟1：設定StorageClass以支援Volume擴充

若要調整NFS PV的大小、管理員必須先將「ow淺Volume Expansion」欄位設定為「true」、以設定儲存類別以允許磁碟區擴充：

```
$ cat storageclass-ontapnas.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ontapnas
provisioner: csi.trident.netapp.io
parameters:
  backendType: ontap-nas
allowVolumeExpansion: true
```

如果您已經建立了沒有此選項的儲存類別、只要使用「kubecl Edit storageclass'」來編輯現有的儲存類別、即可進行磁碟區擴充。

步驟**2**：使用您建立的**StorageClass**建立一個永久虛擬儲存設備

```
$ cat pvc-ontapnas.yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: ontapnas20mb
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 20Mi
  storageClassName: ontapnas
```

Astra Trident應為此PVC建立20MiB NFS PV：

```
$ kubectl get pvc
NAME           STATUS   VOLUME
CAPACITY     ACCESS MODES   STORAGECLASS    AGE
ontapnas20mb   Bound    pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7   20Mi
RWO            ontapnas        9s

$ kubectl get pv pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7
NAME                                       CAPACITY   ACCESS MODES
RECLAIM POLICY   STATUS   CLAIM                  STORAGECLASS    REASON
AGE
pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7   20Mi       RWO
Delete           Bound    default/ontapnas20mb   ontapnas
2m42s
```

步驟**3**：展開**PV**

若要將新建立的20MiB PV調整至1GiB、請編輯PVC並將「siec.resumes.requests.storage」設為1GB：

```
$ kubectl edit pvc ontapnas20mb
# Please edit the object below. Lines beginning with a '#' will be
ignored,
# and an empty file will abort the edit. If an error occurs while saving
this file will be
# reopened with the relevant failures.
#
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  annotations:
    pv.kubernetes.io/bind-completed: "yes"
    pv.kubernetes.io/bound-by-controller: "yes"
    volume.beta.kubernetes.io/storage-provisioner: csi.trident.netapp.io
  creationTimestamp: 2018-08-21T18:26:44Z
  finalizers:
  - kubernetes.io/pvc-protection
  name: ontapnas20mb
  namespace: default
  resourceVersion: "1958015"
  selfLink: /api/v1/namespaces/default/persistentvolumeclaims/ontapnas20mb
  uid: c1bd7fa5-a56f-11e8-b8d7-fa163e59eaab
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
...
```

步驟4：驗證擴充

您可以檢查PVc、PV和Astra Trident Volume的大小、以正確驗證調整大小：

```
$ kubectl get pvc ontapnas20mb
NAME           STATUS   VOLUME
CAPACITY   ACCESS MODES   STORAGECLASS    AGE
ontapnas20mb   Bound    pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7   1Gi
RWO            ontapnas        4m44s

$ kubectl get pv pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7
NAME                                       CAPACITY   ACCESS MODES
RECLAIM POLICY   STATUS   CLAIM                  STORAGECLASS    REASON
AGE
pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7   1Gi        RWO
Delete           Bound    default/ontapnas20mb   ontapnas
5m35s

$ tridentctl get volume pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7 -n
trident
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE   | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-08f3d561-b199-11e9-8d9f-5254004dfdb7 | 1.0 GiB | ontapnas      |
file     | c5a6f6a4-b052-423b-80d4-8fb491a14a22 | online | true    |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
```

匯入磁碟區

您可以使用「tridentcl匯入」將現有的儲存磁碟區匯入為Kubernetes PV。

支援**Volume**匯入的驅動程式

下表說明支援匯入磁碟區的驅動程式、以及這些磁碟區所引進的版本。

驅動程式	版本
「ONTAP-NAS」	19.04
「ONTAP-NAA-flexgroup」	19.04
「olidfire - san」	19.04
《azure-NetApp-fil形》	19.04

驅動程式	版本
《GCP—CVS》	19.04
「ONTAP-SAN」	19.04

為什麼要匯入磁碟區？

將Volume匯入Trident的使用案例有多種：

- 容器化應用程式、並重新使用現有的資料集
- 將資料集的複本用於暫時性應用程式
- 重建故障的Kubernetes叢集
- 在災難恢復期間移轉應用程式資料

匯入如何運作？

Volume匯入程序會使用持續磁碟區宣告（PVc）檔案來建立PVc。至少、PVc檔案應包含名稱、命名空間、存取模式及storageClassName欄位、如下例所示。

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: my_claim
  namespace: my_namespace
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: my_storage_class
```

「tridentctl」用戶端用於匯入現有的儲存磁碟區。Trident會持續儲存Volume中繼資料並建立PVc和PV、以匯入Volume。

```
$ tridentctl import volume <backendName> <volumeName> -f <path-to-pvc-
file>
```

若要匯入儲存磁碟區、請指定包含該磁碟區的Astra Trident後端名稱、以及唯一識別儲存區上該磁碟區的名稱（例如ONTAP FlexVol ：Wsel, Element Volume、CVS Volume path）。儲存磁碟區必須允許讀取/寫入存取、且可由指定的Astra Trident後端存取。需要「-f]字串引數、並指定Yaml或Json PVc檔案的路徑。

當Astra Trident收到匯入磁碟區要求時、現有的磁碟區大小會在PVc中決定及設定。儲存驅動程式匯入磁碟區之後、PV會以PVc的ClaimRef建立。在PV中、回收原則一開始設定為「需要」。Kubernetes成功繫結了PVc和PV之後、系統會更新回收原則以符合儲存類別的回收原則。如果儲存類別的回收原則為「刪除」、則儲存磁碟區會在刪除PV時刪除。

當使用「-NO-Manage」引數匯入磁碟區時、Trident不會在物件生命週期的永久虛擬磁碟或PV上執行任何額外作業。由於Trident會忽略「無管理」物件的PV和PVc事件、因此在刪除PV時不會刪除儲存磁碟區。此外、也會忽略其他作業、例如Volume Clone和Volume resize。如果您想要將Kubernetes用於容器化工作負載、但想要管理Kubernetes以外儲存磁碟區的生命週期、則此選項非常實用。

將註釋新增至PVc和PV、這有兩種用途、表示已匯入磁碟區、以及是否管理了PVc和PV。不應修改或移除此附註。

Trident 19.07及更新版本可處理PV的附加元件、並在匯入磁碟區時掛載磁碟區。對於使用舊版Astra Trident的匯入、資料路徑不會有任何作業、而且磁碟區匯入不會驗證是否可以掛載磁碟區。如果在Volume匯入時發生錯誤（例如StorageClass不正確）、您可以將PV上的回收原則變更為「保留」、刪除PVC和PV、然後重新嘗試Volume匯入命令來恢復。

ONTAP-NAS和ONTAP-NAA-Flexgroup匯入

每個使用「ONTAP-NAS」驅動程式建立的Volume FlexVol 、都是ONTAP 一個在整個叢集上的功能。使用「ONTAP-NAS」驅動程式匯入FlexVols的運作方式相同。已存在於某個叢集上的一個功能、可以匯入為「ONTAP-NAS」的PVc。FlexVol ONTAP同樣地FlexGroup 、也可以將此數據匯入為「ONTAP-NAS-Flexgroup」PVCS。

> 若要由Trident匯入某個類型的Rw。ONTAP如果磁碟區是DP類型、則它是SnapMirror目的地磁碟區；在將磁碟區匯入Trident之前、您應該先中斷鏡射關係。

> 「ONTAP-NAS」驅動程式無法匯入及管理qtree。「ONTAP-NAS」和「ONTAP-NAS-flexgroup」驅動程式不允許重複的Volume名稱。

例如、若要在名為「ONTAP_NAS」的後端匯入名為「manended_volume」的磁碟區、請使用下列命令：

```
$ tridentctl import volume ontap_nas managed_volume -f <path-to-pvc-file>

+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE   | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-bf5ad463-afbb-11e9-8d9f-5254004dfdb7 | 1.0 GiB | standard      |
file     | c5a6f6a4-b052-423b-80d4-8fb491a14a22 | online | true    |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
```

若要匯入名為「Unmanaged _volume」（在「ONTAP_NAS後端」上）的磁碟區（Trident無法管理）、請使用下列命令：

```
$ tridentctl import volume nas_blog unmanaged_volume -f <path-to-pvc-file>
--no-manage

+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE   | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-df07d542-afbc-11e9-8d9f-5254004dfdb7 | 1.0 GiB | standard      |
file     | c5a6f6a4-b052-423b-80d4-8fb491a14a22 | online | false   |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
```

使用「-no－管理」引數時、Trident不會重新命名磁碟區、也不會驗證磁碟區是否已掛載。如果未手動掛載磁碟區、則磁碟區匯入作業會失敗。

> 已修正先前使用自訂Unix權限 匯入磁碟區的錯誤。您可以在您的PVc定義或後端組態中指定unixPermissions、並指示Astra Trident依此匯入磁碟區。

ONTAP-SAN匯入

Astra Trident也能匯入ONTAP 包含單一LUN的SAN FlexVols。這與「ONTAP-SAN」驅動程式一致、因為它會為FlexVol 每個實體磁碟和FlexVol 一個LUN在整個過程中建立一個版本。您可以使用「tridentctl import'」命令、方法與其他情況相同：

- 請加入「ONTAP-SAN」後端名稱。
- 請提供FlexVol 需要匯入的名稱。請記住FlexVol 、這個功能只包含一個必須匯入的LUN。
- 提供必須搭配「-f」旗標使用的PVc定義路徑。
- 您可以選擇管理或不受管理的永久虛擬網路。根據預設、Trident會管理PVc、並在FlexVol 後端重新命名該LUN。若要匯入為非託管磁碟區、請傳遞「-no－管理」旗標。

> 匯入未受管理的「ONTAP-SAN」Volume時、您應確定FlexVol 將位於該功能的LUN命名為「lun0」、並對應至具有所需啟動器的igroup。Astra Trident會自動處理這項作業、以便進行託管匯入。

然後Astra Trident會匯入FlexVol 該等物件、並將其與PVc定義建立關聯。Astra Trident也將FlexVol 該等數據重新命名為「PVC-<uuid>'格式、FlexVol 並將LUN重新命名為「lun0」。

> 建議匯入沒有現有作用中連線的磁碟區。如果您要匯入使用中的Volume、請先複製該Volume、然後再執行匯入。

範例

若要匯入FlexVol 在「ONTAP_san預設」後端上出現的「onta-san託管」功能、請執行「tridentctl匯入」命令：

```
$ tridentctl import volume ontapsan_san_default ontap-san-managed -f pvc-
basic-import.yaml -n trident -d

+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE  | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-d6ee4f54-4e40-4454-92fd-d00fc228d74a | 20 MiB | basic         |
block    | cd394786-ddd5-4470-adc3-10c5ce4ca757 | online | true    |
+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
```

若要由Astra Trident匯入某個類型的RW磁碟區。ONTAP如果磁碟區為DP類型、則為SnapMirror目的地磁碟區；您應該先中斷鏡射關係、再將磁碟區匯入Astra Trident。

元素匯入

您可以使用NetApp Element Trident將支援功能的軟體/NetApp HCI磁碟區匯入Kubernetes叢集。您需要Astra Trident後端的名稱、以及磁碟區的獨特名稱和PVc檔案、作為「tridentctl import'」命令的引數。

```
$ tridentctl import volume element_default element-managed -f pvc-basic-
import.yaml -n trident -d

+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE  | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-970ce1ca-2096-4ecd-8545-ac7edc24a8fe | 10 GiB | basic-element |
block    | d3ba047a-ea0b-43f9-9c42-e38e58301c49 | online | true    |
+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
```

Element驅動程式支援重複的Volume名稱。如果有重複的Volume名稱、Trident的Volume匯入程序會傳回錯誤。因應措施是複製磁碟區、並提供唯一的磁碟區名稱。然後匯入複製的Volume。

GCP-CVS匯入

若要匯入以NetApp Cloud Volumes Service 支援的GCP磁碟區、請使用磁碟區路徑來識別該磁碟區、而非其名稱。

若要在後端匯入名為「gcpcvs_YEppr」的「gcp-CVS」磁碟區、並將磁碟區路徑匯入為「adj統 法社-jolly Swift」、請使用下列命令：

```
$ tridentctl import volume gcpcvs_YEppr adroit-jolly-swift -f <path-to-
pvc-file> -n trident

+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE  | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-a46ccab7-44aa-4433-94b1-e47fc8c0fa55 | 93 GiB | gcp-storage   | file
| e1a6e65b-299e-4568-ad05-4f0a105c888f | online | true    |
+------------------------------------------+--------+---------------
+----------+--------------------------------------+--------+---------+
```

> Volume路徑是Volume匯出路徑的一部分、位於：/之後。例如、如果匯出路徑為「10.0.0.1：/統 法社-jolly Swift」、則Volume路徑為「adj統 法社-jolly Swift」。

Azure與NetApp檔案匯入

若要在後端匯入名為「azurenetappfiles_40517」的「azure-netapp-fil」磁碟區、並使用磁碟區路徑「importvol1」、請執行下列命令：

```
$ tridentctl import volume azurenetappfiles_40517 importvol1 -f <path-to-
pvc-file> -n trident

+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
|                   NAME                   |  SIZE   | STORAGE CLASS |
PROTOCOL |             BACKEND UUID             | STATE  | MANAGED |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
| pvc-0ee95d60-fd5c-448d-b505-b72901b3a4ab | 100 GiB | anf-storage   |
file     | 1c01274f-d94b-44a3-98a3-04c953c9a51e | online | true    |
+------------------------------------------+---------+---------------
+----------+--------------------------------------+--------+---------+
```

> anf磁碟區的磁碟區路徑會出現在裝載路徑中的：/之後。例如、如果掛載路徑為「10.0.0.2：/importvol1」、則磁碟區路徑為「importvol1」。

準備工作節點

Kubernetes叢集中的所有工作節點都必須能夠掛載您已為Pod配置的磁碟區。如果您使用的是「ONTAP-NAS」、「ONTAP-NAS-節約」或「ONTAP-NAS-Flexgroup」驅動程式來支援其中一個後端、則您的工作節點需要NFS工具。否則他們需要iSCSI工具。

最新版本的RedHat CoreOS預設會同時安裝NFS和iSCSI。

> 安裝NFS或iSCSI工具之後、您應該一律重新啟動工作節點、否則將磁碟區附加至容器可能會失敗。

NFS磁碟區

傳輸協定	作業系統	命令
NFS	RHEL/CentOS	「Udo yum install -y ns-utils」
NFS	Ubuntu / DEBIAN	「Udo apt-Get install -y nfs通用」

> 您應確保NFS服務在開機期間啟動。

iSCSI磁碟區

使用iSCSI磁碟區時、請考量下列事項：

- Kubernetes叢集中的每個節點都必須具有唯一的IQN。這是必要的先決條件。
- 如果使用帶有「Poolidfire - san」驅動程式的RHcos 4.5或更新版本、或RHEL或CentOS 8.2或更新版本、請確定在「/etc/iscsi/iscsid.conf」中將CHAP驗證演算法設定為「md5」。

```
sudo sed -i 's/^\(node.session.auth.chap_algs\).*/\1 = MD5/'
/etc/iscsi/iscsid.conf
```

- 使用執行RHEL/RedHat CoreOS搭配iSCSI PV的工作節點時、請務必在StorageClass中指定「disccard」掛載選項、以執行即時空間回收。請參閱 "RedHat的文件"。

傳輸協定	作業系統	命令
iSCSI	RHEL/CentOS	1. 安裝下列系統套件： 「Udo yum install -y lsscsi iscsi-initiator－utils sg3_utils device-mapper-multipath' 2. 檢查iscsite-initier-utils版本是否為6.6.0.874-2.el7或更新版本： 「rpm -q iscsiscsiscs-initier-utils」 3. 將掃描設為手動： 「Udo sed -I 's/^\（node.session.scan\).*/\1 = manual /」/etc/iscsi/iscsid.conf） 4. 啟用多重路徑： 「Udo mpathconf --enable --with _docathd y --fisco_multipaths n」 ⓘ 確保在"default"（錯誤）下"etc/multipath.conf"包含"fappe_multipaths no"。 5. 確保運行的是"iscsid"和"multipathd"： 「啟用Udo systemctl -現在是iscsid多路徑」 6. 啟用並啟動「iSCSI」： 「Udo systemctl enable - NOW iSCSI」

傳輸協定	作業系統	命令
iSCSI	Ubuntu / DEBIAN	1. 安裝下列系統套件： 「Udo apt-Get install -y open-iscsi sg3-utils multipath-tools scsitols」 2. 檢查開放式iSCSI版本是否為2.0.874-5ubuntu2・10或更新版本（適用於雙聲網路）或2.0.874-7.1ubuntu6.1或更新版本（適用於焦點）： 「Dpkg -l open-iscsi」 3. 將掃描設為手動： 「Udo sed -I 's/^\（node.session.scan\).*/\1 = manual /」/etc/iscsi/iscsid.conf） 4. 啟用多重路徑： 「Udo tee /etc/multipath.conf」<←'EOF'預設值｛user_易 記名稱yes focs_multipaths no｝EOF Sudo systemctl enable --NOW multipath-tools.service Sudo服務multipath-ttools restart」 ⓘ 確保在"default"（錯誤）下"etc/multipath.conf"包含"fappe_multipaths no"。 5. 確保已啟用並執行「open-iscsi」和「多路徑工具」： 「Udo systemctl狀態multipath-ttools/」「show systemcl enable -現在open-iscsi.service`「show systemctl狀態open-iscsi」

ⓘ 對於Ubuntu 18.04、您必須先使用「iscsiadmd」探索目標連接埠、然後再啟動「open-iscsi」、iSCSI精靈才能啟動。您也可以修改「iSCSI」服務、以自動啟動「iscsid」。

如果您想要深入瞭解自動工作節點準備（這是試用版功能）、請參閱 "請按這裡"。

自動準備工作節點

Astra Trident可自動在Kubernetes叢集中的節點上安裝所需的「NFS」和「iSCSI」工具。這是*試用版功能*、不適用於*正式作業叢集。目前、此功能適用於執行 CentOS、RHEL及Ubuntu *的節點。

針對這項功能、Astra Trident提供新的安裝旗標：「-enable節點準備」、適用於以「tridentctl」部署的安裝。對於使用Trident運算子的部署、請使用布林選項「enableNodePrep」。

「啟用節點準備」安裝選項可讓Astra Trident安裝、並確保在工作節點上掛載磁碟區時、NFS和iSCSI套件及/或服務會執行。這是*試用版功能*、適用於*不符合*正式作業使用資格的開發/測試環境。

當使用「tridentctl」部署的Astra Trident安裝包含「-enable節點準備」旗標時、會發生以下情況：

1. 在安裝過程中、Astra Trident會登錄其執行的節點。
2. 當提出持續磁碟區宣告（PVc）要求時、Astra Trident會從其管理的其中一個後端建立PV。
3. 在Pod中使用永久虛擬磁碟時、需要使用Astra Trident將磁碟區掛載到執行Pod的節點上。Astra Trident會嘗試安裝所需的NFS/iSCSI用戶端公用程式、並確保所需的服務處於作用中狀態。這是在掛載磁碟區之前完成的。

在第一次嘗試掛載磁碟區時、只需準備一次工作節點。只要Astra Trident以外的任何變更都不會接觸到「NFS」和「iSCSI」公用程式、所有後續的Volume掛載就會成功。

如此一來、Astra Trident就能確保Kubernetes叢集中的所有節點都具備必要的公用程式、以掛載及附加磁碟區。對於NFS磁碟區、匯出原則也應該允許掛載磁碟區。Trident可以自動管理每個後端的匯出原則、也可以管理頻外的匯出原則。

監控Astra Trident

Astra Trident提供一組Prometheus指標端點、可用來監控Astra Trident的效能。

Astra Trident提供的指標可讓您執行下列作業：

- 隨時掌握Astra Trident的健全狀況與組態。您可以檢查作業的成功程度、以及是否能如預期般與後端進行通訊。
- 檢查後端使用資訊、並瞭解後端上配置的磁碟區數量、以及所耗用的空間量等。
- 維護可用後端配置的磁碟區數量對應。
- 追蹤效能。您可以查看Astra Trident與後端及執行作業所需的時間。

根據預設、Trident的指標會顯示在「/度量」端點的目標連接埠「8001」上。安裝Trident時*預設會啟用這些度量。

您需要的是 **#8217** ；需要的是什麼

- 安裝Astra Trident的Kubernetes叢集。

- Prometheus執行個體。這可以是 "容器化Prometheus部署" 或者、您也可以選擇以執行Prometheus "原生應用程式"。

步驟1：定義Prometheus目標

您應該定義Prometheus目標、以收集指標並取得有關後端Astra Trident管理的資訊、以及其建立的磁碟區等資訊。這 "部落格" 說明如何使用Prometheus和Grafana搭配Astra Trident來擷取指標。部落格說明如何在Kubernetes叢集中以營運者的形式執行Prometheus、以及建立ServiceMonitor來取得Astra Trident的指標。

步驟2：建立Prometheus ServiceMonitor

若要使用Trident指標、您應該建立Prometheus ServiceMonitor、以監控「Trident - csi」服務、並在「metrics」連接埠上聆聽。ServiceMonitor範例如下所示：

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: trident-sm
  namespace: monitoring
  labels:
      release: prom-operator
  spec:
    jobLabel: trident
    selector:
      matchLabels:
        app: controller.csi.trident.netapp.io
    namespaceSelector:
      matchNames:
      - trident
    endpoints:
    - port: metrics
      interval: 15s
```

此ServiceMonitor定義會擷取由「Trident -csi」服務傳回的度量、並特別尋找服務的「metrics」端點。因此、Prometheus現在已設定為瞭解Astra Trident的指標。

除了可直接從Astra Trident取得的指標之外、Kibelet還透過自己的指標端點、公開許多「kubelet_volume」*指標。Kubelet可提供有關所附加磁碟區、Pod及其處理的其他內部作業的資訊。請參閱 "請按這裡"。

步驟3：使用PromQL查詢Trident度量

PromQL適用於建立傳回時間序列或表格資料的運算式。

以下是一些您可以使用的PromQL查詢：

取得**Trident**健全狀況資訊

- 來自Astra Trident的HTTP 2XX回應百分比*

```
(sum (trident_rest_ops_seconds_total_count{status_code=~"2.."} OR on()
vector(0)) / sum (trident_rest_ops_seconds_total_count)) * 100
```

- 透過狀態代碼*來自**Astra Trident**的休息回應百分比

```
(sum (trident_rest_ops_seconds_total_count) by (status_code)  / scalar
(sum (trident_rest_ops_seconds_total_count))) * 100
```

- 由**Astra Trident**執行的平均營運持續時間

```
sum by (operation)
(trident_operation_duration_milliseconds_sum{success="true"}) / sum by
(operation)
(trident_operation_duration_milliseconds_count{success="true"})
```

取得**Astra Trident**使用資訊

- 平均Volume大小*

```
trident_volume_allocated_bytes/trident_volume_count
```

- 每個後端配置的Volume空間總計*

```
sum (trident_volume_allocated_bytes) by (backend_uuid)
```

取得個別**Volume**使用量

> 只有同時收集kubelet度量時、才會啟用此功能。

- 每個Volume的已用空間百分比*

```
kubelet_volume_stats_used_bytes / kubelet_volume_stats_capacity_bytes *
100
```

深入瞭解Astra Trident AutoSupport 遙測技術

依預設、Astra Trident會每日傳送Prometheus指標和基本後端資訊給NetApp。

- 若要停止Astra Trident將Prometheus指標和基本後端資訊傳送給NetApp、請在Astra Trident安裝期間、傳遞「-Silence-autosupport」旗標。
- Astra Trident也可透過「tridentctl傳送AutoSupport 功能」、隨需將容器記錄傳送給NetApp Support。您需要觸發Astra Trident來上傳記錄。在您提交記錄之前、您應該接受NetApp的https://www.netapp.com/company/legal/privacy-policy/["隱私權政策"^]。
- 除非另有說明、Astra Trident會從過去24小時擷取記錄。
- 您可以使用「-自」旗標來指定記錄保留時間範圍。例如：「tridentctl傳送AutoSupport 功能不全、自一小時起」。此資訊是透過Astra Trident一起安裝的「Trident -autodupport」容器來收集和傳送。您可以從取得Container映像 "Trident AutoSupport 的"。
- Trident AutoSupport 無法收集或傳輸個人識別資訊（PII）或個人資訊。隨附a "EULA" 這不適用於Trident Container映像本身。您可以深入瞭解NetApp對資料安全性與信任的承諾 "請按這裡"。

Astra Trident傳送的有效負載範例如下：

```
{
  "items": [
    {
      "backendUUID": "ff3852e1-18a5-4df4-b2d3-f59f829627ed",
      "protocol": "file",
      "config": {
        "version": 1,
        "storageDriverName": "ontap-nas",
        "debug": false,
        "debugTraceFlags": null,
        "disableDelete": false,
        "serialNumbers": [
          "nwkvzfanek_SN"
        ],
        "limitVolumeSize": ""
      },
      "state": "online",
      "online": true
    }
  ]
}
```

- 此資訊將傳送至NetApp的「不只是」端點。AutoSupport AutoSupport如果您使用私有登錄來儲存容器映像、可以使用「-image-registry」旗標。
- 您也可以產生安裝Yaml檔案來設定Proxy URL。您可以使用「tridentctl install -generate-custom-yaml」來建立Yaml檔案、並在「trident部署.yaml」中新增「trident -autodupport」容器的「-proxy-URL」引數。

停用Astra Trident度量

要使指標不被報告，您應該生成自定義YAML（使用"-generame-custom-yaml"標誌）並進行編輯，以刪除對"trident－main"容器所調用的"-mication"標誌。

適合Docker的Astra Trident

部署的先決條件

您必須先在主機上安裝並設定必要的傳輸協定先決條件、才能部署Astra Trident。

- 確認您的部署符合所有的 "需求"。
- 確認您安裝的Docker版本受支援。如果您的Docker版本過時、 "安裝或更新"。

```
docker --version
```

- 確認已在您的主機上安裝及設定通訊協定先決條件：

傳輸協定	作業系統	命令
NFS	RHEL/CentOS	「Udo yum install -y ns-utils」
NFS	Ubuntu / DEBIAN	「Udo apt-Get install -y nfs通用」

傳輸協定	作業系統	命令
iSCSI	RHEL/CentOS 7.	1. 安裝下列系統套件： 「Udo yum install -y lsscsi iscsi-initiator－utils sg3_utils device-mapper-multipath' 2. 檢查iscsite-initier-utils版本是否為6.6.0.874-2.el7或更新版本： 「rpm -q iscsiscsiscs-initier-utils」 3. 將掃描設為手動： 「Udo sed -I 's/^\（node.session.scan\).*/\1 = manual /」/etc/iscsi/iscsid.conf） 4. 啟用多重路徑： 「Udo mpathconf --enable --with _docathd y --fisco_multipaths n」 ⓘ 確保在"default"（錯誤）下"etc/multipath.conf"包含"fappe_multipaths no"。 5. 確保運行的是"iscsid"和"multipathd"： 「啟用Udo systemctl -現在是iscsid多路徑」 6. 啟用並啟動「iSCSI」： 「Udo systemctl enable - NOW iSCSI」

傳輸協定	作業系統	命令
iSCSI	Ubuntu	1. 安裝下列系統套件： 「Udo apt-Get install -y open-iscsi sg3-utils multipath-tools scsitols」 2. 檢查開放式iSCSI版本是否為2.0.874-5ubuntu2・10或更新版本（適用於雙聲網路）或2.0.874-7.1ubuntu6.1或更新版本（適用於焦點）： 「Dpkg -l open-iscsi」 3. 將掃描設為手動： 「Udo sed -I 's/^\（node.session.scan\).*/\1 = manual /」/etc/iscsi/iscsid.conf） 4. 啟用多重路徑： 「Udo tee /etc/multipath.conf」<←'EOF'預設值｛user_易 記名稱yes focs_multipaths no｝EOF Sudo systemctl enable --NOW multipath-tools.service Sudo服務multipath-ttools restart」 ⓘ 確保在"default"（錯誤）下"etc/multipath.conf"包含"fappe_multipaths no"。 5. 確保已啟用並執行「open-iscsi」和「多路徑工具」： 「Udo systemctl狀態multipath-ttools/」「show systemcl enable -現在open-iscsi.service`「show systemctl狀態open-iscsi」

部署Astra Trident

適用於Docker的Astra Trident可直接整合適用於NetApp儲存平台的Docker生態系統。它支援從儲存平台到Docker主機的儲存資源資源配置與管理、並提供架構、可在未來新增更多平台。

多個Astra Trident執行個體可同時在同一個主機上執行。這可同時連線至多個儲存系統和儲存類型、並可自訂Docker磁碟區所使用的儲存設備。

請參閱 "部署的先決條件"。在您確定已符合先決條件之後、就可以開始部署Astra Trident。

Docker託管外掛程式方法（1.1/17.03版及更新版本）

> 開始之前
>
> 如果您在傳統精靈方法中使用Astra Trident preer Docker 1.3/17.03、請務必先停止Astra Trident程序、然後重新啟動Docker精靈、再使用託管外掛方法。

1. 停止所有執行中的執行個體：

```
pkill /usr/local/bin/netappdvp
pkill /usr/local/bin/trident
```

2. 重新啟動Docker。

```
systemctl restart docker
```

3. 請確定您已安裝Docker Engine 17.03（全新1.13）或更新版本。

```
docker --version
```

如果您的版本過時、 "安裝或更新安裝"。

步驟

1. 建立組態檔並指定下列選項：
   - 「config」：預設檔案名稱為「config.json」、但您可以使用任何名稱、只要在檔案名稱中指定「config」選項即可。組態檔必須位於主機系統的「/etc/netappdvp」目錄中。
   - 記錄層級：指定記錄層級（「debug」、「info」、「warn」、「f誤差」、「fatal」）。預設值為「資訊」。
   - 「Debug」：指定是否啟用偵錯記錄。預設值為假。如果為true、則會置換記錄層級。
     i. 建立組態檔的位置：

```
sudo mkdir -p /etc/netappdvp
```

ii. 建立組態檔：

```
cat << EOF > /etc/netappdvp/config.json
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "username": "vsadmin",
    "password": "secret",
    "aggregate": "aggr1"
}
EOF
```

2. 使用託管外掛系統啟動Astra Trident。

```
docker plugin install --grant-all-permissions --alias netapp
netapp/trident-plugin:21.07 config=myConfigFile.json
```

3. 開始使用Astra Trident來消耗已設定系統的儲存空間。
   a. 建立名為「firstVolume」的Volume：

   ```
   docker volume create -d netapp --name firstVolume
   ```

   b. 在容器啟動時建立預設Volume：

   ```
   docker run --rm -it --volume-driver netapp --volume
   secondVolume:/my_vol alpine ash
   ```

   c. 移除Volume「firstVolume」：

   ```
   docker volume rm firstVolume
   ```

傳統方法（1.12版或更早版本）

開始之前

1. 請確定您擁有Docker 1.10版或更新版本。

```
docker --version
```

如果您的版本已過時、請更新安裝。

```
curl -fsSL https://get.docker.com/ | sh
```

或者、 "請依照您的經銷指示進行"。

2. 確保已為您的系統設定NFS和/或iSCSI。

步驟

1. 安裝及設定NetApp Docker Volume外掛程式：
   a. 下載並解壓縮應用程式：

   ```
   wget
   https://github.com/NetApp/trident/releases/download/v21.04.0/trident-
   installer-21.07.0.tar.gz
   tar zxf trident-installer-21.07.0.tar.gz
   ```

   b. 移至Bin路徑中的位置：

   ```
   sudo mv trident-installer/extras/bin/trident /usr/local/bin/
   sudo chown root:root /usr/local/bin/trident
   sudo chmod 755 /usr/local/bin/trident
   ```

   c. 建立組態檔的位置：

   ```
   sudo mkdir -p /etc/netappdvp
   ```

   d. 建立組態檔：

```
cat << EOF > /etc/netappdvp/ontap-nas.json
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "username": "vsadmin",
    "password": "secret",
    "aggregate": "aggr1"
}
EOF
```

2. 放置二進位檔並建立組態檔之後、請使用所需的組態檔來啟動Trident精靈。

```
sudo trident --config=/etc/netappdvp/ontap-nas.json
```

> 除非指定、否則Volume驅動程式的預設名稱為「NetApp」。

精靈啟動後、您可以使用Docker CLI介面來建立及管理磁碟區

3. 建立Volume：

```
docker volume create -d netapp --name trident_1
```

4. 在啟動容器時配置Docker Volume：

```
docker run --rm -it --volume-driver netapp --volume trident_2:/my_vol
alpine ash
```

5. 移除Docker Volume：

```
docker volume rm trident_1
docker volume rm trident_2
```

在系統啟動時啟動Astra Trident

可在Git repo的「contrib / trident.service.example`」中找到系統型系統的單元檔案範例。若要搭配CentOS/RHEL使用檔案、請執行下列步驟：

1. 將檔案複製到正確的位置。

如果執行多個執行個體、則應使用單元檔案的唯一名稱。

```
cp contrib/trident.service.example
/usr/lib/systemd/system/trident.service
```

2. 編輯檔案、變更說明（第2行）以符合驅動程式名稱和組態檔案路徑（第9行）、以反映您的環境。
3. 重新載入系統d以擷取變更：

```
systemctl daemon-reload
```

4. 啟用服務。

   此名稱會根據您在「/r/lib/systemd/system]目錄中命名的檔案而有所不同。

```
systemctl enable trident
```

5. 啟動服務。

```
systemctl start trident
```

6. 檢視狀態。

```
systemctl status trident
```

> 每當您修改單元檔案時、請執行「stystemctl daem-reload」命令、以瞭解變更內容。

升級或解除安裝Astra Trident

您可以安全地升級適用於Docker的Astra Trident、而不會對使用中的磁碟區造成任何影響。在升級過程中、針對外掛程式的「docker volume」命令將會短暫失敗、而在外掛程式再次執行之前、應用程式將無法掛載磁碟區。在大多數情況下、這是幾秒鐘的事。

升級

執行下列步驟以升級適用於Docker的Astra Trident。

步驟

1. 列出現有的磁碟區：

```
docker volume ls
DRIVER              VOLUME NAME
netapp:latest       my_volume
```

2. 停用外掛程式：

```
docker plugin disable -f netapp:latest
docker plugin ls
ID                  NAME                DESCRIPTION
ENABLED
7067f39a5df5        netapp:latest       nDVP - NetApp Docker Volume
Plugin   false
```

3. 升級外掛程式：

```
docker plugin upgrade --skip-remote-check --grant-all-permissions
netapp:latest netapp/trident-plugin:21.07
```

> 18.01版的Astra Trident取代了nDVP。您應該直接從「NetApp/ndvp-plugin」映像升級至「NetApp/triident外掛程式」映像。

4. 啟用外掛程式：

```
docker plugin enable netapp:latest
```

5. 確認外掛程式已啟用：

```
docker plugin ls
ID                  NAME                DESCRIPTION
ENABLED
7067f39a5df5        netapp:latest       Trident - NetApp Docker Volume
Plugin   true
```

6. 確認磁碟區可見：

```
docker volume ls
DRIVER              VOLUME NAME
netapp:latest       my_volume
```

> 如果您要從舊版的Astra Trident（20.10之前版本）升級至Astra Trident 20.10或更新版本、可能會發生錯誤。如需詳細資訊、請參閱 "已知問題"。如果您遇到錯誤、請先停用外掛程式、然後移除外掛程式、再傳遞額外的組態參數來安裝所需的Astra Trident版本：「Docker外掛程式安裝NetApp/Trident外掛程式：20.10 -別名NetApp --贈與All權限config=config.json」

解除安裝

執行下列步驟以解除安裝適用於Docker的Astra Trident。

步驟

1. 移除外掛程式所建立的任何磁碟區。
2. 停用外掛程式：

```
docker plugin disable netapp:latest
docker plugin ls
ID                  NAME                DESCRIPTION
ENABLED
7067f39a5df5        netapp:latest       nDVP - NetApp Docker Volume
Plugin   false
```

3. 移除外掛程式：

```
docker plugin rm netapp:latest
```

使用Volume

您可以使用標準的「磁碟區」命令、並在需要時指定Astra Trident驅動程式名稱、輕鬆建立、複製及移除磁碟區。

建立Volume

- 使用預設名稱建立具有驅動程式的磁碟區：

```
docker volume create -d netapp --name firstVolume
```

- 建立具有特定Astra Trident執行個體的Volume：

```
docker volume create -d ntap_bronze --name bronzeVolume
```

> 如果您未指定任何 "選項"，將使用驅動程式的預設值。

- 覆寫預設的Volume大小。請參閱下列範例、以使用驅動程式建立20GiB磁碟區：

```
docker volume create -d netapp --name my_vol --opt size=20G
```

> Volume大小以包含整數值的字串表示、並提供選用單位（例如：10g、20GB、3TiB）。如果未指定單位、則預設值為G大小單位可以表示為2（B、KiB、mib、GiB、TiB）或10（B、KB、MB、GB、TB）的冪。簡寫單元使用2（G = GiB、T = TiB、...）的權力。

移除Volume

- 移除Volume就像移除任何其他Docker Volume一樣：

```
docker volume rm firstVolume
```

> 使用「Poolidfire - san」驅動程式時、上述範例會刪除及清除磁碟區。

執行下列步驟以升級適用於Docker的Astra Trident。

複製磁碟區

使用「ONTAP-NAS」、「ONTAP-SAN」、「Poolidfire - SAN」和「GCP - CVS儲存驅動程式」時、Astra Trident可以複製磁碟區。使用「ONTAP-NAS-flexgroup」或「ONTAP-NAS-節約」驅動程式時、不支援複製。從現有磁碟區建立新磁碟區、將會產生新的快照。

- 檢查磁碟區以列舉快照：

```
docker volume inspect <volume_name>
```

- 從現有的Volume建立新的Volume。這將會產生新的快照：

```
docker volume create -d <driver_name> --name <new_name> -o
from=<source_docker_volume>
```

- 從磁碟區上現有的快照建立新磁碟區。這不會建立新的快照：

```
docker volume create -d <driver_name> --name <new_name> -o
from=<source_docker_volume> -o fromSnapshot=<source_snap_name>
```

範例

```
[me@host ~]$ docker volume inspect firstVolume

[
    {
        "Driver": "ontap-nas",
        "Labels": null,
        "Mountpoint": "/var/lib/docker-volumes/ontap-
nas/netappdvp_firstVolume",
        "Name": "firstVolume",
        "Options": {},
        "Scope": "global",
        "Status": {
            "Snapshots": [
                {
                    "Created": "2017-02-10T19:05:00Z",
                    "Name": "hourly.2017-02-10_1505"
                }
            ]
        }
    }
]

[me@host ~]$ docker volume create -d ontap-nas --name clonedVolume -o
from=firstVolume
clonedVolume

[me@host ~]$ docker volume rm clonedVolume
[me@host ~]$ docker volume create -d ontap-nas --name volFromSnap -o
from=firstVolume -o fromSnapshot=hourly.2017-02-10_1505
volFromSnap

[me@host ~]$ docker volume rm volFromSnap
```

存取外部建立的磁碟區

如果容器沒有分割區、而且Astra Trident支援其檔案系統、則您可以使用Trident * Only *來存取外部建立的區塊裝置（或其複製）（例如：無法透過Astra Trident存取「ext4」格式的「dev/sdc1」）。

驅動程式專屬的**Volume**選項

每個儲存驅動程式都有一組不同的選項、您可以在建立磁碟區時指定、以自訂結果。請參閱下方、以瞭解適用於您所設定儲存系統的選項。

在磁碟區建立作業期間使用這些選項非常簡單。在CLI操作期間、使用「-o」運算子來提供選項和值。這會覆

寫Json組態檔中的任何等效值。

選購的選購配備**ONTAP**

NFS和iSCSI的Volume建立選項包括：

選項	說明
《大小》	Volume的大小、預設為1 GiB。
《保護區》	精簡或完整配置磁碟區、預設為精簡。有效值為「NONE」（精簡配置）和「Volume」（完整配置）。
「快照原則」	這會將快照原則設定為所需的值。預設值為「無」、表示不會自動建立磁碟區的快照。除非您的儲存管理員加以修改、否則ONTAP 所有的各種系統都會有一個名為「預設」的原則、用來建立並保留六個每小時、每天兩個和每週兩個快照。快照中保留的資料可透過瀏覽至磁碟區任何目錄中的「.snapshot」目錄來還原。
「快照保留區」	這會將快照保留設定為所需的百分比。預設值為無值、表示ONTAP 如果您已選取snapshotPolicy、將選取snapshotReserve（通常為5%）、如果snapshotPolicy為無、則選取0%。您可以在組態檔中為所有ONTAP 的支援項目設定預設的snapshotReserve值、也可以將其用作所有ONTAP 不支援ONTAP-NAS-經濟功能的支援項目、作為所有支援項目的磁碟區建立選項。
「PlitOnClone」	當複製Volume時、ONTAP 這會導致停止實體複本、立即將其從父複本分割開來。預設值為「假」。部分複製磁碟區的使用案例、最好是在建立時立即將複本從父複本分割出來、因為不太可能有提高儲存效率的機會。例如，複製空白資料庫可節省大量時間，但儲存空間卻很少，因此最好立即分割實體複本。
加密	這將在新磁碟區上啟用NetApp Volume Encryption（NVE）、預設為「假」。必須在叢集上授權並啟用NVE、才能使用此選項。 Trident目前不支援NetApp Aggregate Encryption（NAE）。
「分層政策」	設定要用於磁碟區的分層原則。這會決定資料在非作用中（冷）時是否移至雲端層。

下列其他選項適用於NFS * Only *：

選項	說明
「unixPermissions」	這會控制Volume本身的權限設定。依預設、權限會設為「-rwxr-x-x'」、或是以數字表示法0755、而「root」則是擁有者。文字或數字格式皆可運作。
「napshotDir	如果將此目錄設為「true」、則會讓存取磁碟區的用戶端看到「.snapshot」目錄。預設值為「假」、表示預設會停用「.snapshot」目錄的可見度。某些影像（例如官方MySQL映像）在出現「.snapshot」目錄時無法如預期般運作。
「匯出政策」	設定要用於磁碟區的匯出原則。預設值為「預設」。
《生態樣式》	設定用於存取磁碟區的安全樣式。預設值為「UNIX」。有效值為「UNIX」和「mixed」。

下列其他選項僅適用於iSCSI *：

選項	說明
「fileSystemType」	設定用於設定iSCSI磁碟區格式的檔案系統。預設值為「ext4」。有效值包括「ext3」、「ext4」和「xfs」。
"paceAllocate（配置）"	如果將此值設為「假」、將會關閉LUN的空間分配功能。預設值為「true」、表示ONTAP 當磁碟區空間不足、且磁碟區中的LUN無法接受寫入時、功能區會通知主機。此選項也可讓ONTAP 支援功能在主機刪除資料時自動回收空間。

範例

請參閱下列範例：

- 建立10GiB Volume：

```
docker volume create -d netapp --name demo -o size=10G -o
encryption=true
```

- 使用快照建立100GiB磁碟區：

```
docker volume create -d netapp --name demo -o size=100G -o
snapshotPolicy=default -o snapshotReserve=10
```

- 建立已啟用setuid位元的磁碟區：

```
docker volume create -d netapp --name demo -o unixPermissions=4755
```

最小Volume大小為20MiB。

如果未指定快照保留、且快照原則為「無」、Trident將使用快照保留0%。

- 建立沒有快照原則且無快照保留的磁碟區：

```
docker volume create -d netapp --name my_vol --opt snapshotPolicy=none
```

- 建立不含快照原則的磁碟區、以及自訂快照保留10%的磁碟區：

```
docker volume create -d netapp --name my_vol --opt snapshotPolicy=none
--opt snapshotReserve=10
```

- 建立具有快照原則和10%自訂快照保留的磁碟區：

```
docker volume create -d netapp --name my_vol --opt
snapshotPolicy=myPolicy --opt snapshotReserve=10
```

- 使用快照原則建立磁碟區、並接受ONTAP的預設快照保留（通常為5%）：

```
docker volume create -d netapp --name my_vol --opt
snapshotPolicy=myPolicy
```

Element軟體Volume選項

元素軟體選項會顯示與磁碟區相關的服務品質（QoS）原則大小和品質。建立磁碟區時、會使用「-o type =service_level」命名法來指定與其相關的QoS原則。

使用元素驅動程式定義QoS服務層級的第一步、是建立至少一種類型、並在組態檔中指定與名稱相關的最小、最大和尖峰IOPS。

其他元素軟體磁碟區建立選項包括：

選項	說明
《大小》	磁碟區大小、預設為1GiB或組態項目… 「預設值」：｛"Size"："5G"｝ 。
「區塊大小」	使用512或4096、預設為512或組態項目預設BlockSizes。

範例

請參閱下列QoS定義範例組態檔：

```
{
    "...": "..."
    "Types": [
        {
            "Type": "Bronze",
            "Qos": {
                "minIOPS": 1000,
                "maxIOPS": 2000,
                "burstIOPS": 4000
            }
        },
        {
            "Type": "Silver",
            "Qos": {
                "minIOPS": 4000,
                "maxIOPS": 6000,
                "burstIOPS": 8000
            }
        },
        {
            "Type": "Gold",
            "Qos": {
                "minIOPS": 6000,
                "maxIOPS": 8000,
                "burstIOPS": 10000
            }
        }
    ]
}
```

在上述組態中、我們有三種原則定義：銅級、銀級和金級。這些名稱為任意名稱。

- 建立10GiB Gold Volume：

```
docker volume create -d solidfire --name sfGold -o type=Gold -o size=10G
```

- 建立100GiB銅級磁碟區：

```
docker volume create -d solidfire --name sfBronze -o type=Bronze -o
size=100G
```

在GCP磁碟區選項上使用CVS

GCP驅動程式上的CVS磁碟區建立選項包括：

選項	說明
《大小》	磁碟區大小、CVS效能磁碟區的預設值為100 GiB、CVS磁碟區的預設值為300 GiB。
《服務層級》	Volume的CVS服務層級預設為標準。有效值包括標準、高級和極高。
「快照保留區」	這會將快照保留設定為所需的百分比。預設值為無值、表示CVS會選取快照保留（通常為0%）。

範例

- 建立2TiB Volume：

```
docker volume create -d netapp --name demo -o size=2T
```

- 建立5TiB優質Volume：

```
docker volume create -d netapp --name demo -o size=5T -o
serviceLevel=premium
```

CVS效能磁碟區的最小Volume大小為100 GiB、CVS磁碟區的最小Volume大小為300 GiB。

選購的選購配備Azure NetApp Files

適用於此驅動程式的Volume Create選項Azure NetApp Files 包括：

選項	說明
《大小》	磁碟區大小、預設為100 GB。

範例

- 建立200GiB Volume：

```
docker volume create -d netapp --name demo -o size=200G
```

最小Volume大小為100 GB。

收集記錄

您可以收集記錄以協助疑難排解。收集記錄的方法會因執行Docker外掛程式的方式而有所不同。

收集記錄以進行疑難排解

步驟

1. 如果您使用建議的託管外掛程式方法執行Astra Trident（亦即使用「docker plugin」命令）、請依照下列方式檢視：

```
# docker plugin ls
ID                  NAME                DESCRIPTION
ENABLED
4fb97d2b956b        netapp:latest       nDVP - NetApp Docker Volume
Plugin   false
# journalctl -u docker | grep 4fb97d2b956b
```

   標準記錄層級應可讓您診斷大多數問題。如果您發現這還不夠、可以啟用偵錯記錄。

2. 若要啟用除錯記錄、請安裝已啟用除錯記錄的外掛程式：

```
docker plugin install netapp/trident-plugin:<version> --alias <alias>
debug=true
```

   或者、當外掛程式已安裝時、請啟用偵錯記錄功能：

```
docker plugin disable <plugin>
docker plugin set <plugin> debug=true
docker plugin enable <plugin>
```

3. 如果您在主機上執行二進位檔本身、則記錄檔可在主機的「/var/log/netappdvp」目錄中使用。若要啟用偵錯記錄、請在執行外掛程式時指定「-DEBUG」。

一般疑難排解秘訣

- 新使用者最常遇到的問題是組態錯誤、導致外掛程式無法初始化。發生這種情況時、當您嘗試安裝或啟用外掛程式時、可能會看到如下訊息：

  「精靈的錯誤回應：請撥打UNIX /run / dock/plugins/<id>/NetApp.sock：Connect：沒有這類檔案或目錄」

  這表示外掛程式無法啟動。幸運的是、外掛程式是以全方位的記錄功能打造而成、可協助您診斷大部分可能遇到的問題。

- 如果將PV掛載到容器時發生問題、請確定已安裝並執行「rpcbind」。使用主機作業系統所需的套件管理程式、檢查「rpcbind」是否正在執行。您可以執行「stystemctl狀態rpcbind」或其等效項目來檢查rpcbind服務

的狀態。

管理多個Astra Trident執行個體

當您想要同時使用多個儲存組態時、需要多個Trident執行個體。多個執行個體的關鍵是在主機上具現化Trident時、使用容器化外掛程式的「-alias」選項或「-volume驅動程式」選項、為它們指定不同的名稱。

Docker託管外掛程式（1.3/17.03版或更新版本）的步驟

1. 啟動第一個指定別名和組態檔的執行個體。

```
docker plugin install --grant-all-permissions --alias silver
netapp/trident-plugin:21.07 config=silver.json
```

2. 啟動第二個執行個體、指定不同的別名和組態檔。

```
docker plugin install --grant-all-permissions --alias gold
netapp/trident-plugin:21.07 config=gold.json
```

3. 建立磁碟區、將別名指定為驅動程式名稱。

   例如、黃金Volume：

```
docker volume create -d gold --name ntapGold
```

   例如、對於銀級Volume：

```
docker volume create -d silver --name ntapSilver
```

傳統的步驟（1.12版或更早版本）

1. 使用自訂驅動程式ID以NFS組態啟動外掛程式：

```
sudo trident --volume-driver=netapp-nas --config=/path/to/config
-nfs.json
```

2. 使用自訂驅動程式ID以iSCSI組態啟動外掛程式：

```
sudo trident --volume-driver=netapp-san --config=/path/to/config
-iscsi.json
```

3. 為每個驅動程式執行個體配置Docker磁碟區：

   例如、對於NFS：

```
docker volume create -d netapp-nas --name my_nfs_vol
```

   例如、對於iSCSI：

```
docker volume create -d netapp-san --name my_iscsi_vol
```

儲存組態選項

請參閱Astra Trident組態的可用組態選項。

全域組態選項

這些組態選項適用於所有Astra Trident組態、無論使用何種儲存平台。

選項	說明	範例
「分度」	組態檔版本編號	1.
「torageDriverName」	儲存驅動程式名稱	"ONTAP-NAS"、"ONTAP-san 、"ONTAP-NAS-經濟"、"ONTAP-NAS- Flexgroup、"Poolidfire - san、 "azure-NetApp-fils"或"GCP-CVS"
「toragePrefix」	Volume名稱的選用首碼。預設值：「netappdvp_」。	接移_
《限制Volume大小》	Volume大小的選擇性限制。預設值：「」（未強制執行）	10g

請勿將「儲存前置詞」（包括預設值）用於元素後端。缺省情況下，"Poolidfire－san "驅動程序將忽略此設置而不使用前綴。我們建議使用特定的TenantId進行Docker Volume對應、或是使用Docker版本、驅動程式資訊和原始名稱填入的屬性資料、以便在可能使用任何名稱標示的情況下使用。

您可以使用預設選項、避免在每個建立的Volume上指定這些選項。「最小化」選項適用於所有控制器類型。如ONTAP 需如何設定預設Volume大小的範例、請參閱「功能區組態」一節。

選項	說明	範例
《大小》	新磁碟區的選用預設大小。預設值：「1G」	10g

組態ONTAP

除了上述全域組態值之外、使用ONTAP 時還提供下列頂層選項。

選項	說明	範例
《馬納格門達利》	IP位址ONTAP ：您可以指定完整網域名稱（FQDN）。	10.0.0.1
「DataLIF」	傳輸協定LIF的IP位址；如果未指定、則會衍生出來。對於「ONTAP-NAS」驅動程式*僅*、您可以指定FQDN、在這種情況下、FQDN將用於NFS掛載作業。對於「ONTAP-san」驅動程式、預設為使用SVM的所有資料LIF IP、並使用iSCSI多重路徑。為「ONTAP-san」驅動程式指定「dataLIF」的IP位址、會強制驅動程式停用多重路徑、並只使用指定的位址。	10.0.0.2
《虛擬機器》	要使用的儲存虛擬機器（如果管理LIF是叢集LIF、則為必要）	SVM_NFS
《使用者名稱》	連線至儲存設備的使用者名稱	vsadmin
密碼	連線至儲存設備的密碼	秘密
《Aggregate》	用於資源配置的Aggregate（選用；如果已設定、則必須指派給SVM）。對於「ONTAP-NAS-flexgroup」驅動程式、此選項會被忽略。指派給SVM的所有集合體均可用來配置FlexGroup 一個SVolume。	Aggr1.
「限制Aggregateusage」	如果使用率高於此百分比、則可選用、失敗的資源配置	75%

選項	說明	範例
「nfsMountOptions」	精細控制NFS掛載選項；預設為「-o nfsver=3」。僅適用於「**ONTAP-NAS**」和「**ONTAP-NAS-**經濟」驅動程式。 "請參閱此處的NFS主機組態資訊"。	-o nfsves=4.
「igroupName」	外掛程式使用的igroup；預設為「netappdvp」。僅適用於「**ONTAP-San'driver**」。	myigroup
《限制Volume大小》	可申請的Volume大小上限和qtree父Volume大小上限。對於「**ONTAP-NAS**經濟」驅動程式、此選項會進一步限制其所建立的**FlexVols**大小。	300克
"qtreesPerFlexvol"	每FlexVol 個邊區最多qtree數、範圍必須為[50、300]、預設值為200。對於「**ONTAP-NAS-P**節約」驅動程式、此選項可自訂每**FlexVol**個版本的**qtree**數量上限。	300

您可以使用預設選項、避免在您建立的每個Volume上指定這些選項：

選項	說明	範例
《保護區》	空間保留模式；「無」（精簡配置）或「Volume」（完整配置）	無
「快照原則」	要使用的Snapshot原則、預設為「無」	無
「快照保留區」	Snapshot保留百分比、預設值為「」以接受ONTAP的預設值	10.
「PlitOnClone」	建立複本時從其父實體分割複本、預設為「假」	錯
加密	啟用NetApp Volume Encryption、預設為「假」	是的
「unixPermissions」	NAS選項適用於已配置的NFS磁碟區、預設為「777」	777
「napshotDir	NAS選項用於存取「.snapshot」目錄、預設為「假」	是的

選項	說明	範例
「匯出政策」	NFS匯出原則使用NAS選項、預設為「預設」	預設
《生態樣式》	NAS選項用於存取已配置的NFS Volume、預設為「UNIX」	混合
「fileSystemType」	SAN選項可選擇檔案系統類型、預設為「ext4」	XFS
「分層政策」	要使用的分層原則、預設為「無」；ONTAP 9.5之前的SVM-DR組態為「純快照」	無

擴充選項

「ONTAP-NAS」和「ONTAP-SAN」驅動程式可為ONTAP FlexVol 每個Docker Volume建立一個支援功能。支援每個叢集節點最多1000個FlexVols、叢集最多12、000個FlexVols。ONTAP如果您的Docker Volume需求符合上述限制、則「ONTAP-NAS」驅動程式是首選的NAS解決方案、因為FlexVols提供的其他功能、例如Docker Volume精細快照和複製。

如果您需要的Docker磁碟區數量超過FlexVol 了《支援》的範圍、請選擇「ONTAP - NAS經濟」或「ONTAP - SAN經濟」驅動程式。

「ONTAP-NAS經濟」驅動程式會在ONTAP 自動管理的FlexVols資源池中、將Docker磁碟區建立為還原樹狀結構。qtree的擴充能力大幅提升、每個叢集節點最多可達100、000個、每個叢集最多可達2、400、000個、而犧牲了部分功能。「ONTAP-NAS-節約」驅動程式不支援Docker Volume精細快照或複製。

> Docker swarm目前不支援「ONTAP-NAS-節約」驅動程式、因為swarm不會協調多個節點之間的磁碟區建立。

「ONTAP-san經濟」驅動程式會在ONTAP 自動管理的FlexVols共用集區內、將Docker Volume建立為還原LUN。如此FlexVol 一來、每個支援不只侷限於一個LUN、而且能為SAN工作負載提供更好的擴充性。根據儲存陣列的不同、ONTAP 每個叢集最多可支援16384個LUN。由於磁碟區是下方的LUN、因此此驅動程式支援Docker磁碟區精細快照和複製。

選擇「ONTAP-NAS-flexgroup」驅動程式、即可將平行處理能力提升至單一磁碟區、以數十億個檔案擴充至PB範圍。FlexGroups的一些理想使用案例包括AI / ML / DL、Big Data和分析、軟體建置、串流、檔案儲存庫等。Trident在配置FlexGroup 完一套SVM時、會使用指派給SVM的所有集合體。支援Trident也有下列考量：FlexGroup

- 需要ONTAP 9.2版或更新版本。
- 截至本文撰寫時、FlexGroups僅支援NFS v3。
- 建議啟用SVM的64位元NFSv3識別碼。
- 建議FlexGroup 的最小內存區大小為100GB。
- 不支援FlexGroup 將複製複製用於實體磁碟區。

如需適用於FlexGroups的FlexGroups與工作負載相關資訊、請參閱 "NetApp FlexGroup 《NetApp》《最佳實務做法與實作指南》"。

若要在同一個環境中獲得進階功能和龐大規模、您可以執行多個Docker Volume外掛程式執行個體、其中一個使用「ONTAP-NAS」、另一個使用「ONTAP-NAS-經濟」。

範例**ONTAP** ：功能組態檔

*「ONTAP-NAS」驅動程式*的NFS範例

```
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "username": "vsadmin",
    "password": "secret",
    "aggregate": "aggr1",
    "defaults": {
      "size": "10G",
      "spaceReserve": "none",
      "exportPolicy": "default"
    }
}
```

*「ONTAP-NAA-flexgroup」驅動程式*的NFS範例

```
{
    "version": 1,
    "storageDriverName": "ontap-nas-flexgroup",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "username": "vsadmin",
    "password": "secret",
    "defaults": {
      "size": "100G",
      "spaceReserve": "none",
      "exportPolicy": "default"
    }
}
```

- ONTAP NAS經濟驅動程式*的NFS範例

```
{
    "version": 1,
    "storageDriverName": "ontap-nas-economy",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.2",
    "svm": "svm_nfs",
    "username": "vsadmin",
    "password": "secret",
    "aggregate": "aggr1"
}
```

- iSCSI的「ONTAP-san」驅動程式*範例

```
{
    "version": 1,
    "storageDriverName": "ontap-san",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.3",
    "svm": "svm_iscsi",
    "username": "vsadmin",
    "password": "secret",
    "aggregate": "aggr1",
    "igroupName": "myigroup"
}
```

*「ONTAP-san經濟」驅動程式*的NFS範例

```
{
    "version": 1,
    "storageDriverName": "ontap-san-economy",
    "managementLIF": "10.0.0.1",
    "dataLIF": "10.0.0.3",
    "svm": "svm_iscsi_eco",
    "username": "vsadmin",
    "password": "secret",
    "aggregate": "aggr1",
    "igroupName": "myigroup"
}
```

元件軟體組態

除了全域組態值之外、使用Element軟體（NetApp HCI / SolidFire）時、也可使用這些選項。

選項	說明	範例
端點	https://<login>:<password>@<mvip>/json-rpc/<element-version>	https://admin:admin@192.168.160.3/json-rpc/8.0
《VIP》	iSCSI IP位址和連接埠	10.0.0.7：3260
《天王名稱》	要使用的SolidFireF租戶（如果找不到、請建立）	「Docker」
《初始器IFACE》	將iSCSI流量限制為非預設介面時、請指定介面	「預設」
《類型》	QoS規格	請參閱以下範例
"LegacyNamePrefix（名前置詞）"	升級版Trident安裝的首碼。如果您使用1.3.2版之前的Trident版本、並使用現有磁碟區執行升級、則必須設定此值、才能存取透過Volume名稱方法對應的舊磁碟區。	「netappdvp-」

「Poolidfire - san」驅動程式不支援Docker swarm。

元素軟體組態檔範例

```
{
    "version": 1,
    "storageDriverName": "solidfire-san",
    "Endpoint": "https://admin:admin@192.168.160.3/json-rpc/8.0",
    "SVIP": "10.0.0.7:3260",
    "TenantName": "docker",
    "InitiatorIFace": "default",
    "Types": [
        {
            "Type": "Bronze",
            "Qos": {
                "minIOPS": 1000,
                "maxIOPS": 2000,
                "burstIOPS": 4000
            }
        },
        {
            "Type": "Silver",
            "Qos": {
                "minIOPS": 4000,
                "maxIOPS": 6000,
                "burstIOPS": 8000
            }
        },
        {
            "Type": "Gold",
            "Qos": {
                "minIOPS": 6000,
                "maxIOPS": 8000,
                "burstIOPS": 10000
            }
        }
    ]
}
```

GCP組態上的資訊（CVS）Cloud Volumes Service

Trident現在支援使用預設CVS服務類型的較小磁碟區 "GCP"。對於以「儲存類別=軟體」建立的後端、現在磁碟區的資源配置大小最小為300 GiB。* NetApp建議客戶使用低於1TiB的磁碟區來處理非正式作業工作負載*。CVS目前在「管制可用度」下提供此功能、並不提供技術支援。

> 註冊以存取低於1TiB的磁碟區 "請按這裡"。

> 使用預設CVS服務類型「儲存類別=軟體」部署後端時、您應該取得GCP上的子1TiB Volume功能、以取得有關專案編號和專案ID的存取權。這是Trident配置子1TiB磁碟區所需的功能。如果沒有、則對於小於600 GiB的PVCs、Volume建立*將會失敗*。使用取得對低於1TiB磁碟區的存取權"這份表格"。

Trident針對預設CVS服務層級所建立的磁碟區將會配置如下：

- 小於300 GiB的PVCS會導致Trident建立300 GiB CVS Volume。
- 在300 GiB到600 GiB之間的PVCS會導致Trident建立一個所需大小的CVS Volume。
- 在600 GiB和1 TiB之間的PVCS會導致Trident建立1TiB CVS Volume。
- 大於1 TiB的PVCS會導致Trident建立所要求大小的CVS Volume。

除了全域組態值、在GCP上使用CVS時、也可使用這些選項。

選項	說明	範例
《apiRegion》	CVS帳戶區域（必填）。為GCP區域、此後端將在其中配置磁碟區。	「美國西部2」
「ProjectNumber」	GCP專案編號（必填）。您可在GCP入口網站的主畫面中找到。	「123456789012」
「hostProjectNumber」	GCP共享VPC主機專案編號（使用共享VPC時需要）	「098765432109」
「apiKey」	具有CVS管理員角色的GCP服務帳戶API金鑰（必填）。是GCP服務帳戶的私密金鑰檔（逐字複製到後端組態檔）的JSON-格式內容。服務帳戶必須具有netappcloudVolumes.admin角色。	（私密金鑰檔案的內容）
「ecretKey」	CVS帳戶秘密金鑰（必填）。可在CVS入口網站的「帳戶設定」>「API存取」中找到。	「預設」
"proxyurl"	Proxy URL（如果需要Proxy伺服器才能連線至CVS帳戶）。Proxy伺服器可以是HTTP Proxy或HTTPS Proxy。若為HTTPS Proxy、則會跳過憑證驗證、以允許在Proxy伺服器中使用自我簽署的憑證。不支援啟用驗證的**Proxy**伺服器。	「http://proxy-server-hostname/”
「nfsMountOptions」	NFS掛載選項；預設為「-o nfsver=3」	「nfsves=3、proto=tcp、timeo=600」

選項	說明	範例
《服務層級》	效能等級（標準、高級、極致）、預設為「標準」	「優質」
網路	用於CVS磁碟區的GCP網路、預設為「預設」	「預設」

> 如果使用共享VPC網路、您應該同時指定「ProjectNumber」和「hostProjectNumber」。在這種情況下、「專案編號」是服務專案、「主機專案編號」則是主機專案。

> NetApp Cloud Volumes Service 適用於GCP的支援不支援大小小於100 GiB的CVS效能磁碟區、或大小小於300 GiB的CVS磁碟區。為了讓部署應用程式更輕鬆、Trident會在要求太小的磁碟區時、自動建立大小最小的磁碟區。

在GCP上使用CVS時、可使用這些預設的Volume選項設定。

選項	說明	範例
「匯出規則」	NFS存取清單（位址和/或CIDR子網路）、預設為「0.00.0.0/0」	「10.0.1.0/24、10.0.2.100」
「napshotDir	控制「.snapshot」目錄的可見度	「假」
「快照保留區」	Snapshot保留百分比、預設值為「」、接受CVS預設值為0	「10」
《大小》	Volume大小、預設為「100GiB」	「10T」

GCP組態檔上的CVS範例

```
{
    "version": 1,
    "storageDriverName": "gcp-cvs",
    "projectNumber": "012345678901",
    "apiRegion": "us-west2",
    "apiKey": {
        "type": "service_account",
        "project_id": "my-gcp-project",
        "private_key_id": "1234567890123456789012345678901234567890",
        "private_key": "-----BEGIN PRIVATE KEY-----
\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZ
srrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisI
sAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSa
PIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZN
```

```
chRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlz
ZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl
/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kw
s8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY
9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHc
zZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHi
sIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOgu
SaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyA
ZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGz
lzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3
bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4
Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5o
jY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nzn
HczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrt
HisIsAbOguSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbO
guSaPIKeyAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKe
yAZNchRAGzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRA
GzlzZE4jK3bl/qp8B4Kws8zX5ojY9m\nznHczZsrrtHisIsAbOguSaPIKeyAZNchRAGzlzZE4j
K3bl/qp8B4Kws8zX5ojY9m\nXsYg6gyxy4zq7OlwWgLwGa==\n-----END PRIVATE
KEY-----\n",
        "client_email": "cloudvolumes-admin-sa@my-gcp-
project.iam.gserviceaccount.com",
        "client_id": "123456789012345678901",
        "auth_uri": "https://accounts.google.com/o/oauth2/auth",
        "token_uri": "https://oauth2.googleapis.com/token",
        "auth_provider_x509_cert_url":
"https://www.googleapis.com/oauth2/v1/certs",
        "client_x509_cert_url":
"https://www.googleapis.com/robot/v1/metadata/x509/cloudvolumes-admin-
sa%40my-gcp-project.iam.gserviceaccount.com"
    },
    "proxyURL": "http://proxy-server-hostname/"
}
```


組態Azure NetApp Files

若要設定及使用 "Azure NetApp Files" 後端、您將需要下列項目：

- Azure訂閱提供的「SubscriptionID」Azure NetApp Files （含功能不支援的功能）
- 來自於的「TenantId」、「clientID」和「clientSecret」 "應用程式註冊" 在Azure Active Directory中、具備Azure NetApp Files 充分的權限執行此功能
- Azure位置、至少包含一個 "委派的子網路"

> 如果Azure NetApp Files 您是第一次使用或是在新的位置使用、則需要進行一些初始組態設定 "快速入門指南" 將帶您逐步完成。

Astra Trident 21.04.0及更早版本不支援手動QoS容量資源池。

選項	說明	預設
「分度」	永遠為1	
「torageDriverName」	「azure-NetApp-Files」	
「後端名稱」	儲存後端的自訂名稱	驅動程式名稱+「_」+隨機字元
《訂閱ID》	Azure訂閱的訂閱ID	
「TenantId」	應用程式註冊的租戶ID	
"clientID"	應用程式註冊的用戶端ID	
「客戶機密」	應用程式註冊的用戶端機密	
《服務層級》	「標準」、「優質」或「超高」之一	「」（隨機）
位置	Azure位置的名稱將會在中建立新的Volume	「」（隨機）
「虛擬化網路」	具有委派子網路的虛擬網路名稱	「」（隨機）
《Subnet》	委派給「icrosoft.Netapp/volumes`」的子網路名稱	「」（隨機）
「nfsMountOptions」	精細控制NFS掛載選項	「-o nfsves=3」
《限制Volume大小》	如果要求的磁碟區大小高於此值、則資源配置失敗	「」（預設不強制執行）

支援大小不低於100 GB的磁碟區的支援服務。Azure NetApp Files為了讓應用程式更容易部署、Trident會在要求較小的磁碟區時自動建立100 GB磁碟區。

您可以在組態的特定區段中、使用這些選項來控制預設配置每個Volume的方式。

選項	說明	預設
「匯出規則」	新磁碟區的匯出規則。必須是以逗號分隔的清單、以CIDR表示法列出所有的IPv4位址或IPv4子網路組合。	「0.00.0.0/0」
「napshotDir	控制「.snapshot」目錄的可見度	「假」
《大小》	新磁碟區的預設大小	「100公克」

範例**Azure NetApp Files**

*範例1：azure-NetApp-files*的最小後端組態

這是絕對最低的後端組態。有了這項組態、Trident將會探索您在全球各地委派給ANF的所有NetApp帳戶、容量資源池和子網路、並隨機將新磁碟區放在其中一個上。

當您剛開始使用ANF和試用產品時、此組態非常實用。 但實際上、您想要為您所配置的磁碟區提供額外的範圍、以確保它們具有您想要的特性、並最終建立在接近使用該磁碟區之運算的網路上。如需詳細資料、請參閱後續範例。

```
{
    "version": 1,
    "storageDriverName": "azure-netapp-files",
    "subscriptionID": "9f87c765-4774-fake-ae98-a721add45451",
    "tenantID": "68e4f836-edc1-fake-bff9-b2d865ee56cf",
    "clientID": "dd043f63-bf8e-fake-8076-8de91e5713aa",
    "clientSecret": "SECRET"
}
```

*範例2：azure-NetApp-files*的單一位置和特定服務層級

此後端組態可將磁碟區放置在Azure的「東向」位置、位於「優質」容量資源池中。Trident會自動探索委派給該位置ANF的所有子網路、並隨機在其中一個子網路上放置新磁碟區。

```
{
    "version": 1,
    "storageDriverName": "azure-netapp-files",
    "subscriptionID": "9f87c765-4774-fake-ae98-a721add45451",
    "tenantID": "68e4f836-edc1-fake-bff9-b2d865ee56cf",
    "clientID": "dd043f63-bf8e-fake-8076-8de91e5713aa",
    "clientSecret": "SECRET",
    "location": "eastus",
    "serviceLevel": "Premium"
}
```

*範例3：azure-NetApp-files*的進階組態

此後端組態可進一步將磁碟區放置範圍縮小至單一子網路、並修改部分Volume資源配置預設值。

```
{
    "version": 1,
    "storageDriverName": "azure-netapp-files",
    "subscriptionID": "9f87c765-4774-fake-ae98-a721add45451",
    "tenantID": "68e4f836-edc1-fake-bff9-b2d865ee56cf",
    "clientID": "dd043f63-bf8e-fake-8076-8de91e5713aa",
    "clientSecret": "SECRET",
    "location": "eastus",
    "serviceLevel": "Premium",
    "virtualNetwork": "my-virtual-network",
    "subnet": "my-subnet",
    "nfsMountOptions": "nfsvers=3,proto=tcp,timeo=600",
    "limitVolumeSize": "500Gi",
    "defaults": {
        "exportRule": "10.0.0.0/24,10.0.1.0/24,10.0.2.100",
        "size": "200Gi"
    }
}
```

範例**4**：使用**azure-NetApp-fil**等 檔案的虛擬儲存資源池

此後端組態定義多個 "儲存資源池" 在單一檔案中。當您有多個容量集區支援不同的服務層級、而且想要在Kubernetes中建立代表這些層級的儲存類別時、這很有用。

這只是在虛擬儲存資源池及其標籤的威力表面留下一層痕跡。

```
{
    "version": 1,
    "storageDriverName": "azure-netapp-files",
    "subscriptionID": "9f87c765-4774-fake-ae98-a721add45451",
    "tenantID": "68e4f836-edc1-fake-bff9-b2d865ee56cf",
    "clientID": "dd043f63-bf8e-fake-8076-8de91e5713aa",
    "clientSecret": "SECRET",
    "nfsMountOptions": "nfsvers=3,proto=tcp,timeo=600",
    "labels": {
        "cloud": "azure"
    },
    "location": "eastus",

    "storage": [
        {
            "labels": {
                "performance": "gold"
            },
            "serviceLevel": "Ultra"
        },
        {
            "labels": {
                "performance": "silver"
            },
            "serviceLevel": "Premium"
        },
        {
            "labels": {
                "performance": "bronze"
            },
            "serviceLevel": "Standard",
        }
    ]
}
```

已知問題與限制

尋找使用Astra Trident搭配Docker時已知問題和限制的相關資訊。

從舊版升級**Trident Docker Volume**外掛程式至**20.10**及更新版本、會導致升級失敗、且不會發生此類檔案或目錄錯誤。

因應措施

1. 停用外掛程式。

```
docker plugin disable -f netapp:latest
```

2. 移除外掛程式。

```
docker plugin rm -f netapp:latest
```

3. 提供額外的「config」參數、重新安裝外掛程式。

```
docker plugin install netapp/trident-plugin:20.10 --alias netapp --grant
-all-permissions config=config.json
```

Volume名稱長度必須至少2個字元。

> 這是Docker用戶端的限制。用戶端會將單一字元名稱解譯為Windows路徑。 "請參閱錯誤25773"。

Docker swarm具有某些行為、可防止Astra Trident在每個儲存設備和驅動程式組合中支援它。

- Docker swarm目前使用Volume名稱、而非Volume ID做為其唯一的Volume識別碼。
- Volume要求會同時傳送至swarm叢集中的每個節點。
- Volume外掛程式（包括Astra Trident）必須在swarm叢集中的每個節點上獨立執行。由於功能不一、ONTAP「ONTAP-NAS」和「ONTAP-SAN」驅動程式的運作方式、因此它們是唯一能夠在這些限制範圍內運作的驅動程式。

其餘的驅動程式可能會受到競爭狀況等問題的影響、導致在單一要求中建立大量磁碟區、而不需要明確的「贏家」；例如、Element的功能可讓磁碟區擁有相同名稱但不同ID。

NetApp已向Docker團隊提供意見回饋、但沒有任何未來追索的跡象。

如果配置的是某個功能區、則如果第二個功能區的一個或多個集合體與要配置的功能區相同、則不提供第二個功能區。FlexGroup ONTAP FlexGroup FlexGroup FlexGroup

常見問題集

尋找有關安裝、設定、升級及疑難排解Astra Trident的常見問題解答。

一般問題

Astra Trident多久發佈一次？

Astra Trident每三個月發佈一次：1月、4月、7月和10月。這是Kubernetes發表後的一個月。

Astra Trident是否支援特定Kubernetes版本中發行的所有功能？

Astra Trident通常不支援Kubernetes的Alpha功能。Trident可能支援Kubernetes試用版之後的兩個Trident版本中的試用版功能。

Astra Trident是否因其運作而依賴其他NetApp產品？

Astra Trident並不依賴其他NetApp軟體產品、而是以獨立應用程式的形式運作。不過、您應該擁有NetApp後端儲存設備。

如何取得完整的Astra Trident組態詳細資料？

使用「tridentctl Get」命令可取得有關Astra Trident組態的詳細資訊。

我可以取得Astra Trident如何配置儲存設備的指標嗎？

是的。Trident 20.01推出Prometheus端點、可用來收集有關Astra Trident作業的資訊、例如受管理的後端數目、已配置的磁碟區數目、已用位元組等。您也可以使用Cloud Insights 此功能進行監控與分析。

使用Astra Trident做為「csi資源配置工具」時、使用者是否會有變更？

不可以就使用者體驗和功能而言、沒有任何改變。使用的置備程式名稱是「csi.trident.netapp.io`」。如果您想要使用目前和未來版本所提供的所有新功能、建議使用這種安裝Astra Trident的方法。

在Kubernetes叢集上安裝及使用Astra Trident

支援的etcd版本為何？

Astra Trident不再需要「etcd」。它使用客戶需求日來維持狀態。

Astra Trident是否支援從私有登錄進行離線安裝？

可以、Astra Trident可以離線安裝。請參閱 "請按這裡"。

我可以從遠端安裝Astra Trident嗎？

是的。Astra Trident 18.10及更新版本可從任何對叢集具有「kubectl」存取權的機器、支援遠端安裝功能。驗

證「kubecll」存取後（例如、從遠端機器啟動「kubecll Get nodes」命令進行驗證）、請遵循安裝指示。

我可以使用**Astra Trident**設定高可用度嗎？

Astra Trident是以單一執行個體安裝為Kubernetes部署（ReplicaSet）、因此內建HA。您不應增加部署中的複本數量。如果安裝Astra Trident的節點遺失、或Pod無法存取、Kubernetes會自動將Pod重新部署至叢集中的健全節點。Astra Trident僅適用於控制面板、因此若重新部署Astra Trident、目前安裝的Pod不會受到影響。

Astra Trident是否需要存取**Kube**系統命名空間？

Astra Trident從Kubernetes API伺服器讀取、以判斷應用程式何時要求新的PVCS、因此需要存取Kube-system。

Astra Trident使用哪些角色和權限？

Trident安裝程式會建立Kubernetes ClusterRole、該叢集具有特定存取權、可存取叢集的PersistentVolume、PersistentVolume Claim、StorageClass及Kubernetes叢集的Secret資源。請參閱 "請按這裡"。

我可以在本機產生安裝所用的確切資訊清單檔案嗎？

如有需要、您可以在本機產生及修改安裝所使用的確切資訊清單檔案Astra Trident。請參閱 "請按這裡"。

我是否可以針對**ONTAP** 兩個獨立**Kubernetes**叢集的兩個**Astra Trident**執行個體、共用相同的支援功能後端**SVM**？

雖然不建議使用、但您可以針對兩個Astra Trident執行個體使用相同的後端SVM。在安裝期間為每個執行個體指定唯一的Volume名稱、並（或）在「Setup/backend.json」檔案中指定唯一的「儲存前置字元」參數。這是為了確保FlexVol 兩個執行個體都不會使用相同的功能。

是否能在**ContainerLinux**（前身為**CoreOS**）下安裝**Astra Trident**？

Astra Trident只是Kubernetes Pod、可安裝在Kubernetes執行的任何位置。

我可以將**Astra Trident**搭配**NetApp Cloud Volumes ONTAP** 功能嗎？

是的、AWS、Google Cloud和Azure均支援Astra Trident。

Astra Trident是否能與**Cloud Volumes Services**搭配運作？

是的、Astra Trident不僅支援Azure NetApp Files Azure中的「不二」服務、Cloud Volumes Service 也支援GCP中的「不二」功能。

疑難排解與支援

NetApp是否支援**Astra Trident**？

儘管Astra Trident是開放原始碼且免費提供、但只要您的NetApp後端受到支援、NetApp就能提供完整支援。

如何提出支援案例？

若要提出支援案例、請執行下列其中一項：

1. 請聯絡您的支援客戶經理、以取得索取機票的協助。
2. 請聯絡以提出支援案例 "NetApp支援"。

如何產生支援記錄套裝組合？

您可以執行「tridentctl logs -A」來建立支援服務組合。除了在套裝組合中擷取的記錄之外、請擷取kubelet記錄、以診斷Kubernetes端的掛載問題。取得Kubernetet記錄的指示會根據Kubernetes的安裝方式而有所不同。

如果我需要提出新功能的要求、該怎麼辦？

在上建立問題 "Trident Github" 並在問題的主題和說明中提及* RFE*。

我該在哪裡提出瑕疵？

在上建立問題 "Astra Trident Github"。請務必附上與問題相關的所有必要資訊和記錄。

如果我有關於**Astra Trident**的快速問題、而我需要澄清、會發生什麼情況？是否有社群或論壇？

如果您有任何問題、問題或要求、請透過我們的聯絡我們 "可寬延" 團隊或GitHub。

我的儲存系統密碼已變更、**Astra Trident**已無法運作、我該如何恢復？

使用「tridentctl update backend myBackend -f」（三重更新後端myBackend -f）來更新後端的密碼。以您的後端名稱取代範例中的「myBackend」、並以正確的「backend.json」檔案路徑取代「/path/to_new_backend.json」。

Astra Trident找不到**Kubernetes**節點。如何修正此問題？

Astra Trident找不到Kubernetes節點的原因可能有兩種。這可能是因為Kubernetes內的網路問題或DNS問題。在每個Kubernetes節點上執行的Trident節點取消影像集、必須能夠與Trident控制器通訊、才能在Trident中登錄節點。如果在安裝Astra Trident之後發生網路變更、您只會遇到新增至叢集的Kubernetes節點的問題。

如果**Trident Pod**毀損、我會遺失資料嗎？

如果Trident Pod遭到破壞、資料將不會遺失。Trident的中繼資料儲存在CRD物件中。所有由Trident提供的PV均可正常運作。

升級**Astra Trident**

我可以直接從舊版本升級至新版本（跳過幾個版本）嗎？

NetApp支援將Astra Trident從一個重大版本升級至下一個重大版本。您可以從11.xx版升級至19.xx、19.xx版升級至20.xx版、依此類推。在正式作業部署之前、您應該先在實驗室中測試升級。

是否能將Trident降級至先前的版本？

如果您想要降級、有許多因素需要評估。請參閱 "降級一節"。

管理後端和磁碟區

我是否需要在ONTAP 一個後端定義檔案中定義管理和資料生命期？

NetApp建議在後端定義檔中同時使用這兩個檔案。不過、管理LIF是唯一必須執行的功能。

Astra Trident是否能設定CHAP以ONTAP 供後端使用？

是的。從20.04開始、Astra Trident支援雙向CHAP以利ONTAP 實現後端。這需要在後端組態中設定「useCHAP=true」。

如何使用Astra Trident管理匯出原則？

Astra Trident可從20.04版起、動態建立及管理匯出原則。如此一來、儲存管理員就能在其後端組態中提供一或多個CIDR區塊、並將位於這些範圍內的Trident新增節點IP、加入其所建立的匯出原則。如此一來、Astra Trident就能自動管理新增和刪除在指定CIDR內具有IP的節點規則。此功能需要「csi Trident」。

我們可以在DataLIF中指定連接埠嗎？

Astra Trident 19.01及更新版本支援在DataLIF中指定連接埠。在「backend.json」檔案中將其設定為「managementLIF」：<IP位址>:<port>」。例如、如果管理LIF的IP位址為192.0.2、連接埠為1000、請設定「managementLIF」：「192.0.2：2.1：1000」。

IPv6位址是否可用於管理和資料生命量？

是的。Astra Trident 20.01支援定義用於管理的IPv6位址LIF和ONTAP 用於支援不支援的dataLIF參數。您應該確保地址遵循IPv6語義、且管理LIF是在方括弧內定義（例如、「」[ec0d:6504：a9c1：ae67：53d1：4bdf：ab32：e233]）。您也應該確保使用「-use IPv6」旗標來安裝Astra Trident、以便透過IPv6運作。

是否能在後端更新管理LIF？

可以、您可以使用「tridentctl update backend」命令來更新後端管理LIF。

是否能在後端更新Data LIF？

否、無法在後端更新Data LIF。

我可以在Astra Trident中為Kubernetes建立多個後端嗎？

Astra Trident可同時支援多個後端、無論是使用相同的驅動程式或不同的驅動程式。

Astra Trident如何儲存後端認證資料？

Astra Trident將後端認證儲存為Kubernetes Secrets。

Astra Trident如何選擇特定的後端？

如果後端屬性無法用於自動選擇某個類的正確池，則可使用"scoragePools"和"additionalStoragePools"參數來選擇特定的池集區集區集區集區。

如何確保Astra Trident不會從特定後端進行資源配置？

使用「排除StoragePools」參數來篩選Astra Trident將用於資源配置的資源池集區集區、並移除任何相符的資源池。

如果有多個相同類型的後端、Astra Trident如何選擇要使用的後端？

如果有多個相同類型的已設定後端、Astra Trident會根據「儲存類別」和「PeristentVolume Claim」中的參數、選取適當的後端。例如、如果有多個ONTAP-NAS驅動程式後端、Astra Trident會嘗試搭配「torageClass」和「PeristentVolume Claim」中的參數、並搭配後端、以滿足「torageClass」和「PeristentVolume Claim」中列出的需求。如果有多個後端符合要求、則Astra Trident會隨機從其中一個後端選取。

Astra Trident是否支援採用Element / SolidFire的雙向CHAP？

是的。

Astra Trident如何將qtree部署在ONTAP 一個邊角捲上？單一磁碟區可部署多少qtree？

「ONTAP-NAS-節約」驅動程式可在同FlexVol 一個範圍內建立多達200個qtree（可設定為50到300個）、每個叢集節點可建立100、000個qtree、每個叢集可建立240萬個qtree。當您輸入經濟型驅動程式所提供的全新「PersistentVolume Claim」時、駕駛會查看FlexVol 是否已存在可為新Qtree提供服務的功能。如果FlexVol 不存在能夠服務Qtree的功能、FlexVol 就會建立新的功能。

我要如何為ONTAP 以NAS配置的Volume設定Unix權限？

您可以在後端定義檔中設定參數、以設定Astra Trident所佈建的Volume上的Unix權限。

如何在ONTAP 配置Volume時、設定一組明確的靜態NFS掛載選項？

依預設、Astra Trident不會使用Kubernetes將掛載選項設為任何值。若要在Kubernetes儲存類別中指定掛載選項、請遵循所提供的範例 "請按這裡"。

如何將已配置的磁碟區設定為特定的匯出原則？

若要允許適當的主機存取磁碟區、請使用後端定義檔中設定的「exportPolicy」參數。

如何透過Astra Trident搭配ONTAP 使用才能設定Volume加密？

您可以使用後端定義檔中的加密參數、在Trident所提供的磁碟區上設定加密。

什麼是透過ONTAP Astra Trident實作QoS for Sfor Sfor Sfor the S星？

使用「儲存類」來實作ONTAP QoS以利實現。

如何透過Astra Trident指定精簡或完整的資源配置？

支援精簡或密集資源配置的支援。ONTAP此功能預設為精簡配置。ONTAP如果需要完整資源配置、您應該設定後端定義檔或「儲存類別」。如果兩者都已設定、則「儲存類別」優先。設定ONTAP 下列項目以供參考：

1. 在「儲存類別」上、將「資源配置類型」屬性設為「完整」。
2. 在後端定義檔中、將「backend spaceReserve參數」設為Volume、以啟用厚磁碟區。

如何確保即使意外刪除了PVC,也不會刪除使用中的磁碟區？

Kubernetes從1.10版開始自動啟用PVc保護。

我可以擴充由Astra Trident所建立的NFS PVCs嗎？

是的。您可以擴充由Astra Trident所建立的永久虛電路。請注意、Volume自動擴充ONTAP 是不適用於Trident的功能。

如果我的磁碟區是在Astra Trident以外建立的、我可以將它匯入Astra Trident嗎？

從19.04開始、您可以使用Volume匯入功能將磁碟區帶入Kubernetes。

我可以在磁碟區處於SnapMirror資料保護（DP）或離線模式時匯入該磁碟區嗎？

如果外部磁碟區處於DP模式或離線、則磁碟區匯入會失敗。您會收到下列錯誤訊息：

```
Error: could not import volume: volume import failed to get size of
volume: volume <name> was not found (400 Bad Request) command terminated
with exit code 1.
Make sure to remove the DP mode or put the volume online before importing
the volume.
```

我可以擴充由Astra Trident建立的iSCSI PVCs嗎？

Trident 19.10支援使用csi資源配置程式擴充iSCSI PV。

資源配額如何轉譯至NetApp叢集？

只要NetApp儲存設備具備容量、Kubernetes儲存資源配額就能運作。當NetApp儲存設備因為容量不足而無法遵守Kubernetes配額設定時、Astra Trident會嘗試進行資源配置、但卻發生錯誤。

我可以使用Astra Trident建立Volume Snapshot嗎？

是的。Astra Trident支援從快照建立隨需磁碟區快照和持續磁碟區。若要從快照建立PV、請確定已啟用「Volume SnapshotDataSource」功能閘道。

哪些驅動程式支援Astra Trident Volume快照？

到目前爲止、我們的「ONTAP-NAS」、「ONTAP-NAS-flexgroup」、「ONTAP-SAN」、「ONTAP-san經濟型」、「Poolidfire SAN」、「GCP-CVS」、以及「azure-NetApp-fil」後端驅動程式。

我要如何針對Astra Trident提供ONTAP 的含有「支援」功能的磁碟區進行快照備份？

這可在「ONTAP-NAS」、「ONTAP-SAN」及「ONTAP-NAA-flexgroup」等驅動程式上使用。您也可以針對FlexVol 「ontap-san經濟」驅動程式指定「快照原則」、以利執行此作業。

這也可在「ONTAP-NAS-節約」驅動程式中找到、但FlexVol 可在「不」於qtree層級精細度上找到。若要讓Astra Trident提供的磁碟區快照、請將後端參數選項「快照原則」設為ONTAP 在支援後端上定義的所需快照原則。Astra Trident不知道儲存控制器所拍攝的任何快照。

我可以為透過Astra Trident佈建的磁碟區設定快照保留百分比嗎？

是的、您可以在後端定義檔中設定「快照保留區」屬性、保留特定百分比的磁碟空間、以便透過Astra Trident來儲存快照複本。如果您在後端定義檔中設定了「快照原則」和「快照保留」、則快照保留百分比會根據後端檔案中所述的「快照保留」百分比來設定。如果未提及「快照保留」百分比數字、ONTAP 則根據預設、將快照保留百分比設為5。如果將「快照原則」選項設為「無」、則快照保留百分比會設為0。

我可以直接存取Volume Snapshot目錄並複製檔案嗎？

是的、您可以在後端定義檔中設定「shapshotDir’參數、以存取Trident所佈建之磁碟區上的Snapshot目錄。

我可以透過Astra Trident為磁碟區設定SnapMirror嗎？

目前、SnapMirror必須使用ONTAP CLI或OnCommand 《系統管理程式》從外部設定。

如何將持續磁碟區還原至特定ONTAP 的不還原快照？

若要將磁碟區還原ONTAP 成一個無法修復的快照、請執行下列步驟：

1. 靜止使用持續磁碟區的應用程式Pod。
2. 透過ONTAP NetApp CLI或OnCommand 《系統管理程式》回復至所需的快照。
3. 重新啟動應用程式Pod。

是否能在已設定負載共享鏡射的SVM上、對磁碟區進行Trident資源配置？

您可以為透過NFS提供資料的SVM根磁碟區建立負載共享鏡像。針對Trident所建立的磁碟區、自動更新負載共享鏡像。ONTAP這可能會導致掛載磁碟區延遲。使用Trident建立多個磁碟區時、資源配置磁碟區會仰賴ONTAP於更新負載共享鏡像。

如何區分每位客戶/租戶的儲存類別使用量？

Kubernetes不允許命名空間中的儲存類別。不過、您可以使用Kubernetes來限制每個命名空間的特定儲存類別使用量、方法是使用儲存資源配額（每個命名空間）。若要拒絕特定儲存設備的特定命名空間存取、請將該儲存類別的資源配額設為0。

支援

Astra Trident是正式支援的NetApp專案。您可以使用任何標準機制聯絡NetApp、取得所需的企業級支援。

在「Container」通路上、也有一個充滿活力的公共社群、由容器使用者（包括Astra Trident開發人員）所共同參與 "NetApp的Slack工作"。這是您提出專案相關一般問題、並與志同道合的同儕討論相關主題的好地方。

疑難排解

請使用此處提供的指標來疑難排解安裝及使用Astra Trident時可能遇到的問題。

> 如需Astra Trident的協助、請使用「tridentctl logs -A -n trident」建立支援套裝組合、並將其傳送至「NetApp支援<取得協助>」。

> 如需疑難排解文章的完整清單、請參閱 "NetApp知識庫（需要登入）"。您也可以找到Astra相關疑難排解問題的相關資訊 "請按這裡"。

一般疑難排解

- 如果Trident pod無法正常啟動（例如、當Trident pod卡在「ContainerCreating」階段、且只有兩個可用的容器）、則執行「kubechtl -n trident描述部署Trident」和「kubectl -n trident描述pod trident -」 提供更多洞見。取得kubelet記錄（例如透過「journalctl -xeu kubelet」）也很有幫助。
- 如果Trident記錄中的資訊不足、您可以根據安裝選項、將「-d」旗標傳給安裝參數、嘗試啟用Trident的偵錯模式。

  然後使用「/tridentctl logs -n triident」確認偵錯設定、並在記錄中搜尋「level =偵錯msg」。

  與營運者一起安裝

  ```
  # kubectl patch torc trident -n <namespace> --type=merge -p
  '{"spec":{"debug":true}}'
  ```

  這會重新啟動所有Trident Pod、可能需要數秒鐘的時間。您可以查看輸出「kubecll Get pod - n triident」中的「年齡」欄位來檢查。

  對於Astra Trident 20.07和20.10、請使用「tprov」取代「torc」。

  與**Helm**一起安裝

  ```
  $ helm upgrade <name> trident-operator-21.07.1-custom.tgz --set
  tridentDebug=true`
  ```

  安裝試用版

  ```
  ./tridentctl uninstall -n trident
  ./tridentctl install -d -n trident
  ```

- 您也可以在後端定義中加入「debugTraceFlags」、以取得每個後端的偵錯記錄。例如、在Trident記錄中加入「debugTraceFlags：｛"API"：true、「method」：true、｝」以取得API呼叫和方法反向。現有的後端可設定「debugTraceFlags」、設定為「tridentctl後端更新」。
- 使用RedHat CoreOS時、請確定已在工作節點上啟用「iscsid」、並預設為啟動。您可以使用OpenShift機器組態或修改點火模板來完成此作業。

- 使用Trident時可能會遇到的常見問題 "Azure NetApp Files" 當租戶和用戶端機密來自權限不足的應用程式登錄時。如需Trident需求的完整清單、請參閱 "Azure NetApp Files" 組態：
- 如果將PV掛載到容器時發生問題、請確定已安裝並執行「rpcbind」。使用主機作業系統所需的套件管理程式、檢查「rpcbind」是否正在執行。您可以執行「stystemctl狀態rpcbind」或其等效項目、來檢查「rpcbind」服務的狀態。
- 如果Trident後端回報雖然曾經工作、但仍處於「失敗」狀態、則可能是因為變更與後端相關的SVM/admin認證資料所致。使用「tridentctl update backend」更新後端資訊、或是退回Trident pod、將可修正此問題。
- 如果您要升級Kubernetes叢集和（或）Trident以使用beta Volume Snapshot、請確定已完全移除所有現有的Alpha Snapshot CRS。然後、您可以使用「tridentctl obliviate alpha-snapsnapshot crd」命令來刪除Alpha Snapshot CRD。請參閱 "這篇部落格" 瞭解移轉Alpha快照所需的步驟。
- 如果在容器執行時間安裝Trident with Docker時遇到權限問題、請嘗試使用「-in cluster =fals'」旗標來安裝Trident。這不會使用安裝程式Pod、也不會因為「Trident安裝程式」使用者而造成權限問題。
- 使用「uninstall參數<uninstalling Trident >」來在執行失敗後進行清理。根據預設、指令碼不會移除Trident所建立的客戶需求日、即使在執行中的部署中、也能安全地解除安裝及再次安裝。
- 如果您想要降級至舊版Trident、請先執行「tridenctl uninstall」命令來移除Trident。下載所需的 "Trident版本" 並使用「tridentctl install」命令進行安裝。只有在沒有建立新的PV、且尚未對現有的PV/後端/儲存類別進行任何變更時、才考慮降級。由於Trident現在使用CRD來維護狀態、因此所有建立的儲存實體（後端、儲存類別、PV和Volume Snapshot）都有「Associated CRD objects <Kubernetes CustomResourcesDefinition Objects>」、而非寫入先前安裝的Trident版本所使用之PV的資料。移回舊版時、新建立的**PV**將無法使用。*對物件所做的變更、例如後端、PV、儲存類別和Volume快照（已建立/更新/刪除）、在降級*時、Trident將無法看到。Trident安裝的舊版Trident所使用的PV、仍可供Trident查看。返回舊版不會中斷已使用舊版建立之PV的存取、除非已升級。
- 若要完全移除Trident、請執行「tridentctl obliviate crd」命令。這會移除所有CRD物件、並取消定義客戶需求日。Trident將不再管理任何已配置的PV。

  > 之後需要從頭重新設定Trident。

- 成功安裝之後、如果某個永久虛擬磁碟卡在「Pending」（擱置）階段、執行「KECBECTL描述永久虛擬磁碟」可提供有關Trident為何無法為此永久虛擬磁碟配置PV的其他資訊。

使用操作員疑難排解不成功的**Trident**部署

如果您使用運算子來部署Trident、則「TridentOrchestrator」的狀態會從「安裝」變更為「安裝」。如果您看到「失敗」狀態、而且操作員本身無法恢復、您應該執行下列命令來檢查操作員的記錄：

```
tridentctl logs -l trident-operator
```

追蹤Trident運算子容器的記錄可以指出問題所在。例如、其中一個問題可能是無法從無線環境中的上游登錄擷取所需的容器映像。

若要瞭解Trident安裝失敗的原因、您應該看看「TridentOrchestrator」狀態。

```
$ kubectl describe torc trident-2
Name:         trident-2
Namespace:
Labels:       <none>
Annotations:  <none>
API Version:  trident.netapp.io/v1
Kind:         TridentOrchestrator
...
Status:
  Current Installation Params:
    IPv6:
    Autosupport Hostname:
    Autosupport Image:
    Autosupport Proxy:
    Autosupport Serial Number:
    Debug:
    Enable Node Prep:
    Image Pull Secrets:         <nil>
    Image Registry:
    k8sTimeout:
    Kubelet Dir:
    Log Format:
    Silence Autosupport:
    Trident Image:
  Message:                      Trident is bound to another CR 'trident'
  Namespace:                    trident-2
  Status:                       Error
  Version:
Events:
  Type     Reason  Age                From                        Message
  ----     ------  ----               ----                        -------
  Warning  Error   16s (x2 over 16s)  trident-operator.netapp.io  Trident
is bound to another CR 'trident'
```

此錯誤表示已存在用於安裝Trident的「TridentOrchestrator」。由於每個Kubernetes叢集只能有一個Trident執行個體、因此營運者可確保在任何指定時間只存在一個可建立的作用中「TridentOrchestrator」。

此外、觀察Trident Pod的狀態、通常會指出是否有不正確的情況。

```
$ kubectl get pods -n trident

NAME                                READY   STATUS             RESTARTS
AGE
trident-csi-4p5kq                   1/2     ImagePullBackOff   0
5m18s
trident-csi-6f45bfd8b6-vfrkw        4/5     ImagePullBackOff   0
5m19s
trident-csi-9q5xc                   1/2     ImagePullBackOff   0
5m18s
trident-csi-9v95z                   1/2     ImagePullBackOff   0
5m18s
trident-operator-766f7b8658-ldzsv   1/1     Running            0
8m17s
```

您可以清楚看到、由於未擷取一或多個容器映像、所以Pod無法完全初始化。

若要解決此問題、您應該編輯「TridentOrchestrator」。或者、您也可以刪除「TridentOrchestrator」、然後使用修改後的準確定義來建立新定義。

疑難排解使用tridentctl進行不成功的Trident部署

為了協助您找出問題所在、您可以使用「-d'」引數再次執行安裝程式、這會開啟偵錯模式、並協助您瞭解問題所在：

```
./tridentctl install -n trident -d
```

在解決此問題之後、您可以依照下列步驟清理安裝、然後再次執行「tridentctl install」命令：

```
./tridentctl uninstall -n trident
INFO Deleted Trident deployment.
INFO Deleted cluster role binding.
INFO Deleted cluster role.
INFO Deleted service account.
INFO Removed Trident user from security context constraint.
INFO Trident uninstallation succeeded.
```

最佳實務做法與建議

部署

部署Astra Trident時、請使用此處列出的建議。

部署至專屬命名空間

"命名空間" 在不同的應用程式之間提供管理分離、是資源共用的障礙。例如、某個命名空間的某個永久虛電路無法從另一個命名空間使用。Astra Trident為Kubernetes叢集中的所有命名空間提供PV資源、並因此運用具有較高權限的服務帳戶。

此外、存取Trident Pod可能會讓使用者存取儲存系統認證和其他敏感資訊。請務必確保應用程式使用者和管理應用程式無法存取Trident物件定義或Pod本身。

使用配額和範圍限制來控制儲存使用量

Kubernetes有兩項功能、一旦結合、就能提供強大的機制來限制應用程式的資源使用量。。 "儲存配額機制" 可讓系統管理員針對每個命名空間來實作全域和儲存類別的容量和物件數使用限制。此外、請使用 "範圍限制" 確保在將要求轉送至資源配置程式之前、永久虛擬機器要求的最小值和最大值都在內。

這些值是以每個命名空間為基礎來定義、這表示每個命名空間都應該定義符合其資源需求的值。如需相關資訊、請參閱此處 "如何運用配額"。

儲存組態

NetApp產品組合中的每個儲存平台都有獨特的功能、無論應用程式是否為容器化的應用程式帶來好處。Trident可搭配ONTAP 使用沒有一個平台比其他平台更適合所有應用程式和案例、不過在選擇平台時、應考慮應用程式和管理裝置團隊的需求。

您應該遵循主機作業系統的基礎最佳實務做法、以及您所使用的傳輸協定。或者、您可能想要考慮在可用的情況下、將應用程式最佳實務做法與後端、儲存類別和永久虛擬基礎架構設定整合、以最佳化特定應用程式的儲存。

最佳實務做法**ONTAP Cloud Volumes ONTAP**

瞭解設定ONTAP 適用於Cloud Volumes ONTAP Trident的功能性及功能性的最佳實務做法。

以下建議是設定ONTAP 以容器化工作負載為基礎的功能指南、這些功能會消耗Trident動態配置的磁碟區。每個項目都應考量及評估是否適合您的環境。

使用**Trident**專用的**SVM**

儲存虛擬機器（SVM）可隔離ONTAP 及管理各個客戶在一個系統上的區隔。將SVM專用於應用程式可委派權限、並可套用最佳實務做法來限制資源使用量。

SVM管理有多種選項可供選擇：

- 在後端組態中提供叢集管理介面、以及適當的認證、然後指定SVM名稱。
- 使用ONTAP 支援功能的支援中心或CLI、為SVM建立專屬的管理介面。

- 與NFS資料介面共用管理角色。

在每種情況下、介面都應該位於DNS中、而且在設定Trident時、應該使用DNS名稱。這有助於推動一些DR案例、例如不使用網路身分保留功能的SVM-DR。

不過、您並不偏好為SVM設定專屬或共享的管理LIF、不過您應該確保網路安全性原則符合您選擇的方法。無論如何、管理LIF都應透過DNS存取、以達到最大的靈活度 "SVM-DR" 與Trident搭配使用。

限制最大**Volume**數

根據軟體版本和硬體平台、系統可提供最大的Volume數。ONTAP請參閱 "NetApp Hardware Universe" 針對您的特定平台和ONTAP 版本、決定確切的限制。當磁碟區數用盡時、資源配置作業不僅會針對Trident、也會針對所有儲存要求失敗。

Trident的「ONTAP-NAS」和「ONTAP-SAN」驅動程式會為每個建立的Kubernetes持續Volume（PV）提供FlexVolume。「ONTAP-NAS-P節約」驅動程式可為每200個PV建立約一個FlexVolume（可設定為50到300個）。「ONTAP-san經濟型」驅動程式可為每100個PV建立約一個FlexVolume（可設定為50到200個）。若要避免Trident佔用儲存系統上的所有可用磁碟區、您應該在SVM上設定限制。您可以從命令列執行此動作：

```
vserver modify -vserver <svm_name> -max-volumes <num_of_volumes>
```

「最大用量（max volume）的值會根據您環境的幾項特定條件而有所不同：

- 在叢集中現有的Volume數量ONTAP
- 您預期在Trident外部配置其他應用程式的Volume數量
- Kubernetes應用程式預期會使用的持續磁碟區數量

「最大容量」值是ONTAP 指在整個叢集中所有節點上配置的總Volume、而非在個別ONTAP 的節點上配置的總Volume。因此ONTAP 、您可能會遇到一些情況、例如、某個叢集節點的資源配置量可能遠高於或低於其他節點。

舉例ONTAP 來說、雙節點的「叢集」最多可裝載2000個FlexVolumes。將最大Volume數設為1250似乎非常合理。不過、如果只有 "集合體" 從某個節點指派給SVM、或是從某個節點指派的集合體無法根據（例如、由於容量）進行資源配置、則另一個節點會成為所有Trident資源配置磁碟區的目標。這表示在達到「最大磁碟區」值之前、該節點可能會達到磁碟區限制、進而影響使用該節點的Trident和其他Volume作業。您可以確保叢集中每個節點的集合體都指派給**Trident**使用的**SVM**、數量相等、藉此避免這種情況。

限制**Trident**所建立的**Volume**大小上限

若要設定Trident可建立的磁碟區大小上限、請在「backend.json」定義中使用「limitVolume Sizes」參數。

除了控制儲存陣列的磁碟區大小、您也應該善用Kubernetes功能。

設定**Trident**使用雙向**CHAP**

您可以在後端定義中指定CHAP啟動器和目標使用者名稱和密碼、並在SVM上啟用Trident啟用CHAP。Trident使用後端組態中的「useCHAP」參數來驗證iSCSI連線、以ONTAP CHAP作為後端點。Trident 20.04及更新版本提供雙向CHAP支援。

建立並使用**SVM QoS**原則

運用ONTAP 套用至SVM的SVM的SQoS原則、限制Trident佈建磁碟區所耗用的IOPS數量。這對我們有幫助 "預防欺凌" 或失控的容器、避免影響Trident SVM以外的工作負載。

您可以在幾個步驟中建立SVM的QoS原則。如ONTAP 需最準確的資訊、請參閱您的版次更新文件。以下範例建立QoS原則、將SVM可用的總IOPS限制為5000。

```
# create the policy group for the SVM
qos policy-group create -policy-group <policy_name> -vserver <svm_name>
-max-throughput 5000iops

# assign the policy group to the SVM, note this will not work
# if volumes or files in the SVM have existing QoS policies
vserver modify -vserver <svm_name> -qos-policy-group <policy_name>
```

此外、如果ONTAP 您的版本支援此功能、您可以考慮使用QoS下限來保證容器化工作負載的處理量。調適性QoS與SVM層級原則不相容。

容器化工作負載專用的IOPS數量取決於許多層面。其中包括：

- 使用儲存陣列的其他工作負載。如果有其他工作負載與Kubernetes部署無關、請善用儲存資源、確保這些工作負載不會意外受到不良影響。
- 預期的工作負載會在容器中執行。如果將在容器中執行高IOPS需求的工作負載、低QoS原則會導致不良體驗。

請務必記住、在SVM層級指派的QoS原則會導致所有已配置給SVM的磁碟區共用相同的IOPS集區。如果其中一種或少數幾種容器化應用程式的IOPS需求較高、可能會成為其他容器化工作負載的一大功臣。如果是這種情況、您可能需要考慮使用外部自動化來指派每個Volume QoS原則。

> 如果您的版本早於ONTAP 9.8、您應該將QoS原則群組指派給SVM * Only *。

為**Trident**建立**QoS**原則群組

服務品質（QoS）可確保關鍵工作負載的效能不會因競爭工作負載而降級。支援QoS原則群組的QoS選項可用於磁碟區、並可讓使用者定義一或多個工作負載的處理量上限。ONTAP如需QoS的詳細資訊、請參閱 "保證QoS的處理量"。您可以在後端或儲存資源池中指定QoS原則群組、並將其套用至該資源池或後端中建立的每個磁碟區。

包含兩種QoS原則群組：傳統和可調適。ONTAP傳統原則群組可在IOPS中提供最大（或最小）的單位處理量（在較新版本中）。調適性QoS會自動將處理量調整至工作負載大小、並隨著工作負載大小變更、維持IOPS與TBs的比率。當您在大型部署中管理數百個或數千個工作負載時、這項優勢就相當顯著。

建立QoS原則群組時、請考量下列事項：

- 您應該在後端組態的「故障」區塊中設定「qosPolicy」金鑰。請參閱下列後端組態範例：

```
{
  "version": 1,
  "storageDriverName": "ontap-nas",
  "managementLIF": "0.0.0.0",
  "dataLIF": "0.0.0.0",
  "svm": "svm0",
  "username": "user",
  "password": "pass",
  "defaults": {
    "qosPolicy": "standard-pg"
  },
  "storage": [
    {
      "labels": {"performance": "extreme"},
      "defaults": {
        "adaptiveQosPolicy": "extremely-adaptive-pg"
      }
    },
    {
      "labels": {"performance": "premium"},
      "defaults": {
        "qosPolicy": "premium-pg"
      }
    }
  ]
}
```

- 您應該為每個Volume套用原則群組、以便每個Volume都能獲得原則群組指定的整個處理量。不支援共用原則群組。

如需QoS原則群組的詳細資訊、請參閱 "Sof 9.8 QoS命令ONTAP"。

限制**Kubernetes**叢集成員存取儲存資源

限制存取Trident所建立的NFS磁碟區和iSCSI LUN、是Kubernetes部署安全態勢的重要元件。這樣做可防止非Kubernetes叢集一部分的主機存取磁碟區、並可能意外修改資料。

請務必瞭解命名空間是Kubernetes中資源的邏輯邊界。假設相同命名空間中的資源可以共用、但重要的是、沒有跨命名空間功能。這表示即使PV是全域物件、但只有在同一個命名空間中的Pod才能存取它們。確保命名空間在適當時用於提供分隔是非常重要的。

大多數組織對於Kubernetes內容中的資料安全性、主要關注的是、容器中的程序可以存取掛載到主機的儲存設備、但不適用於容器。 "命名空間" 旨在防止這類入侵。不過、有一個例外：特殊權限容器。

與正常情況相比、特權容器的執行主機層級權限大幅增加。依預設不會拒絕這些功能、因此請務必使用停用該功能 "Pod安全性原則"。

對於需要從Kubernetes和外部主機存取的磁碟區、儲存設備應以傳統方式進行管理、由系統管理員引進PV、而非由Trident管理。這可確保只有在Kubernetes和外部主機中斷連線且不再使用磁碟區時、才會銷毀儲存磁碟區。此外、也可以套用自訂匯出原則、以便從Kubernetes叢集節點和Kubernetes叢集以外的目標伺服器存取。

對於具有專屬基礎架構節點（例如OpenShift）或其他無法排程給使用者應用程式的節點的部署、應使用個別的匯出原則來進一步限制對儲存資源的存取。這包括為部署至這些基礎架構節點的服務（例如OpenShift Metrics和記錄服務）、以及部署至非基礎架構節點的標準應用程式建立匯出原則。

使用專屬的匯出原則

您應該確保每個後端都有一個匯出原則、只允許存取Kubernetes叢集中的節點。Trident可從20.04版開始自動建立及管理匯出原則。如此一來、Trident就能限制對Kubernetes叢集中節點所配置之磁碟區的存取、並簡化節點的新增/刪除作業。

或者、您也可以手動建立匯出原則、並以一或多個匯出規則填入、以處理每個節點存取要求：

- 使用「vserver匯出原則建立」ONTAP 的flexcli命令來建立匯出原則。
- 使用「vserver匯出原則規則create」ONTAP 的CLI命令、將規則新增至匯出原則。

執行這些命令可讓您限制哪些Kubernetes節點可以存取資料。

停用應用程式**SVM**的**showmount**

「show mount」功能可讓NFS用戶端查詢SVM、以取得可用的NFS匯出清單。部署到Kubernetes叢集的Pod可針對資料LIF發出「show mount -e」命令、並接收可用掛載的清單、包括無法存取的掛載。雖然這本身並不是安全威脅、但它確實提供不必要的資訊、可能有助於未獲授權的使用者連線至NFS匯出。

您應該使用SVM層級ONTAP 的CLI命令來停用「show mount」：

```
vserver nfs modify -vserver <svm_name> -showmount disabled
```

最佳實務做法**SolidFire**

瞭解設定SolidFire Trident之用的功能完善的功能。

建立**SolidFire** 支援帳戶

每SolidFire 個驗證帳戶都代表唯一的磁碟區擁有者、並會收到自己的挑戰握手驗證傳輸協定（CHAP）認證資料。您可以使用帳戶名稱和相對CHAP認證、或是透過Volume存取群組、來存取指派給帳戶的磁碟區。帳戶最多可指派2、000個磁碟區、但一個磁碟區只能屬於一個帳戶。

建立**QoS**原則

如果您想建立並儲存可套用至許多Volume的標準化服務品質設定、請使用SolidFire 「服務品質（QoS）」原則。

您可以設定每個Volume的QoS參數。設定三個可設定的參數來定義QoS、以確保每個Volume的效能：最小IOPS、最大IOPS和爆發IOPS。

以下是4KB區塊大小的可能最小、最大和尖峰IOPS值。

IOPS參數	定義	最小價值	預設值	最大價值（4KB）
最小IOPS	保證磁碟區效能等級。	50	50	15000
最大IOPS	效能不會超過此限制。	50	15000	20萬
暴增IOPS	在短時間暴增案例中允許的最大IOPS。	50	15000	20萬

> 雖然最大IOPS和爆發IOPS可設定為高達20、000、但實際的Volume最大效能卻受到叢集使用量和每節點效能的限制。

區塊大小和頻寬會直接影響IOPS的數量。隨著區塊大小增加、系統會將頻寬增加至處理較大區塊大小所需的層級。隨著頻寬增加、系統能夠達到的IOPS數量也隨之減少。請參閱 "服務品質SolidFire" 如需QoS和效能的詳細資訊、請參閱。

驗證SolidFire

Element支援兩種驗證方法：CHAP和Volume Access Groups（VAG）。CHAP使用CHAP傳輸協定驗證主機到後端的驗證。Volume存取群組可控制對其所配置之Volume的存取。NetApp建議使用CHAP進行驗證、因為它更簡單、而且沒有擴充限制。

> Trident搭配增強的csi佈置程式、可支援使用CHAP驗證。VAG只能在傳統的非csi操作模式下使用。

CHAP驗證（驗證啟動器是否為預定的Volume使用者）僅支援帳戶型存取控制。如果您使用CHAP進行驗證、則有兩個選項可供使用：單向CHAP和雙向CHAP。單向CHAP使用SolidFire 驗證帳戶名稱和啟動器密碼來驗證Volume存取。雙向CHAP選項提供最安全的驗證磁碟區方法、因為磁碟區會透過帳戶名稱和啟動器密碼來驗證主機、然後主機會透過帳戶名稱和目標密碼來驗證磁碟區。

但是、如果無法啟用CHAP且需要VAG、請建立存取群組、然後將主機啟動器和磁碟區新增至存取群組。您新增至存取群組的每個IQN都可以使用或不使用CHAP驗證來存取群組中的每個磁碟區。如果iSCSI啟動器設定為使用CHAP驗證、則會使用帳戶型存取控制。如果iSCSI啟動器未設定為使用CHAP驗證、則會使用Volume Access Group存取控制。

哪裡可以找到更多資訊？

以下列出部分最佳實務做法文件。搜尋 "NetApp資料庫" 適用於最新版本。

《》ONTAP

- "NFS最佳實務與實作指南"
- "SAN管理指南" （適用於iSCSI）
- "適用於RHEL的iSCSI Express組態"

元件軟體

- "設定SolidFire 適用於Linux的功能"

《》NetApp HCI

- "部署先決條件NetApp HCI"
- "存取NetApp部署引擎"

應用程式最佳實務做法資訊

- "MySQL ONTAP 的最佳實務做法"
- "MySQL SolidFire 的最佳實務做法"
- "NetApp SolidFire 的功能與Cassandra"
- "Oracle SolidFire 的最佳實務做法"
- "PostgreSQL SolidFire 的最佳實務做法"

並非所有應用程式都有特定的準則、請務必與您的NetApp團隊合作並使用 "NetApp資料庫" 以尋找最新的文件。

整合Astra Trident

若要整合Astra Trident、下列設計與架構元素需要整合：驅動程式選擇與部署、儲存類別設計、虛擬儲存資源池設計、持續磁碟區宣告（PVc）、使用Astra Trident對儲存資源配置、磁碟區作業及OpenShift服務部署的影響。

驅動程式選擇與部署

選擇**ONTAP** 一個後端驅動程式以供使用

共有四種不同的後端驅動程式可供ONTAP 使用。這些驅動程式會因使用的傳輸協定以及儲存系統上的磁碟區配置方式而有所差異。因此、請仔細考量要部署的驅動程式。

較高層級的應用程式若有需要共用儲存設備的元件（多個Pod存取相同的PVc）、則以NAS為基礎的驅動程式將是預設選擇、而區塊型iSCSI驅動程式則可滿足非共用儲存設備的需求。根據應用程式的需求、以及儲存設備和基礎架構團隊的舒適度來選擇傳輸協定。一般而言、大多數應用程式的差異不大、因此通常是根據是否需要共用儲存設備（如果有多個Pod需要同時存取）來決定。

以下列出五個ONTAP 驅動程式供作幕後之用：

- 「ONTAP-NAS」：每個提供的PV都是ONTAP 完整的FlexVolume。
- 「ONTAP-NAS-EAS'：每個已配置的PV都是qtree、每個FlexVolume可設定的qtree數量（預設值為200）。
- 「ONTAP-NAS-flexgroup」：每個PV均以ONTAP FlexGroup 完整的形式配置、並使用指派給SVM的所有集合體。
- 「ONTAP-san」：每個已配置的PV都是其FlexVolume內的LUN。
- 「ONTAP-san經濟」：每個配置的PV都是LUN、每個FlexVolume有可設定的LUN數量（預設值為100）。

在這三種NAS驅動程式之間選擇、會對應用程式可用的功能產生一些影響。

請注意、在下表中、並非所有功能都透過Astra Trident公開。如果需要這些功能、儲存管理員必須在資源配置後套用部分功能。上標註可區分每項功能和驅動程式的功能。

ASNAS驅動程式**ONTAP**	快照	複製	動態匯出原則	多重附加	**QoS**	調整大小	複寫
「ONTAP-NAS」	是的	是的	是註腳：5[]	是的	是註腳：1[]	是的	是註腳：1[]
《ONTAP-NANAS經濟》	是註腳：3[]	是註腳：3[]	是註腳：5[]	是的	是註腳：3[]	是的	是註腳：3[]
「ONTAP-NAA-flexgroup」	是註腳：1[]	否	是註腳：5[]	是的	是註腳：1[]	是的	是註腳：1[]

Astra Trident提供2個ONTAP SAN驅動程式來支援功能如下所示。

支援**SAN**驅動程式**ONTAP**	快照	複製	多重附加	雙向**CHAP**	**QoS**	調整大小	複寫
「ONTAP-SAN」	是的	是的	是註腳：4[]	是的	是註腳：1[]	是的	是註腳：1[]
《ONTAP-san經濟》	是的	是的	是註腳：4[]	是的	是註腳：3[]	是註腳：1[]	是註腳：3[]

> 上表的註腳：是註腳：1[]：不受Astra Trident管理是註腳：2[]：由Astra Trident管理、但不受PV精細控制是註腳：3[]：不受Astra Trident管理而非PV精細控制是註腳：4[]：支援原始區塊Volume是註腳：5[]：受Ci Trident支援

非PV精細的功能會套用至整個FlexVolume、而所有PV（即共享FlexVols中的qtree或LUN）都會共用一個共同排程。

如以上表格所示、「ONTAP-NAS」與「ONTAP-NAS經濟」之間的大部分功能都相同。不過、由於「ONTAP - NAS經濟」驅動程式限制了以每個PV精細度控制排程的能力、因此這可能會特別影響您的災難恢復與備份規劃。對於想要在ONTAP 不支援的儲存設備上使用永久虛擬複製功能的開發團隊、只有在使用「ONTAP-NAS」、「ONTAP-SAN」或「ONTAP-SAN經濟」驅動程式時、才有可能做到這一點。

> 「Poolidfire - san」驅動程式也能複製PVCS。

選擇**Cloud Volumes ONTAP** 一個後端驅動程式以供使用

支援資料控管功能、並提供企業級的儲存功能、適用於各種使用案例、包括檔案共用、區塊層級儲存設備（NFS、SMB / CIFS及iSCSI）Cloud Volumes ONTAP 。Cloud Volume ONTAP 的相容驅動程式為「ONTAP-NAS」、「ONTAP-NAS經濟」、「ONTAP-SAN」和「ONTAP-SAN經濟」。適用於ONTAP Azure的Cloud Volume供應、適用於ONTAP GCP的Cloud Volume供應。

選擇**Amazon FSX ONTAP for Sfor Sfa**的後端驅動程式

Amazon FSX ONTAP for VMware可讓客戶運用熟悉的NetApp功能、效能和管理功能、同時充分發揮儲存AWS資料的簡易性、敏捷度、安全性和擴充性。FSX ONTAP for FSfor支援ONTAP的許多檔案系統功能和管理API。Cloud Volume ONTAP 的相容驅動程式為「ONTAP-NAS」、「ONTAP-NAAS-經濟」、「ONTAP-NAS-flexgroup」、「ONTAP-SAN」和「ONTAP-san經濟」。

選擇**NetApp HCI / SolidFire**的後端驅動程式

NetApp HCI / SolidFire平台搭配使用的「Poolidfire」驅動程式、可協助管理員根據QoS限制、為Trident設定元素後端。如果您想要設計後端、以便針對Trident提供的磁碟區設定特定的QoS限制、請在後端檔案中使用「type」參數。管理員也可以使用「limitVolume Siz」參數、限制儲存設備上可建立的磁碟區大小。目前、磁碟區大小調整和磁碟區複寫等元素儲存功能不支援使用「Poolidfire - san」驅動程式。這些作業應透過Element Software Web UI手動完成。

驅動程式**SolidFire**	快照	複製	多重附加	**CHAP**	**QoS**	調整大小	複寫
「olidfire - san」	是的	是的	是註腳：2[]	是的	是的	是的	是註腳：1[]

> 註腳：是註腳：1[]：不受Astra Trident管理是註腳：2[]：支援原始區塊磁碟區

選擇**Azure NetApp Files** 一個後端驅動程式以供使用

Astra Trident使用「azure-NetApp-files」驅動程式來管理 "Azure NetApp Files" 服務：

如需此驅動程式及其設定方式的詳細資訊、請參閱 "Astra Trident的Azure NetApp Files 後端組態、適用於"。

驅動程式**Azure NetApp Files**	快照	複製	多重附加	**QoS**	展開	複寫
《azure-NetApp-fil形》	是的	是的	是的	是的	是的	是註腳：1[]

> 註腳：是註腳：1[]：不由Astra Trident管理

選擇一套後端驅動程式**Cloud Volumes Service** 以供運用**GCP**進行功能

Astra Trident使用「GCP-CVS」驅動程式、在Cloud Volumes Service GCP後端連結到該功能。若要在Trident上設定GCP後端、您必須在後端檔案中指定「ProjectNumber」、「apiRegion」和「apiKey」。專案編號可在GCP入口網站找到、API金鑰必須取自您在GCP上設定Cloud Volumes API存取時所建立的服務帳戶私密金鑰檔案。Astra Trident可在兩個磁碟區中建立CVS磁碟區 "服務類型"：

1. * CVS：基礎CVS服務類型、提供高分區可用度、效能等級有限/中等。
2. * CVS效能*：效能最佳化的服務類型、最適合重視效能的正式作業工作負載。您可以從三種獨特的服務層級中進行選擇：「標準」、「優質」和「極致」。目前、要配置的CVS效能磁碟區大小下限為100 GiB、而CVS磁碟區則必須至少為300 GiB。CVS未來版本可能會移除此限制。

> 使用預設CVS服務類型["scorageClass=software"部署後端時、使用者*必須取得GCP上子1TiB Volume功能的存取權*、才能取得相關專案編號和專案ID的存取權。這是Trident配置子1TiB磁碟區所需的功能。如果沒有、則對於小於600 GiB的PVCs、Volume建立*將會失敗*。使用 "這份表格" 以取得對低於1TiB磁碟區的存取權。

CVS for GCP驅動程式	快照	複製	多重附加	**QoS**	展開	複寫
《GCP—CVS》	是的	是的	是的	是的	是的	是註腳：1[]

> 註腳：是註腳：1[]：不由Astra Trident管理

「GCP-CVS」驅動程式使用虛擬儲存資源池。虛擬儲存池會將後端抽象化、讓Astra Trident決定磁碟區的放置位置。系統管理員會在backend.json檔案中定義虛擬儲存池。儲存類別會使用標籤來識別虛擬儲存資源池。

儲存層級設計

需要設定並套用個別的儲存類別、才能建立Kubernetes儲存類別物件。本節將討論如何為應用程式設計儲存類別。

專為特定後端使用率而設計的儲存類別

篩選功能可在特定的儲存類別物件內使用、以決定要搭配該特定儲存類別使用的儲存資源池或集區集區集區。儲存類別可設定三組篩選器：「儲存設備」、「其他儲存設備」及/或「排除儲存設備」。

「儲存池」參數有助於將儲存區限制在符合任何指定屬性的集區集區集區。「addionalStoragePools」參數可用來擴充Astra Trident用來進行資源配置的資源池集區集區、以及由屬性和「儲存池」參數所選取的資源池集區集區集區集區集區集區集區集區。您可以單獨使用參數或同時使用兩者、以確保已選取適當的儲存資源池集區集區。

「exclude StoragePools」參數是用來明確排除列出的符合屬性的集區集區集區集區。

模擬**QoS**原則的儲存類別設計

如果您想設計儲存類別來模擬服務品質原則、請建立儲存類別、並將「媒體」屬性設定為「HDD」或「SD」。根據儲存類別中提及的「媒體」屬性、Trident會選擇適當的後端、以提供「HDD」或「sd」集合體、以符合媒體屬性、然後將磁碟區的資源配置導向特定的集合體。因此、我們可以建立儲存等級Premium、將「媒體」屬性設為「sd」、可歸類為優質QoS原則。我們可以建立另一個儲存類別標準、將媒體屬性設為「HDD」、並將其歸類為標準QoS原則。我們也可以使用儲存類別中的「IOPS」屬性、將資源配置重新導向至可定義為QoS原則的元素應用裝置。

儲存等級設計、可根據特定功能來使用後端

儲存類別可設計用於將Volume資源配置導向特定後端、啟用精簡與完整資源配置、快照、複製及加密等功能。若要指定要使用的儲存設備、請建立儲存設備類別、以指定啟用所需功能的適當後端。

虛擬儲存資源池的儲存等級設計

所有Astra Trident後端均可使用虛擬儲存資源池。您可以使用任何Astra Trident提供的驅動程式、為任何後端定義虛擬儲存池。

虛擬儲存資源池可讓系統管理員在後端建立抽象層級、以便透過儲存類別進行參考、以提高磁碟區在後端的靈活度與效率。不同的後端可以使用相同的服務類別來定義。此外、您也可以在相同的後端上建立多個儲存資源池、但其特性不同。當儲存類別設定為具有特定標籤的選取器時、Astra Trident會選擇符合所有選取器標籤的後端來放置磁碟區。如果「儲存類別」選取器標籤符合多個儲存資源池、Astra Trident會選擇其中一個來配置磁碟區。

虛擬儲存資源池設計

建立後端時、您通常可以指定一組參數。系統管理員無法以相同的儲存認證和一組不同的參數來建立另一個後端。隨著虛擬儲存資源池的推出、此問題已獲得緩和。虛擬儲存資源池是後端與Kubernetes儲存類別之間引進的層級抽象、可讓系統管理員定義參數及標籤、並以不受後端限制的方式透過Kubernetes儲存類別做為選取元來參照。您可以使用Astra Trident為所有支援的NetApp後端定義虛擬儲存池。這份清單包括SolidFire/NetApp HCI、ONTAP 《關於Cloud Volumes Service GCP的功能、功能、功能、功能Azure NetApp Files 、功能、以及

> 定義虛擬儲存集區時、建議您不要嘗試重新排列後端定義中現有虛擬集區的順序。此外、建議您不要編輯/修改現有虛擬資源池的屬性、改為定義新的虛擬資源池。

設計虛擬儲存資源池、以模擬不同的服務層級/**QoS**

您可以設計虛擬儲存池來模擬服務類別。使用適用於Azure NetApp Files 支援功能的Cloud Volume Service for效益的虛擬資源池實作、讓我們來看看如何設定不同的服務類別。使用代表不同效能等級的多個標籤來設定ANF後端。將「最大效能」設定為適當的效能層級、並在每個標籤下加入其他必要的層面。現在請建立不同的Kubernetes儲存類別、以便對應至不同的虛擬儲存資源池。使用「parameters.selector`」欄位、每個StorageClass都會呼叫哪些虛擬資源池可用於裝載磁碟區。

設計虛擬集區以指派特定的層面集區

可從單一儲存後端設計多個具有特定層面的虛擬儲存集區。若要這麼做、請使用多個標籤來設定後端、並在每個標籤下設定所需的層面。現在、請使用「parameters.selector`」欄位來建立不同的Kubernetes儲存類別、該欄位會對應至不同的虛擬儲存資源池。在後端上進行資源配置的磁碟區、將會在所選的虛擬儲存資源池中定義各個層面。

會影響儲存資源配置的永久儲存設備特性

超出所要求儲存類別的部分參數、可能會影響Astra Trident在建立永久虛擬儲存設備時的資源配置決策程序。

存取模式

透過永久虛擬網路申請儲存時、其中一個必填欄位是存取模式。所需的模式可能會影響所選的後端、以裝載儲存要求。

Astra Trident會嘗試將所使用的儲存傳輸協定與根據下列對照表所指定的存取方法配對。這與基礎儲存平台無關。

	ReadWriteOnce	**ReadOnlyMany**	**ReadWriteMany**
iSCSI	是的	是的	是（原始區塊）
NFS	是的	是的	是的

如果要求將ReadWriteMany永久虛擬磁碟提交至Trident部署、但未設定NFS後端、則不會配置任何磁碟區。因此、申請者應使用適合其應用程式的存取模式。

Volume作業

修改持續磁碟區

持續磁碟區除了兩個例外、都是Kubernetes中不可變的物件。建立後、即可修改回收原則和大小。不過、這並不妨礙在Kubernetes外部修改磁碟區的某些部分。這可能是理想的做法、以便針對特定應用程式自訂磁碟區、確保容量不會意外耗用、或是單純地將磁碟區移至不同的儲存控制器。

> Kubernetes樹狀目錄內建資源配置程式目前不支援NFS或iSCSI PV的磁碟區大小調整作業。Astra Trident支援同時擴充NFS和iSCSI磁碟區。

PV的連線詳細資料無法在建立後修改。

建立隨需磁碟區快照

Astra Trident支援隨需磁碟區快照建立、並使用csi架構從快照建立PVCS。Snapshot提供便利的方法來維護資料的時間點複本、並使Kubernetes中的來源PV在生命週期上獨立不受影響。這些快照可用於複製PVCS。

從快照建立磁碟區

Astra Trident也支援從Volume快照建立PersistentVolumes。為達成此目標、只要建立一個PersistentVolume Claim、並將「資料來源」提及為需要建立磁碟區的必要快照即可。Astra Trident會利用快照上的資料建立磁碟區、以處理此永久虛擬磁碟。有了這項功能、您可以跨區域複製資料、建立測試環境、完整取代毀損或毀損的正式作業磁碟區、或擷取特定檔案和目錄、然後將它們傳輸到其他附加磁碟區。

在叢集中移動磁碟區

儲存管理員能夠在ONTAP 整個叢集中的集合體和控制器之間、不中斷營運地將磁碟區移至儲存使用者。此作業不會影響Astra Trident或Kubernetes叢集、只要目的地Aggregate是Astra Trident所使用的SVM能夠存取的集合體。重要的是、如果新將Aggregate新增至SVM、則需要重新將其新增至Astra Trident來重新整理後端。這會觸發Astra Trident重新清查SVM、以便辨識新的Aggregate。

然而、Astra Trident並不支援跨後端移動磁碟區。這包括在同一個叢集內的SVM之間、叢集之間或不同的儲存平台（即使該儲存系統是連接至Astra Trident的儲存系統）。

如果將磁碟區複製到其他位置、則磁碟區匯入功能可用於將目前的磁碟區匯入Astra Trident。

展開**Volume**

Astra Trident支援調整NFS和iSCSI PV的大小。這可讓使用者透過Kubernetes層直接調整磁碟區大小。所有主要的NetApp儲存平台皆可進行Volume擴充、包括ONTAP ：NetApp、SolidFire/NetApp HCI及Cloud Volumes Service 背後端點。若要稍後允許擴充、請在與磁碟區相關的StorageClass中將「owVolume Expansion」設為「true」。每當需要調整「持續Volume」的大小時、請在「持續Volume」中編輯「s.pec.resistices.resistices.storage」註釋、使其符合所需的Volume大小。Trident會自動調整儲存叢集上的磁碟區大小。

將現有磁碟區匯入**Kubernetes**

Volume匯入功能可將現有的儲存磁碟區匯入Kubernetes環境。目前支援的驅動程式有「ONTAP-NAS」、「ONTAP-NAs-flexgroup」、「Poolidfire - san」、「azure-NetApp-fil卻」和「GCP - CVS」。當將現有應用程式移轉至Kubernetes或發生災難恢復時、此功能非常實用。

使用ONTAP 支援功能的支援功能和「Poolidfire - san」驅動程式時、請使用命令「tridentctl import volume <backend-name><volume-name>-f /path/PVC.yaml」、將現有的磁碟區匯入要由Astra Trident管理的Kubernetes。匯入Volume命令中使用的PVc Yaml或Json檔案會指向儲存類別、以將Astra Trident識別為資源配置程式。使用NetApp HCI / SolidFire後端時、請確定磁碟區名稱是唯一的。如果磁碟區名稱重複、請將磁碟區複製成唯一名稱、以便磁碟區匯入功能能夠區分它們。

如果使用「azure-netapp-files」或「gcp-CVS」驅動程式、請使用命令「tridentctl匯入Volume <後端名稱><Volume path>-f /path/pcv.yaml」、將磁碟區匯入要由Astra Trident管理的Kubernetes。如此可確保唯一的Volume參考。

執行上述命令時、Astra Trident會在後端找到磁碟區並讀取其大小。它會自動新增（必要時覆寫）設定的PVc Volume大小。Astra Trident接著會建立新的PV、Kubernetes則會將PVc繫結至PV。

如果部署的容器需要特定匯入的PVc、則會保持擱置狀態、直到PVC/PV配對透過Volume匯入程序繫結為止。

在PVC/PV配對繫結之後、如果沒有其他問題、則應啟動容器。

部署OpenShift服務

OpenShift加值叢集服務可為叢集管理員和託管的應用程式提供重要功能。這些服務所使用的儲存設備可以使用節點本機資源進行資源配置、但這通常會限制服務的容量、效能、可恢復性及永續性。運用企業儲存陣列來提供這些服務的容量、可大幅改善服務品質、不過OpenShift和儲存管理員應該密切合作、以決定每個服務的最佳選項。Red Hat文件應充分運用、以判斷需求、並確保符合規模調整與效能需求。

登錄服務

登錄的儲存設備部署與管理已記錄在中 "NetApp.IO" 在中 "部落格"。

記錄服務

如同其他OpenShift服務、記錄服務是使用Ansible搭配庫存檔案所提供的組態參數（即k.a.）來部署主機、提供給教戰手冊。其中包括兩種安裝方法：在初始OpenShift安裝期間部署記錄、以及在安裝OpenShift之後部署記錄。

> 從Red Hat OpenShift版本3.9起、官方文件建議您不要使用NFS來執行記錄服務、因為您擔心資料毀損。這是以Red Hat測試其產品為基礎。ONTAP的NFS伺服器沒有這些問題、可以輕鬆地回溯記錄部署。最後、記錄服務的通訊協定選擇取決於您、只要知道兩者在使用NetApp平台時都能順利運作、而且如果您偏好NFS、就沒有理由不使用NFS。

如果您選擇使用NFS搭配記錄服務、則必須將Ansible變數「openshift_enable _unsupported_configurations」設為「true」、以避免安裝程式失敗。

開始使用

記錄服務可選擇性地同時部署給應用程式、以及OpenShift叢集本身的核心作業。如果您選擇部署作業記錄、將變數「openshift_logging_use」指定為「true」、就會建立兩個服務執行個體。控制作業記錄執行個體的變數包含「ops」、而應用程式執行個體則不包含。

根據部署方法設定Ansible變數非常重要、因為這樣才能確保基礎服務使用正確的儲存設備。讓我們來看看每種部署方法的選項。

> 下表僅包含與記錄服務相關的儲存組態變數。您可以在中找到其他選項 "RedHat OpenShift記錄文件" 應根據您的部署情況來審查、設定及使用。

下表中的變數會使用提供的詳細資料、產生Ansible教戰手冊、為記錄服務建立PV和PVc。這種方法的彈性遠低於OpenShift安裝後使用元件安裝方針、不過如果您有現有的磁碟區可用、這是一個選項。

變動	詳細資料
"openshift_logging_storage _gin"	設定為「NFS」、讓安裝程式為記錄服務建立NFS PV。
"openshift_logging_storage主機"	NFS主機的主機名稱或IP位址。這應該設定為虛擬機器的資料LIF。
"openshift_logging_storage、nfs_directory"	NFS匯出的掛載路徑。例如、如果磁碟區已連接為「/openshift_logging」、您就會將該路徑用於此變數。

變動	詳細資料
"openshift_logging_storage磁碟區名稱"	要建立之PV的名稱、例如「PV_ose記錄」。
"openshift_logging_storage磁碟區大小"	NFS匯出的大小、例如「100Gi」。

如果您的OpenShift叢集已在執行中、因此已部署及設定Trident、則安裝程式可以使用動態資源配置來建立磁碟區。需要設定下列變數。

變動	詳細資料
「openshift_logging_es _PVC_Dynamic」	設為true可使用動態資源配置的磁碟區。
「openshift_logging_es _PVC_storage _class_name」	將在PVc中使用的儲存類別名稱。
「openshift_logging_es _PVC_size」	在永久虛擬磁碟中要求的磁碟區大小。
「openshift_logging_es _PVC_prefix」	記錄服務使用的PVCS前置詞。
「openshift_logging_es _ops_PVC_Dynamic」	設為「true」、以動態配置的磁碟區用於作業記錄執行個體。
「openshift_logging_es _ops_PVC_storage儲存設備類別名稱」	作業記錄執行個體的儲存類別名稱。
「openshift_logging_es _ops_PVC_Size'	作業執行個體的Volume要求大小。
「openshift_logging_es_ops_PVC_prefix」	ops執行個體PVCS的前置詞。

部署記錄堆疊

如果您將記錄部署為初始OpenShift安裝程序的一部分、則只需遵循標準部署程序即可。Ansible會設定及部署所需的服務和OpenShift物件、以便在可執行的完成後立即提供服務。

不過、如果您在初始安裝之後進行部署、Ansible將需要使用元件方針。不同版本的OpenShift可能會稍微改變此程序、因此請務必閱讀並遵循 "RedHat OpenShift Container Platform 3.11文件" 適用於您的版本。

度量服務

度量服務可針對OpenShift叢集的狀態、資源使用率及可用度、提供寶貴的資訊給系統管理員。此外、也需要Pod自動擴充功能、許多組織會使用度量服務的資料來支付費用和/或顯示應用程式。

如同記錄服務和OpenShift整體、Ansible可用於部署度量服務。此外、如同記錄服務、度量服務也可在叢集初始設定期間或使用元件安裝方法運作之後進行部署。下表包含在設定度量服務的持續儲存時、重要的變數。

> 下表僅包含與度量服務相關的儲存組態相關變數。文件中還有許多其他選項、您應該根據部署情況來檢閱、設定及使用。

變動	詳細資料
"openshift_imization_storage類型"	設定為「NFS」、讓安裝程式為記錄服務建立NFS PV。
"openshift_imization_storage主機"	NFS主機的主機名稱或IP位址。這應該設定為SVM的資料LIF。

變動	詳細資料
"openshift_imization_storage、nfs_directory"	NFS匯出的掛載路徑。例如、如果磁碟區已連接為「/openshift_度量」、您就會使用該路徑來處理此變數。
"openshift_imization_storage磁碟區名稱"	要建立之PV的名稱、例如「PV_ose度量」。
"openshift_imization_storage磁碟區大小"	NFS匯出的大小、例如「100Gi」。

如果您的OpenShift叢集已在執行中、因此已部署及設定Trident、則安裝程式可以使用動態資源配置來建立磁碟區。需要設定下列變數。

變動	詳細資料
"openshift_imization_cassandra _PVC_prefix"	用於度量PVCS的前置詞。
"openshift_imization_cassandra _PVC_Size"	要要求的磁碟區大小。
"openshift_imensits_cassandra儲存設備類型"	用於度量的儲存類型、必須設定為動態、Ansible才能建立具有適當儲存類別的PVCS。
"openshift_imization_cassanda_PVC_storage _class_name"	要使用的儲存類別名稱。

部署度量服務

在您的主機/庫存檔案中定義適當的可Ansible變數後、使用Ansible部署服務。如果您是在OpenShift安裝時間進行部署、則會自動建立及使用PV。如果您使用元件教戰手冊進行部署、則在OpenShift安裝之後、Ansible會建立任何需要的PVCS、並在Astra Trident為其配置儲存設備之後、部署該服務。

上述變數及部署程序可能會隨OpenShift的每個版本而變更。請務必檢閱並遵循 "RedHat的OpenShift部署指南" 以供您的環境使用。

資料保護

瞭解NetApp儲存平台提供的資料保護與可恢復性選項。Astra Trident可配置磁碟區、以充分利用其中的部分功能。對於每個應用程式、您都應該有持續性需求的資料保護與還原策略。

備份**etcd**叢集資料

Astra Trident將其中繼資料儲存在Kubernetes叢集的「etcd」資料庫中。定期備份「etcd」叢集資料、對於在災難情況下恢復Kubernetes叢集而言、非常重要。

步驟

1. 使用「etcctll snapshot SAVE」命令、您可以擷取「etcd」叢集的時間點快照：

```
sudo docker run --rm -v /backup:/backup \
  --network host \
  -v /etc/kubernetes/pki/etcd:/etc/kubernetes/pki/etcd \
  --env ETCDCTL_API=3 \
  k8s.gcr.io/etcd-amd64:3.2.18 \
  etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/healthcheck-client.crt \
  --key=/etc/kubernetes/pki/etcd/healthcheck-client.key \
  snapshot save /backup/etcd-snapshot.db
```

此命令可藉由旋轉etcd容器來建立etcd快照、並將其儲存在「/Backup」目錄中。

2. 發生災難時、您可以使用etcd快照來啟動Kubernetes叢集。使用「etcctll snapshot還原」命令、還原擷取至「/var/lib/etcd」資料夾的特定快照。還原之後、請確認「/var/lib/etcd」資料夾是否已填入「member」資料夾。以下是「etcdctl snapshot還原」命令的範例：

```
# etcdctl snapshot restore '/backup/etcd-snapshot-latest.db' ; mv
/default.etcd/member/ /var/lib/etcd/
```

3. 初始化Kubernetes叢集之前、請複製所有必要的憑證。
4. 使用「-ignore預先執行錯誤= DirAvailable」-var-lib-etcd標誌建立叢集。
5. 叢集啟動後、請確定已啟動kube系統Pod。
6. 使用「kubectl Get crd」命令來驗證Trident所建立的自訂資源是否存在、並擷取Trident物件、以確保所有資料都可用。

使用**ONTAP** 不含資訊的快照來恢復日期

快照提供應用程式資料的時間點還原選項、因此扮演著重要角色。不過、快照本身並不是備份、因此無法防止儲存系統故障或其他災難。但是、在大多數情況下、它們都是一種方便、快速且簡單的資料恢復方法。瞭ONTAP解如何使用「支援資料快照」技術來備份磁碟區、以及如何還原。

- 如果未在後端定義快照原則、則預設使用「無」原則。如此一ONTAP 來、我們便無法自動擷取快照。不過、儲存管理員可以手動擷取快照、或是透過ONTAP 功能完善的管理介面變更快照原則。這不會影響Trident作業。
- Snapshot目錄預設為隱藏。這有助於實現使用「ONTAP-NAS」和「ONTAP-NAS經濟」驅動程式所配置之磁碟區的最大相容性。使用「ONTAP-NAS」和「ONTAP-NAS經濟」驅動程式時、請啟用「.snapshot」目錄、讓應用程式能夠直接從快照中恢復資料。
- 使用「Volume Snapshot Restore」ONTAP （Volume Snapshot還原）CLI命令、將磁碟區還原至先前快照中記錄的狀態。還原快照複本時、還原作業會覆寫現有的Volume組態。建立Snapshot複本之後、對磁碟區中資料所做的任何變更都會遺失。

```
cluster1::*> volume snapshot restore -vserver vs0 -volume vol3 -snapshot
vol3_snap_archive
```

使用ONTAP 功能複寫資料

複寫資料對於防止儲存陣列故障所造成的資料遺失、扮演著重要的角色。

> 如需ONTAP 深入瞭解有關資訊複寫技術的資訊、請參閱 "本文檔 ONTAP"。

SnapMirror儲存虛擬機器（**SVM**）複寫

您可以使用 "SnapMirror" 複寫完整的SVM、其中包括其組態設定及其磁碟區。發生災難時、您可以啟動SnapMirror目的地SVM、開始提供資料服務。還原系統時、您可以切換回主要系統。

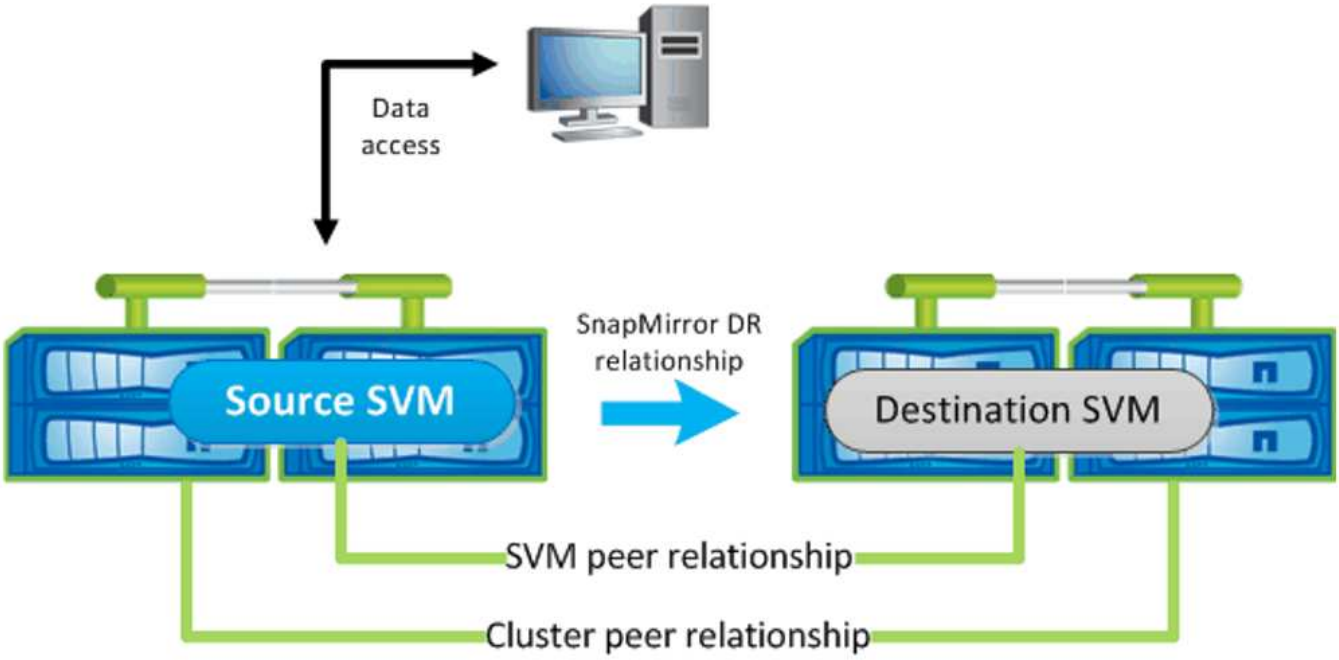
Astra Trident無法自行設定複寫關係、因此儲存管理員可以使用ONTAP的SnapMirror SVM複寫功能、將磁碟區自動複寫到災難恢復（DR）目的地。

如果您打算使用SnapMirror SVM複寫功能、或是目前正在使用此功能、請考慮下列事項：

- 您應該為每個啟用SVM災難恢復的SVM建立不同的後端。
- 您應該設定儲存類別、以便在需要時、不要選取複寫的後端。這一點非常重要、因為如此一來、您就不必將不需要保護複寫關係的磁碟區配置到支援SVM-DR的後端。
- 應用程式管理員應瞭解複寫資料所需的額外成本與複雜度、並在使用資料複寫之前、先決定恢復計畫。
- 啟動SnapMirror目的地SVM之前、請先停止所有排程的SnapMirror傳輸、中止所有進行中的SnapMirror傳輸、中斷複寫關係、停止來源SVM、然後啟動SnapMirror目的地SVM。
- Astra Trident不會自動偵測SVM故障。因此，當發生故障時，系統管理員應該執行「tridentctl後端更新」命令，以觸發Trident的容錯移轉到新的後端。

以下是SVM設定步驟的總覽：

- 在來源與目的地叢集與SVM之間設定對等關係。
- 使用「副類型DP目的地」選項建立目的地SVM。
- 建立複寫工作排程、以確保複寫會在所需的時間間隔執行。
- 使用「idente-preserve true」選項、建立從目的地SVM到來源SVM的SnapMirror複寫、以確保來源SVM組態和來源SVM介面複製到目的地。從目的地SVM初始化SnapMirror SVM複寫關係。



Trident的災難恢復工作流程

Astra Trident 19.07及更新版本使用Kubernetes CRD來儲存及管理自己的狀態。它使用Kubernetes叢集的「etcd」來儲存中繼資料。我們假設Kubernetes的「etcd」資料檔案和憑證都儲存在NetApp FlexVolume上。此FlexVolume位於SVM中、與次要站台的目的地SVM有SnapMirror SVM-DR關係。

下列步驟說明如何在發生災難時、使用Astra Trident來還原單一主Kubernetes叢集：

1. 如果來源SVM故障、請啟動SnapMirror目的地SVM。為達成此目的、您應該停止排程的SnapMirror傳輸、中止進行中的SnapMirror傳輸、中斷複寫關係、停止來源SVM、以及啟動目的地SVM。
2. 從目的地SVM、將包含Kubernetes「etcd」資料檔案和憑證的磁碟區掛載到主機上、主機將設定為主節點。
3. 複製所有必要的Kubernetes叢集憑證、這些憑證位於「/etc/Kubernetes /PKI」下、而etcd「m十二」檔案則位於「/var/lib/etcd」下。
4. 使用帶有「-ignore preflas-errors = DirAvailable - var-lib-etcd」旗標的「kubeadm init」命令建立Kubernetes叢集。Kubernetes節點使用的主機名稱應與來源Kubernetes叢集相同。
5. 執行「kubectl Get crd」命令、確認所有Trident自訂資源是否都已啟動並擷取Trident物件、以驗證所有資料是否可用。
6. 執行「/tridentctl update backend <backend-name>-f <backend-jser-file>-n <namedes>'命令、更新所有必要的後端、以反映新的目的地SVM名稱。

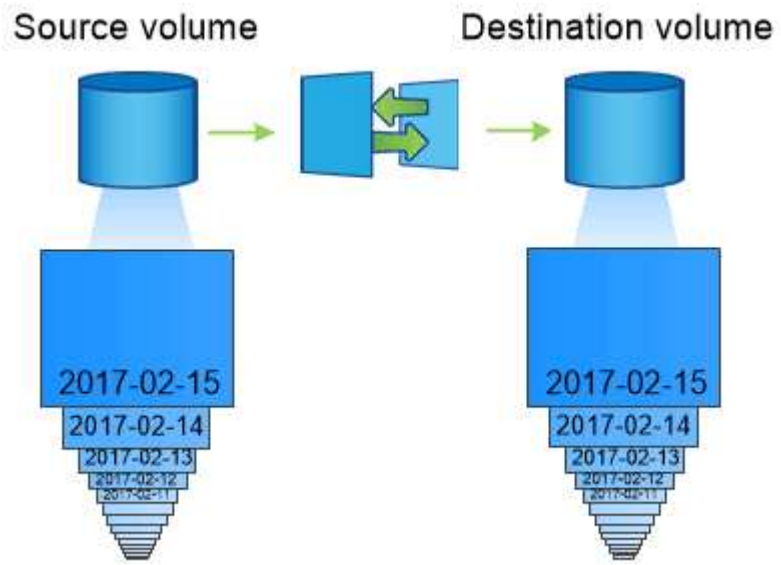
> ⓘ 對於應用程式持續磁碟區、當目的地SVM啟動時、Trident所配置的所有磁碟區都會開始提供資料。使用上述步驟在目的地端設定Kubernetes叢集之後、所有的部署和Pod都會啟動、而且容器化應用程式應該能順利執行、不會發生任何問題。

SnapMirror Volume複寫

SnapMirror Volume複寫是一項災難恢復功能、可從Volume層級的主要儲存設備進行容錯移轉至目的地儲存設備。ONTAPSnapMirror透過同步快照、在次要儲存設備上建立主要儲存設備的Volume複本或鏡射。

以下是ONTAP 關於SnapMirror Volume複寫設定步驟的總覽：

- 在磁碟區所在的叢集與從磁碟區提供資料的SVM之間設定對等關係。
- 建立SnapMirror原則、以控制關係的行為、並指定該關係的組態屬性。
- 使用建立目的地Volume與來源Volume之間的SnapMirror關係 "d9934e78a9254dde4a227181c30fa2d2" 並指派適當的SnapMirror原則。
- 建立SnapMirror關係之後、請初始化關係、以便完成從來源磁碟區到目的地磁碟區的基礎傳輸。



適用於**Trident**的**SnapMirror Volume**災難恢復工作流程

下列步驟說明如何使用Astra Trident來復原單一主Kubernetes叢集。

1. 發生災難時、請停止所有排程的SnapMirror傳輸、並中止所有進行中的SnapMirror傳輸。中斷目的地與來源磁碟區之間的複寫關係、使目的地磁碟區變成讀取/寫入。
2. 從目的地SVM、將包含Kubernetes「etcd」資料檔案和憑證的磁碟區掛載到主機上、主機將設定為主節點。
3. 複製所有必要的Kubernetes叢集憑證、這些憑證位於「/etc/Kubernetes /PKI」下、而etcd「m十二」檔案則位於「/var/lib/etcd」下。
4. 使用帶有"-ignore preflas-errors = DirAvailable - var-lib-etcd"標記的"kubeadm init"命令來建立Kubernetes叢集。主機名稱應與來源Kubernetes叢集相同。
5. 執行「kubectl Get crd」命令、確認所有Trident自訂資源是否都已啟動並擷取Trident物件、以確保所有資料都可用。
6. 清理先前的後端、並在Trident上建立新的後端。指定目的地SVM的新管理與資料LIF、新SVM名稱及密碼。

應用程式持續磁碟區的災難恢復工作流程

下列步驟說明在發生災難時、如何為容器化工作負載提供SnapMirror目的地磁碟區：

1. 停止所有排程的SnapMirror傳輸、並中止所有進行中的SnapMirror傳輸。中斷目的地與來源磁碟區之間的複寫關係、使目的地磁碟區變成讀取/寫入。清除使用與來源SVM上磁碟區連結之PVc的部署。

2. 使用上述步驟在目的地端設定Kubernetes叢集之後、請從Kubernetes叢集清除部署、PVCS和PV。
3. 指定新的管理與資料LIF、新的SVM名稱和目的地SVM密碼、在Trident上建立新的後端。
4. 使用「Trident匯入」功能、將所需的磁碟區匯入為與新的PVc繫結的PV。
5. 使用新建立的PVCS重新部署應用程式部署。

使用元素快照來恢復資料

設定磁碟區的快照排程、並確保每隔一段時間擷取快照、以備份元素磁碟區上的資料。您應該使用元素UI或API來設定快照排程。目前無法透過「Poolidfire - san」驅動程式、將快照排程設定為磁碟區。

在資料毀損的情況下、您可以使用元素UI或API、選擇特定的快照、然後手動將磁碟區復原至快照。這會還原自建立快照以來對磁碟區所做的任何變更。

安全性

請使用此處列出的建議、確保您的Astra Trident安裝安全無虞。

在自己的命名空間中執行**Astra Trident**

防止應用程式、應用程式管理員、使用者及管理應用程式存取Astra Trident物件定義或Pod、以確保可靠的儲存並封鎖潛在的惡意活動、這一點非常重要。

若要將其他應用程式和使用者與Astra Trident區隔開、請務必在Kubernetes命名空間（「trident」）中安裝Astra Trident。將Astra Trident放在自己的命名空間中、可確保只有Kubernetes管理人員能夠存取Astra Trident Pod、以及儲存在命名式CRD物件中的成品（例如後端和CHAP機密）。您應該確保只允許系統管理員存取Astra Trident命名空間、進而存取「tridentctl」應用程式。

使用**CHAP**驗證搭配**ONTAP** 使用支援**SAN**的功能

Astra Trident支援ONTAP 以CHAP為基礎的驗證功能、適用於各種SAN工作負載（使用「ONTAP-SAN」和「ONTAP-san經濟」驅動程式）。NetApp建議使用雙向CHAP搭配Astra Trident進行主機與儲存後端之間的驗證。

針對使用SAN儲存驅動程式的幕後作業、Astra Trident可設定雙向CHAP、並透過「tridentctl」管理CHAP使用者名稱和機密。ONTAP請參閱 "請按這裡" 瞭解Astra Trident如何在ONTAP 不景點上設定CHAP。

> Trident 20.04及更新版本提供對支援不支援的CHAP ONTAP 。

使用**CHAP**驗證**NetApp HCI** 搭配不景和**SolidFire** 不景的後端

NetApp建議部署雙向CHAP、以確保主機與NetApp HCI 支援功能及SolidFire 支援功能之間的驗證。Astra Trident使用每個租戶包含兩個CHAP密碼的秘密物件。當Trident安裝為「csi佈建程式」時、它會管理CHAP機密、並將其儲存在個別PV的「tridentvolume」（Tridentvolume）CR物件中。建立PV時、「csi Astra Trident」會使用CHAP機密來啟動iSCSI工作階段、NetApp HCI 並透過CHAP與該系統進行通訊SolidFire 。

> 由csi Trident建立的磁碟區不會與任何Volume存取群組相關聯。

在非csi前端中、磁碟區作為工作節點上的裝置附加作業是由Kubernetes處理。建立磁碟區之後、Astra Trident會

呼叫NetApp HCI / SolidFire系統、以便在該租戶的機密資訊尚未存在時擷取機密資訊。然後、Astra Trident將機密傳給Kubernetes。位於每個節點上的Kubelet可透過Kubernetes API存取機密資料、並使用它們在存取磁碟區的每個節點與磁碟區所在的NetApp HCI / SolidFire系統之間執行/啟用CHAP。

參考資料

Astra Trident連接埠

深入瞭解Astra Trident透過哪些連接埠進行通訊。

Astra Trident透過下列連接埠進行通訊：

連接埠	目的
8443.	後端通道HTTPS
8001	Prometheus指標端點
8000	Trident REST伺服器
17546	Trident取消安裝套件所使用的活動/整備度探針連接埠

在安裝期間、可使用「-Probe-port'」旗標來變更活動力/整備度探針連接埠。請務必確認工作節點上的其他程序並未使用此連接埠。

Astra Trident REST API

而 "tridentctl命令和選項" 這是與Astra Trident REST API互動最簡單的方法、您可以視需要直接使用REST端點。

這對於在非Kubernetes部署中使用Astra Trident做為獨立二進位檔的進階安裝來說非常有用。

為了提高安全性、Astra Trident的「REST API」在Pod內部執行時、預設僅限使用localhost。若要變更此行為、您必須在Pod組態中設定Astra Trident的「位址」引數。

API的運作方式如下：

取得

- 「Get <trident位址>/trident / v1/<object-type >」：列出該類型的所有物件。
- 「Get <trident位址>/trident / v1/<object-typed>/<object-name>」：取得命名物件的詳細資料。

貼文

「POST <trident位址>/trident / v1/<object-type>」：建立指定類型的物件。

- 需要Json組態才能建立物件。如需每種物件類型的規格、請參閱 "ce87eb03803d5633c163541464e9e7f2"。
- 如果物件已經存在、行為會有所不同：後端會更新現有物件、而其他所有物件類型都會使作業失敗。

刪除

「刪除<trident位址>/trident / v1/<object-typed>/<object-name>：刪除命名的資源。

> 與後端或儲存類別相關聯的磁碟區將繼續存在、必須分別刪除。如需詳細資訊、請參閱 "ce87eb03803d5633c163541464e9e7f2"。

如需這些API的名稱範例、請傳遞DEBUG（`-d'）旗標。如需詳細資訊、請參閱 "ce87eb03803d5633c163541464e9e7f2"。

命令列選項

Astra Trident提供Trident Orchestrator的數個命令列選項。您可以使用這些選項來修改部署。

記錄

- debug'：啟用除錯輸出。
- 記錄層級：設定記錄層級（偵錯、資訊、警告、錯誤、嚴重）。預設為資訊。

Kubernetes

- 「-k8s_pod」：使用此選項或「-k8s_API_server'」來啟用Kubernetes支援。設定此選項會使Trident使用內含Pod的Kubernetes服務帳戶認證、來聯絡API伺服器。這只有當Trident在Kubernetes叢集中以Pod形式執行、且已啟用服務帳戶時才會運作。
- k8s_API_server <ininside-address:inimite-port>:使用此選項或`-k8s_pod'來啟用Kubernetes支援。如果指定、Trident會使用提供的不安全位址和連接埠、連線至Kubernetes API伺服器。這可讓Trident部署在Pod外部、但它只支援不安全的API伺服器連線。若要安全連線、請使用「-k8s_pod」選項、在Pod中部署Trident。
- 「-k8s_config_path <file>'：必要；您必須指定KubeConfig檔案的路徑。

Docker

- -`-volume驅動程式<name>：登錄Docker外掛程式時使用的驅動程式名稱。預設為「NetApp」。
- -`driver_port <port-number>'：偵聽此連接埠、而非UNIX網域通訊端。
- 「-config」：必要；您必須指定後端組態檔的路徑。

休息

- -`-address<ip-or主機>'：指定Trident的REST伺服器應接聽的位址。預設為localhost。當偵聽localhost並在Kubernetes Pod內部執行時、無法從Pod外部直接存取REST介面。使用「-address」（位址）可讓REST介面從Pod IP位址存取。

> Trident REST介面可設定為偵聽、僅適用於127.0.0.1（適用於IPV4）或[:1]（適用於IPv6）。

- -`-port <port-number>'：指定Trident的REST伺服器應接聽的連接埠。預設為8000。

- 「-REST」：啟用REST介面。預設為true。

與Kubernetes整合的NetApp產品

NetApp儲存產品組合可與Kubernetes叢集的許多不同層面整合、提供進階資料管理功能、強化Kubernetes部署的功能、功能、效能及可用度。

Astra

"Astra" 讓企業更輕鬆地管理、保護及搬移在公有雲內部及內部部署的Kubernetes上執行的資料豐富的容器化工作負載。Astra採用NetApp在公有雲和內部部署環境中備受肯定且廣泛的儲存產品組合Trident、提供持續的容器儲存設備。此外、它還提供一組豐富的進階應用程式感知資料管理功能、例如Snapshot、備份與還原、活動記錄、以及資料保護、災難/資料恢復、資料稽核、以及Kubernetes工作負載的移轉使用案例的主動複製。

ONTAP

NetApp的多重傳輸協定、統一化儲存作業系統可為任何應用程式提供進階資料管理功能。ONTAP支援所有Flash、混合式或全硬碟組態的系統、可提供多種不同的部署模式、包括工程設計硬體（英文版）、白箱（英文版）和僅雲端（英文版）ONTAP FAS AFF ONTAP Select Cloud Volumes ONTAP 。

> Trident支援上述ONTAP 所有的功能齊全的部署模式。

Cloud Volumes ONTAP

"Cloud Volumes ONTAP" 是一款純軟體的儲存應用裝置、可在ONTAP 雲端上執行功能完善的資料管理軟體。您可以使用Cloud Volumes ONTAP 支援功能來執行正式作業工作負載、災難恢復、DevOps、檔案共享和資料庫管理。它提供儲存效率、高可用度、資料複寫、資料分層和應用程式一致性、將企業儲存設備延伸至雲端。

Amazon FSX for NetApp ONTAP 產品

"Amazon FSX for NetApp ONTAP 產品" 是一項完整管理的AWS服務、可讓客戶啟動及執行採用NetApp ONTAP的一套資訊儲存作業系統的檔案系統。FSX for ONTAP VMware可讓客戶運用熟悉的NetApp功能、效能和管理功能、同時充分發揮儲存AWS資料的簡易性、敏捷度、安全性和擴充性。FSX ONTAP for FSfor支援ONTAP的許多檔案系統功能和管理API。

Element軟體

"元素" 儲存管理員可藉由保證效能、並簡化及簡化儲存設備佔用空間、來整合工作負載。搭配API來實現儲存管理所有層面的自動化、Element可讓儲存管理員以更少的心力完成更多工作。

NetApp HCI

"NetApp HCI" 將例行工作自動化、讓基礎架構管理員能夠專注於更重要的功能、進而簡化資料中心的管理與規模。

Trident完全支援此功能。NetApp HCITrident可直接針對底層NetApp HCI 的資訊儲存平台、為容器化應用程式配置及管理儲存設備。

Azure NetApp Files

"Azure NetApp Files" 是採用NetApp技術的企業級Azure檔案共享服務。您可以在Azure原生環境中執行最嚴苛的檔案型工作負載、並享有NetApp所提供的效能與豐富資料管理功能。

適用於 Google Cloud Cloud Volumes Service

"適用於Cloud Volumes Service Google Cloud的NetApp解決方案" 是雲端原生檔案服務、可透過NFS和SMB提供NAS磁碟區的All Flash效能。此服務可讓任何工作負載（包括舊版應用程式）在GCP雲端上執行。它提供完整的託管服務、提供一致的高效能、即時複製、資料保護、以及安全存取Google Compute Engine（GCE）執行個體。

Kubernetes和Trident物件

您可以透過讀取和寫入資源物件、使用REST API與Kubernetes和Trident互動。Kubernetes與Trident、Trident與Storage、Kubernetes與儲存設備之間有幾個資源物件、分別是它們之間的關係。其中有些物件是透過Kubernetes進行管理、其他物件則是透過Trident進行管理。

物件如何彼此互動？

瞭解物件、物件的適用範圍及其互動方式、最簡單的方法可能是遵循Kubernetes使用者的單一儲存要求：

1. 使用者會建立一個「PeristentVolume Claim」、要求系統管理員先前設定的Kubernetes「torageClass」中的特定大小的新「PersistentVolume」。
2. Kubernetes「torageClass」可將Trident識別為其資源配置程式、並包含可告知Trident如何為所要求的類別資源配置Volume的參數。
3. Trident查看自己的「儲存類」、其名稱與用來為類別配置磁碟區的「後端」和「儲存類」相符。
4. Trident會在相符的後端上配置儲存設備、並建立兩個物件：Kubernetes的「PeristentVolume」、告訴Kubernetes如何尋找、掛載及處理Volume、以及Trident中保留「PeristentVolume」與實際儲存設備之間關係的Volume。
5. Kubernetes將「PeristentVolume Claim」連結到新的「PeristentVolume」。在執行的任何主機上、包含PeristentVolume的「PeristentVolume Claim」掛載的Pod。
6. 使用者使用指向Trident的「Volume SnapshotClass」建立現有的永久虛擬磁碟的「Volume Snapshot」。
7. Trident會識別與該PVc相關聯的磁碟區、並在其後端建立磁碟區快照。它也會建立「Volume SnapshotContent」、指示Kubernetes如何識別快照。
8. 使用者可以使用「Volume Snapshot」作為來源來建立「PeristentVolume Claim」。
9. Trident會識別所需的快照、並執行建立「PeristentVolume」和「Volume」所需的相同步驟。

如需進一步瞭解Kubernetes物件、我們強烈建議您閱讀 "持續磁碟區" Kubernetes文件的一節。

Kubernetes PeristentVolume Claim物件

Kubernetes「PeristentVolume Claim」物件是Kubernetes叢集使用者所提出的儲存要求。

除了標準規格之外、Trident還可讓使用者指定下列Volume專屬附註、以覆寫您在後端組態中設定的預設值：

註釋	**Volume**選項	支援的驅動程式
trident.netapp.io/fileSystem	檔案系統	ONTAP-SAN、solidfire-san、Eseries -iSCSI、ONTAP-san經濟型
trident.netapp.io/cloneFromPVC	cloneSourceVolume	ONTAP-NAS、ONTAP-SAN、solidfire-san、azure-NetApp-Files、GCP-CVS、 ONTAP-san經濟型
trident.netapp.io/splitOnClone	分岔OnClone	ONTAP-NAS、ONTAP-SAN
trident.netapp.io/protocol	傳輸協定	任何
trident.netapp.io/exportPolicy	匯出原則	ONTAP-NAS、ONTAP-NAS-經濟型、ONTAP-NAS- Flexgroup
trident.netapp.io/snapshotPolicy	Snapshot原則	ONTAP-NAS、ONTAP-NAS-經濟型、ONTAP-NAS-flexgroup、ONTAP-SAN
trident.netapp.io/snapshotReserve	Snapshot保留區	ONTAP-NAS、ONTAP-NAs-flexgroup、ONTAP-SAN、GCP-CVS
trident.netapp.io/snapshotDirectory	Snapshot目錄	ONTAP-NAS、ONTAP-NAS-經濟型、ONTAP-NAS- Flexgroup
trident.netapp.io/unixPermissions	unix權限	ONTAP-NAS、ONTAP-NAS-經濟型、ONTAP-NAS- Flexgroup
trident.netapp.io/blockSize	區塊大小	solidfire-san

如果建立的PV具有「刪除」回收原則、則當PV釋出時（亦即使用者刪除PVc時）、Trident會同時刪除PV和備用Volume。如果刪除動作失敗、Trident會將PV標示為這樣、並定期重試該作業、直到成功或手動刪除PV為止。如果PV使用「+Retain +」原則、Trident會忽略它、並假設系統管理員會從Kubernetes和後端進行清理、以便在移除之前備份或檢查磁碟區。請注意、刪除PV並不會導致Trident刪除背板Volume。您應該使用REST API（「tridentctl」）將其移除。

Trident支援使用csi規格建立Volume Snapshot：您可以建立Volume Snapshot、並將其作為資料來源來複製現有的PVCS。如此一來、PV的時間點複本就能以快照形式呈現給Kubernetes。快照可用來建立新的PV。請參閱「隨需磁碟區快照」、瞭解這項功能的運作方式。

Trident也提供「cloneFromPVc」和「shplitOnClone」註釋、以建立複本。您可以使用這些附註來複製一個PVC,而不需要使用csi實作（在Kubernetes 1.13及更早版本上）或Kubernetes版本不支援beta Volume Snapshot（Kubernetes 1.16及更早版本）。請記住、Trident 19.10支援從PVc複製的csi工作流程。

> 您可以將「cloneFromPVc」和「shplitOnClone」註解搭配「csi Trident」以及傳統的非csi前端使用。

以下是一個範例：如果使用者已經有一個名為「mysql"的PVC,則使用者可以使用「trident.netapp.io/cloneFromPVC: mySQL」之類的註解來建立一個名為「mysqlclone」的新PVC.使用此註釋集、Trident會複製對應於mySQL PVc的磁碟區、而非從頭開始配置磁碟區。

請考量以下幾點：

- 我們建議您複製閒置的Volume。
- 一個PVc及其複本應位於相同的Kubernetes命名空間中、且具有相同的儲存類別。
- 有了「ONTAP-NAS」和「ONTAP-SAN」驅動程式、可能需要將「trident.netapp.io/splitOnClone`」標註與「trident.netapp.io/cloneFromPVC`」一起設定。Trident將trident.netapp.io/splitOnClone`設為「true」、將複製的磁碟區從父磁碟區分割出來、因此將複製的磁碟區的生命週期與其父磁碟區完全分離、而犧牲部分儲存效率。如果不將「trident.netapp.io/splitOnClone`」設定為「假」、則會減少後端的空間使用量、而犧牲父磁碟區與複製磁碟區之間的相依性、使父磁碟區無法刪除、除非先刪除複本。分割實體複製是合理的做法、是將空的資料庫磁碟區複製到磁碟區及其實體複製環境、以大幅分散差異、而非ONTAP 受益於由NetApp提供的儲存效率。

「shample-input」目錄包含可與Trident搭配使用的PVc定義範例。請參閱Trident Volume物件、以取得與Trident Volume相關的參數和設定完整說明。

Kubernetes PersistentVolume物件

Kubernetes「PersistentVolume」物件代表Kubernetes叢集可用的儲存設備。它的生命週期與使用它的Pod無關。

> Trident會建立「PeristentVolume」物件、並根據其所配置的磁碟區、自動在Kubernetes叢集上登錄。您不需要自行管理。

當您建立參照Trident型「TorageClass」的PVc時、Trident會使用對應的儲存類別來配置新的Volume、並針對該Volume登錄新的PV。在設定已配置的Volume和對應的PV時、Trident遵循下列規則：

- Trident會產生Kubernetes的PV名稱、以及用來配置儲存設備的內部名稱。在這兩種情況下、都是確保名稱在其範圍內是唯一的。
- 磁碟區的大小會盡可能接近在室早中所要求的大小、不過視平台而定、磁碟區可能會四捨五入至最接近的可分配數量。

Kubernetes StorageClass物件

Kubernetes的「torageClass」物件是以名稱在「PeristentVolume Claims」中指定、以一組內容來配置儲存設備。儲存類別本身會識別要使用的資源配置程式、並根據資源配置程式所瞭解的方式來定義該組內容。

這是需要由系統管理員建立及管理的兩個基本物件之一。另一個是Trident後端物件。

使用Trident的Kubernetes「torageClass」物件看起來像這樣：

```yaml
apiVersion: storage.k8s.io/v1beta1
kind: StorageClass
metadata:
  name: <Name>
provisioner: csi.trident.netapp.io
mountOptions: <Mount Options>
parameters:
  <Trident Parameters>
allowVolumeExpansion: true
volumeBindingMode: Immediate
```

這些參數是Trident專屬的、可告訴Trident如何為類別配置Volume。

儲存類別參數包括：

屬性	類型	必要	說明
屬性	map[stric]字串	否	請參閱以下「屬性」一節
storagePools	map[stringList	否	將後端名稱對應至中的儲存資源池清單
其他StoragePools	map[stringList	否	將後端名稱對應至中的儲存資源池清單
排除StoragePools	map[stringList	否	將後端名稱對應至中的儲存資源池清單

儲存屬性及其可能值可分類為儲存資源池選擇屬性和Kubernetes屬性。

儲存資源池選擇屬性

這些參數決定應使用哪些Trident託管儲存資源池來配置特定類型的磁碟區。

屬性	類型	價值	優惠	申請	支援者
媒體1^	字串	HDD、混合式、SSD	資源池包含此類型的媒體、混合式表示兩者	指定的媒體類型	ONTAP-NAS、ONTAP-NAS-經濟型、ONTAP-NAS-flexgroup、ONTAP-SAN、solidfire-san
資源配置類型	字串	纖薄、厚實	Pool支援此資源配置方法	指定的資源配置方法	厚：全ONTAP 系列支援iSCSI；薄型：全ONTAP 系列支援整合式SAN

屬性	類型	價值	優惠	申請	支援者
後端類型	字串	ONTAP-NAS、ONTAP-NAS-經濟型、ONTAP-NAS-flexgroup、ONTAP-SAN、solidfire-san、 Eseries -iSCSI、GCP-CVS、azure-NetApp-Files、ONTAP-san經濟型	集區屬於此類型的後端	指定後端	所有驅動程式
快照	布爾	對、錯	集區支援具有快照的磁碟區	已啟用快照的Volume	ONTAP-NAS、ONTAP-SAN、Solidfire-SAN、GCP-CVS
複製	布爾	對、錯	資源池支援複製磁碟區	已啟用複本的Volume	ONTAP-NAS、ONTAP-SAN、Solidfire-SAN、GCP-CVS
加密	布爾	對、錯	資源池支援加密磁碟區	已啟用加密的Volume	ONTAP-NAS、ONTAP-NAS-經濟型、ONTAP-NAS-FlexGroups、ONTAP-SAN
IOPS	內部	正整數	集區能夠保證此範圍內的IOPS	Volume保證這些IOPS	solidfire-san

[1]：ONTAP Select 不受支援

在大多數情況下、所要求的值會直接影響資源配置、例如、要求完整資源配置會導致資源配置較為密集的Volume。不過、元素儲存資源池會使用其提供的IOPS下限和上限來設定QoS值、而非所要求的值。在此情況下、要求的值僅用於選取儲存資源池。

理想情況下、您可以單獨使用「屬性」來建構儲存設備的品質、以滿足特定類別的需求。Trident會自動探索並選取符合您指定「屬性」的_all_儲存集區。

如果您發現自己無法使用「屬性」來自動選取適合某個類別的資源池、您可以使用「儲存池」和「其他儲存池」參數來進一步精簡資源池、甚至選取特定的資源池集區。

您可以使用「儲存池」參數、進一步限制符合任何指定「屬性」的集區集區集區。換句話說、Trident會使用由「屬性」和「儲存庫」參數所識別的資源池交會來進行資源配置。您可以單獨使用參數、也可以同時使用兩者。

您可以使用「addionalStoragePools」參數來擴充Trident用來資源配置的資源池集區集區集區、而不論「attributes」和「scoragePools」參數所選取的任何資源池為何。

您可以使用「排除StoragePools」參數來篩選Trident用於資源配置的資源池集區集區。使用此參數會移除任何相

符的集區。

在「儲存池」和「其他儲存池」參數中、每個項目的格式均為「<backender>:<storagePoollist>'」、其中「<storagePoollist>'」是以逗號分隔的儲存池清單、用於指定的後端。例如、「addionalStoragePools」的值可能會像是「ontapnas_192.168.1.100:solidgr1、aggr2、aggrfire、192.168.1.101：Bronze」。這些清單接受後端值和清單值的regex值。您可以使用「tridentctl Get backend」來取得後端及其資源池的清單。

Kubernetes屬性

這些屬性在動態資源配置期間、不會影響Trident選擇儲存資源池/後端。相反地、這些屬性只會提供Kubernetes持續磁碟區所支援的參數。工作節點負責檔案系統建立作業、可能需要檔案系統公用程式、例如xfsprogs。

屬性	類型	價值	說明	相關驅動因素	**Kubernetes**版本
FSType	字串	ext4、ext3、xfs等	區塊磁碟區的檔案系統類型	solidfire-san、ontap、nap、nap、nas經濟、ontap、nas、flexgroup、ontap、san、ONTAP-san經濟型、E系列-iSCSI	全部
owVolume擴充	布林值	對、錯	啟用或停用對增加PVc大小的支援	ONTAP-NAS、ONTAP-NAS-經濟型、ONTAP-NAS-flexgroup、ONTAP-SAN、ONTAP-san經濟型、solidfire-san、gcp-CVS、azure-netapp檔案	1.11+
Volume BindingMode	字串	立即、WaitForFirst消費者	選擇何時進行磁碟區繫結和動態資源配置	全部	1.18 - 1.24

- 「FSType」參數用於控制SAN LUN所需的檔案系統類型。此外、Kubernetes也會在儲存類別中使用「FSType」表示檔案系統存在。只有在設定「FSType」時、才能使用Pod的「fsGroup」安全內容來控制Volume擁有權。請參閱 "Kubernetes：設定Pod或Container的安全內容" 如需使用「fsGroup」內容設定磁碟區所有權的總覽。Kubernetes只會在下列情況下套用「fsGroup」值：
  - 「FSType」是在儲存類別中設定的。
  - PVc存取模式為rwo。

  對於NFS儲存驅動程式、檔案系統已存在做為NFS匯出的一部分。為了使用「fsGroup」、儲存類別仍需指定「FSType」。您可以將其設定為「NFS」或任何非null值。
- 請參閱 "展開Volume" 如需磁碟區擴充的詳細資料、
- Trident安裝程式套裝組合提供多個範例儲存類別定義、可與Trident搭配使用、位於「sham-INPUT /儲存設備類別-*。yaml」。刪除Kubernetes儲存類別也會刪除對應的Trident儲存類別。

Kubernetes Volume SnapshotClass物件

Kubernetes的「Volume SnapshotClass」物件類似於「儲存類別」。它們有助於定義多種儲存類別、並由Volume Snapshot參考、以將快照與所需的Snapshot類別建立關聯。每個Volume Snapshot都與單一Volume Snapshot類別相關聯。

系統管理員應定義「Volume SnapshotClass」、以建立快照。建立具有下列定義的Volume Snapshot類別：

```
apiVersion: snapshot.storage.k8s.io/v1beta1
kind: VolumeSnapshotClass
metadata:
  name: csi-snapclass
driver: csi.trident.netapp.io
deletionPolicy: Delete
```

對Kubernetes而言、「driver」是指Trident處理「Cig-snapClass」類別的Volume快照要求。「刪除原則」指定必須刪除快照時要採取的動作。當「刪除原則」設定為「刪除」時、刪除快照時、就會移除儲存叢集上的Volume Snapshot物件和基礎Snapshot。或者、將其設為「保留」、表示保留「Volume SnapshotContent」和實體快照。

Kubernetes Volume Snapshot物件

Kubernetes「Volume Snapshot」物件是建立磁碟區快照的要求。就像使用者針對磁碟區所提出的要求一樣、磁碟區快照是使用者建立現有虛擬磁碟快照的要求。

當磁碟區快照要求出現時、Trident會自動管理後端磁碟區的快照建立、並建立獨特的「Volume SnapshotContent」物件來公開快照。您可以從現有的PVCS建立快照、並在建立新的PVCS時、將快照作為DataSource使用。

> Volume Snapshot的生命週期與來源PVCs無關：即使刪除來源PVCs、快照仍會持續存在。刪除具有相關快照的永久虛擬磁碟時、Trident會將此永久虛擬磁碟的備份磁碟區標示為*刪除*狀態、但不會將其完全移除。刪除所有相關的快照時、即會移除該磁碟區。

Kubernetes Volume SnapshotContent物件

Kubernetes「Volume SnapshotContent」物件代表從已配置的磁碟區擷取的快照。它類似於「PersistentVolume」、代表儲存叢集上已配置的快照。與「PeristentVolume Claim」和「PeristentVolume」物件類似、建立快照時、「Volume SnapshotContent」物件會維持一對一的對應、以對應「Volume Snapshot」物件、該物件已要求建立快照。

> Trident會建立「Volume SnapshotContent」物件、並根據其所配置的磁碟區、自動在Kubernetes叢集上登錄。您不需要自行管理。

「Volume SnapshotContent」物件包含可唯一識別快照的詳細資料、例如「快照資料」。此「快照處理」是PV名稱與「Volume SnapshotContent」物件名稱的獨特組合。

當快照要求出現時、Trident會在後端建立快照。建立快照之後、Trident會設定「Volume SnapshotContent」物件、並將快照公開給Kubernetes API。

Kubernetes Custom資源 定義物件

Kubernetes自訂資源是Kubernetes API中由系統管理員定義的端點、用於將類似物件分組。Kubernetes支援建立自訂資源來儲存物件集合。您可以執行「kubecl Get crds」來取得這些資源定義。

自訂資源定義（CRD）及其相關的物件中繼資料會由Kubernetes儲存在其中繼資料儲存區中。如此一來、您就不需要另外建立Trident的儲存區。

從19.07版開始、Trident使用許多「CustomResourceDefinition」物件來保留Trident物件的身分、例如Trident後端、Trident儲存類別和Trident Volume。這些物件由Trident管理。此外、「csi Volume Snapshot」架構也引進了定義Volume快照所需的部分CRD。

CRD是Kubernetes建構。上述資源的物件是由Trident所建立。例如、當使用「tridentctl」建立後端時、Kubernetes會建立一個對應的「tridentbackend」CRD物件供其使用。

以下是Trident客戶需求日的幾點重點：

- 安裝Trident時、會建立一組客戶需求日、並可像使用任何其他資源類型一樣使用。
- 從舊版Trident（使用「etcd」來維護狀態）升級時、Trident安裝程式會從「etcd」金鑰值資料儲存區移轉資料、並建立對應的CRD物件。
- 使用「tridentctl uninstall」命令解除安裝Trident時、會刪除Trident Pod、但不會清除建立的CRD。請參閱 "解除安裝Trident" 瞭解如何徹底移除Trident並從頭重新設定。

Trident StorageClass物件

Trident會為Kubernetes「torageClass」物件建立相符的儲存類別、並在其資源配置程式欄位中指定「csi.trident.netapp.io`/`netapp.io/trident`」。儲存類別名稱與Kubernetes「torageClass」物件的名稱相符。

使用Kubernetes、當Kubernetes「torageClass」以Trident做為資源配置程式登錄時、就會自動建立這些物件。

儲存類別包含一組磁碟區需求。Trident會將這些需求與每個儲存資源池中的屬性相符；如果符合、則該儲存資源池是使用該儲存類別來配置磁碟區的有效目標。

您可以使用REST API建立儲存類別組態、以直接定義儲存類別。不過、在Kubernetes部署中、我們預期在登錄新的Kubernetes「torageClass」物件時、會建立這些物件。

Trident後端物件

後端代表儲存供應商、其中Trident會配置磁碟區；單一Trident執行個體可管理任何數量的後端。

這是您自己建立和管理的兩種物件類型之一。另一個是Kubernetes的「torageClass」物件。

如需如何建構這些物件的詳細資訊、請參閱後端組態。

Trident StoragePool物件

儲存資源池代表可在每個後端上進行資源配置的不同位置。就支援而言ONTAP 、這些項目對應於SVM中的集合體。對於NetApp HCI / SolidFire、這些服務會對應到系統管理員指定的QoS頻段。就架構而言、這些項目對應於雲端供應商所在的地區。Cloud Volumes Service每個儲存資源池都有一組獨特的儲存屬性、可定義其效能特性和資料保護特性。

與此處的其他物件不同、儲存資源池候選項目一律會自動探索及管理。

Trident Volume物件

Volume是資源配置的基本單位、包含NFS共用和iSCSI LUN等後端端點。在Kubernetes中、這些內容直接對應到「PersistentVolumes」。建立磁碟區時、請確定它有一個儲存類別、決定該磁碟區可以配置的位置及大小。

在Kubernetes中、會自動管理這些物件。您可以檢視這些資源、以查看資源配置的Trident內容。

刪除具有相關快照的PV時、對應的Trident Volume會更新為*刪除*狀態。若要刪除Trident磁碟區、您應該移除該磁碟區的快照。

Volume組態會定義已配置磁碟區應具備的內容。

屬性	類型	必要	說明
版本	字串	否	Trident API版本（「1」）
名稱	字串	是的	要建立的Volume名稱
storageClass	字串	是的	配置Volume時使用的儲存類別
尺寸	字串	是的	要配置的磁碟區大小（以位元組為單位）

屬性	類型	必要	說明
傳輸協定	字串	否	要使用的傳輸協定類型；「檔案」或「區塊」
內部名稱	字串	否	儲存系統上的物件名稱；由Trident產生
cloneSourceVolume	字串	否	Sname（NAS、SAN）& S--*：要複製的磁碟區名稱ONTAP SolidFire
分岔OnClone	字串	否	例（NAS、SAN）：從父實體分割複本ONTAP
Snapshot原則	字串	否	S--*：快照原則ONTAP
Snapshot保留區	字串	否	Sing-*：保留給快照的磁碟區百分比ONTAP
匯出原則	字串	否	ONTAP-NAS*：要使用的匯出原則
Snapshot目錄	布爾	否	ONTAP-NAS*：快照目錄是否可見
unix權限	字串	否	ONTAP-NAS*：初始UNIX權限
區塊大小	字串	否	S--*：區塊/區段大小SolidFire
檔案系統	字串	否	檔案系統類型

Trident在建立磁碟區時會產生「內部名稱」。這包括兩個步驟。首先、它會將儲存前置詞（預設的「Trident」或後端組態中的前置詞）預先加上磁碟區名稱、以「<prefix>-<volume名稱>」格式命名。然後、它會繼續清理名稱、取代後端不允許的字元。對於後端、它會以底線取代連字號（因此內部名稱會變成「<prefix>_<volume名稱>」）ONTAP 。對於元素後端、它會以連字號取代底線。

您可以使用Volume組態、使用REST API直接配置磁碟區、但在Kubernetes部署中、我們預期大多數使用者都會使用標準的Kubernetes「PeristentVolume Claim」方法。Trident會自動建立此Volume物件、做為資源配置程序的一部分。

Trident Snapshot物件

快照是磁碟區的時間點複本、可用來配置新的磁碟區或還原狀態。在Kubernetes中、這些物件會直接對應到「Volume SnapshotContent」物件。每個快照都與一個Volume相關聯、該磁碟區是快照資料的來源。

每個「napshot」物件都包含下列內容：

屬性	類型	必要	說明
版本	字串	是的	Trident API版本（「1」）
名稱	字串	是的	Trident Snapshot物件的名稱

屬性	類型	必要	說明
內部名稱	字串	是的	儲存系統上Trident Snapshot物件的名稱
Volume名稱	字串	是的	為其建立快照的持續Volume名稱
Volume內部名稱	字串	是的	儲存系統上相關Trident Volume物件的名稱

在Kubernetes中、會自動管理這些物件。您可以檢視這些資源、以查看資源配置的Trident內容。

當Kubernetes「Volume Snapshot」物件要求建立時、Trident會在備份儲存系統上建立Snapshot物件。此快照物件的「內部名稱」是將前置詞「sfapshot-」與「Volume Snapshot」物件的「UID」（例如、「sfapshot-e8d8a0ca-9826-11e9-9807-525400f3f660」）結合在一起產生的。「Volume Name」（Volume名稱）和「Volume InternalName」（磁碟區內部名稱）會透過取得備用磁碟區的詳細資料來填入資料。

tridentctl命令和選項

。 "Trident安裝程式套裝組合" 內含命令列公用程式「tridentctl」、可讓您輕鬆存取Astra Trident。擁有足夠權限的Kubernetes使用者可以使用它來安裝Astra Trident、以及直接與它互動、以管理包含Astra Trident Pod的命名空間。

如需使用資訊、請執行「tridentctl --help"。

可用的命令和全域選項包括：

```
Usage:
  tridentctl [command]
```

可用命令：

- 「Create」：新增資源至Astra Trident。
- 「刪除」：從Astra Trident移除一或多個資源。
- 「Get」（取得）：從Astra Trident取得一或多項資源。
- 「說明」：任何命令的相關說明。
- 「影像」：列印容器影像Astra Trident需求的表格。
- 「匯入」：將現有資源匯入Astra Trident。
- 「安裝」：安裝Astra Trident。
- 「記錄」：從Astra Trident列印記錄。
- 「結束」：從Astra Trident傳送資源。
- 「解除安裝」：解除安裝Astra Trident。
- 「更新」：修改Astra Trident中的資源。

- 「升級」：升級Astra Trident的資源。
- 《版本》：印出Astra Trident的版本。

旗標：

- 「-d、--debug'：偵錯輸出。
- "h-h,--help"：對"tridentctl"的幫助。
- 「-n、--namespace字串」：Astra Trident部署的命名空間。
- 「-o」、「-outputstr字 串」：輸出格式。json之一| yaml| name| wl| ps（預設）。
- 「-s、--server字串」：Astra Trident REST介面的位址/連接埠。

> Trident REST介面可設定為偵聽、僅適用於127.0.0.1（適用於IPV4）或[:1]（適用於IPv6）。

> Trident REST介面可設定為偵聽、僅適用於127.0.0.1（適用於IPV4）或[:1]（適用於IPv6）。

建立

您可以使用執行「create」命令、將資源新增至Astra Trident。

```
Usage:
  tridentctl create [option]
```

可用選項：「後端」：新增後端至Astra Trident。

刪除

您可以執行「刪除」命令、從Astra Trident移除一或多個資源。

```
Usage:
  tridentctl delete [option]
```

可用選項：

- 「後端」：從Astra Trident刪除一或多個儲存後端。
- 「節點」：從Astra Trident刪除一或多個SCSI節點。
- 「napshot」：從Astra Trident刪除一或多個Volume快照。
- 「torageclass'：從Astra Trident刪除一或多個儲存類別。
- 「Volume」（磁碟區）：從Astra Trident刪除一或多個儲存磁碟區。

取得

您可以執行「Get」命令、從Astra Trident取得一或多個資源。

```
Usage:
  tridentctl get [option]
```

可用選項：

- 「後端」：從Astra Trident取得一或多個儲存後端。
- 「napshot」：從Astra Trident取得一或多個快照。
- 「torageclass'：從Astra Trident取得一或多個儲存課程。
- 「Volume」：從Astra Trident取得一或多個Volume。

映像

您可以執行「影像」旗標、以列印容器影像Astra Trident需求的表格。

```
Usage:
  tridentctl images [flags]
```

旗標：、**-help'**：影像說明。`-v、--k8s-version字串：語義版本的Kubernetes叢集。

匯入Volume

您可以執行「匯入Volume」命令、將現有磁碟區匯入Astra Trident。

```
Usage:
  tridentctl import volume <backendName> <volumeName> [flags]
```

別名：「Volume、v」

旗標：

- `-f、--FileName字串：Yaml或Json PVc檔案的路徑。
- 「-h、-help'：Volume說明。
- 「無管理」：僅建立PV/PVc。不要假設磁碟區生命週期管理。

安裝

您可以執行「install」旗標來安裝Astra Trident。

```
Usage:
  tridentctl install [flags]
```

旗標：

- 「-autosup-image字串」：AutoSupport 適用於「遙測」的容器影像（預設為「NetApp/triident autosupport：20.07.0」）。
- 「-autosupport-proxy]：代理伺服器的位址/連接埠、用於傳送AutoSupport 「遙測」。
- 「-csi」：安裝csi Trident（僅適用於Kubernetes 1.13的置換功能、需要功能閘道）。
- 「-enable節點準備」：嘗試在節點上安裝所需的套件。
- 「-generame-custom-yaml」：產生Yaml檔案、無需安裝任何內容。
- 「-h、-help'：安裝說明。
- 「-http-requer-timeout」：覆寫Trident控制器REST API的HTTP要求逾時（預設值為1m30s）。
- 「-image-registry字串」：內部映像登錄的位址/連接埠。
- 「-k8s-timeout duration」（k8s-timeout持續時間）：所有Kubernetes作業的逾時時間（預設為3個月）。
- 「-kubelet-dir字串」：kubelet內部狀態的主機位置（預設為「/var/lib/kubelet」）。
- 「-log-format字串」：Astra Trident記錄格式（文字、json）（預設為「text」）。
- 「-PV字串」：Astra Trident使用的舊PV名稱、可確保不存在（預設為「Trident」）。
- 「-PVc字串」：Astra Trident使用的舊版PVc名稱、可確保不存在（預設為「Trident」）。
- 「-Silence-autosupport」：請勿AutoSupport 自動將支援套裝組合傳送至NetApp（預設為true）。
- 「-silent」：安裝期間停用大部分的輸出。
- 「-trident映像字串」：要安裝的Astra Trident映像。
- 「用途自訂yaml」：使用安裝目錄中現有的任何Yaml檔案。
- 《使用IPv6》：使用IPv6進行Astra Trident的通訊。

記錄

您可以執行「記錄」旗標、從Astra Trident列印記錄。

```
Usage:
  tridentctl logs [flags]
```

旗標：

- 「-a、-archive」：除非另有說明、否則請使用所有記錄建立支援歸檔。
- 「-h、-help'：記錄說明。
- 「-l、-log字串」：要顯示的Astra Trident記錄。其中一個trident | auto| trident運算子| all（預設為「自動」）。

- 「節點字串」：Kubernetes節點名稱、用於收集節點pod記錄。
- 「-p、- previous」：取得前一個Container執行個體的記錄（如果存在）。
- "-sidecars"：取得sidecar容器的記錄。

傳送

您可以執行「shend」命令、從Astra Trident傳送資源。

```
Usage:
  tridentctl send [option]
```

可用選項：AutoSupport 「無法用」：傳送AutoSupport 一份不全的檔案給NetApp。

解除安裝

您可以執行「解除安裝」旗標來解除安裝Astra Trident。

```
Usage:
  tridentctl uninstall [flags]
```

旗標：、**-help'**：解除安裝說明。「-silent」：解除安裝期間停用大部分的輸出。

更新

您可以執行「update」命令來修改Astra Trident中的資源。

```
Usage:
  tridentctl update [option]
```

可用選項：「後端」：更新Astra Trident的後端。

升級

您可以執行「升級」命令來升級Astra Trident中的資源。

```
Usage:
tridentctl upgrade [option]
```

可用選項：「Volume」（Volume）：將一個或多個持續磁碟區從NFS/iSCSI升級至SCSI。

版本

您可以執行「vrsion」旗標來列印「tridentctl」版本和執行中的Trident服務。

```
Usage:
  tridentctl version [flags]
```

旗標：、用戶端：僅限用戶端版本（不需伺服器）。「-h、--help'：版本說明。

較早版本的文件

如果您未執行Astra Trident 22.04、則可取得先前版本的文件。

若為21：07之前的版本、您將被重新導向至舊版文件網站。

- "Astra Trident 22.01"
- "Astra Trident 21.10"
- "Astra Trident 21.07"
- "Astra Trident 21.04."
- "Astra Trident 21.01."
- "Astra Trident 20.10"
- "Astra Trident 20.07"
- "Astra Trident 20.04"
- "Astra Trident 20.01"
- "Astra Trident 19.10"
- "Astra Trident 19.07"
- "Astra Trident 19.04"
- "Astra Trident 19.01"

法律聲明

法律聲明提供版權聲明、商標、專利等存取權限。

版權

http://www.netapp.com/us/legal/copyright.aspx

商標

NetApp 、 NetApp 標誌及 NetApp 商標頁面上列出的標章均為 NetApp 、 Inc. 的商標。其他公司與產品名稱可能為其各自所有者的商標。

http://www.netapp.com/us/legal/netapptmlist.aspx

專利

如需最新的 NetApp 擁有專利清單、請參閱：

https://www.netapp.com/us/media/patents-page.pdf

隱私權政策

https://www.netapp.com/us/legal/privacypolicy/index.aspx

開放原始碼

您可以在每個版本的通知檔案中檢閱NetApp軟體中用於Astra Trident的協力廠商版權與授權、網址為 https://github.com/NetApp/trident/。

版權資訊

Copyright © 2022 NetApp, Inc. 版權所有。台灣印製。非經版權所有人事先書面同意，不得將本受版權保護文件的任何部分以任何形式或任何方法（圖形、電子或機械）重製，包括影印、錄影、錄音或儲存至電子檢索系統中。

由 NetApp 版權資料衍伸之軟體必須遵守下列授權和免責聲明：

此軟體以 NETAPP「原樣」提供，不含任何明示或暗示的擔保，包括但不限於有關適售性或特定目的適用性之擔保，特此聲明。於任何情況下，就任何已造成或基於任何理論上責任之直接性、間接性、附隨性、特殊性、懲罰性或衍生性損害（包括但不限於替代商品或服務之採購；使用、資料或利潤上的損失；或企業營運中斷），無論是在使用此軟體時以任何方式所產生的契約、嚴格責任或侵權行為（包括疏忽或其他）等方面，NetApp 概不負責，即使已被告知有前述損害存在之可能性亦然。

NetApp 保留隨時變更本文所述之任何產品的權利，恕不另行通知。NetApp 不承擔因使用本文所述之產品而產生的責任或義務，除非明確經過 NetApp 書面同意。使用或購買此產品並不會在依據任何專利權、商標權或任何其他 NetApp 智慧財產權的情況下轉讓授權。

本手冊所述之產品受到一項（含）以上的美國專利、國外專利或申請中專利所保障。

有限權利說明：政府機關的使用、複製或公開揭露須受 DFARS 252.227-7013（2014 年 2 月）和 FAR 52.227-19（2007 年 12 月）中的「技術資料權利 - 非商業項目」條款 (b)(3) 小段所述之限制。

此處所含屬於商業產品和 / 或商業服務（如 FAR 2.101 所定義）的資料均為 NetApp, Inc. 所有。根據本協議提供的所有 NetApp 技術資料和電腦軟體皆屬於商業性質，並且完全由私人出資開發。 美國政府對於該資料具有非專屬、非轉讓、非轉授權、全球性、有限且不可撤銷的使用權限，僅限於美國政府為傳輸此資料所訂合約所允許之範圍，並基於履行該合約之目的方可使用。除非本文另有規定，否則未經 NetApp Inc. 事前書面許可，不得逕行使用、揭露、重製、修改、履行或展示該資料。美國政府授予國防部之許可權利，僅適用於 DFARS 條款 252.227-7015(b)（2014 年 2 月）所述權利。

商標資訊

NETAPP、NETAPP 標誌及 http://www.netapp.com/TM 所列之標章均為 NetApp, Inc. 的商標。文中所涉及的所有其他公司或產品名稱，均為其各自所有者的商標，不得侵犯。